For a **0–5 year experience Full-Stack Developer**,

1. SQL Fundamentals

These are the **absolute basics every developer must know**.

- What is SQL
- What is a **Relational Database**
- Tables, Rows, Columns
- Primary Key
- Foreign Key
- Data Types (INT, VARCHAR, DATE, BOOLEAN, etc.)
- NULL values
- Constraints
    - NOT NULL
    - UNIQUE
    - DEFAULT
    - CHECK

2. Basic SQL Queries

Core operations used every day.

- `SELECT`
- `WHERE`
- `ORDER BY`
- `DISTINCT`
- `LIMIT / OFFSET`
- `LIKE`
- `BETWEEN`
- `IN`
- `IS NULL / IS NOT NULL`

3. Data Manipulation Language (DML)

- `INSERT`
- `UPDATE`
- `DELETE`
- `MERGE`
- Bulk Insert

4. Data Definition Language (DDL)

- `CREATE DATABASE`
- `CREATE TABLE`
- `ALTER TABLE`
- `DROP TABLE`
- `TRUNCATE`
- Index creation

5. Aggregate Functions

- `COUNT()`
- `SUM()`
- `AVG()`
- `MIN()`
- `MAX()`
- `GROUP BY`
- `HAVING`

6. SQL Joins (VERY IMPORTANT)

- `INNER JOIN`
- `LEFT JOIN`
- `RIGHT JOIN`
- `FULL JOIN`
- `CROSS JOIN`
- Self Join

7. Subqueries

- Scalar subquery
- Correlated subquery
- Nested subquery
- Subquery in `SELECT`
- Subquery in `WHERE`

8. Set Operations

- `UNION`
- `UNION ALL`
- `INTERSECT`
- `EXCEPT`

9. Indexing (Performance)

- What is an Index
- Clustered Index
- Non-clustered Index
- Composite Index
- When to use indexes
- When indexes hurt performance

10. Database Normalization

Important for **database design**.

- 1NF
- 2NF

- 3NF
- BCNF
- Denormalization

11. Transactions

- ACID properties
- `BEGIN`
- `COMMIT`
- `ROLLBACK`
- Savepoints
- Transaction isolation levels

12. Views

- Create view
- Update view
- Materialized views
- Security benefits

13. Stored Procedures

- Creating procedures
- Parameters
- Calling procedures
- Error handling

14. Triggers

- BEFORE triggers
- AFTER triggers
- Audit triggers
- Event triggers

15. Window Functions (Modern SQL)

Very common in **data-heavy applications**.

- `ROW_NUMBER()`
- `RANK()`
- `DENSE_RANK()`
- `LEAD()`
- `LAG()`
- `OVER(PARTITION BY)`
- Running totals

16. Advanced Query Techniques

- Common Table Expressions (CTE)

- Recursive queries
- Pivot / Unpivot
- JSON queries
- Dynamic SQL

17. Performance Optimization

- Query execution plan
- Query tuning
- Index tuning
- Avoiding N+1 queries
- Partitioning
- Caching strategies

18. Security in SQL

- SQL Injection prevention
- Prepared statements
- Roles & permissions
- Encryption

19. Database Design for Applications

- ER diagrams
- One-to-One relationship
- One-to-Many relationship
- Many-to-Many relationship
- Junction tables

20. Backend Integration

A full-stack developer should also know how SQL works with backend frameworks.

Example:

- Node.js + SQL
- ORM concepts
- Connection pooling
- Migrations
- Transactions in backend code

Common SQL databases used in industry:

- MySQL
- PostgreSQL
- Microsoft SQL Server
- SQLite

21. ORM Concepts (Important for Full-Stack)

- What is ORM
- Entity mapping
- Lazy vs eager loading
- Migration tools

Examples:

- Sequelize
- Prisma
- Hibernate

22. Practical Skills Companies Expect

You should be able to:

- Design a database schema
- Write complex joins
- Optimize slow queries
- Write transactions
- Integrate SQL with backend APIs
- Debug production issues

✅ **Reality check:**
Most **0-5 year developers are tested mainly on:**

- Joins
- Group By
- Subqueries
- Window functions
- Indexes
- Transactions
- Database design

SQL Fundamentals: Complete Theory Notes with Practical Examples

1. What is SQL?

SQL (Structured Query Language) is a standardized programming language specifically designed for managing and manipulating relational databases. It allows you to create, read, update, and delete database records.

Key Characteristics:

- **Declarative Language**: You specify WHAT you want, not HOW to get it
- **Case-Insensitive**: SQL keywords can be written in uppercase or lowercase (but uppercase is conventional)
- **Standardized**: ANSI and ISO standards exist, but different databases have slight variations

SQL Command Categories:

Category	Purpose	Examples
DDL (Data Definition Language)	Define structure	CREATE, ALTER, DROP
DML (Data Manipulation Language)	Manage data	SELECT, INSERT, UPDATE, DEL
DCL (Data Control Language)	Control access	GRANT, REVOKE
TCL (Transaction Control Language)	Manage transactions	COMMIT, ROLLBACK

Practical Example:

```sql
```

```sql
-- Simple SQL query to retrieve data
SELECT first_name, last_name
FROM employees
WHERE department = 'IT';
```

2. What is a Relational Database?

A **relational database** organizes data into tables that are related to each other through common fields. It's based on the relational model proposed by E.F. Codd in 1970.

Core Concepts:

- **Data organized in tables** (relations)
- **Relationships between tables** through keys
- **ACID compliance** (Atomicity, Consistency, Isolation, Durability)
- **Data integrity** through constraints

Popular Relational Databases:

- **MySQL/MariaDB**: Open-source, widely used for web applications
- **PostgreSQL**: Advanced open-source, feature-rich
- **Oracle Database**: Enterprise-grade, commercial
- **Microsoft SQL Server**: Microsoft's enterprise solution
- **SQLite**: Lightweight, file-based, perfect for mobile/embedded

Example of Relational Structure:

```text
Database: CompanyDB
├── employees table
├── departments table
└── projects table
Relationships:
- employees belong to departments (one-to-many)
- employees work on projects (many-to-many)
```

3. Tables, Rows, and Columns

Table

A **table** is a collection of related data organized in rows and columns. Think of it like a spreadsheet.

Row (Record/Tuple)

A **row** represents a single, complete record in a table. Each row contains data for all columns defined in the table.

Column (Field/Attribute)

A **column** represents a specific type of information that is the same for all rows in the table.

Visual Representation:

text

```
Table: employees
┌──────────────┬──────────────┬──────────────┐
│ employee_id  │ first_name   │ last_name    │  ← Column Headers
├──────────────┼──────────────┼──────────────┤
│ 1            │ John         │ Doe          │  ← Row/Record
├──────────────┼──────────────┼──────────────┤
│ 2            │ Jane         │ Smith        │  ← Row/Record
├──────────────┼──────────────┼──────────────┤
│ 3            │ Bob          │ Johnson      │  ← Row/Record
└──────────────┴──────────────┴──────────────┘
        ↑             ↑              ↑
     Column        Column         Column
```

Practical Example - Creating a Table:

sql

```sql
-- Creating a table with columns
```

```sql
CREATE TABLE students (
    student_id INT,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    enrollment_date DATE
);

-- Inserting rows (records)
INSERT INTO students VALUES
(1, 'Alice', 'Brown', '2024-01-15'),
(2, 'Bob', 'Wilson', '2024-01-16'),
(3, 'Charlie', 'Davis', '2024-01-20');

-- Selecting specific columns
SELECT first_name, last_name FROM students;

-- Selecting entire rows
SELECT * FROM students;
```

4. Primary Key

A **Primary Key** is a column (or set of columns) that uniquely identifies each row in a table.

Key Characteristics:

- **Uniqueness**: No two rows can have the same primary key value
- **Not NULL**: Primary key cannot contain NULL values
- **Immutable**: Should not change over time
- **Single per table**: Each table can have only ONE primary key

Types of Primary Keys:

1. **Single-column Primary Key**: Uses one column
2. **Composite Primary Key**: Uses multiple columns together
3. **Surrogate Key**: System-generated (like auto-increment integers)
4. **Natural Key**: Uses existing data that's naturally unique (like SSN, Email)

Practical Examples:

sql

```sql
-- Example 1: Single-column Primary Key with AUTO_INCREMENT
CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(100) UNIQUE,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Example 2: Composite Primary Key
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT,
    price DECIMAL(10,2),
    PRIMARY KEY (order_id, product_id)  -- Both columns together form the primary
key
);

-- Example 3: Natural Key as Primary Key
CREATE TABLE countries (
    country_code CHAR(2) PRIMARY KEY,  -- Using ISO country codes (US, IN, UK)
    country_name VARCHAR(100)
);

-- Example 4: Adding Primary Key to existing table
ALTER TABLE employees
ADD PRIMARY KEY (employee_id);

-- Example 5: Dropping Primary Key
ALTER TABLE customers
DROP PRIMARY KEY;
```

Real-world Scenario:

sql

```sql
-- Creating a users table with proper primary key
CREATE TABLE users (
```

```sql
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- This will work
INSERT INTO users (username, email, password_hash)
VALUES ('john_doe', 'john@example.com', 'hash123');

-- This will FAIL (duplicate user_id would be generated by AUTO_INCREMENT,
-- but duplicate username would violate UNIQUE constraint)
INSERT INTO users (username, email, password_hash)
VALUES ('john_doe', 'another@example.com', 'hash456');
```

---

5. Foreign Key

A **Foreign Key** is a column (or set of columns) that creates a link between data in two tables. It references the PRIMARY KEY of another table.

Key Concepts:

- **Establishes relationships** between tables
- **Enforces referential integrity**
- **Can be NULL** (unlike primary keys)
- **A table can have multiple foreign keys**

Types of Relationships:

Relationship	Description	Example
One-to-One	One record in Table A relates to one record in Table B	User ↔ UserProfile

Relationship	Description	Example
One-to-Many	One record in Table A relates to many records in Table B	Department ↔ Employees
Many-to-Many	Many records relate to many records (requires junction table)	Students ↔ Cours

Referential Actions:

When a parent record is updated or deleted:

- **CASCADE**: Automatically update/delete child records
- **SET NULL**: Set foreign key to NULL in child records
- **RESTRICT**: Prevent the operation if child records exist
- **NO ACTION**: Similar to RESTRICT (check occurs at different time)
- **SET DEFAULT**: Set to default value (not widely supported)

Practical Examples:

sql

```sql
-- Example 1: Basic One-to-Many Relationship
-- Parent table
CREATE TABLE departments (
    dept_id INT AUTO_INCREMENT PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL
);

-- Child table with Foreign Key
CREATE TABLE employees (
    emp_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

```sql
-- Example 2: Foreign Key with CASCADE
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- Example 3: Multiple Foreign Keys
CREATE TABLE reviews (
    review_id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT,
    user_id INT,
    rating INT CHECK (rating >= 1 AND rating <= 5),
    comment TEXT,
    FOREIGN KEY (product_id) REFERENCES products(product_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

-- Example 4: Many-to-Many relationship using junction table
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100)
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    title VARCHAR(100)
);

-- Junction table with composite foreign keys
CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

Inserting Data with Foreign Keys:

sql

```sql
-- First, insert into parent table
INSERT INTO departments (dept_name) VALUES
('IT'), ('HR'), ('Finance');

-- Then insert into child table
INSERT INTO employees (emp_name, dept_id) VALUES
('Alice', 1),  -- Works: dept_id 1 exists
('Bob', 2),    -- Works: dept_id 2 exists
('Charlie', 3); -- Works: dept_id 3 exists

-- This will FAIL (dept_id 99 doesn't exist in departments)
INSERT INTO employees (emp_name, dept_id) VALUES
('David', 99);
```

---

6. Data Types

Data types define what kind of data can be stored in a column. Choosing the correct data type is crucial for data integrity and performance.

Numeric Data Types:

Type	Range/Description	Storage	Use Case
INT/INTEGER	-2^31 to 2^31-1	4 bytes	IDs, counters, age
TINYINT	-128 to 127 or 0-255	1 byte	Small numbers, boolean flags
SMALLINT	-32768 to 32767	2 bytes	Small ranges

Type	Range/Description	Storage	Use Case
BIGINT	-2^63 to 2^63-1	8 bytes	Large counters
DECIMAL(p,s)	Exact numeric	Variable	Money, precise calculations
FLOAT/DOUBLE	Approximate numeric	4/8 bytes	Scientific calculations

String Data Types:

Type	Description	Max Length	Use Case
CHAR(n)	Fixed-length	255 chars	Codes, fixed formats (e.g., phone)
VARCHAR(n)	Variable-length	65535 chars	Names, emails, addresses
TEXT	Long text	64KB	Articles, comments
MEDIUMTEXT	Longer text	16MB	Books, long documents
LONGTEXT	Longest text	4GB	Very large text
ENUM	Predefined values	65535 values	Status, category

Date/Time Data Types:

Type	Format	Range	Use Case
DATE	YYYY-MM-DD	1000-01-01 to 9999-12-31	Birthdays, events
TIME	HH:MM:SS	-838:59:59 to 838:59:59	Durations, schedules
DATETIME	YYYY-MM-DD HH:MM:SS	1000-01-01 to 9999-12-31	Complete timestamps
TIMESTAMP	YYYY-MM-DD HH:MM:SS	1970-01-01 to 2038-01-19	Audit fields
YEAR	YYYY	1901 to 2155	Year only

Boolean Data Type:

Type	Values	Storage	Notes
BOOLEAN/BOOL	TRUE, FALSE	1 byte	Synonym for TINYINT(1)

Binary Data Types:

Type	Description	Use Case
BLOB	Binary Large Object	Images, files
BINARY(n)	Fixed binary	Hashes, encryption keys

Special Data Types:

Type	Description	Use Case
JSON	JSON data	Modern apps, flexible schemas
UUID	Universal Unique Identifier	Distributed systems
IP ADDRESS	Network addresses	Networking apps
GEOMETRY	Spatial data	Maps, location apps

Practical Examples with Data Types:

sql

```sql
-- Example 1: Comprehensive table with various data types
CREATE TABLE products (
    -- Numeric types
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    sku BIGINT UNIQUE,
    quantity_in_stock SMALLINT DEFAULT 0,
    weight DECIMAL(5,2),  -- 5 total digits, 2 after decimal (e.g., 123.45)
    price DECIMAL(10,2) NOT NULL,
    rating FLOAT,

    -- String types
    product_code CHAR(10) NOT NULL,  -- Fixed length, always 10 chars
    product_name VARCHAR(200) NOT NULL,
    description TEXT,
    status ENUM('active', 'inactive', 'discontinued') DEFAULT 'active',

    -- Date/Time types
    manufacture_date DATE,
    expiry_date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME ON UPDATE CURRENT_TIMESTAMP,
```

```sql
    -- Boolean (using TINYINT)
    is_featured BOOLEAN DEFAULT FALSE,
    is_taxable TINYINT(1) DEFAULT 1,

    -- JSON (MySQL 5.7+)
    attributes JSON
);

-- Example 2: Inserting different data types
INSERT INTO products (
    sku, quantity_in_stock, weight, price, rating,
    product_code, product_name, description, status,
    manufacture_date, expiry_date,
    is_featured, attributes
) VALUES (
    1234567890123,  -- BIGINT
    100,            -- SMALLINT
    1.5,            -- DECIMAL(5,2)
    29.99,          -- DECIMAL(10,2)
    4.5,            -- FLOAT
    'PROD001A',     -- CHAR(10) - fixed
    'Wireless Mouse',  -- VARCHAR(200)
    'Ergonomic wireless mouse with USB receiver',  -- TEXT
    'active',       -- ENUM
    '2024-01-01',   -- DATE
    '2026-01-01',   -- DATE
    TRUE,           -- BOOLEAN
    '{"color": "black", "buttons": 6, "wireless": true}'  -- JSON
);

-- Example 3: Choosing wrong data type - DON'T DO THIS
-- Bad: Using VARCHAR for numeric data
CREATE TABLE bad_example (
    age VARCHAR(3),  -- Should be TINYINT
    price VARCHAR(10),  -- Should be DECIMAL
    date_added VARCHAR(20)  -- Should be DATE/TIMESTAMP
);

-- Good: Using appropriate data types
CREATE TABLE good_example (
    age TINYINT UNSIGNED,  -- Age can't be negative
    price DECIMAL(10,2),   -- Exact for money
```

```
    date_added TIMESTAMP    -- Proper date/time
);
```

Choosing Data Types - Best Practices:

sql

```sql
-- Example 4: User profile table with appropriate data types
CREATE TABLE user_profiles (
    user_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,

    -- Strings
    username VARCHAR(50) NOT NULL,
    email VARCHAR(255) NOT NULL,
    password_hash CHAR(60) NOT NULL,  -- bcrypt hash is always 60 chars

    -- Numeric
    age TINYINT UNSIGNED,  -- 0-255, can't be negative
    height_cm DECIMAL(5,1),  -- e.g., 175.5 cm
    account_balance DECIMAL(12,2) DEFAULT 0.00,  -- For money

    -- Date/Time
    birth_date DATE,
    last_login TIMESTAMP,
    account_expires DATE,

    -- ENUM for limited options
    subscription_tier ENUM('free', 'basic', 'premium', 'enterprise') DEFAULT 'free
',

    -- Boolean flags
    is_email_verified BOOLEAN DEFAULT FALSE,
    is_active BOOLEAN DEFAULT TRUE,

    -- JSON for flexible attributes
    preferences JSON,

    -- Indexes for performance
    INDEX idx_email (email),
    INDEX idx_username (username)
);
```

7. NULL Values

NULL represents the absence of a value or unknown data. It's important to understand that NULL is not the same as zero, empty string, or false.

Key Concepts:

- **NULL means "unknown" or "no value"**
- **NULL is not equal to anything**, not even another NULL
- **Any arithmetic operation with NULL results in NULL**
- **NULL behaves differently in comparisons and aggregations**

Common Misconceptions:

Expression	Result	Explanation
NULL = NULL	NULL	NULL is not equal to itself
NULL != NULL	NULL	Same as above
NULL = 0	NULL	NULL is not zero
NULL = ''	NULL	NULL is not empty string
NULL + 5	NULL	Any operation with NULL yields NULL

Working with NULL:

sql

```sql
-- Example 1: Creating table with NULL and NOT NULL columns
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,  -- Must have value
    email VARCHAR(100),  -- Can be NULL
```

```sql
    phone VARCHAR(20),    -- Can be NULL
    salary DECIMAL(10,2) NOT NULL DEFAULT 0  -- Has default, so not NULL
);

-- Example 2: Inserting NULL values
INSERT INTO employees VALUES
(1, 'John Doe', 'john@email.com', '123-456-7890', 50000),
(2, 'Jane Smith', NULL, NULL, 45000),  -- email and phone are NULL
(3, 'Bob Wilson', 'bob@email.com', NULL, 48000);  -- phone is NULL

-- Example 3: NULL in WHERE clause - WRONG WAY
-- This returns NO rows (even though there are NULL emails)
SELECT * FROM employees WHERE email = NULL;  -- INCORRECT

-- Example 4: NULL in WHERE clause - CORRECT WAY
SELECT * FROM employees WHERE email IS NULL;  -- Returns Jane
SELECT * FROM employees WHERE email IS NOT NULL;  -- Returns John and Bob

-- Example 5: NULL in comparisons
SELECT
    emp_name,
    email,
    CASE
        WHEN email IS NULL THEN 'No email provided'
        ELSE email
    END as email_display
FROM employees;

-- Example 6: NULL in calculations
SELECT
    emp_name,
    salary,
    commission,
    salary + commission as total_compensation  -- NULL if commission is NULL
FROM employees;

-- Example 7: Handling NULL with COALESCE
SELECT
    emp_name,
    salary,
    COALESCE(commission, 0) as commission,  -- Replace NULL with 0
    salary + COALESCE(commission, 0) as total_compensation
```

```sql
FROM employees;

-- Example 8: NULL in aggregate functions
SELECT
    COUNT(*) as total_employees,        -- Counts all rows
    COUNT(email) as employees_with_email,  -- Counts non-NULL emails
    COUNT(phone) as employees_with_phone,  -- Counts non-NULL phones
    AVG(salary) as avg_salary,           -- Ignores NULL in calculation
    AVG(commission) as avg_commission    -- Only averages non-NULL values
FROM employees;

-- Example 9: NULLIF function
SELECT
    NULLIF(status, 'inactive') as active_status  -- Returns NULL if status = 'inac
tive'
FROM orders;
```

NULL in Different Databases:

sql

```sql
-- MySQL: NULL handling
SELECT IFNULL(column, 'default') FROM table;  -- MySQL
SELECT COALESCE(column, 'default') FROM table;  -- Standard SQL

-- PostgreSQL: NULL handling
SELECT COALESCE(column, 'default') FROM table;  -- Same
SELECT column IS DISTINCT FROM 'value' FROM table;  -- NULL-aware comparison

-- SQL Server: NULL handling
SELECT ISNULL(column, 'default') FROM table;  -- SQL Server specific
```

Best Practices with NULL:

sql

```sql
-- Example 10: Good practices
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    order_date DATE NOT NULL,  -- Always know when order was placed
    shipped_date DATE NULL,    -- NULL means not shipped yet
    cancelled_date DATE NULL,  -- NULL means not cancelled
```

```sql
    -- Use NULL meaningfully: NULL for not yet, actual date for completed
    status VARCHAR(20) DEFAULT 'pending',

    -- Avoid ambiguous NULLs
    -- Bad: delivery_date DATE NULL (NULL could mean not yet, or no delivery)
    -- Good: has_delivery BOOLEAN, delivery_date DATE NULL
    has_delivery BOOLEAN DEFAULT FALSE,
    delivery_date DATE NULL
);

-- Example 11: Querying with meaning
-- Find orders that haven't been shipped
SELECT * FROM orders
WHERE shipped_date IS NULL
AND cancelled_date IS NULL;

-- Find orders shipped but not delivered
SELECT * FROM orders
WHERE shipped_date IS NOT NULL
AND has_delivery = TRUE
AND delivery_date IS NULL;
```

8. Constraints

Constraints are rules enforced on data columns to ensure data integrity and accuracy.

Types of Constraints:

1. **NOT NULL**: Ensures column cannot have NULL values
2. **UNIQUE**: Ensures all values in a column are different
3. **DEFAULT**: Provides a default value when none is specified
4. **CHECK**: Ensures values meet a specific condition
5. **PRIMARY KEY**: Combines NOT NULL and UNIQUE
6. **FOREIGN KEY**: Ensures referential integrity

8.1 NOT NULL Constraint

Ensures that a column cannot have NULL values.

Theory:

- Every row must have a value for this column
- Cannot insert NULL
- Cannot update existing NULL to remain NULL
- Essential for mandatory data fields

Practical Examples:

sql

```sql
-- Example 1: Basic NOT NULL constraint
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,  -- Must provide username
    email VARCHAR(100) NOT NULL,    -- Must provide email
    password_hash VARCHAR(255) NOT NULL,  -- Must provide password
    bio TEXT  -- Optional, can be NULL
);

-- This works
INSERT INTO users (username, email, password_hash)
VALUES ('john_doe', 'john@email.com', 'hash123');

-- This FAILS (username cannot be NULL)
INSERT INTO users (username, email, password_hash)
VALUES (NULL, 'john@email.com', 'hash123');

-- This FAILS (email cannot be NULL)
INSERT INTO users (username, email, password_hash)
VALUES ('john_doe', NULL, 'hash123');

-- Example 2: Adding NOT NULL to existing table
-- First, ensure no NULL values exist
UPDATE users SET bio = '' WHERE bio IS NULL;

-- Then add constraint
ALTER TABLE users
```

```sql
MODIFY bio TEXT NOT NULL;

-- Example 3: Removing NOT NULL constraint
ALTER TABLE users
MODIFY bio TEXT NULL;

-- Example 4: Multiple NOT NULL columns
CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(200) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT NOT NULL DEFAULT 0,
    description TEXT,  -- Can be NULL
    image_url VARCHAR(500)  -- Can be NULL
);
```

8.2 UNIQUE Constraint

Ensures that all values in a column (or combination of columns) are different.

Theory:

- Prevents duplicate values
- Can have multiple UNIQUE constraints per table
- NULL values can be in UNIQUE columns (database dependent)
- Different from PRIMARY KEY (only one PRIMARY KEY, no NULLs)

Practical Examples:

sql

```sql
-- Example 1: Single column UNIQUE constraint
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,  -- Usernames must be unique
    email VARCHAR(100) UNIQUE NOT NULL,    -- Emails must be unique
    full_name VARCHAR(100)  -- Can have duplicates
);

-- This works
INSERT INTO users (username, email, full_name)
```

```sql
VALUES ('john123', 'john@email.com', 'John Doe');

-- This FAILS (duplicate username)
INSERT INTO users (username, email, full_name)
VALUES ('john123', 'jane@email.com', 'Jane Doe');

-- This FAILS (duplicate email)
INSERT INTO users (username, email, full_name)
VALUES ('jane456', 'john@email.com', 'Jane Doe');

-- Example 2: Composite UNIQUE constraint (unique combination)
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT,
    price DECIMAL(10,2),
    UNIQUE KEY unique_order_product (order_id, product_id)  -- Each product appear
s once per order
);

-- This works
INSERT INTO order_items VALUES (1, 100, 2, 29.99);
INSERT INTO order_items VALUES (1, 101, 1, 49.99);  -- Different product, same ord
er

-- This FAILS (product 100 already in order 1)
INSERT INTO order_items VALUES (1, 100, 3, 29.99);

-- Example 3: UNIQUE with NULL values (MySQL behavior)
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    ssn CHAR(11) UNIQUE,  -- Social Security Number (can be NULL)
    email VARCHAR(100) UNIQUE
);

-- Multiple NULLs are allowed (in MySQL)
INSERT INTO employees VALUES (1, NULL, 'emp1@company.com');
INSERT INTO employees VALUES (2, NULL, 'emp2@company.com');  -- Works, multiple NU
LLs allowed

-- Example 4: Adding UNIQUE constraint to existing table
-- Check for duplicates first
```

```sql
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;

-- Remove duplicates (example strategy)
DELETE u1 FROM users u1
INNER JOIN users u2
WHERE u1.user_id > u2.user_id
AND u1.email = u2.email;

-- Add unique constraint
ALTER TABLE users
ADD CONSTRAINT unique_email UNIQUE (email);

-- Example 5: Dropping UNIQUE constraint
ALTER TABLE users
DROP INDEX unique_email;

-- Example 6: Real-world scenario - unique across active records only
CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    sku VARCHAR(50),
    is_active BOOLEAN DEFAULT TRUE,
    -- Create a unique index that only applies to active products
    UNIQUE KEY unique_active_sku (sku, is_active)
);

-- This works
INSERT INTO products (sku, is_active) VALUES ('SKU123', TRUE);
INSERT INTO products (sku, is_active) VALUES ('SKU123', FALSE);  -- Same SKU but i
nactive
INSERT INTO products (sku, is_active) VALUES ('SKU123', FALSE);  -- Another inacti
ve, allowed

-- This FAILS (trying to activate duplicate SKU)
INSERT INTO products (sku, is_active) VALUES ('SKU123', TRUE);
```

8.3 DEFAULT Constraint

Provides a default value for a column when no value is specified in an INSERT statement.

Theory:

- Automatically populates column with default value
- Can be used with or without NOT NULL
- Default value must match column data type
- Can be a constant or an expression (function)

Practical Examples:

sql

```sql
-- Example 1: Basic DEFAULT constraints
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,  -- Default to active
    role VARCHAR(20) DEFAULT 'user',  -- Default role
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  -- Default to current time
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    login_count INT DEFAULT 0,  -- Start at 0
    last_login DATETIME  -- No default, can be NULL
);

-- Insert without specifying default columns
INSERT INTO users (username) VALUES ('john_doe');
-- This creates:
-- username: 'john_doe'
-- is_active: TRUE
-- role: 'user'
-- created_at: current timestamp
-- login_count: 0

-- Example 2: Overriding defaults
INSERT INTO users (username, is_active, role)
VALUES ('admin_user', TRUE, 'admin');
-- Uses provided values instead of defaults
```

```sql
-- Example 3: DEFAULT with expressions
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    order_date DATE DEFAULT (CURRENT_DATE),
    order_number VARCHAR(50) DEFAULT (CONCAT('ORD-', UUID())),
    tax_rate DECIMAL(5,2) DEFAULT 10.00,
    status VARCHAR(20) DEFAULT 'pending'
);

-- Example 4: Complex DEFAULT scenarios
CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(200) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    discount_price DECIMAL(10,2) DEFAULT NULL,  -- NULL means no discount
    stock_quantity INT NOT NULL DEFAULT 0,
    is_available BOOLEAN GENERATED ALWAYS AS (stock_quantity > 0) STORED,  -- Comp
uted column
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Example 5: Setting DEFAULT on existing table
ALTER TABLE users
ALTER COLUMN login_count SET DEFAULT 0;

-- Example 6: Removing DEFAULT
ALTER TABLE users
ALTER COLUMN login_count DROP DEFAULT;

-- Example 7: DEFAULT with ENUM
CREATE TABLE tasks (
    task_id INT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    priority ENUM('low', 'medium', 'high', 'critical') DEFAULT 'medium',
    status ENUM('pending', 'in_progress', 'completed', 'cancelled') DEFAULT 'pendi
ng'
);

-- Example 8: Real-world application
CREATE TABLE audit_log (
    log_id BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```sql
    table_name VARCHAR(100) NOT NULL,
    record_id INT NOT NULL,
    action ENUM('INSERT', 'UPDATE', 'DELETE') NOT NULL,
    changed_by VARCHAR(100) DEFAULT CURRENT_USER,  -- Database user
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    old_value JSON DEFAULT NULL,
    new_value JSON DEFAULT NULL
);

-- Insert example
INSERT INTO audit_log (table_name, record_id, action, old_value, new_value)
VALUES ('users', 123, 'UPDATE',
        '{"name": "John"}',
        '{"name": "John Doe"}');
-- changed_by and changed_at get default values automatically
```

8.4 CHECK Constraint

Ensures that values in a column satisfy a specific condition.

Theory:

- Validates data before insert or update
- Can reference multiple columns
- Can use logical operators (AND, OR, NOT)
- Can use subqueries (database dependent)
- Expression must evaluate to TRUE or UNKNOWN for the operation to succeed

Practical Examples:

sql

```sql
-- Example 1: Basic CHECK constraints
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    age INT CHECK (age >= 18 AND age <= 65),  -- Must be between 18 and 65
    salary DECIMAL(10,2) CHECK (salary > 0),  -- Must be positive
    email VARCHAR(255) CHECK (email LIKE '%@%.%')  -- Basic email format
);
```

```sql
-- This works
INSERT INTO employees VALUES (1, 'John Doe', 25, 50000, 'john@company.com');

-- This FAILS (age < 18)
INSERT INTO employees VALUES (2, 'Jane Smith', 16, 50000, 'jane@company.com');

-- This FAILS (salary <= 0)
INSERT INTO employees VALUES (3, 'Bob Wilson', 30, -1000, 'bob@company.com');

-- Example 2: Multiple column CHECK constraint
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(200) NOT NULL,
    list_price DECIMAL(10,2) NOT NULL,
    selling_price DECIMAL(10,2) NOT NULL,
    CHECK (selling_price <= list_price),  -- Selling price can't exceed list price
    CHECK (selling_price > 0),  -- Positive selling price
    CHECK (list_price > 0)  -- Positive list price
);

-- This works
INSERT INTO products VALUES (1, 'Laptop', 1000, 899.99);

-- This FAILS (selling price > list price)
INSERT INTO products VALUES (2, 'Phone', 500, 599.99);

-- Example 3: Complex CHECK with multiple conditions
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    order_date DATE NOT NULL,
    shipped_date DATE,
    cancelled_date DATE,
    status VARCHAR(20) NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    -- Complex validation rules
    CHECK (
        (status = 'pending' AND shipped_date IS NULL AND cancelled_date IS NULL) O
R
        (status = 'shipped' AND shipped_date IS NOT NULL AND cancelled_date IS NUL
L) OR
        (status = 'cancelled' AND cancelled_date IS NOT NULL AND shipped_date IS N
ULL) OR
```

```sql
        (status = 'delivered' AND shipped_date IS NOT NULL AND cancelled_date IS N
ULL)
    ),
    CHECK (shipped_date IS NULL OR shipped_date >= order_date),  -- Can't ship bef
ore order
    CHECK (cancelled_date IS NULL OR cancelled_date >= order_date),  -- Can't canc
el before order
    CHECK (total_amount >= 0)
);

-- Example 4: CHECK with pattern matching
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    username VARCHAR(50) CHECK (username ~ '^[a-zA-Z0-9_]{3,20}$'),  -- PostgreSQL
regex
    email VARCHAR(255) CHECK (email LIKE '%@%.%' AND LENGTH(email) >= 5),
    phone VARCHAR(20) CHECK (phone ~ '^[0-9+\-\s()]{10,}$'),  -- Basic phone forma
t
    zip_code VARCHAR(10) CHECK (zip_code ~ '^[0-9]{5}(-[0-9]{4})?$')  -- US zip co
de
);

-- Example 5: Named CHECK constraints (easier to drop/modify)
CREATE TABLE accounts (
    account_id INT PRIMARY KEY,
    balance DECIMAL(15,2) NOT NULL,
    account_type VARCHAR(20) NOT NULL,
    minimum_balance DECIMAL(15,2),
    CONSTRAINT check_positive_balance CHECK (balance >= 0),
    CONSTRAINT check_valid_type CHECK (account_type IN ('checking', 'savings', 'mo
ney_market')),
    CONSTRAINT check_minimum_balance CHECK (
        (account_type = 'checking' AND minimum_balance >= 100) OR
        (account_type = 'savings' AND minimum_balance >= 500) OR
        (account_type = 'money_market' AND minimum_balance >= 2500)
    )
);

-- Example 6: CHECK with date validation
CREATE TABLE events (
    event_id INT PRIMARY KEY,
    event_name VARCHAR(200) NOT NULL,
```

```sql
    start_date DATETIME NOT NULL,
    end_date DATETIME NOT NULL,
    registration_deadline DATETIME NOT NULL,
    max_attendees INT CHECK (max_attendees > 0 AND max_attendees <= 1000),
    CHECK (start_date < end_date),  -- Event must end after it starts
    CHECK (registration_deadline <= start_date)  -- Registration must close before
event starts
);

-- Example 7: Adding CHECK to existing table
-- First, check if any existing data violates the constraint
SELECT * FROM employees WHERE age < 18 OR age > 65;

-- Add constraint (MySQL syntax)
ALTER TABLE employees
ADD CONSTRAINT check_age_range CHECK (age >= 18 AND age <= 65);

-- Example 8: Dropping CHECK constraint
ALTER TABLE employees
DROP CHECK check_age_range;

-- Example 9: CHECK with boolean logic
CREATE TABLE articles (
    article_id INT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    content TEXT NOT NULL,
    is_published BOOLEAN DEFAULT FALSE,
    published_date DATETIME,
    CHECK (
        (is_published = FALSE AND published_date IS NULL) OR  -- Not published = n
o date
        (is_published = TRUE AND published_date IS NOT NULL)  -- Published = must
have date
    )
);

-- Example 10: Real-world e-commerce validation
CREATE TABLE order_items (
    item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
```

```sql
    unit_price DECIMAL(10,2) NOT NULL,
    discount_percent DECIMAL(5,2) DEFAULT 0,
    final_price DECIMAL(10,2) NOT NULL,
    -- Validation rules
    CONSTRAINT check_quantity_positive CHECK (quantity > 0),
    CONSTRAINT check_unit_price_positive CHECK (unit_price > 0),
    CONSTRAINT check_discount_range CHECK (discount_percent >= 0 AND discount_perc
ent <= 100),
    CONSTRAINT check_final_price_valid CHECK (
        final_price = ROUND(unit_price * quantity * (1 - discount_percent/100), 2)
    ),
    CONSTRAINT check_max_quantity CHECK (quantity <= 100)  -- Max 100 items per pr
oduct per order
);

-- Example 11: Database-specific CHECK implementations
-- MySQL (actually doesn't enforce CHECK until MySQL 8.0.16+)
CREATE TABLE test_mysql (
    id INT,
    age INT CHECK (age >= 18)  -- May be ignored in older MySQL versions
) ENGINE=InnoDB;

-- PostgreSQL (fully supports CHECK)
CREATE TABLE test_postgres (
    id SERIAL PRIMARY KEY,
    age INT CONSTRAINT age_check CHECK (age >= 18)
);

-- SQL Server
CREATE TABLE test_sqlserver (
    id INT PRIMARY KEY,
    age INT,
    CONSTRAINT CK_age CHECK (age >= 18)
);
```

Summary of All Constraints Together:

sql

```sql
-- Complete example with all constraints
CREATE TABLE students (
    -- PRIMARY KEY constraint
```

```sql
    student_id INT AUTO_INCREMENT PRIMARY KEY,

    -- NOT NULL constraints
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,

    -- UNIQUE constraint
    email VARCHAR(100) UNIQUE NOT NULL,
    student_number CHAR(10) UNIQUE NOT NULL,

    -- DEFAULT constraint
    enrollment_date DATE DEFAULT (CURRENT_DATE),
    status VARCHAR(20) DEFAULT 'active',

    -- CHECK constraint
    age INT CHECK (age >= 16 AND age <= 100),
    gpa DECIMAL(3,2) CHECK (gpa >= 0.0 AND gpa <= 4.0),

    -- FOREIGN KEY constraint (if referencing another table)
    department_id INT,

    -- Composite constraints
    CONSTRAINT unique_full_name UNIQUE (first_name, last_name),
    CONSTRAINT check_status CHECK (status IN ('active', 'graduated', 'suspended',
'withdrawn')),

    -- Foreign key definition
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

-- Now try various INSERT statements
-- This works (all constraints satisfied)
INSERT INTO students (first_name, last_name, email, student_number, age, gpa, depa
rtment_id)
VALUES ('John', 'Doe', 'john.doe@university.edu', 'S12345', 20, 3.5, 1);

-- This FAILS (duplicate email - UNIQUE constraint)
INSERT INTO students (first_name, last_name, email, student_number, age, gpa)
VALUES ('Jane', 'Smith', 'john.doe@university.edu', 'S12346', 19, 3.8);

-- This FAILS (age < 16 - CHECK constraint)
INSERT INTO students (first_name, last_name, email, student_number, age, gpa)
```

```sql
VALUES ('Bob', 'Wilson', 'bob@university.edu', 'S12347', 15, 3.0);

-- This works (uses DEFAULT for enrollment_date and status)
INSERT INTO students (first_name, last_name, email, student_number, age, gpa)
VALUES ('Alice', 'Brown', 'alice@university.edu', 'S12348', 22, 3.9);
```

Best Practices for Constraints:

1. **Always use PRIMARY KEY** for unique record identification
2. **Use NOT NULL for required fields** to ensure data completeness
3. **Add UNIQUE constraints** for fields that must be distinct
4. **Use CHECK constraints** to validate data at database level (not just application level)
5. **Set appropriate DEFAULT values** to simplify inserts
6. **Name your constraints** for easier maintenance
7. **Consider performance** - too many constraints can slow down inserts/updates
8. **Test constraints** with edge cases before deploying to production

Basic SQL Queries: Complete Theory Notes with Practical Examples

Introduction to Basic SQL Queries

These are the fundamental operations you'll use every day as a developer. Mastering these will allow you to retrieve and filter data effectively from any database.

---

1. SELECT Statement

The **SELECT** statement is used to select data from a database. The data returned is stored in a result table, called the result-set.

Theory:

- **SELECT** specifies which columns to retrieve
- Can retrieve one, multiple, or all columns
- Can include expressions, calculations, and functions
- Always returns a result set (even if empty)

Syntax:

sql

```sql
-- Basic syntax
SELECT column1, column2, ...
FROM table_name;

-- Select all columns
SELECT * FROM table_name;

-- Select with expressions
SELECT expression AS alias FROM table_name;
```

Practical Examples:

sql

```sql
-- Example 1: Sample table for all examples
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    department VARCHAR(50),
    salary DECIMAL(10,2),
    hire_date DATE,
    is_active BOOLEAN
);

INSERT INTO employees VALUES
(1, 'John', 'Doe', 'john.doe@company.com', 'IT', 75000, '2020-01-15', TRUE),
(2, 'Jane', 'Smith', 'jane.smith@company.com', 'HR', 65000, '2019-03-20', TRUE),
(3, 'Bob', 'Johnson', 'bob.johnson@company.com', 'IT', 82000, '2021-06-10', TRUE),
(4, 'Alice', 'Brown', 'alice.brown@company.com', 'Finance', 70000, '2020-11-01', F
ALSE),
(5, 'Charlie', 'Wilson', 'charlie.wilson@company.com', 'IT', 90000, '2018-09-05',
TRUE),
(6, 'Diana', 'Miller', 'diana.miller@company.com', 'HR', 58000, '2022-01-12', TRUE
),
(7, 'Eve', 'Davis', NULL, 'Finance', 72000, '2019-07-19', TRUE);

-- Example 2: Basic SELECT - specific columns
SELECT first_name, last_name, email
FROM employees;

-- Result:
-- first_name | last_name | email
-- John       | Doe       | john.doe@company.com
-- Jane       | Smith     | jane.smith@company.com
-- ...

-- Example 3: SELECT all columns
SELECT * FROM employees;

-- Example 4: SELECT with expressions
SELECT
```

```sql
    first_name,
    last_name,
    CONCAT(first_name, ' ', last_name) AS full_name,
    salary,
    salary * 1.1 AS increased_salary,  -- 10% increase calculation
    YEAR(hire_date) AS hire_year
FROM employees;

-- Example 5: SELECT with DISTINCT (covered in detail later)
SELECT DISTINCT department FROM employees;

-- Example 6: SELECT with functions
SELECT
    COUNT(*) AS total_employees,
    AVG(salary) AS average_salary,
    MAX(salary) AS highest_salary,
    MIN(salary) AS lowest_salary
FROM employees;

-- Example 7: SELECT with aliases (AS keyword)
SELECT
    first_name AS "First Name",  -- Column alias with spaces
    last_name AS "Last Name",
    salary AS "Annual Salary"
FROM employees;

-- Example 8: SELECT without FROM (for calculations)
SELECT
    NOW() AS current_datetime,
    PI() AS pi_value,
    RAND() AS random_number,
    USER() AS current_user;

-- Example 9: SELECT with CASE statement
SELECT
    first_name,
    last_name,
    salary,
    CASE
        WHEN salary < 60000 THEN 'Entry Level'
        WHEN salary BETWEEN 60000 AND 80000 THEN 'Mid Level'
        WHEN salary > 80000 THEN 'Senior Level'
```

```
        ELSE 'Unknown'
    END AS salary_level
FROM employees;
```

2. WHERE Clause

The **WHERE** clause is used to filter records. It extracts only those records that fulfill a specified condition.

Theory:

- Filters rows based on conditions
- Can use various operators: =, <>, >, <, >=, <=
- Can combine multiple conditions with AND, OR, NOT
- Supports pattern matching with LIKE
- Works with text, numbers, dates

Operator Reference:

Operator	Description	Example
=	Equal	`WHERE age = 25`
<> or !=	Not equal	`WHERE age != 25`
>	Greater than	`WHERE salary > 50000`
<	Less than	`WHERE salary < 50000`
>=	Greater than or equal	`WHERE age >= 18`
<=	Less than or equal	`WHERE age <= 65`

Operator	Description	Example
AND	Logical AND	`WHERE age > 18 AND age < 65`
OR	Logical OR	`WHERE dept = 'IT' OR dept = 'HR'`
NOT	Logical NOT	`WHERE NOT dept = 'IT'`

Practical Examples:

sql

```sql
-- Example 1: WHERE with equality operator
-- Find employee with specific ID
SELECT first_name, last_name, department
FROM employees
WHERE emp_id = 3;

-- Result: Bob Johnson, IT

-- Example 2: WHERE with comparison operators
-- Find employees with salary > 75000
SELECT first_name, last_name, salary
FROM employees
WHERE salary > 75000;

-- Result:
-- Bob | Johnson | 82000
-- Charlie | Wilson | 90000

-- Find employees with salary between ranges
SELECT first_name, last_name, salary
FROM employees
WHERE salary >= 65000 AND salary <= 80000;

-- Example 3: WHERE with text matching
-- Find employees in IT department
SELECT first_name, last_name, department
FROM employees
```

```sql
WHERE department = 'IT';

-- Find employees NOT in IT
SELECT first_name, last_name, department
FROM employees
WHERE department <> 'IT';

-- Example 4: WHERE with multiple conditions (AND)
-- Find IT employees with salary > 80000
SELECT first_name, last_name, department, salary
FROM employees
WHERE department = 'IT' AND salary > 80000;

-- Example 5: WHERE with multiple conditions (OR)
-- Find employees in IT OR HR
SELECT first_name, last_name, department
FROM employees
WHERE department = 'IT' OR department = 'HR';

-- Example 6: WHERE with date comparisons
-- Find employees hired after 2020
SELECT first_name, last_name, hire_date
FROM employees
WHERE hire_date > '2020-01-01';

-- Find employees hired between specific dates
SELECT first_name, last_name, hire_date
FROM employees
WHERE hire_date >= '2019-01-01' AND hire_date <= '2020-12-31';

-- Example 7: WHERE with boolean
-- Find active employees
SELECT first_name, last_name, is_active
FROM employees
WHERE is_active = TRUE;

-- Find inactive employees
SELECT first_name, last_name, is_active
FROM employees
WHERE is_active = FALSE;

-- Example 8: WHERE with NOT operator
```

```sql
-- Find employees NOT in IT
SELECT first_name, last_name, department
FROM employees
WHERE NOT department = 'IT';

-- Find employees NOT active
SELECT first_name, last_name, is_active
FROM employees
WHERE NOT is_active;

-- Example 9: WHERE with calculations
-- Find employees whose salary is above average
SELECT first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Example 10: WHERE with string functions
-- Find employees whose first name starts with 'J'
SELECT first_name, last_name
FROM employees
WHERE LEFT(first_name, 1) = 'J';

-- Find employees with email containing 'john'
SELECT first_name, last_name, email
FROM employees
WHERE email LIKE '%john%';  -- Covered in LIKE section
```

3. ORDER BY Clause

The **ORDER BY** keyword is used to sort the result-set in ascending or descending order.

Theory:

- **ASC** for ascending (default, can be omitted)
- **DESC** for descending
- Can sort by multiple columns
- Can sort by column name, alias, or position
- NULL values: typically appear first in ASC, last in DESC (database dependent)

Syntax:

sql

```sql
-- Basic syntax
SELECT column1, column2
FROM table_name
ORDER BY column1 ASC, column2 DESC;

-- Sort by column position
SELECT first_name, last_name, salary
FROM employees
ORDER BY 3 DESC;  -- Sort by third column (salary)
```

Practical Examples:

sql

```sql
-- Example 1: Basic ORDER BY (ascending - default)
-- Sort employees by salary (low to high)
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary ASC;
-- ASC is optional, same as: ORDER BY salary

-- Result:
-- Diana | Miller | 58000
-- Jane | Smith | 65000
-- Alice | Brown | 70000
-- John | Doe | 75000
-- Bob | Johnson | 82000
-- Charlie | Wilson | 90000

-- Example 2: ORDER BY descending
-- Sort employees by salary (high to low)
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary DESC;

-- Example 3: ORDER BY multiple columns
-- Sort by department first, then by salary
SELECT department, first_name, last_name, salary
FROM employees
```

```sql
ORDER BY department ASC, salary DESC;

-- Result:
-- Finance | Alice | Brown | 70000
-- Finance | Eve | Davis | 72000
-- HR | Jane | Smith | 65000
-- HR | Diana | Miller | 58000
-- IT | Charlie | Wilson | 90000
-- IT | Bob | Johnson | 82000
-- IT | John | Doe | 75000

-- Example 4: ORDER BY with text
-- Sort employees alphabetically by last name
SELECT first_name, last_name
FROM employees
ORDER BY last_name;

-- Sort by last name descending
SELECT first_name, last_name
FROM employees
ORDER BY last_name DESC;

-- Example 5: ORDER BY with dates
-- Sort by hire date (newest first)
SELECT first_name, last_name, hire_date
FROM employees
ORDER BY hire_date DESC;

-- Example 6: ORDER BY with expression
-- Sort by calculated column
SELECT
    first_name,
    last_name,
    salary,
    salary * 1.1 AS bonus
FROM employees
ORDER BY bonus DESC;

-- Example 7: ORDER BY with column alias
-- Sort by full name
SELECT
    first_name,
```

```sql
    last_name,
    CONCAT(last_name, ', ', first_name) AS full_name
FROM employees
ORDER BY full_name;

-- Example 8: ORDER BY with column position (not recommended for production)
-- Sort by third column
SELECT first_name, last_name, salary
FROM employees
ORDER BY 3 DESC;  -- 3 refers to salary column

-- Example 9: ORDER BY with NULL handling
-- Create table to demonstrate NULL sorting
CREATE TABLE test_nulls (id INT, value INT);
INSERT INTO test_nulls VALUES (1, 100), (2, NULL), (3, 50), (4, NULL);

-- NULLs typically appear first in ASC
SELECT * FROM test_nulls ORDER BY value ASC;
-- Result: NULL, NULL, 50, 100

-- NULLs typically appear last in DESC
SELECT * FROM test_nulls ORDER BY value DESC;
-- Result: 100, 50, NULL, NULL

-- Example 10: ORDER BY with CASE (custom sort order)
-- Custom sort order: IT first, then Finance, then HR
SELECT first_name, last_name, department
FROM employees
ORDER BY
    CASE department
        WHEN 'IT' THEN 1
        WHEN 'Finance' THEN 2
        WHEN 'HR' THEN 3
        ELSE 4
    END;

-- Example 11: ORDER BY with different data types
-- Sort by department (string), then by salary (number), then by hire_date (date)
SELECT
    department,
    salary,
    hire_date,
```

```sql
    first_name,
    last_name
FROM employees
ORDER BY
    department,
    salary DESC,
    hire_date;
```

4. DISTINCT Keyword

The **DISTINCT** keyword is used to return only distinct (different) values.

Theory:

- Eliminates duplicate rows from the result
- Works on one or multiple columns
- NULL is considered a distinct value
- Can be used with aggregate functions

Syntax:

sql

```sql
-- Single column
SELECT DISTINCT column1 FROM table_name;

-- Multiple columns (unique combinations)
SELECT DISTINCT column1, column2 FROM table_name;

-- With aggregate
SELECT COUNT(DISTINCT column1) FROM table_name;
```

Practical Examples:

sql

```sql
-- Example 1: DISTINCT on single column
-- Get all unique departments
SELECT DISTINCT department
FROM employees
```

```sql
ORDER BY department;

-- Result:
-- Finance
-- HR
-- IT

-- Without DISTINCT (would show duplicates)
SELECT department FROM employees;
-- Result: IT, HR, IT, Finance, IT, HR, Finance

-- Example 2: DISTINCT on multiple columns
-- Get unique department and active status combinations
SELECT DISTINCT department, is_active
FROM employees
ORDER BY department, is_active;

-- Result:
-- Finance | FALSE (Alice is inactive)
-- Finance | TRUE (Eve is active)
-- HR | TRUE
-- IT | TRUE

-- Example 3: COUNT with DISTINCT
-- Count unique departments
SELECT COUNT(DISTINCT department) AS unique_departments
FROM employees;

-- Result: 3

-- Count unique combinations
SELECT COUNT(DISTINCT department, is_active) AS unique_combinations
FROM employees;

-- Example 4: DISTINCT with NULL values
-- Create table with NULLs
CREATE TABLE test_distinct (category VARCHAR(50));
INSERT INTO test_distinct VALUES ('A'), ('B'), (NULL), ('A'), (NULL);

-- NULL is considered a distinct value
SELECT DISTINCT category FROM test_distinct;
-- Result: A, B, NULL
```

```sql
-- Example 5: DISTINCT in subqueries
-- Find departments that have employees with salary > 80000
SELECT DISTINCT department
FROM employees
WHERE emp_id IN (
    SELECT DISTINCT emp_id
    FROM employees
    WHERE salary > 80000
);

-- Example 6: DISTINCT with JOINs
-- Get distinct customers who have placed orders
SELECT DISTINCT c.customer_id, c.customer_name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;

-- Example 7: DISTINCT vs GROUP BY
-- Same result, different approach
SELECT DISTINCT department FROM employees;
-- vs
SELECT department FROM employees GROUP BY department;

-- Example 8: DISTINCT with multiple columns and sorting
SELECT DISTINCT department, is_active
FROM employees
ORDER BY department DESC, is_active;

-- Example 9: DISTINCT with date functions
-- Get distinct years employees were hired
SELECT DISTINCT YEAR(hire_date) AS hire_year
FROM employees
ORDER BY hire_year;

-- Example 10: Real-world scenario - product categories
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    category VARCHAR(50),
    supplier VARCHAR(50),
    price DECIMAL(10,2)
);
```

```sql
INSERT INTO products VALUES
(1, 'Laptop', 'Electronics', 'Dell', 999.99),
(2, 'Mouse', 'Electronics', 'Logitech', 29.99),
(3, 'Desk', 'Furniture', 'IKEA', 349.99),
(4, 'Chair', 'Furniture', 'IKEA', 199.99),
(5, 'Monitor', 'Electronics', 'Dell', 299.99);

-- Get unique categories
SELECT DISTINCT category FROM products;

-- Get unique supplier and category combinations
SELECT DISTINCT supplier, category
FROM products
ORDER BY supplier, category;
```

5. LIMIT and OFFSET

LIMIT restricts the number of rows returned. **OFFSET** specifies where to start returning rows. These are crucial for pagination.

Theory:

- **LIMIT**: Maximum number of rows to return
- **OFFSET**: Number of rows to skip before starting to return rows
- Different databases have different syntax:
  - MySQL, PostgreSQL: `LIMIT [offset,] row_count` or `LIMIT row_count OFFSET offset`
  - SQL Server: `TOP` or `OFFSET-FETCH`
  - Oracle: `ROWNUM` or `OFFSET-FETCH`

Syntax:

sql

```sql
-- MySQL/PostgreSQL syntax
SELECT column1, column2
FROM table_name
LIMIT row_count;
```

```sql
SELECT column1, column2
FROM table_name
LIMIT offset, row_count;  -- offset, count

SELECT column1, column2
FROM table_name
LIMIT row_count OFFSET offset;

-- SQL Server syntax
SELECT TOP row_count column1, column2
FROM table_name;

SELECT column1, column2
FROM table_name
ORDER BY column
OFFSET offset ROWS
FETCH NEXT row_count ROWS ONLY;
```

Practical Examples:

```sql
-- Example 1: Basic LIMIT
-- Get first 3 employees
SELECT emp_id, first_name, last_name
FROM employees
LIMIT 3;

-- Result:
-- 1 | John | Doe
-- 2 | Jane | Smith
-- 3 | Bob | Johnson

-- Example 2: LIMIT with ORDER BY
-- Get top 3 highest paid employees
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;

-- Result:
```

```sql
-- Charlie | Wilson | 90000
-- Bob | Johnson | 82000
-- John | Doe | 75000

-- Example 3: LIMIT with OFFSET for pagination
-- Page 1: first 3 employees
SELECT first_name, last_name
FROM employees
ORDER BY emp_id
LIMIT 3 OFFSET 0;  -- or LIMIT 0, 3

-- Page 2: next 3 employees (skip first 3)
SELECT first_name, last_name
FROM employees
ORDER BY emp_id
LIMIT 3 OFFSET 3;  -- or LIMIT 3, 3

-- Page 3: next 3 employees (skip first 6)
SELECT first_name, last_name
FROM employees
ORDER BY emp_id
LIMIT 3 OFFSET 6;  -- or LIMIT 6, 3

-- Example 4: Pagination formula
-- For page number N with page size P
-- OFFSET = (N - 1) * P

-- Page 1, size 3: OFFSET 0
-- Page 2, size 3: OFFSET 3
-- Page 3, size 3: OFFSET 6

-- Example 5: Get second highest salary
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;  -- Skip first, return second

-- Example 6: LIMIT with WHERE clause
-- Get 2 highest paid IT employees
SELECT first_name, last_name, department, salary
FROM employees
WHERE department = 'IT'
```

```sql
ORDER BY salary DESC
LIMIT 2;

-- Result:
-- Charlie | Wilson | IT | 90000
-- Bob | Johnson | IT | 82000

-- Example 7: LIMIT to get sample data
-- Get 3 random employees
SELECT first_name, last_name
FROM employees
ORDER BY RAND()
LIMIT 3;

-- Example 8: LIMIT with aggregate functions
-- Get departments with highest average salary (top 2)
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC
LIMIT 2;

-- Example 9: OFFSET without LIMIT (rare, but possible in some DBs)
-- Skip first 2, return all remaining
SELECT first_name, last_name
FROM employees
ORDER BY emp_id
OFFSET 2 ROWS;  -- Not valid in MySQL, works in PostgreSQL/SQL Server

-- Example 10: SQL Server syntax
-- SQL Server equivalent of LIMIT 3 OFFSET 2
SELECT first_name, last_name
FROM employees
ORDER BY emp_id
OFFSET 2 ROWS
FETCH NEXT 3 ROWS ONLY;

-- Example 11: Find nth highest salary (n=3)
-- Method 1: Using LIMIT and OFFSET
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
```

```sql
LIMIT 1 OFFSET 2;  -- 3rd highest (0-indexed: 0=1st, 1=2nd, 2=3rd)

-- Method 2: Get complete employee with 3rd highest salary
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;

-- Example 12: Pagination with search/filter
-- Get page 2 of IT employees (page size = 2)
SELECT first_name, last_name, salary
FROM employees
WHERE department = 'IT'
ORDER BY salary DESC
LIMIT 2 OFFSET 2;  -- Skip first 2 IT employees

-- Example 13: Checking if more pages exist
-- Get one extra record to check if there's a next page
SELECT first_name, last_name
FROM employees
ORDER BY emp_id
LIMIT 4;  -- Request page size + 1 to check for "next page" existence

-- Example 14: LIMIT with UPDATE/DELETE (careful!)
-- Delete oldest 2 employees
DELETE FROM employees
ORDER BY hire_date
LIMIT 2;

-- Update first 3 employees with small raise
UPDATE employees
SET salary = salary * 1.05
ORDER BY salary
LIMIT 3;
```

6. LIKE Operator

The **LIKE** operator is used in a WHERE clause to search for a specified pattern in a column.

Theory:

- Uses wildcards for pattern matching
- **%** - represents zero, one, or multiple characters
- _ - represents a single character
- Case-insensitive or sensitive depends on database collation
- Can be combined with NOT for negative matching

Wildcard Reference:

Wildcard	Description	Example
%	Any number of characters	`'J%'` - starts with J
_	Single character	`'J_hn'` - J?hn (John, Jahn)
[charlist]	Any single character in list (SQL Server)	`'[JT]%'` - starts with J or T
[^charlist]	Any single character not in list	`'[^JT]%'` - doesn't start with J/T

Practical Examples:

sql

```sql
-- Example 1: Basic LIKE patterns
-- Names starting with 'J'
SELECT first_name, last_name
FROM employees
WHERE first_name LIKE 'J%';

-- Result:
-- John | Doe
-- Jane | Smith

-- Names ending with 'n'
SELECT first_name, last_name
```

```sql
FROM employees
WHERE first_name LIKE '%n';

-- Result:
-- John | Doe
-- Bob | Johnson? No (first_name Bob doesn't end with n)

-- Example 2: LIKE with % (anywhere in string)
-- Names containing 'o'
SELECT first_name, last_name
FROM employees
WHERE first_name LIKE '%o%';

-- Result:
-- John | Doe
-- Bob | Johnson

-- Example 3: LIKE with _ (single character)
-- Names with 4 letters, starting with J and ending with n
SELECT first_name, last_name
FROM employees
WHERE first_name LIKE 'J__n';  -- J + 2 chars + n

-- Result:
-- John | Doe (matches: J o h n)

-- Names with 5 letters
SELECT first_name
FROM employees
WHERE first_name LIKE '_____';  -- 5 underscores

-- Example 4: LIKE with email patterns
-- Find employees with Gmail
SELECT first_name, last_name, email
FROM employees
WHERE email LIKE '%@gmail.com';

-- Find employees with company email
SELECT first_name, last_name, email
FROM employees
WHERE email LIKE '%@company.com';
```

```sql
-- Find employees with email containing 'john'
SELECT first_name, last_name, email
FROM employees
WHERE email LIKE '%john%';

-- Example 5: NOT LIKE
-- Find employees whose names don't start with J
SELECT first_name, last_name
FROM employees
WHERE first_name NOT LIKE 'J%';

-- Example 6: LIKE with escape characters
-- Find strings containing % (need to escape)
CREATE TABLE test_escape (code VARCHAR(50));
INSERT INTO test_escape VALUES ('10% off'), ('20% off'), ('full price'), ('sale 30
%');

-- Find codes with % sign (MySQL)
SELECT * FROM test_escape
WHERE code LIKE '%\%%' ESCAPE '\';

-- Or without ESCAPE if % is the only wildcard
SELECT * FROM test_escape
WHERE code LIKE '%$%%' ESCAPE '$';  -- $ is custom escape

-- Example 7: LIKE with underscores in data
-- If you need to search for actual underscore
CREATE TABLE product_codes (code VARCHAR(50));
INSERT INTO product_codes VALUES ('ABC_123'), ('ABC-123'), ('XYZ_789');

-- Find codes with underscore
SELECT * FROM product_codes
WHERE code LIKE '%\_%' ESCAPE '\';

-- Example 8: LIKE with multiple patterns
-- Names starting with J or B
SELECT first_name, last_name
FROM employees
WHERE first_name LIKE 'J%' OR first_name LIKE 'B%';

-- Example 9: LIKE with case sensitivity
-- MySQL (case insensitive by default)
```

```sql
SELECT first_name FROM employees WHERE first_name LIKE 'j%';  -- Finds John, Jane

-- PostgreSQL (case sensitive)
SELECT first_name FROM employees WHERE first_name LIKE 'J%';  -- Only uppercase J
SELECT first_name FROM employees WHERE first_name ILIKE 'j%';  -- PostgreSQL case-
insensitive

-- Example 10: Real-world search scenarios
-- Search employees by partial name (search box functionality)
SELECT first_name, last_name, email
FROM employees
WHERE
    first_name LIKE '%oh%' OR
    last_name LIKE '%oh%' OR
    email LIKE '%oh%';

-- Example 11: LIKE with date patterns (convert to string first)
-- Find employees hired in 2020
SELECT first_name, last_name, hire_date
FROM employees
WHERE hire_date LIKE '2020%';  -- Works if date is stored as string

-- Better approach for dates
SELECT first_name, last_name, hire_date
FROM employees
WHERE YEAR(hire_date) = 2020;

-- Example 12: LIKE with phone numbers
CREATE TABLE contacts (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    phone VARCHAR(20)
);

INSERT INTO contacts VALUES
(1, 'John', '123-456-7890'),
(2, 'Jane', '(123) 456-7890'),
(3, 'Bob', '123.456.7890'),
(4, 'Alice', '1234567890');

-- Find all contacts with area code 123 regardless of format
SELECT * FROM contacts
```

```sql
WHERE phone LIKE '%123%456%7890%';

-- Example 13: Performance considerations
-- Good for performance (starts with known prefix)
SELECT * FROM employees WHERE last_name LIKE 'Sm%';  -- Can use index

-- Bad for performance (starts with wildcard)
SELECT * FROM employees WHERE last_name LIKE '%son';  -- Cannot use index

-- Example 14: LIKE with [] character list (SQL Server only)
-- In SQL Server:
-- SELECT * FROM employees
-- WHERE last_name LIKE '[A-C]%';  -- Last names starting with A, B, or C

-- SELECT * FROM employees
-- WHERE last_name LIKE '[^A-C]%';  -- Last names NOT starting with A, B, or C
```

7. BETWEEN Operator

The **BETWEEN** operator selects values within a given range. The values can be numbers, text, or dates.

Theory:

- **BETWEEN** is inclusive: includes begin and end values
- Equivalent to: `value >= low AND value <= high`
- Works with numbers, dates, and strings
- Can be combined with NOT for exclusion

Syntax:

sql

```sql
SELECT column1, column2
FROM table_name
WHERE column BETWEEN value1 AND value2;
```

Practical Examples:

```sql
-- Example 1: BETWEEN with numbers
-- Find employees with salary between 65000 and 80000
SELECT first_name, last_name, salary
FROM employees
WHERE salary BETWEEN 65000 AND 80000;

-- This is equivalent to:
SELECT first_name, last_name, salary
FROM employees
WHERE salary >= 65000 AND salary <= 80000;

-- Example 2: NOT BETWEEN
-- Find employees with salary outside range
SELECT first_name, last_name, salary
FROM employees
WHERE salary NOT BETWEEN 65000 AND 80000;

-- Example 3: BETWEEN with dates
-- Find employees hired in 2020
SELECT first_name, last_name, hire_date
FROM employees
WHERE hire_date BETWEEN '2020-01-01' AND '2020-12-31';

-- Find employees hired in last 30 days
SELECT first_name, last_name, hire_date
FROM employees
WHERE hire_date BETWEEN DATE_SUB(CURDATE(), INTERVAL 30 DAY) AND CURDATE();

-- Example 4: BETWEEN with strings
-- Find employees whose names are alphabetically between 'B' and 'E'
SELECT first_name, last_name
FROM employees
WHERE first_name BETWEEN 'B' AND 'E'
ORDER BY first_name;

-- Result:
-- Bob, Charlie, Diana (not Eve because 'Eve' > 'E')

-- Note: BETWEEN 'B' and 'E' includes strings starting with B, C, D
```

```sql
-- but not E because 'E' is the upper bound and 'Eve' > 'E'

-- Example 5: BETWEEN with timestamps
-- Find records created in specific time range
SELECT first_name, last_name, created_at
FROM employees
WHERE created_at BETWEEN '2024-01-01 00:00:00' AND '2024-01-31 23:59:59';

-- Example 6: BETWEEN with decimals
-- Find products in price range
SELECT product_name, price
FROM products
WHERE price BETWEEN 10.50 AND 50.99;

-- Example 7: BETWEEN with NULL handling
-- NULL values are not included in BETWEEN
INSERT INTO employees (emp_id, first_name, salary) VALUES (8, 'Test', NULL);
SELECT * FROM employees WHERE salary BETWEEN 50000 AND 100000;  -- Test not includ
ed

-- Example 8: BETWEEN in CASE statements
SELECT
    first_name,
    last_name,
    salary,
    CASE
        WHEN salary BETWEEN 0 AND 50000 THEN 'Low'
        WHEN salary BETWEEN 50001 AND 80000 THEN 'Medium'
        WHEN salary BETWEEN 80001 AND 100000 THEN 'High'
        ELSE 'Very High'
    END AS salary_category
FROM employees;

-- Example 9: BETWEEN with JOINs
-- Find orders placed in Q1 2024
SELECT o.order_id, o.order_date, c.customer_name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_date BETWEEN '2024-01-01' AND '2024-03-31';

-- Example 10: BETWEEN with multiple columns (not standard)
-- Some databases support row value comparisons
```

```sql
-- Find employees with (department, salary) in range (not standard SQL)
-- SELECT * FROM employees
-- WHERE (department, salary) BETWEEN ('A', 50000) AND ('Z', 100000);

-- Example 11: Age calculation with BETWEEN
-- Find employees aged 25-35
SELECT
    first_name,
    last_name,
    TIMESTAMPDIFF(YEAR, birth_date, CURDATE()) AS age
FROM employees
WHERE TIMESTAMPDIFF(YEAR, birth_date, CURDATE()) BETWEEN 25 AND 35;

-- Example 12: BETWEEN with EXTRACT
-- Find employees hired in Q2 (April-June)
SELECT first_name, last_name, hire_date
FROM employees
WHERE MONTH(hire_date) BETWEEN 4 AND 6;

-- Example 13: Dynamic BETWEEN
-- Set variables for range
SET @min_salary = 60000;
SET @max_salary = 80000;

SELECT first_name, last_name, salary
FROM employees
WHERE salary BETWEEN @min_salary AND @max_salary;

-- Example 14: BETWEEN with subquery
-- Find employees with salary between the average and the maximum
SELECT first_name, last_name, salary
FROM employees
WHERE salary BETWEEN
    (SELECT AVG(salary) FROM employees)
    AND
    (SELECT MAX(salary) FROM employees);

-- Example 15: Performance considerations
-- BETWEEN is generally optimized and may use indexes effectively
CREATE INDEX idx_salary ON employees(salary);
-- This query can use the index:
SELECT * FROM employees WHERE salary BETWEEN 50000 AND 80000;
```

8. IN Operator

The **IN** operator allows you to specify multiple values in a WHERE clause. It's a shorthand for multiple OR conditions.

Theory:

- Checks if value matches any value in a list
- More readable than multiple OR conditions
- Can use with subqueries
- Can be combined with NOT for exclusion
- NULL handling: if list contains NULL, it doesn't affect matching (NULL = NULL is unknown)

Syntax:

sql

```sql
-- With value list
SELECT column1, column2
FROM table_name
WHERE column IN (value1, value2, ...);

-- With subquery
SELECT column1, column2
FROM table_name
WHERE column IN (SELECT column FROM another_table);
```

Practical Examples:

```sql
-- Example 1: Basic IN with list of values
-- Find employees in specific departments
SELECT first_name, last_name, department
FROM employees
WHERE department IN ('IT', 'Finance');

-- This is equivalent to:
```

```sql
SELECT first_name, last_name, department
FROM employees
WHERE department = 'IT' OR department = 'Finance';

-- Example 2: NOT IN
-- Find employees NOT in IT or Finance
SELECT first_name, last_name, department
FROM employees
WHERE department NOT IN ('IT', 'Finance');

-- Example 3: IN with numbers
-- Find employees with specific IDs
SELECT first_name, last_name, emp_id
FROM employees
WHERE emp_id IN (1, 3, 5, 7);

-- Example 4: IN with subquery
-- Find employees in departments that have high average salary
SELECT first_name, last_name, department
FROM employees
WHERE department IN (
    SELECT department
    FROM employees
    GROUP BY department
    HAVING AVG(salary) > 70000
);

-- Example 5: IN with subquery returning single column
-- Find employees who have the same salary as anyone in IT
SELECT first_name, last_name, salary
FROM employees
WHERE salary IN (
    SELECT DISTINCT salary
    FROM employees
    WHERE department = 'IT'
);

-- Example 6: IN with dates
-- Find employees hired in specific months
SELECT first_name, last_name, hire_date
FROM employees
WHERE MONTH(hire_date) IN (1, 6, 12);  -- January, June, December
```

```sql
-- Example 7: IN with ENUM or status values
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    status VARCHAR(20)
);

INSERT INTO orders VALUES
(1, 'pending'),
(2, 'shipped'),
(3, 'delivered'),
(4, 'cancelled'),
(5, 'pending');

-- Find active orders (not cancelled or delivered)
SELECT * FROM orders
WHERE status IN ('pending', 'shipped', 'processing');

-- Example 8: NOT IN with subquery and NULL handling (CAUTION!)
-- Create tables to demonstrate NULL issue
CREATE TABLE dept_blacklist (dept_name VARCHAR(50));
INSERT INTO dept_blacklist VALUES ('HR'), (NULL);

-- This query will return NO rows because of NULL in subquery
SELECT * FROM employees
WHERE department NOT IN (SELECT dept_name FROM dept_blacklist);
-- Reason: NULL in subquery makes NOT IN behave unpredictably

-- Safe way: exclude NULLs
SELECT * FROM employees
WHERE department NOT IN (
    SELECT dept_name
    FROM dept_blacklist
    WHERE dept_name IS NOT NULL
);

-- Example 9: IN with JOIN (sometimes better performance)
-- Instead of IN subquery:
SELECT * FROM employees
WHERE department IN (
    SELECT department
    FROM dept_budget
```

```sql
    WHERE budget > 1000000
);

-- Consider using JOIN:
SELECT DISTINCT e.*
FROM employees e
JOIN dept_budget d ON e.department = d.department
WHERE d.budget > 1000000;

-- Example 10: IN with multiple columns (some databases support)
-- Find employees with specific (department, salary) combinations
-- Not standard SQL in all databases
-- SELECT * FROM employees
-- WHERE (department, salary) IN (('IT', 75000), ('HR', 65000));

-- Example 11: IN with array of values (dynamic)
-- For application use with prepared statements
-- Suppose you have an array of IDs from frontend: [1, 2, 3]
-- In application code, you'd generate:
-- SELECT * FROM employees WHERE emp_id IN (?, ?, ?)

-- Example 12: IN with LIKE (can't directly combine)
-- Wrong: WHERE name IN ('J%', 'B%')  -- Doesn't work with wildcards
-- Correct approach:
SELECT * FROM employees
WHERE first_name LIKE 'J%' OR first_name LIKE 'B%';

-- Example 13: IN with EXISTS - sometimes better for large datasets
-- Find employees who have orders
SELECT * FROM employees e
WHERE e.emp_id IN (SELECT DISTINCT customer_id FROM orders);  -- IN version

SELECT * FROM employees e
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = e.emp_id);  -- EXISTS v
ersion (often faster)

-- Example 14: IN with literal values
SELECT
    first_name,
    last_name,
    CASE
        WHEN department IN ('IT', 'Engineering') THEN 'Tech'
```

```sql
        WHEN department IN ('HR', 'Admin') THEN 'Support'
        WHEN department IN ('Sales', 'Marketing') THEN 'Revenue'
        ELSE 'Other'
    END AS department_category
FROM employees;

-- Example 15: IN with large lists (performance)
-- For very large lists, consider:
-- 1. Temporary tables
CREATE TEMPORARY TABLE temp_ids (id INT);
INSERT INTO temp_ids VALUES (1), (2), (3), (4), (5);

SELECT e.*
FROM employees e
JOIN temp_ids t ON e.emp_id = t.id;

-- 2. Using EXISTS
-- 3. Using IN with indexed column is usually fine for lists up to few thousand

-- Example 16: IN with different data types
-- Works with any data type that can be compared
SELECT * FROM products
WHERE category IN ('Electronics', 'Computers')
   OR price IN (29.99, 49.99, 99.99)
   OR product_code IN ('ABC123', 'XYZ789');

-- Example 17: Real-world scenario - filter by multiple criteria
-- Find employees who:
-- Work in IT or Finance AND
-- Have salary in top ranges AND
-- Were hired in specific years
SELECT first_name, last_name, department, salary, YEAR(hire_date) as hire_year
FROM employees
WHERE department IN ('IT', 'Finance')
  AND salary IN (75000, 82000, 90000)
  AND YEAR(hire_date) IN (2018, 2020, 2021);
```

9. IS NULL / IS NOT NULL

These operators are used to test for NULL values. NULL represents missing or unknown data.

Theory:

- **IS NULL** checks if value is NULL
- **IS NOT NULL** checks if value is NOT NULL
- Cannot use = NULL or != NULL (these always return NULL)
- NULL is not equal to anything, not even another NULL
- Important in data quality checks and reporting

Why not = NULL?

sql

```sql
-- These are ALWAYS FALSE (actually, they return NULL, not FALSE)
SELECT * FROM employees WHERE email = NULL;     -- Returns nothing
SELECT * FROM employees WHERE email != NULL;    -- Returns nothing

-- Always use:
SELECT * FROM employees WHERE email IS NULL;    -- Correct
SELECT * FROM employees WHERE email IS NOT NULL; -- Correct
```

Practical Examples:

sql

```sql
-- Example 1: Basic IS NULL
-- Find employees without email
SELECT first_name, last_name, email
FROM employees
WHERE email IS NULL;

-- Example 2: Basic IS NOT NULL
-- Find employees with email
SELECT first_name, last_name, email
FROM employees
WHERE email IS NOT NULL;
```

```sql
-- Example 3: NULL in WHERE clause with other conditions
-- Find IT employees with missing email
SELECT first_name, last_name, department, email
FROM employees
WHERE department = 'IT'
  AND email IS NULL;

-- Example 4: NULL handling in calculations
-- Calculate total compensation with potential NULL bonus
CREATE TABLE compensation (
    emp_id INT,
    base_salary DECIMAL(10,2),
    bonus DECIMAL(10,2),
    commission DECIMAL(10,2)
);

INSERT INTO compensation VALUES
(1, 50000, 5000, 2000),
(2, 60000, NULL, 3000),
(3, 55000, 4000, NULL),
(4, 52000, NULL, NULL);

-- Problem: NULL values break calculations
SELECT
    emp_id,
    base_salary,
    bonus,
    commission,
    base_salary + bonus + commission AS total_compensation  -- NULL if any part is
NULL
FROM compensation;

-- Solution: Use COALESCE or IFNULL
SELECT
    emp_id,
    base_salary,
    COALESCE(bonus, 0) AS bonus,
    COALESCE(commission, 0) AS commission,
    base_salary + COALESCE(bonus, 0) + COALESCE(commission, 0) AS total_compensati
on
FROM compensation;
```

```sql
-- Example 5: NULL in aggregate functions
SELECT
    COUNT(*) AS total_rows,
    COUNT(bonus) AS rows_with_bonus,  -- COUNT ignores NULLs
    SUM(bonus) AS total_bonus,         -- SUM ignores NULLs
    AVG(bonus) AS avg_bonus,            -- AVG ignores NULLs
    MIN(bonus) AS min_bonus,            -- MIN ignores NULLs
    MAX(bonus) AS max_bonus             -- MAX ignores NULLs
FROM compensation;

-- Example 6: NULL in ORDER BY
-- NULLs typically appear first in ASC, last in DESC
SELECT emp_id, bonus
FROM compensation
ORDER BY bonus ASC;
-- Results: NULL, NULL, 4000, 5000 (or NULLs first)

SELECT emp_id, bonus
FROM compensation
ORDER BY bonus DESC;
-- Results: 5000, 4000, NULL, NULL (NULLs last)

-- Some databases allow controlling NULL position
-- PostgreSQL: ORDER BY bonus NULLS LAST;
-- Oracle: ORDER BY bonus NULLS FIRST;

-- Example 7: NULL in DISTINCT
-- NULL is considered a distinct value
INSERT INTO compensation (emp_id, bonus) VALUES (5, NULL), (6, NULL);
SELECT DISTINCT bonus FROM compensation;
-- Result: NULL, 4000, 5000

-- Example 8: NULL in JOINs
-- Create two tables
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE
);

CREATE TABLE customers (
```

```sql
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100),
    email VARCHAR(100)
);

INSERT INTO orders VALUES (1, 101, '2024-01-01'), (2, NULL, '2024-01-02');
INSERT INTO customers VALUES (101, 'John Doe', 'john@email.com'), (102, 'Jane Smit
h', 'jane@email.com');

-- LEFT JOIN with NULL
SELECT o.order_id, o.customer_id, c.customer_name
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id;
-- order_id 2 will have NULL for customer_name

-- Example 9: NULL in UNIQUE constraints
CREATE TABLE unique_test (
    id INT PRIMARY KEY,
    code VARCHAR(10) UNIQUE
);

INSERT INTO unique_test VALUES (1, 'A');
INSERT INTO unique_test VALUES (2, NULL);  -- Works
INSERT INTO unique_test VALUES (3, NULL);  -- Works in MySQL, fails in some databa
ses
-- In MySQL: multiple NULLs allowed in UNIQUE column
-- In PostgreSQL: multiple NULLs allowed
-- In SQL Server: multiple NULLs allowed

-- Example 10: NULL in CHECK constraints
CREATE TABLE employees_check (
    emp_id INT PRIMARY KEY,
    termination_date DATE,
    CHECK (termination_date IS NULL OR termination_date >= hire_date)
    -- Validates: if termination exists, it must be after hire date
);

-- Example 11: NULL in CASE statements
SELECT
    first_name,
    last_name,
    CASE
```

```sql
        WHEN email IS NULL THEN 'No email provided'
        WHEN email IS NOT NULL THEN email
        ELSE 'Unknown'  -- This case never happens
    END AS email_status
FROM employees;

-- Example 12: NULL handling with IFNULL/COALESCE
-- MySQL: IFNULL
SELECT
    first_name,
    IFNULL(email, 'No email') AS email
FROM employees;

-- Standard SQL: COALESCE (works with multiple values)
SELECT
    first_name,
    COALESCE(email, phone, 'No contact info') AS contact
FROM employees;

-- Example 13: NULL in boolean expressions
-- NULL in AND/OR logic
-- NULL AND TRUE = NULL
-- NULL AND FALSE = FALSE
-- NULL OR TRUE = TRUE
-- NULL OR FALSE = NULL
-- NOT NULL = NULL

SELECT
    NULL AND TRUE,    -- NULL
    NULL AND FALSE,   -- FALSE
    NULL OR TRUE,     -- TRUE
    NULL OR FALSE,    -- NULL
    NOT NULL          -- NULL
FROM dual;

-- Example 14: NULLIF function
-- NULLIF returns NULL if two expressions are equal
SELECT
    emp_id,
    bonus,
    NULLIF(bonus, 0) AS non_zero_bonus  -- Returns NULL if bonus = 0
FROM compensation;
```

```sql
-- Example 15: Data quality checks with IS NULL
-- Find incomplete records
SELECT * FROM employees
WHERE first_name IS NULL
   OR last_name IS NULL
   OR email IS NULL;

-- Find fully complete records
SELECT * FROM employees
WHERE first_name IS NOT NULL
  AND last_name IS NOT NULL
  AND email IS NOT NULL;

-- Example 16: NULL in string concatenation
SELECT
    CONCAT(first_name, ' ', last_name) AS full_name,  -- Works fine with NULL?
    CONCAT(first_name, ' ', COALESCE(last_name, '')) AS safe_full_name
FROM employees;

-- Example 17: NULL in date comparisons
-- Find orders that haven't been shipped (ship_date IS NULL)
-- AND aren't cancelled (cancelled_date IS NULL)
SELECT * FROM orders
WHERE shipped_date IS NULL
  AND cancelled_date IS NULL;

-- Find orders shipped but not delivered
SELECT * FROM orders
WHERE shipped_date IS NOT NULL
  AND delivered_date IS NULL;

-- Example 18: NULL with GROUP BY
-- NULL forms its own group
SELECT
    department,
    COUNT(*) as emp_count,
    AVG(salary) as avg_salary
FROM employees
GROUP BY department;
-- NULL department will be a separate group if any
```

```sql
-- Example 19: NULL in window functions
SELECT
    first_name,
    salary,
    LAG(salary) OVER (ORDER BY emp_id) AS previous_salary,  -- NULL for first row
    LEAD(salary) OVER (ORDER BY emp_id) AS next_salary      -- NULL for last row
FROM employees;

-- Example 20: Practical scenario - optional fields
CREATE TABLE user_profiles (
    user_id INT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,  -- Required
    full_name VARCHAR(100),          -- Optional
    bio TEXT,                        -- Optional
    profile_picture VARCHAR(500),    -- Optional
    last_login TIMESTAMP,             -- Will be NULL until first login
    deleted_at TIMESTAMP NULL         -- NULL means not deleted
);

-- Find active users (not deleted)
SELECT * FROM user_profiles WHERE deleted_at IS NULL;

-- Find users who never logged in
SELECT * FROM user_profiles WHERE last_login IS NULL;

-- Find users with complete profiles
SELECT * FROM user_profiles
WHERE full_name IS NOT NULL
  AND bio IS NOT NULL
  AND profile_picture IS NOT NULL;
```

Summary: Quick Reference Table

Operator	Purpose	Example	Notes
SELECT	Retrieve data	`SELECT name FROM users`	Can select columns, expressions, functions

Operator	Purpose	Example	Notes
WHERE	Filter rows	`WHERE age > 18`	Use with AND/OR/NOT
ORDER BY	Sort results	`ORDER BY name DESC`	ASC (default) or DESC
DISTINCT	Remove duplicates	`SELECT DISTINCT city`	Works on column combinations
LIMIT	Restrict rows	`LIMIT 10`	Use with OFFSET for pagination
OFFSET	Skip rows	`OFFSET 20`	Usually with LIMIT
LIKE	Pattern matching	`WHERE name LIKE 'J%'`	% = multiple, _ = single
BETWEEN	Range check	`WHERE age BETWEEN 18 AND 65`	Inclusive
IN	Multiple values	`WHERE city IN ('NYC', 'LA')`	Works with subqueries
IS NULL	Check for NULL	`WHERE email IS NULL`	Never use = NULL
IS NOT NULL	Check for non-NULL	`WHERE email IS NOT NULL`	For required fields

Data Manipulation Language (DML) and Data Definition Language (DDL): Complete Theory Notes with Practical Examples

Part 1: Data Manipulation Language (DML)

DML commands are used to manage data within database objects. These commands handle the actual data manipulation - adding, modifying, and removing records.

1. INSERT Statement

The **INSERT** statement is used to add new rows of data to a table.

Theory:

- Adds one or more rows to a table
- Can insert values for all columns or specific columns
- Can insert data from another table using SELECT
- Must respect all constraints (NOT NULL, UNIQUE, FOREIGN KEY, etc.)
- Auto-increment columns are automatically generated

Syntax Variations:

sql

```sql
-- Insert single row (all columns)
INSERT INTO table_name VALUES (value1, value2, ...);

-- Insert single row (specific columns)
INSERT INTO table_name (column1, column2) VALUES (value1, value2);

-- Insert multiple rows
INSERT INTO table_name (column1, column2) VALUES
    (value1, value2),
    (value3, value4),
    (value5, value6);
```

```sql
-- Insert from another table
INSERT INTO table_name (column1, column2)
SELECT column1, column2 FROM another_table WHERE condition;
```

Practical Examples:

sql

```sql
-- Example 1: Create sample table
CREATE TABLE students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    age INT CHECK (age >= 16),
    enrollment_date DATE DEFAULT (CURRENT_DATE),
    is_active BOOLEAN DEFAULT TRUE
);

-- Example 2: Basic INSERT - all columns
INSERT INTO students
VALUES (1, 'John', 'Doe', 'john.doe@email.com', 20, '2024-01-15', TRUE);

-- Example 3: INSERT with AUTO_INCREMENT (skip student_id)
INSERT INTO students (first_name, last_name, email, age, enrollment_date, is_activ
e)
VALUES ('Jane', 'Smith', 'jane.smith@email.com', 22, '2024-01-16', TRUE);

-- Example 4: INSERT with DEFAULT values
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Bob', 'Johnson', 'bob.johnson@email.com', 19);
-- enrollment_date uses DEFAULT (current date), is_active uses DEFAULT (TRUE)

-- Example 5: INSERT multiple rows
INSERT INTO students (first_name, last_name, email, age) VALUES
    ('Alice', 'Brown', 'alice.brown@email.com', 21),
    ('Charlie', 'Wilson', 'charlie.wilson@email.com', 23),
    ('Diana', 'Miller', 'diana.miller@email.com', 20);

-- Example 6: INSERT with SELECT (copy data)
CREATE TABLE student_archive LIKE students;
```

```sql
INSERT INTO student_archive (first_name, last_name, email, age)
SELECT first_name, last_name, email, age
FROM students
WHERE age > 21;

-- Example 7: INSERT IGNORE (MySQL) - skip errors
-- Skip duplicate email instead of failing
INSERT IGNORE INTO students (first_name, last_name, email, age)
VALUES ('Eve', 'Davis', 'john.doe@email.com', 24);  -- Duplicate email, will be ig
nored

-- Example 8: INSERT ON DUPLICATE KEY UPDATE (MySQL)
-- Update if exists, insert if not
INSERT INTO students (student_id, first_name, last_name, email, age)
VALUES (1, 'John', 'Doe', 'john.new@email.com', 21)
ON DUPLICATE KEY UPDATE
    email = VALUES(email),
    age = VALUES(age);

-- Example 9: INSERT with NULL values
INSERT INTO students (first_name, last_name, email, age)
VALUES ('Frank', 'Thomas', NULL, 25);  -- email can be NULL

-- Example 10: INSERT with functions
INSERT INTO students (first_name, last_name, email, age, enrollment_date)
VALUES (
    'Grace',
    'Lee',
    LOWER('Grace.Lee@email.com'),
    FLOOR(RAND() * 10) + 18,
    DATE_SUB(CURDATE(), INTERVAL 30 DAY)
);

-- Example 11: INSERT with subquery
CREATE TABLE student_summary AS SELECT * FROM students WHERE 1=0;  -- Create empty
copy

INSERT INTO student_summary (student_id, full_name, email, age_group)
SELECT
    student_id,
    CONCAT(first_name, ' ', last_name) AS full_name,
```

```sql
    email,
    CASE
        WHEN age < 20 THEN 'Teen'
        WHEN age BETWEEN 20 AND 25 THEN 'Young Adult'
        ELSE 'Adult'
    END AS age_group
FROM students
WHERE is_active = TRUE;

-- Example 12: INSERT with SELECT from multiple tables
CREATE TABLE enrollment_report (
    report_id INT AUTO_INCREMENT PRIMARY KEY,
    student_name VARCHAR(100),
    course_name VARCHAR(100),
    enrollment_date DATE
);

INSERT INTO enrollment_report (student_name, course_name, enrollment_date)
SELECT
    CONCAT(s.first_name, ' ', s.last_name),
    c.course_name,
    e.enrollment_date
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;

-- Example 13: Conditional INSERT using WHERE NOT EXISTS
-- Insert only if student doesn't already exist
INSERT INTO students (first_name, last_name, email, age)
SELECT 'Henry', 'Ford', 'henry.ford@email.com', 30
WHERE NOT EXISTS (
    SELECT 1 FROM students WHERE email = 'henry.ford@email.com'
);

-- Example 14: INSERT with timestamp
CREATE TABLE audit_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    action VARCHAR(50),
    table_name VARCHAR(50),
    record_id INT,
    changed_by VARCHAR(50),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```sql
);

INSERT INTO audit_log (action, table_name, record_id, changed_by)
VALUES ('INSERT', 'students', LAST_INSERT_ID(), USER());

-- Example 15: INSERT with JSON data (MySQL 5.7+)
CREATE TABLE product_reviews (
    review_id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT,
    review_data JSON
);

INSERT INTO product_reviews (product_id, review_data) VALUES
(1, '{"rating": 5, "comment": "Excellent product!", "verified": true}'),
(2, '{"rating": 4, "comment": "Good value", "verified": false}');

-- Example 16: INSERT with ON CONFLICT (PostgreSQL)
-- PostgreSQL syntax for handling conflicts
-- INSERT INTO students (first_name, last_name, email, age)
-- VALUES ('Ivan', 'Petrov', 'ivan@email.com', 28)
-- ON CONFLICT (email)
-- DO UPDATE SET age = EXCLUDED.age;

-- Example 17: INSERT with RETURNING (PostgreSQL)
-- INSERT INTO students (first_name, last_name, email, age)
-- VALUES ('Julia', 'Roberts', 'julia@email.com', 35)
-- RETURNING student_id, CONCAT(first_name, ' ', last_name) AS full_name;
```

2. UPDATE Statement

The **UPDATE** statement is used to modify existing records in a table.

Theory:

- Modifies data in existing rows
- Can update one or multiple columns
- Should always use WHERE clause (otherwise updates ALL rows)
- Can use expressions and subqueries in SET clause
- Respects constraints and triggers

Syntax:

```sql
-- Basic update
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;

-- Update with expression
UPDATE table_name
SET column1 = column1 * 1.1  -- Increase by 10%
WHERE condition;

-- Update with subquery
UPDATE table_name
SET column1 = (SELECT expression FROM another_table)
WHERE condition;
```

Practical Examples:

```sql
-- Example 1: Basic UPDATE
-- Update single column for specific student
UPDATE students
SET email = 'john.doe.updated@email.com'
WHERE student_id = 1;

-- Example 2: UPDATE multiple columns
UPDATE students
SET
    age = 21,
    is_active = FALSE
WHERE student_id = 2;

-- Example 3: UPDATE without WHERE (DANGEROUS!)
-- This updates ALL rows - use with caution!
UPDATE students SET is_active = FALSE;
-- Better to always include WHERE

-- Example 4: UPDATE with expression
-- Give 10% age increase? No, that doesn't make sense. Better:
```

```sql
-- Increase age by 1 year for all active students
UPDATE students
SET age = age + 1
WHERE is_active = TRUE;

-- Example 5: UPDATE with calculated value
-- Set email domain to new company
UPDATE students
SET email = REPLACE(email, '@email.com', '@newdomain.com')
WHERE email LIKE '%@email.com';

-- Example 6: UPDATE with CASE statement
UPDATE students
SET age_group = CASE
    WHEN age < 20 THEN 'Teen'
    WHEN age BETWEEN 20 AND 25 THEN 'Young'
    WHEN age BETWEEN 26 AND 35 THEN 'Adult'
    ELSE 'Senior'
END;

-- Example 7: UPDATE with subquery
-- Update student status based on enrollment count
UPDATE students s
SET s.status = CASE
    WHEN (SELECT COUNT(*) FROM enrollments e WHERE e.student_id = s.student_id) >
3
    THEN 'active'
    ELSE 'inactive'
END;

-- Example 8: UPDATE with JOIN
-- Update students with their department name
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);

ALTER TABLE students ADD COLUMN dept_id INT;
ALTER TABLE students ADD COLUMN dept_name VARCHAR(50);

UPDATE students s
JOIN departments d ON s.dept_id = d.dept_id
```

```sql
SET s.dept_name = d.dept_name;

-- Example 9: UPDATE with LIMIT (MySQL)
-- Update only first 3 matching records
UPDATE students
SET is_active = FALSE
WHERE age > 25
LIMIT 3;

-- Example 10: UPDATE with ORDER BY (MySQL)
-- Update oldest students first
UPDATE students
SET priority = 'high'
WHERE is_active = TRUE
ORDER BY age DESC
LIMIT 5;

-- Example 11: UPDATE with COALESCE for NULL handling
UPDATE students
SET email = COALESCE(email, CONCAT(first_name, '.', last_name, '@default.com'))
WHERE email IS NULL;

-- Example 12: UPDATE with date functions
-- Set enrollment date to current date for inactive students
UPDATE students
SET enrollment_date = CURDATE()
WHERE is_active = FALSE;

-- Example 13: UPDATE with multiple conditions
UPDATE students
SET
    status = 'graduated',
    graduation_date = CURDATE()
WHERE age >= 22
  AND is_active = TRUE
  AND status != 'graduated';

-- Example 14: UPDATE with RANDOM values
-- Assign random age to students with NULL age
UPDATE students
SET age = FLOOR(18 + RAND() * 30)
WHERE age IS NULL;
```

```sql
-- Example 15: UPDATE with string functions
UPDATE students
SET
    first_name = CONCAT(UPPER(LEFT(first_name, 1)), LOWER(SUBSTRING(first_name, 2)
)),
    last_name = CONCAT(UPPER(LEFT(last_name, 1)), LOWER(SUBSTRING(last_name, 2)));

-- Example 16: UPDATE with numeric operations
-- Apply salary increase with conditions
UPDATE employees
SET salary = CASE
    WHEN department = 'IT' THEN salary * 1.10  -- 10% raise
    WHEN department = 'HR' THEN salary * 1.05  -- 5% raise
    ELSE salary * 1.03                         -- 3% raise
END
WHERE is_active = TRUE;

-- Example 17: UPDATE with EXISTS
-- Update only students who have enrollments
UPDATE students s
SET s.has_enrollment = TRUE
WHERE EXISTS (SELECT 1 FROM enrollments e WHERE e.student_id = s.student_id);

-- Example 18: UPDATE with RETURNING (PostgreSQL)
-- UPDATE students
-- SET is_active = FALSE
-- WHERE student_id = 5
-- RETURNING student_id, first_name, last_name, is_active;

-- Example 19: Safe UPDATE pattern
-- Always SELECT first to verify
SELECT * FROM students WHERE student_id = 10;

-- Then UPDATE
UPDATE students SET age = 25 WHERE student_id = 10;

-- Verify the UPDATE
SELECT * FROM students WHERE student_id = 10;

-- Example 20: UPDATE with transaction for safety
START TRANSACTION;
```

```sql
UPDATE students
SET age = 30
WHERE student_id = 100;

-- Check if correct
SELECT * FROM students WHERE student_id = 100;

-- If correct:
COMMIT;

-- If wrong:
ROLLBACK;
```

3. DELETE Statement

The **DELETE** statement removes existing records from a table.

Theory:

- Removes rows from a table
- Always use WHERE clause (otherwise deletes ALL rows)
- Can be rolled back if in transaction
- Triggers DELETE triggers
- Does NOT reset AUTO_INCREMENT counter
- Slower than TRUNCATE for removing all rows

Syntax:

sql

```sql
-- Basic delete
DELETE FROM table_name WHERE condition;

-- Delete all rows
DELETE FROM table_name;  -- No WHERE clause

-- Delete with subquery
DELETE FROM table_name
```

```sql
WHERE column IN (SELECT column FROM another_table WHERE condition);

-- Delete with LIMIT (MySQL)
DELETE FROM table_name WHERE condition LIMIT number;
```

Practical Examples:

```sql
-- Example 1: Basic DELETE
-- Delete specific student
DELETE FROM students
WHERE student_id = 1;

-- Example 2: DELETE with condition
-- Delete inactive students
DELETE FROM students
WHERE is_active = FALSE;

-- Example 3: DELETE without WHERE (DANGEROUS!)
-- This deletes ALL rows!
DELETE FROM students;
-- Use with extreme caution!

-- Example 4: DELETE with multiple conditions
DELETE FROM students
WHERE age < 18
   OR enrollment_date < '2020-01-01'
   AND is_active = FALSE;

-- Example 5: DELETE with subquery
-- Delete students with no enrollments
DELETE FROM students
WHERE student_id NOT IN (
    SELECT DISTINCT student_id
    FROM enrollments
    WHERE student_id IS NOT NULL
);

-- Example 6: DELETE with JOIN (MySQL)
-- Delete students and their records from related tables
DELETE s, e
```

```sql
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
WHERE s.student_id = 5;

-- Example 7: DELETE with LIMIT (MySQL)
-- Delete oldest 10 inactive students
DELETE FROM students
WHERE is_active = FALSE
ORDER BY enrollment_date
LIMIT 10;

-- Example 8: DELETE with EXISTS
DELETE FROM students s
WHERE EXISTS (
    SELECT 1
    FROM enrollments e
    WHERE e.student_id = s.student_id
    AND e.grade = 'F'
);

-- Example 9: Cascade DELETE (if foreign key with CASCADE)
-- If foreign key is set with ON DELETE CASCADE
DELETE FROM departments
WHERE dept_id = 10;
-- This will automatically delete all employees in that department

-- Example 10: DELETE with date condition
-- Delete records older than 1 year
DELETE FROM audit_log
WHERE changed_at < DATE_SUB(NOW(), INTERVAL 1 YEAR);

-- Example 11: DELETE with NULL check
-- Delete records with missing data
DELETE FROM students
WHERE email IS NULL AND age IS NULL;

-- Example 12: DELETE with IN operator
DELETE FROM students
WHERE student_id IN (100, 101, 102, 103);

-- Example 13: DELETE with BETWEEN
DELETE FROM students
```

```sql
WHERE age BETWEEN 30 AND 40
AND is_active = FALSE;

-- Example 14: DELETE with pattern matching
-- Delete temporary users
DELETE FROM users
WHERE username LIKE 'temp_%'
AND created_at < DATE_SUB(NOW(), INTERVAL 7 DAY);

-- Example 15: Safe DELETE pattern with transaction
START TRANSACTION;

-- First, count records to be deleted
SELECT COUNT(*) FROM students WHERE is_active = FALSE AND age > 60;

-- Review a sample
SELECT * FROM students WHERE is_active = FALSE AND age > 60 LIMIT 5;

-- If correct, delete
DELETE FROM students WHERE is_active = FALSE AND age > 60;

-- Check result
SELECT ROW_COUNT() AS deleted_count;

-- Commit if happy
COMMIT;
-- Or rollback if not
-- ROLLBACK;

-- Example 16: DELETE with RETURNING (PostgreSQL)
-- DELETE FROM students
-- WHERE is_active = FALSE
-- RETURNING student_id, first_name, last_name;

-- Example 17: DELETE with ORDER BY and LIMIT for batch processing
-- Delete in batches to avoid locks
DELETE FROM large_table
WHERE created_at < '2020-01-01'
ORDER BY id
LIMIT 1000;
-- Run this multiple times until no rows affected
```

```sql
-- Example 18: DELETE vs TRUNCATE comparison
-- DELETE (slower, can be rolled back, doesn't reset AUTO_INCREMENT)
DELETE FROM students;

-- TRUNCATE (faster, can't rollback in some DBs, resets AUTO_INCREMENT)
TRUNCATE TABLE students;

-- Example 19: DELETE with foreign key considerations
-- First check for dependencies
SELECT * FROM enrollments WHERE student_id = 10;

-- Then decide action
-- Option 1: Delete dependencies first
DELETE FROM enrollments WHERE student_id = 10;
DELETE FROM students WHERE student_id = 10;

-- Option 2: Update to NULL or default
UPDATE enrollments SET student_id = NULL WHERE student_id = 10;
DELETE FROM students WHERE student_id = 10;

-- Example 20: Clean up orphaned records
-- Delete enrollments without valid students
DELETE FROM enrollments
WHERE student_id NOT IN (SELECT student_id FROM students)
   OR student_id IS NULL;
```

4. MERGE Statement (UPSERT)

The **MERGE** statement (also known as UPSERT) performs INSERT, UPDATE, or DELETE operations based on source data.

Theory:

- Combines INSERT, UPDATE, and DELETE in one statement
- Also called UPSERT (Update + Insert)
- Useful for synchronizing tables
- Different databases have different syntax:
  - **MERGE** (SQL Server, Oracle, PostgreSQL)

- **INSERT ... ON DUPLICATE KEY UPDATE** (MySQL)
- **INSERT ... ON CONFLICT** (PostgreSQL)
- **REPLACE INTO** (MySQL)

Syntax (Standard SQL):

```sql
MERGE INTO target_table AS target
USING source_table AS source
ON target.key = source.key
WHEN MATCHED THEN
    UPDATE SET target.column = source.column
WHEN NOT MATCHED THEN
    INSERT (column1, column2) VALUES (source.value1, source.value2)
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

Practical Examples:

```sql
-- Example 1: Prepare tables for MERGE
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2),
    stock INT,
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE product_updates (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2),
    stock INT
);

-- Insert sample data
INSERT INTO products VALUES
(1, 'Laptop', 999.99, 10, NOW()),
(2, 'Mouse', 29.99, 50, NOW()),
```

```sql
(3, 'Keyboard', 79.99, 30, NOW());

INSERT INTO product_updates VALUES
(1, 'Laptop', 949.99, 8),      -- Update price and stock
(2, 'Mouse', 29.99, 45),        -- Update stock only
(4, 'Monitor', 299.99, 15);     -- New product

-- Example 2: MERGE (SQL Server/Oracle/PostgreSQL syntax)
-- MERGE INTO products AS p
-- USING product_updates AS u
-- ON p.product_id = u.product_id
-- WHEN MATCHED THEN
--     UPDATE SET
--         p.price = u.price,
--         p.stock = u.stock,
--         p.last_updated = CURRENT_TIMESTAMP
-- WHEN NOT MATCHED THEN
--     INSERT (product_id, product_name, price, stock, last_updated)
--     VALUES (u.product_id, u.product_name, u.price, u.stock, CURRENT_TIMESTAMP);

-- Example 3: MySQL alternative - INSERT ON DUPLICATE KEY UPDATE
INSERT INTO products (product_id, product_name, price, stock, last_updated)
SELECT
    product_id,
    product_name,
    price,
    stock,
    CURRENT_TIMESTAMP
FROM product_updates
ON DUPLICATE KEY UPDATE
    price = VALUES(price),
    stock = VALUES(stock),
    last_updated = VALUES(last_updated);

-- Example 4: MySQL - REPLACE INTO (simpler but less control)
-- REPLACE deletes and inserts, so auto-increment changes
REPLACE INTO products (product_id, product_name, price, stock, last_updated)
SELECT * FROM product_updates;

-- Example 5: PostgreSQL - INSERT ON CONFLICT
-- INSERT INTO products (product_id, product_name, price, stock, last_updated)
-- SELECT * FROM product_updates
```

```
-- ON CONFLICT (product_id)
-- DO UPDATE SET
--     price = EXCLUDED.price,
--     stock = EXCLUDED.stock,
--     last_updated = EXCLUDED.last_updated;

-- Example 6: Complex MERGE with conditions
-- MERGE INTO inventory i
-- USING sales_data s
-- ON i.product_id = s.product_id
-- WHEN MATCHED AND s.quantity > i.quantity THEN
--     UPDATE SET i.status = 'backorder'
-- WHEN MATCHED THEN
--     UPDATE SET i.quantity = i.quantity - s.quantity
-- WHEN NOT MATCHED THEN
--     INSERT (product_id, quantity, status)
--     VALUES (s.product_id, s.quantity, 'new')
-- WHEN NOT MATCHED BY SOURCE AND i.status = 'discontinued' THEN
--     DELETE;

-- Example 7: MERGE with multiple sources
-- MERGE INTO employee_target t
-- USING (
--     SELECT e.emp_id, e.salary, d.bonus_percent
--     FROM employees e
--     JOIN departments d ON e.dept_id = d.dept_id
-- ) s
-- ON t.emp_id = s.emp_id
-- WHEN MATCHED THEN
--     UPDATE SET
--         t.salary = s.salary,
--         t.bonus = s.salary * s.bonus_percent / 100;

-- Example 8: Data warehouse slowly changing dimension (Type 1)
-- MERGE INTO dim_customer d
-- USING staging_customer s
-- ON d.customer_id = s.customer_id
-- WHEN MATCHED AND (
--     d.name != s.name OR
--     d.address != s.address OR
--     d.phone != s.phone
-- ) THEN
```

```sql
--     UPDATE SET
--         d.name = s.name,
--         d.address = s.address,
--         d.phone = s.phone,
--         d.updated_date = CURRENT_DATE
-- WHEN NOT MATCHED THEN
--     INSERT (customer_id, name, address, phone, created_date)
--     VALUES (s.customer_id, s.name, s.address, s.phone, CURRENT_DATE);

-- Example 9: MySQL with multiple row handling
CREATE TEMPORARY TABLE temp_updates (
    id INT,
    value INT
);

INSERT INTO temp_updates VALUES (1, 100), (2, 200), (3, 300);

INSERT INTO target_table (id, value, updated_at)
SELECT id, value, NOW() FROM temp_updates
ON DUPLICATE KEY UPDATE
    value = VALUES(value),
    updated_at = VALUES(updated_at);

-- Example 10: MERGE with delete condition
-- MERGE INTO active_products a
-- USING products p
-- ON a.product_id = p.product_id
-- WHEN MATCHED AND p.stock = 0 THEN
--     DELETE
-- WHEN NOT MATCHED AND p.stock > 0 THEN
--     INSERT (product_id, product_name, stock)
--     VALUES (p.product_id, p.product_name, p.stock);
```

5. Bulk Insert Operations

Bulk insert operations are used to efficiently insert large amounts of data.

Theory:

- Much faster than individual INSERT statements

- Reduces network round trips
- Can load from files
- Different methods per database
- May disable constraints temporarily for speed

Methods:

1. **INSERT with multiple VALUES** (standard)
2. **LOAD DATA INFILE** (MySQL)
3. **BULK INSERT** (SQL Server)
4. **COPY** (PostgreSQL)
5. **INSERT INTO ... SELECT** (standard)
6. **Batch processing with transactions**

Practical Examples:

sql

```sql
-- Example 1: Multiple VALUES INSERT (standard)
INSERT INTO students (first_name, last_name, email, age) VALUES
('User1', 'Test1', 'user1@test.com', 20),
('User2', 'Test2', 'user2@test.com', 21),
('User3', 'Test3', 'user3@test.com', 22),
-- ... can be thousands of rows
('User1000', 'Test1000', 'user1000@test.com', 25);

-- Example 2: MySQL - LOAD DATA INFILE
-- Load from CSV file
LOAD DATA INFILE '/tmp/students.csv'
INTO TABLE students
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS  -- Skip header
(first_name, last_name, email, age, @dummy, @dummy)
SET
    enrollment_date = CURDATE(),
    is_active = TRUE;
```

```sql
-- Example 3: MySQL - LOAD DATA LOCAL INFILE (from client)
LOAD DATA LOCAL INFILE 'C:\\data\\students.csv'
INTO TABLE students
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\r\n'
(first_name, last_name, email, age);

-- Example 4: PostgreSQL - COPY command
-- COPY students(first_name, last_name, email, age)
-- FROM '/tmp/students.csv'
-- DELIMITER ','
-- CSV HEADER;

-- Example 5: SQL Server - BULK INSERT
-- BULK INSERT students
-- FROM 'C:\data\students.csv'
-- WITH (
--     FIELDTERMINATOR = ',',
--     ROWTERMINATOR = '\n',
--     FIRSTROW = 2,
--     BATCHSIZE = 1000
-- );

-- Example 6: INSERT INTO ... SELECT for copying between tables
INSERT INTO student_history (student_id, first_name, last_name, email, age, archiv
ed_date)
SELECT
    student_id,
    first_name,
    last_name,
    email,
    age,
    CURDATE()
FROM students
WHERE is_active = FALSE;

-- Example 7: Bulk insert with transaction (for rollback ability)
START TRANSACTION;

-- Insert in batches
INSERT INTO large_table (col1, col2)
SELECT col1, col2 FROM source_table WHERE id BETWEEN 1 AND 10000;
```

```sql
INSERT INTO large_table (col1, col2)
SELECT col1, col2 FROM source_table WHERE id BETWEEN 10001 AND 20000;

-- Commit if all good
COMMIT;
-- Or rollback if error
-- ROLLBACK;

-- Example 8: MySQL - INSERT with SELECT and UNION for generated data
INSERT INTO numbers (num, square, cube)
SELECT n, n*n, n*n*n
FROM (
    SELECT 1 AS n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5
    UNION SELECT 6 UNION SELECT 7 UNION SELECT 8 UNION SELECT 9 UNION SELECT 10
) AS nums;

-- Example 9: Bulk insert with prepared statements (application level)
-- In application code (Python example):
-- import mysql.connector
-- data = [(f'User{i}', f'Test{i}', f'user{i}@test.com', 20 + i%10) for i in range
(1000)]
-- cursor.executemany(
--     "INSERT INTO students (first_name, last_name, email, age) VALUES (%s, %s, %
s, %s)",
--     data
-- )

-- Example 10: MySQL - INSERT with IGNORE for error handling
INSERT IGNORE INTO students (first_name, last_name, email, age) VALUES
('User1', 'Test1', 'user1@test.com', 20),
('User2', 'Test2', 'duplicate@test.com', 21),  -- Duplicate email will be ignored
('User3', 'Test3', 'user3@test.com', 22);

-- Example 11: Bulk insert with temporary disabling of constraints (careful!)
-- Disable foreign key checks (MySQL)
SET FOREIGN_KEY_CHECKS = 0;

-- Bulk insert
LOAD DATA INFILE '/tmp/bigdata.csv' INTO TABLE big_table;

-- Re-enable
```

```sql
SET FOREIGN_KEY_CHECKS = 1;

-- Example 12: Batch insert with progress tracking
SET @batch_size = 1000;
SET @counter = 0;

INSERT INTO log_table (message, created_at)
SELECT
    CONCAT('Processing batch ', @counter := @counter + 1),
    NOW()
FROM information_schema.tables
LIMIT 10;

-- Example 13: MySQL - INSERT with SELECT from another database
INSERT INTO current_db.students (first_name, last_name, email)
SELECT first_name, last_name, email
FROM old_db.students
WHERE enrollment_year < 2020;

-- Example 14: Bulk insert with data transformation
LOAD DATA INFILE '/tmp/raw_data.csv'
INTO TABLE processed_data
FIELDS TERMINATED BY ','
(first_name, last_name, @birth_date, @salary)
SET
    full_name = CONCAT(first_name, ' ', last_name),
    birth_date = STR_TO_DATE(@birth_date, '%m/%d/%Y'),
    age = TIMESTAMPDIFF(YEAR, birth_date, CURDATE()),
    salary = REPLACE(@salary, '$', '') * 1.0;

-- Example 15: Performance comparison
-- Slow way (individual inserts):
-- INSERT INTO t VALUES (1); INSERT INTO t VALUES (2); ...

-- Fast way (batch insert):
INSERT INTO t VALUES (1), (2), (3), (4), (5);  -- Much faster

-- Even faster for huge data: LOAD DATA INFILE
```

Part 2: Data Definition Language (DDL)

DDL commands are used to define, modify, and remove database structures.

---

1. CREATE DATABASE

Creates a new database.

Theory:

- Database is a container for tables and other objects
- Can specify character set and collation
- Different databases have different options
- Requires appropriate privileges

Syntax:

sql

```sql
-- Basic
CREATE DATABASE database_name;

-- With options
CREATE DATABASE database_name
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

-- Create if not exists
CREATE DATABASE IF NOT EXISTS database_name;
```

Practical Examples:

sql

```sql
-- Example 1: Basic database creation
CREATE DATABASE company_db;
```

```sql
-- Example 2: Create with character set (MySQL)
CREATE DATABASE university_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

-- Example 3: Create if not exists
CREATE DATABASE IF NOT EXISTS school_db;

-- Example 4: PostgreSQL specific
-- CREATE DATABASE ecommerce_db
-- ENCODING 'UTF8'
-- LC_COLLATE 'en_US.UTF-8'
-- LC_CTYPE 'en_US.UTF-8';

-- Example 5: SQL Server specific
-- CREATE DATABASE inventory_db
-- ON PRIMARY (
--     NAME = 'inventory_data',
--     FILENAME = 'C:\data\inventory.mdf',
--     SIZE = 10MB,
--     MAXSIZE = 100MB,
--     FILEGROWTH = 5MB
-- )
-- LOG ON (
--     NAME = 'inventory_log',
--     FILENAME = 'C:\data\inventory.ldf',
--     SIZE = 5MB,
--     MAXSIZE = 50MB,
--     FILEGROWTH = 5MB
-- );

-- Example 6: Use database
USE company_db;

-- Example 7: Show/create/use pattern
CREATE DATABASE IF NOT EXISTS test_db;
USE test_db;

-- Example 8: Create database with specific tablespace (Oracle)
-- CREATE TABLESPACE app_data
-- DATAFILE 'app_data.dbf' SIZE 100M;
```

```sql
-- CREATE DATABASE app_db
-- DATAFILE 'app_system.dbf' SIZE 100M
-- SYSAUX DATAFILE 'app_sysaux.dbf' SIZE 50M
-- DEFAULT TABLESPACE app_data
-- DEFAULT TEMPORARY TABLESPACE temp;

-- Example 9: Drop database (careful!)
-- DROP DATABASE IF EXISTS test_db;

-- Example 10: Alter database
ALTER DATABASE company_db
CHARACTER SET latin1;

-- Example 11: Database with specific options (MySQL)
CREATE DATABASE IF NOT EXISTS analytics_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_bin
ENCRYPTION='Y';

-- Example 12: View database creation script
SHOW CREATE DATABASE company_db;  -- MySQL
-- \l+ company_db in PostgreSQL

-- Example 13: Set default database
USE company_db;
-- Now all operations default to this database

-- Example 14: Create database with read-only option
-- MySQL: Can't directly set read-only at creation
-- But can set after:
CREATE DATABASE reporting_db;
ALTER DATABASE reporting_db READ ONLY = 1;  -- MySQL 8.0+

-- Example 15: Database with case sensitivity settings
-- MySQL: lower_case_table_names setting affects how table names are stored
-- Set at server level, not per database
```

2. CREATE TABLE

Creates a new table in the database.

Theory:

- Defines table structure (columns, data types)
- Specifies constraints
- Can create from existing table
- Can be temporary
- Can have storage options

Syntax:

sql

```sql
-- Basic
CREATE TABLE table_name (
    column1 datatype constraints,
    column2 datatype constraints,
    ...
    table_constraints
);

-- Create from existing
CREATE TABLE new_table AS
SELECT * FROM existing_table;

-- Create temporary
CREATE TEMPORARY TABLE temp_table (...);
```

Practical Examples:

sql

```sql
-- Example 1: Simple table
CREATE TABLE employees (
    emp_id INT,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
```

```sql
    hire_date DATE
);

-- Example 2: Table with all constraints
CREATE TABLE employees (
    -- Primary key
    emp_id INT AUTO_INCREMENT PRIMARY KEY,

    -- NOT NULL constraints
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,

    -- UNIQUE constraint
    email VARCHAR(100) UNIQUE NOT NULL,

    -- DEFAULT constraint
    hire_date DATE DEFAULT (CURRENT_DATE),
    is_active BOOLEAN DEFAULT TRUE,

    -- CHECK constraint
    salary DECIMAL(10,2) CHECK (salary > 0),
    age INT CHECK (age >= 18 AND age <= 65),

    -- FOREIGN KEY (assuming departments table exists)
    dept_id INT,

    -- Table-level constraints
    CONSTRAINT emp_email_unique UNIQUE (email),
    CONSTRAINT emp_name_unique UNIQUE (first_name, last_name),
    CONSTRAINT fk_department
        FOREIGN KEY (dept_id)
        REFERENCES departments(dept_id)
        ON DELETE SET NULL
);

-- Example 3: Table with composite primary key
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (order_id, product_id),
```

```sql
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- Example 4: Table with ENUM and SET (MySQL)
CREATE TABLE tasks (
    task_id INT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    priority ENUM('low', 'medium', 'high', 'critical') DEFAULT 'medium',
    status ENUM('pending', 'in_progress', 'completed', 'cancelled') DEFAULT 'pendi
ng',
    tags SET('urgent', 'bug', 'feature', 'documentation')  -- MySQL specific
);

-- Example 5: Table with JSON column
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    attributes JSON,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_attributes ((CAST(attributes->>'$.category' AS CHAR(50))))  -- Virtu
al index
);

-- Example 6: Create table with SELECT (CTAS)
CREATE TABLE employee_archive AS
SELECT emp_id, first_name, last_name, email, hire_date
FROM employees
WHERE hire_date < '2020-01-01';

-- Example 7: Create temporary table
CREATE TEMPORARY TABLE temp_sales (
    sale_id INT,
    product_id INT,
    amount DECIMAL(10,2),
    sale_date DATE
);

-- Example 8: Table with indexes
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    email VARCHAR(100) UNIQUE,
```

```sql
    last_name VARCHAR(50),
    first_name VARCHAR(50),
    city VARCHAR(50),
    state VARCHAR(2),
    zip VARCHAR(10),
    created_at TIMESTAMP,

    -- Index definitions
    INDEX idx_name (last_name, first_name),
    INDEX idx_location (state, city),
    INDEX idx_created (created_at)
);

-- Example 9: Table with partitions (MySQL)
CREATE TABLE orders (
    order_id INT,
    order_date DATE,
    customer_id INT,
    amount DECIMAL(10,2)
)
PARTITION BY RANGE (YEAR(order_date)) (
    PARTITION p2019 VALUES LESS THAN (2020),
    PARTITION p2020 VALUES LESS THAN (2021),
    PARTITION p2021 VALUES LESS THAN (2022),
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION p_future VALUES LESS THAN MAXVALUE
);

-- Example 10: Table with auto-increment starting value
CREATE TABLE sequences (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50)
) AUTO_INCREMENT = 1000;

-- Example 11: Table with comments
CREATE TABLE departments (
    dept_id INT PRIMARY KEY COMMENT 'Unique department identifier',
    dept_name VARCHAR(100) NOT NULL COMMENT 'Full department name',
    budget DECIMAL(15,2) COMMENT 'Annual budget in USD',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT 'Record creation timest
amp'
) COMMENT='Department information table';
```

```sql
-- Example 12: Create table with default character set
CREATE TABLE international_data (
    id INT PRIMARY KEY,
    content TEXT
) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Example 13: Table with generated columns
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    price DECIMAL(10,2),
    tax_rate DECIMAL(5,2) DEFAULT 10.00,
    price_with_tax DECIMAL(10,2) GENERATED ALWAYS AS (price * (1 + tax_rate/100))
STORED,
    -- Virtual generated column (not stored)
    price_category VARCHAR(20) GENERATED ALWAYS AS (
        CASE
            WHEN price < 50 THEN 'Budget'
            WHEN price < 200 THEN 'Standard'
            ELSE 'Premium'
        END
    ) VIRTUAL
);

-- Example 14: Table with foreign key cascade
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE CASCADE  -- Delete orders when customer deleted
        ON UPDATE CASCADE  -- Update orders when customer_id changes
);

-- Example 15: Table with check constraint across columns
CREATE TABLE events (
    event_id INT PRIMARY KEY,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    registration_deadline DATE,
    CONSTRAINT valid_dates CHECK (
```

```sql
        start_date <= end_date AND
        (registration_deadline IS NULL OR registration_deadline <= start_date)
    )
);

-- Example 16: Create table like another (MySQL)
CREATE TABLE employees_copy LIKE employees;

-- Example 17: Create table with storage engine (MySQL)
CREATE TABLE log_entries (
    log_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    message TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=ARCHIVE;  -- Archive engine for high compression

-- Example 18: PostgreSQL - table with inheritance
-- CREATE TABLE person (
--     person_id SERIAL PRIMARY KEY,
--     name VARCHAR(100)
-- );
--
-- CREATE TABLE employee (
--     salary DECIMAL(10,2),
--     hire_date DATE
-- ) INHERITS (person);

-- Example 19: Table with default values from functions
CREATE TABLE sessions (
    session_id CHAR(36) PRIMARY KEY DEFAULT (UUID()),  -- MySQL 8.0+
    user_id INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP DEFAULT (CURRENT_TIMESTAMP + INTERVAL 1 HOUR)
);

-- Example 20: Complete real-world example
CREATE TABLE IF NOT EXISTS blog_posts (
    post_id INT AUTO_INCREMENT PRIMARY KEY,
    author_id INT NOT NULL,
    title VARCHAR(255) NOT NULL,
    slug VARCHAR(255) UNIQUE NOT NULL,
    content TEXT,
    excerpt TEXT,
```

```sql
    status ENUM('draft', 'published', 'archived') DEFAULT 'draft',
    view_count INT DEFAULT 0,
    published_at DATETIME,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    -- Indexes
    INDEX idx_author (author_id),
    INDEX idx_status (status),
    INDEX idx_published (published_at),
    FULLTEXT INDEX idx_content (title, content),  -- For full-text search

    -- Foreign key
    CONSTRAINT fk_author
        FOREIGN KEY (author_id)
        REFERENCES users(user_id)
        ON DELETE RESTRICT,

    -- Check constraints
    CONSTRAINT valid_status_transition CHECK (
        (status = 'published' AND published_at IS NOT NULL) OR
        (status != 'published')
    ),
    CONSTRAINT valid_slug CHECK (slug REGEXP '^[a-z0-9-]+$')
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

3. ALTER TABLE

Modifies an existing table structure.

Theory:

- Can add, modify, or drop columns
- Can add or drop constraints
- Can rename table or columns
- Can change storage options
- May lock table during operation

Syntax:

```sql
-- Add column
ALTER TABLE table_name ADD column_name datatype constraints;

-- Modify column
ALTER TABLE table_name MODIFY column_name new_datatype constraints;

-- Drop column
ALTER TABLE table_name DROP COLUMN column_name;

-- Rename column
ALTER TABLE table_name RENAME COLUMN old_name TO new_name;

-- Add constraint
ALTER TABLE table_name ADD CONSTRAINT constraint_name constraint_type (column);

-- Drop constraint
ALTER TABLE table_name DROP CONSTRAINT constraint_name;

-- Rename table
ALTER TABLE table_name RENAME TO new_table_name;
```

Practical Examples:

```sql
-- Example 1: Add single column
ALTER TABLE employees
ADD COLUMN middle_name VARCHAR(50);

-- Example 2: Add column with default value
ALTER TABLE employees
ADD COLUMN nationality VARCHAR(50) DEFAULT 'USA';

-- Example 3: Add multiple columns
ALTER TABLE employees
ADD COLUMN birth_date DATE,
ADD COLUMN phone VARCHAR(20),
ADD COLUMN emergency_contact VARCHAR(100);
```

```sql
-- Example 4: Add column with position (MySQL)
ALTER TABLE employees
ADD COLUMN title VARCHAR(100) AFTER last_name;

ALTER TABLE employees
ADD COLUMN emp_code CHAR(10) FIRST;  -- Add as first column

-- Example 5: Modify column data type
ALTER TABLE employees
MODIFY COLUMN phone VARCHAR(30);  -- MySQL

-- ALTER TABLE employees
-- ALTER COLUMN phone TYPE VARCHAR(30);  -- PostgreSQL

-- Example 6: Modify column with constraints
ALTER TABLE employees
MODIFY COLUMN email VARCHAR(255) NOT NULL UNIQUE;

-- Example 7: Change column name and definition
-- MySQL
ALTER TABLE employees
CHANGE COLUMN phone mobile_phone VARCHAR(25);

-- PostgreSQL
-- ALTER TABLE employees
-- RENAME COLUMN phone TO mobile_phone;

-- Example 8: Drop column
ALTER TABLE employees
DROP COLUMN middle_name;

-- Example 9: Drop column with cascade (if referenced)
ALTER TABLE employees
DROP COLUMN department_id CASCADE;

-- Example 10: Add PRIMARY KEY
ALTER TABLE employees
ADD PRIMARY KEY (emp_id);

-- Example 11: Add FOREIGN KEY
ALTER TABLE employees
ADD CONSTRAINT fk_department
```

```sql
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
ON DELETE SET NULL;

-- Example 12: Add UNIQUE constraint
ALTER TABLE employees
ADD CONSTRAINT unique_email UNIQUE (email);

-- Example 13: Add CHECK constraint
ALTER TABLE employees
ADD CONSTRAINT check_age CHECK (age >= 18 AND age <= 65);

-- Example 14: Add DEFAULT value
ALTER TABLE employees
ALTER COLUMN is_active SET DEFAULT TRUE;  -- PostgreSQL

-- MySQL
ALTER TABLE employees
ALTER COLUMN is_active SET DEFAULT TRUE;

-- Example 15: Drop DEFAULT
ALTER TABLE employees
ALTER COLUMN is_active DROP DEFAULT;

-- Example 16: Drop constraint
ALTER TABLE employees
DROP CONSTRAINT unique_email;

ALTER TABLE employees
DROP FOREIGN KEY fk_department;  -- MySQL for foreign key

-- Example 17: Rename table
ALTER TABLE employees
RENAME TO staff_members;

-- Example 18: Enable/Disable constraints (useful for bulk load)
ALTER TABLE employees
DISABLE KEYS;  -- MySQL temporary disable non-unique indexes

ALTER TABLE employees
ENABLE KEYS;

-- PostgreSQL
```

```sql
-- ALTER TABLE employees
-- DISABLE TRIGGER ALL;

-- ALTER TABLE employees
-- ENABLE TRIGGER ALL;

-- Example 19: Set auto-increment value
ALTER TABLE employees
AUTO_INCREMENT = 1000;  -- MySQL

-- Example 20: Change table character set
ALTER TABLE employees
CONVERT TO CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Example 21: Add INDEX
ALTER TABLE employees
ADD INDEX idx_last_name (last_name);

ALTER TABLE employees
ADD INDEX idx_name (last_name, first_name);

-- Example 22: Drop INDEX
ALTER TABLE employees
DROP INDEX idx_last_name;

-- Example 23: Modify column NULL/NOT NULL
ALTER TABLE employees
MODIFY COLUMN email VARCHAR(255) NOT NULL;  -- MySQL

-- First ensure no NULLs
UPDATE employees SET email = '' WHERE email IS NULL;
ALTER TABLE employees ALTER COLUMN email SET NOT NULL;  -- PostgreSQL

-- Example 24: Add generated column
ALTER TABLE employees
ADD COLUMN full_name VARCHAR(101)
GENERATED ALWAYS AS (CONCAT(first_name, ' ', last_name)) STORED;

-- Example 25: Multiple alterations in one statement (MySQL)
ALTER TABLE employees
ADD COLUMN twitter_handle VARCHAR(50),
MODIFY COLUMN phone VARCHAR(30),
```

```sql
DROP COLUMN emergency_contact,
ADD INDEX idx_twitter (twitter_handle);

-- Example 26: Set column comment
ALTER TABLE employees
MODIFY COLUMN emp_id INT COMMENT 'Employee identifier';

-- Example 27: Change column order (MySQL)
ALTER TABLE employees
MODIFY COLUMN email VARCHAR(255) AFTER last_name;

-- Example 28: Add column with complex default
ALTER TABLE employees
ADD COLUMN uuid CHAR(36) DEFAULT (UUID());  -- MySQL 8.0+

-- Example 29: Rename index (MySQL)
ALTER TABLE employees
RENAME INDEX idx_last_name TO idx_surname;

-- Example 30: Table-level operations
-- Check table structure
DESCRIBE employees;
-- Or
SHOW CREATE TABLE employees;

-- Check existing indexes
SHOW INDEX FROM employees;
```

---

4. DROP TABLE

Removes a table from the database.

Theory:

- Permanently deletes table and data
- Cannot be rolled back (in most databases)
- Also drops indexes, triggers, constraints

- May need CASCADE to drop dependent objects

Syntax:

sql

```sql
-- Basic
DROP TABLE table_name;

-- Drop if exists
DROP TABLE IF EXISTS table_name;

-- With cascade
DROP TABLE table_name CASCADE CONSTRAINTS;  -- Oracle
DROP TABLE table_name CASCADE;  -- PostgreSQL
```

Practical Examples:

sql

```sql
-- Example 1: Basic drop
DROP TABLE temp_employees;

-- Example 2: Drop if exists (prevents error)
DROP TABLE IF EXISTS temp_employees;

-- Example 3: Drop multiple tables
DROP TABLE table1, table2, table3;

-- Example 4: Drop with CASCADE (PostgreSQL)
-- DROP TABLE departments CASCADE;
-- This drops departments and any objects depending on it (like foreign keys)

-- Example 5: Oracle - drop with purge (bypass recycle bin)
-- DROP TABLE employees PURGE;

-- Example 6: MySQL - check if table exists before drop
DROP TABLE IF EXISTS backup_log;

-- Example 7: Safe drop pattern
-- First, check dependencies
SELECT * FROM information_schema.KEY_COLUMN_USAGE
```

```sql
WHERE REFERENCED_TABLE_NAME = 'employees';

-- Then drop
DROP TABLE IF EXISTS employees;

-- Example 8: Drop temporary table
DROP TEMPORARY TABLE temp_data;

-- Example 9: PostgreSQL - drop with RESTRICT (default)
-- DROP TABLE employees RESTRICT;
-- Fails if there are dependent objects

-- Example 10: SQL Server - check existence
-- IF OBJECT_ID('dbo.employees', 'U') IS NOT NULL
-- DROP TABLE dbo.employees;

-- Example 11: Drop and recreate (for resetting)
DROP TABLE IF EXISTS log_table;
CREATE TABLE log_table (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    message TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Example 12: Drop with foreign key considerations
-- If table is referenced, you may need to:
-- Option 1: Drop child tables first
DROP TABLE child_table;
DROP TABLE parent_table;
-- Option 2: Remove foreign key constraints first
ALTER TABLE child_table DROP FOREIGN KEY fk_name;
DROP TABLE parent_table;
-- Example 13: Drop table in another database
DROP TABLE old_database.obsolete_table;
-- Example 14: Drop with error handling in script
DROP TABLE IF EXISTS staging_data;
-- Example 15: PostgreSQL - drop schema with all tables
-- DROP SCHEMA public CASCADE;
-- CREATE SCHEMA public;
```

5. TRUNCATE TABLE

Removes all rows from a table without logging individual row deletions.

Theory:

- Removes ALL rows (cannot use WHERE)
- Faster than DELETE without WHERE
- Resets AUTO_INCREMENT counter
- Cannot be rolled back (in most databases)
- Does not fire triggers
- DDL command (not DML)

Syntax:

```sql
TRUNCATE TABLE table_name;
```

Practical Examples:

```sql
-- Example 1: Basic truncate
TRUNCATE TABLE log_entries;

-- Example 2: TRUNCATE vs DELETE comparison
-- DELETE (DML) - slower, logs each row, can rollback, doesn't reset auto_incremen
t
DELETE FROM log_entries;
-- Can rollback: ROLLBACK;

-- TRUNCATE (DDL) - faster, minimal logging, can't rollback (usually), resets auto
_increment
TRUNCATE TABLE log_entries;
-- Can't rollback in most databases

-- Example 3: Check auto_increment before and after
INSERT INTO test_table (name) VALUES ('Test');
SELECT LAST_INSERT_ID();  -- 1
```

```sql
TRUNCATE TABLE test_table;

INSERT INTO test_table (name) VALUES ('Test');
SELECT LAST_INSERT_ID();  -- 1 again (reset)

-- DELETE doesn't reset:
INSERT INTO test_table (name) VALUES ('Test');  -- ID 1
DELETE FROM test_table;
INSERT INTO test_table (name) VALUES ('Test');  -- ID 2 (continues)

-- Example 4: TRUNCATE with foreign keys
-- If table has foreign key references, truncate may fail
-- Solution: Disable foreign key checks (MySQL)
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE parent_table;
SET FOREIGN_KEY_CHECKS = 1;

-- Example 5: PostgreSQL - TRUNCATE with CASCADE
-- TRUNCATE TABLE parent_table CASCADE;
-- Also truncates child tables

-- Example 6: TRUNCATE multiple tables
TRUNCATE TABLE table1, table2, table3;  -- MySQL

-- Example 7: Check if table exists before truncate
-- MySQL doesn't have IF EXISTS for TRUNCATE
-- Workaround:
DROP TABLE IF EXISTS temp_table;
CREATE TABLE temp_table LIKE original_table;
-- Or use stored procedure

-- Example 8: TRUNCATE with transaction (MySQL)
START TRANSACTION;
TRUNCATE TABLE log_table;  -- This commits implicitly in MySQL
ROLLBACK;  -- Won't work, TRUNCATE already committed

-- Example 9: TRUNCATE vs DROP + CREATE
-- TRUNCATE is usually faster
TRUNCATE TABLE big_table;

-- Equivalent but slower:
```

```sql
DROP TABLE big_table;
CREATE TABLE big_table (...);

-- Example 10: PostgreSQL - TRUNCATE with RESTART IDENTITY
-- TRUNCATE TABLE users RESTART IDENTITY;
-- Resets sequences and truncates

-- Example 11: Use case - reset staging table
-- ETL process: load, transform, then truncate
-- Load data
INSERT INTO staging_sales SELECT * FROM daily_sales;

-- Process data
UPDATE staging_sales SET processed = TRUE;

-- After processing, truncate for next day
TRUNCATE TABLE staging_sales;

-- Example 12: TRUNCATE with partitions (MySQL)
ALTER TABLE partitioned_table TRUNCATE PARTITION p2020;

-- Example 13: TRUNCATE temporary tables
CREATE TEMPORARY TABLE temp_data (id INT);
INSERT INTO temp_data VALUES (1), (2), (3);
TRUNCATE TABLE temp_data;  -- Works

-- Example 14: Performance test
-- Measure time for DELETE vs TRUNCATE
SET @start = NOW();
DELETE FROM big_table;
SELECT TIMEDIFF(NOW(), @start) AS delete_time;

SET @start = NOW();
TRUNCATE TABLE big_table;
SELECT TIMEDIFF(NOW(), @start) AS truncate_time;

-- Example 15: Safe truncate pattern
-- 1. Backup if needed
CREATE TABLE backup_table AS SELECT * FROM table_to_truncate;

-- 2. Check row count
SELECT COUNT(*) FROM table_to_truncate;
```

```sql
-- 3. Truncate
TRUNCATE TABLE table_to_truncate;

-- 4. Verify
SELECT COUNT(*) FROM table_to_truncate;  -- Should be 0
```

6. Index Creation

Indexes are used to speed up data retrieval.

Theory:

- Improves SELECT query performance
- Slows down INSERT, UPDATE, DELETE
- Creates additional storage overhead
- Types:
  - **Primary Key Index**: Automatically created
  - **Unique Index**: Ensures uniqueness
  - **Regular Index**: Speeds up searches
  - **Composite Index**: Multiple columns
  - **Full-text Index**: For text search
  - **Spatial Index**: For geographic data
  - **Partial Index**: Only on subset of rows (PostgreSQL)

Syntax:

```sql
-- Create index
CREATE INDEX index_name ON table_name (column1, column2);

-- Unique index
CREATE UNIQUE INDEX index_name ON table_name (column);

-- Full-text index (MySQL)
CREATE FULLTEXT INDEX index_name ON table_name (column);
```

```sql
-- Drop index
DROP INDEX index_name ON table_name;  -- MySQL
-- DROP INDEX index_name;  -- PostgreSQL
```

Practical Examples:

sql

```sql
-- Example 1: Simple index on one column
CREATE INDEX idx_last_name ON employees (last_name);

-- Example 2: Composite index (multiple columns)
-- Order matters! For queries with both columns or just first column
CREATE INDEX idx_name ON employees (last_name, first_name);

-- Good for: WHERE last_name = 'Smith'
-- Good for: WHERE last_name = 'Smith' AND first_name = 'John'
-- Not good for: WHERE first_name = 'John' (can't use first column efficiently)

-- Example 3: Unique index (like UNIQUE constraint)
CREATE UNIQUE INDEX idx_email ON employees (email);

-- Example 4: Index with descending order
CREATE INDEX idx_hire_date ON employees (hire_date DESC);

-- Example 5: Composite index with mixed order
CREATE INDEX idx_salary_hire ON employees (salary DESC, hire_date ASC);

-- Example 6: Full-text index for text search (MySQL)
CREATE FULLTEXT INDEX idx_content ON articles (title, body);

-- Use with MATCH AGAINST
SELECT * FROM articles
WHERE MATCH(title, body) AGAINST('database optimization' IN NATURAL LANGUAGE MODE)
;

-- Example 7: Partial index (PostgreSQL)
-- CREATE INDEX idx_active_users ON users (email) WHERE is_active = TRUE;
-- Only indexes active users, smaller and faster

-- Example 8: Expression index (PostgreSQL)
```

```sql
-- CREATE INDEX idx_lower_email ON users (LOWER(email));
-- Allows fast case-insensitive search

-- Example 9: MySQL - prefix index (index only first N characters)
CREATE INDEX idx_phone ON customers (phone(10));  -- Only index first 10 chars

-- Example 10: Spatial index (MySQL)
CREATE SPATIAL INDEX idx_location ON stores (location);

-- Example 11: Invisible index (MySQL 8.0+) - for testing
CREATE INDEX idx_test ON employees (salary) INVISIBLE;
-- Can make visible when ready
ALTER TABLE employees ALTER INDEX idx_test VISIBLE;

-- Example 12: Composite index covering query (covering index)
-- If all needed columns are in index, no need to access table
CREATE INDEX idx_covering ON employees (last_name, first_name, email);

-- Query that can be satisfied by index only:
SELECT last_name, first_name, email FROM employees WHERE last_name = 'Smith';

-- Example 13: Index on JSON column (MySQL 8.0+)
CREATE INDEX idx_category ON products ((CAST(attributes->>'$.category' AS CHAR(50)
)));

-- Example 14: Drop index
DROP INDEX idx_last_name ON employees;  -- MySQL
-- DROP INDEX idx_last_name;  -- PostgreSQL

-- Example 15: Check existing indexes
SHOW INDEX FROM employees;  -- MySQL
-- \di employees* in PostgreSQL

-- Example 16: Index with table creation
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    status VARCHAR(20),
    INDEX idx_customer (customer_id),
    INDEX idx_date (order_date DESC),
    INDEX idx_status (status)
```

```sql
);

-- Example 17: Bitmap index (Oracle) - for low cardinality columns
-- CREATE BITMAP INDEX idx_gender ON employees (gender);
-- Example 18: Hash index (MySQL MEMORY engine)
CREATE INDEX idx_hash ON memory_table (customer_id) USING HASH;
-- Example 19: Index on UDF (User Defined Function)
-- MySQL doesn't support directly, but can use generated column
ALTER TABLE employees
ADD COLUMN hire_year INT GENERATED ALWAYS AS (YEAR(hire_date)) STORED,
ADD INDEX idx_hire_year (hire_year);
-- Example 20: When to use indexes - scenarios
-- Good for indexing:
CREATE INDEX idx_customer_id ON orders (customer_id);  -- Foreign key
CREATE INDEX idx_order_date ON orders (order_date);    -- Range queries
CREATE INDEX idx_status ON orders (status);            -- Filtering

-- Not good for indexing:
-- Low cardinality (e.g., gender column with only M/F)
-- Very small tables
-- Columns rarely used in WHERE
-- Example 21: Composite index design
-- Query: WHERE department = 'IT' AND salary > 80000
-- Good index: (department, salary) - can filter department then salary
CREATE INDEX idx_dept_salary ON employees (department, salary);
-- Query: WHERE salary > 80000 AND department = 'IT'
-- Same index still works (optimizer reorders conditions)
-- Example 22: Index maintenance
-- Analyze index usage (MySQL)
SELECT * FROM sys.schema_index_statistics WHERE table_name = 'employees';
-- Rebuild index (MySQL)
ALTER TABLE employees ENGINE=InnoDB;  -- Rebuilds all indexes
-- Or optimize table
OPTIMIZE TABLE employees;

-- Example 23: Functional index (PostgreSQL)
-- CREATE INDEX idx_upper_name ON employees (UPPER(last_name));
-- Query: SELECT * FROM employees WHERE UPPER(last_name) = 'SMITH';
-- Example 24: Conditional unique index (PostgreSQL)
-- CREATE UNIQUE INDEX idx_unique_active_email ON users (email) WHERE is_active =
TRUE;
-- Only active users must have unique email
```

```sql
-- Example 25: Index for ORDER BY
-- For query: ORDER BY last_name, first_name
CREATE INDEX idx_name_sort ON employees (last_name, first_name);
-- Example 26: Index for GROUP BY
-- For query: GROUP BY department
CREATE INDEX idx_dept ON employees (department);
-- Example 27: Multiple column index vs multiple single indexes
-- Better: One composite index
CREATE INDEX idx_customer_date ON orders (customer_id, order_date);
-- Instead of:
-- CREATE INDEX idx_customer ON orders (customer_id);
-- CREATE INDEX idx_date ON orders (order_date);

-- Example 28: Check if index is being used (MySQL)
EXPLAIN SELECT * FROM employees WHERE last_name = 'Smith';
-- Example 29: Rename index (MySQL 8.0+)
ALTER TABLE employees RENAME INDEX idx_old TO idx_new;
-- Example 30: Real-world indexing strategy
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATETIME NOT NULL,
    status ENUM('pending', 'processing', 'shipped', 'delivered') NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    shipping_zip VARCHAR(10),
    -- Indexes for common query patterns
    INDEX idx_customer_orders (customer_id, order_date DESC),  -- Customer order h
istory
    INDEX idx_status_date (status, order_date),                 -- Status-based qu
eries
    INDEX idx_date_amount (order_date, total_amount),           -- Reporting queri
es
    INDEX idx_zip (shipping_zip)                                 -- Location-based
queries
);

-- Query patterns:
-- 1. Customer order history: WHERE customer_id = 123 ORDER BY order_date DESC
-- 2. Pending orders today: WHERE status = 'pending' AND order_date >= CURDATE()
-- 3. Daily sales report: GROUP BY order_date, SUM(total_amount)
-- 4. Orders by zip code: WHERE shipping_zip = '90210'
```

Summary: DML vs DDL Comparison

Aspect	DML (Data Manipulation)	DDL (Data Definition)
Commands	INSERT, UPDATE, DELETE, MERGE	CREATE, ALTER, DROP, TRUNCATE
Purpose	Manage data	Manage structure
Transaction	Can be rolled back	Auto-commit (mostly)
Undo	Possible with ROLLBACK	Difficult/impossible
Triggers	Fire triggers	Don't fire triggers
Performance	Variable, can be slow	Usually fast
Locking	Row/table locks	Table/schema locks
Logging	Extensive logging	Minimal logging
Examples	Add/edit/remove records	Create/modify tables

Aggregate Functions and SQL Joins: Complete Theory Notes with Practical Examples

Part 1: Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single result. They are essential for data analysis and reporting.

Introduction to Aggregate Functions

Theory:

- Operate on sets of rows
- Return a single value per group
- Ignore NULL values (except COUNT(*))
- Often used with GROUP BY
- Cannot be used in WHERE clause (use HAVING instead)

Common Aggregate Functions:

Function	Description	NULL Handling
COUNT()	Counts rows	COUNT(*) includes NULLs, COUNT(column) ignores NU
SUM()	Total of numeric column	Ignores NULLs
AVG()	Average of numeric column	Ignores NULLs
MIN()	Minimum value	Ignores NULLs

Function	Description	NULL Handling
MAX()	Maximum value	Ignores NULLs

Sample Data for All Examples

sql

```sql
-- Create sample database for examples
CREATE DATABASE IF NOT EXISTS company_db;
USE company_db;

-- Create departments table
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50),
    location VARCHAR(50)
);

-- Create employees table
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    dept_id INT,
    salary DECIMAL(10,2),
    commission DECIMAL(10,2),
    hire_date DATE,
    job_title VARCHAR(50),
    manager_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

-- Insert sample data
INSERT INTO departments VALUES
(1, 'IT', 'New York'),
```

```sql
(2, 'HR', 'London'),
(3, 'Finance', 'New York'),
(4, 'Marketing', 'Chicago'),
(5, 'Sales', 'Chicago');

INSERT INTO employees VALUES
(101, 'John', 'Doe', 'john.doe@company.com', 1, 75000.00, 5000.00, '2020-01-15', '
Developer', NULL),
(102, 'Jane', 'Smith', 'jane.smith@company.com', 2, 65000.00, 3000.00, '2019-03-20
', 'HR Manager', NULL),
(103, 'Bob', 'Johnson', 'bob.johnson@company.com', 1, 82000.00, 7000.00, '2021-06-
10', 'Senior Developer', 101),
(104, 'Alice', 'Brown', 'alice.brown@company.com', 3, 70000.00, 4000.00, '2020-11-
01', 'Accountant', NULL),
(105, 'Charlie', 'Wilson', 'charlie.wilson@company.com', 1, 90000.00, 8000.00, '20
18-09-05', 'Team Lead', NULL),
(106, 'Diana', 'Miller', 'diana.miller@company.com', 2, 58000.00, 2000.00, '2022-0
1-12', 'Recruiter', 102),
(107, 'Eve', 'Davis', NULL, 3, 72000.00, NULL, '2019-07-19', 'Financial Analyst',
104),
(108, 'Frank', 'Thomas', 'frank.thomas@company.com', 4, 63000.00, 3500.00, '2021-0
3-08', 'Marketing Specialist', NULL),
(109, 'Grace', 'Lee', 'grace.lee@company.com', 5, 68000.00, 6000.00, '2020-08-22',
'Sales Rep', NULL),
(110, 'Henry', 'Ford', 'henry.ford@company.com', 5, 71000.00, 5500.00, '2019-11-14
', 'Sales Rep', 109),
(111, 'Ivy', 'Chen', 'ivy.chen@company.com', 4, 59000.00, 2800.00, '2022-05-30', '
Marketing Assistant', 108),
(112, 'Jack', 'Williams', 'jack.williams@company.com', 1, 95000.00, 9000.00, '2017
-12-03', 'Architect', 105);
```

---

1. COUNT() Function

Counts the number of rows in a result set.

Theory:

- **COUNT(*)**: Counts all rows, including NULLs and duplicates
- **COUNT(column)**: Counts non-NULL values in specific column
- **COUNT(DISTINCT column)**: Counts unique non-NULL values
- Returns BIGINT type

Practical Examples:

sql

```sql
-- Example 1: COUNT(*) - count all rows
SELECT COUNT(*) AS total_employees
FROM employees;
-- Result: 12

-- Example 2: COUNT(column) - count non-NULL values
SELECT
    COUNT(*) AS total_employees,
    COUNT(email) AS employees_with_email,  -- Excludes NULL emails
    COUNT(commission) AS employees_with_commission  -- Excludes NULL commissions
FROM employees;
-- Result: total:12, with_email:11, with_commission:10

-- Example 3: COUNT(DISTINCT) - count unique values
SELECT
    COUNT(DISTINCT dept_id) AS unique_departments,
    COUNT(DISTINCT job_title) AS unique_job_titles
FROM employees;
-- Result: unique_depts:4, unique_titles:? (depends on data)

-- Example 4: COUNT with WHERE clause
SELECT COUNT(*) AS it_employees
FROM employees
WHERE dept_id = 1;
-- Result: 4 (John, Bob, Charlie, Jack)

-- Example 5: COUNT with multiple conditions
SELECT COUNT(*) AS high_salary_employees
FROM employees
WHERE salary > 80000 AND dept_id = 1;
```

```sql
-- Result: 3 (Bob, Charlie, Jack)

-- Example 6: COUNT with GROUP BY
SELECT
    dept_id,
    COUNT(*) AS employee_count
FROM employees
GROUP BY dept_id
ORDER BY dept_id;
-- Result: dept1:4, dept2:2, dept3:2, dept4:2, dept5:2

-- Example 7: COUNT with JOIN
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS employee_count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
-- Result: Shows all departments, even those with 0 employees

-- Example 8: Conditional COUNT using CASE
SELECT
    COUNT(*) AS total,
    COUNT(CASE WHEN salary > 70000 THEN 1 END) AS high_salary,
    COUNT(CASE WHEN salary <= 70000 THEN 1 END) AS low_salary
FROM employees;

-- Example 9: COUNT with HAVING
SELECT
    dept_id,
    COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 2;
-- Result: Only dept_id 1 (has 4 employees)

-- Example 10: COUNT percentage
SELECT
    dept_id,
    COUNT(*) AS dept_count,
    COUNT(*) * 100.0 / (SELECT COUNT(*) FROM employees) AS percentage
FROM employees
```

```sql
GROUP BY dept_id;
```

---

2. SUM() Function

Calculates the total sum of a numeric column.

Theory:

- Works only with numeric data types
- Ignores NULL values
- Returns same data type as column (or larger for safety)
- SUM(column * 1.0) for decimal precision

Practical Examples:

sql

```sql
-- Example 1: Basic SUM
SELECT SUM(salary) AS total_salary
FROM employees;
-- Result: Sum of all salaries

-- Example 2: SUM with WHERE
SELECT SUM(salary) AS it_total_salary
FROM employees
WHERE dept_id = 1;

-- Example 3: Multiple SUMs
SELECT
    SUM(salary) AS total_salary,
    SUM(commission) AS total_commission,
    SUM(salary) + SUM(commission) AS total_compensation
FROM employees;
-- Note: This adds sums, not row by row

-- Example 4: SUM with NULL handling
SELECT
```

```sql
    SUM(commission) AS total_commission,  -- Ignores NULLs
    SUM(COALESCE(commission, 0)) AS total_commission_with_default
FROM employees;

-- Example 5: SUM with GROUP BY
SELECT
    d.dept_name,
    SUM(e.salary) AS total_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY total_salary DESC;

-- Example 6: SUM with DISTINCT
SELECT
    SUM(DISTINCT salary) AS sum_unique_salaries,  -- Each salary value counted onc
e
    SUM(salary) AS sum_all_salaries               -- All salaries added
FROM employees;

-- Example 7: Conditional SUM
SELECT
    SUM(CASE WHEN dept_id = 1 THEN salary ELSE 0 END) AS it_salary,
    SUM(CASE WHEN dept_id = 2 THEN salary ELSE 0 END) AS hr_salary,
    SUM(salary) AS total_salary
FROM employees;

-- Example 8: Running total (with window function)
SELECT
    emp_id,
    salary,
    SUM(salary) OVER (ORDER BY hire_date) AS running_total
FROM employees;

-- Example 9: SUM with date condition
SELECT
    SUM(CASE
        WHEN YEAR(hire_date) = 2020 THEN salary
        ELSE 0
    END) AS salary_2020_hires
FROM employees;
```

```sql
-- Example 10: SUM with mathematical operations
SELECT
    SUM(salary * 1.1) AS salary_with_raise,  -- Sum after 10% increase
    SUM(salary) / COUNT(*) AS avg_salary
FROM employees;

-- Example 11: SUM over expression
SELECT
    SUM(salary + COALESCE(commission, 0)) AS total_compensation
FROM employees;

-- Example 12: SUM with percentage of total
SELECT
    dept_id,
    SUM(salary) AS dept_salary,
    SUM(salary) / (SELECT SUM(salary) FROM employees) * 100 AS percentage
FROM employees
GROUP BY dept_id;

-- Example 13: SUM with multiple groupings
SELECT
    dept_id,
    job_title,
    SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id, job_title
ORDER BY dept_id, job_title;

-- Example 14: SUM with HAVING
SELECT
    dept_id,
    SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id
HAVING SUM(salary) > 200000;

-- Example 15: SUM for inventory/value calculation
SELECT
    SUM(quantity * unit_price) AS total_inventory_value
FROM products;
```

---

3. AVG() Function

Calculates the average (mean) of a numeric column.

Theory:

- SUM(column) / COUNT(column) (ignores NULLs)
- Returns decimal value even for integer columns
- Ignores NULL values
- AVG(DISTINCT) averages unique values

Practical Examples:

sql

```sql
-- Example 1: Basic AVG
SELECT AVG(salary) AS average_salary
FROM employees;

-- Example 2: AVG with WHERE
SELECT AVG(salary) AS it_avg_salary
FROM employees
WHERE dept_id = 1;

-- Example 3: Multiple AVGs
SELECT
    AVG(salary) AS avg_salary,
    AVG(commission) AS avg_commission,  -- Only non-NULL commissions
    AVG(COALESCE(commission, 0)) AS avg_commission_including_zero
FROM employees;

-- Example 4: AVG with DISTINCT
SELECT
    AVG(salary) AS regular_avg,
    AVG(DISTINCT salary) AS avg_unique_salaries
FROM employees;
```

```sql
-- Example 5: AVG with GROUP BY
SELECT
    d.dept_name,
    AVG(e.salary) AS avg_salary,
    COUNT(*) AS emp_count
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY avg_salary DESC;

-- Example 6: AVG with NULL handling
SELECT
    dept_id,
    AVG(commission) AS avg_commission,        -- Only those with commission
    AVG(COALESCE(commission, 0)) AS avg_commission_all
FROM employees
GROUP BY dept_id;

-- Example 7: Conditional AVG
SELECT
    AVG(CASE WHEN dept_id = 1 THEN salary END) AS it_avg,
    AVG(CASE WHEN dept_id = 2 THEN salary END) AS hr_avg,
    AVG(salary) AS overall_avg
FROM employees;

-- Example 8: AVG with date functions
SELECT
    AVG(DATEDIFF(CURDATE(), hire_date)) AS avg_days_employed
FROM employees;

-- Example 9: AVG with HAVING
SELECT
    dept_id,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 70000;

-- Example 10: Weighted average
SELECT
    SUM(salary * years_of_experience) / SUM(years_of_experience) AS weighted_avg_s
alary
```

```sql
FROM employees;

-- Example 11: Moving average (with window functions)
SELECT
    emp_id,
    salary,
    hire_date,
    AVG(salary) OVER (ORDER BY hire_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
AS moving_avg_3
FROM employees;

-- Example 12: AVG with decimal precision
SELECT
    ROUND(AVG(salary), 2) AS avg_salary_2dec,
    ROUND(AVG(salary), 0) AS avg_salary_whole
FROM employees;

-- Example 13: Compare AVG with median (approximation)
SELECT
    AVG(salary) AS mean_salary,
    (
        SELECT salary
        FROM employees e2
        WHERE (SELECT COUNT(*) FROM employees WHERE salary <= e2.salary)
            >= (SELECT COUNT(*) FROM employees) / 2
        ORDER BY salary
        LIMIT 1
    ) AS approximate_median
FROM employees;

-- Example 14: AVG with zero handling
SELECT
    AVG(NULLIF(salary, 0)) AS avg_non_zero,  -- Exclude zeros
    AVG(salary) AS avg_including_zero
FROM employees;

-- Example 15: Department comparison
SELECT
    d.dept_name,
    AVG(e.salary) AS dept_avg,
    (SELECT AVG(salary) FROM employees) AS company_avg,
    AVG(e.salary) - (SELECT AVG(salary) FROM employees) AS diff_from_company
```

```sql
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

---

4. MIN() Function

Finds the minimum value in a column.

Theory:

- Works with numeric, string, and date types
- For strings: alphabetical minimum
- For dates: earliest date
- Ignores NULL values
- MIN(DISTINCT) finds minimum of unique values

Practical Examples:

sql

```sql
-- Example 1: MIN with numbers
SELECT MIN(salary) AS lowest_salary
FROM employees;

-- Example 2: MIN with dates
SELECT
    MIN(hire_date) AS first_hire_date,
    MIN(CURDATE() - hire_date) AS shortest_employment_days?
    -- Better:
    DATEDIFF(CURDATE(), MIN(hire_date)) AS days_since_first_hire
FROM employees;

-- Example 3: MIN with strings
SELECT MIN(last_name) AS first_alphabetically
FROM employees;
```

```sql
-- Example 4: MIN with WHERE
SELECT
    MIN(salary) AS lowest_salary_it
FROM employees
WHERE dept_id = 1;

-- Example 5: Multiple MINs
SELECT
    MIN(salary) AS min_salary,
    MIN(hire_date) AS earliest_hire,
    MIN(last_name) AS first_name_alpha
FROM employees;

-- Example 6: MIN with GROUP BY
SELECT
    d.dept_name,
    MIN(e.salary) AS lowest_salary,
    MIN(e.hire_date) AS first_hire
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

-- Example 7: MIN with DISTINCT
SELECT
    MIN(DISTINCT salary) AS min_unique_salary,  -- Same as MIN usually
    MIN(salary) AS min_all_salary
FROM employees;
-- Usually same unless all salaries are NULL

-- Example 8: Find employee with minimum salary
SELECT
    emp_id,
    first_name,
    last_name,
    salary
FROM employees
WHERE salary = (SELECT MIN(salary) FROM employees);

-- Example 9: MIN with NULL handling
SELECT
    MIN(commission) AS min_commission,  -- Only non-NULL
    MIN(COALESCE(commission, 0)) AS min_commission_including_zeros
```

```sql
FROM employees;

-- Example 10: MIN with date parts
SELECT
    MIN(YEAR(hire_date)) AS earliest_hire_year,
    MIN(MONTH(hire_date)) AS earliest_hire_month
FROM employees;

-- Example 11: MIN with CASE
SELECT
    MIN(CASE WHEN dept_id = 1 THEN salary END) AS min_it_salary,
    MIN(CASE WHEN dept_id = 2 THEN salary END) AS min_hr_salary
FROM employees;

-- Example 12: Running minimum (window function)
SELECT
    emp_id,
    salary,
    hire_date,
    MIN(salary) OVER (ORDER BY hire_date) AS min_salary_so_far
FROM employees;

-- Example 13: MIN with string length
SELECT
    MIN(LENGTH(first_name)) AS shortest_name_length,
    MIN(LENGTH(last_name)) AS shortest_lastname_length
FROM employees;

-- Example 14: Department with lowest average salary
SELECT
    d.dept_name,
    AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY avg_salary
LIMIT 1;

-- Example 15: MIN across related tables
SELECT
    d.dept_name,
    MIN(e.salary) AS min_salary,
```

```sql
    (SELECT MIN(salary) FROM employees) AS company_min,
    MIN(e.salary) - (SELECT MIN(salary) FROM employees) AS diff_from_min
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

---

5. MAX() Function

Finds the maximum value in a column.

Theory:

- Works with numeric, string, and date types
- For strings: alphabetical maximum
- For dates: latest date
- Ignores NULL values
- MAX(DISTINCT) finds maximum of unique values

Practical Examples:

sql

```sql
-- Example 1: MAX with numbers
SELECT MAX(salary) AS highest_salary
FROM employees;

-- Example 2: MAX with dates
SELECT
    MAX(hire_date) AS most_recent_hire,
    DATEDIFF(CURDATE(), MAX(hire_date)) AS days_since_last_hire
FROM employees;

-- Example 3: MAX with strings
SELECT MAX(last_name) AS last_alphabetically
FROM employees;
-- Returns the string that would come last in alphabetical order
```

```sql
-- Example 4: MAX with WHERE
SELECT
    MAX(salary) AS highest_salary_it
FROM employees
WHERE dept_id = 1;

-- Example 5: Multiple MAXs
SELECT
    MAX(salary) AS max_salary,
    MAX(hire_date) AS latest_hire,
    MAX(last_name) AS last_name_alpha
FROM employees;

-- Example 6: MAX with GROUP BY
SELECT
    d.dept_name,
    MAX(e.salary) AS highest_salary,
    MAX(e.hire_date) AS latest_hire
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

-- Example 7: Find employee with maximum salary
SELECT
    emp_id,
    first_name,
    last_name,
    salary
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);

-- Example 8: Second highest salary
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Example 9: MAX with NULL handling
SELECT
    MAX(commission) AS max_commission,
    MAX(COALESCE(commission, 0)) AS max_commission_including_zeros
FROM employees;
```

```sql
-- Example 10: MAX with date parts
SELECT
    MAX(YEAR(hire_date)) AS latest_hire_year,
    MAX(MONTH(hire_date)) AS latest_hire_month
FROM employees;

-- Example 11: MAX with CASE
SELECT
    MAX(CASE WHEN dept_id = 1 THEN salary END) AS max_it_salary,
    MAX(CASE WHEN dept_id = 2 THEN salary END) AS max_hr_salary
FROM employees;

-- Example 12: Running maximum (window function)
SELECT
    emp_id,
    salary,
    hire_date,
    MAX(salary) OVER (ORDER BY hire_date) AS max_salary_so_far
FROM employees;

-- Example 13: MAX with string length
SELECT
    MAX(LENGTH(first_name)) AS longest_name_length,
    MAX(LENGTH(last_name)) AS longest_lastname_length
FROM employees;

-- Example 14: Department with highest average salary
SELECT
    d.dept_name,
    AVG(e.salary) AS avg_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY avg_salary DESC
LIMIT 1;

-- Example 15: Salary range by department
SELECT
    d.dept_name,
    MIN(e.salary) AS min_salary,
    MAX(e.salary) AS max_salary,
```

```sql
    MAX(e.salary) - MIN(e.salary) AS salary_range
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

---

6. GROUP BY Clause

Groups rows that have the same values in specified columns.

Theory:

- Divides rows into groups
- Each group returns one row
- Used with aggregate functions
- Columns in SELECT must be in GROUP BY or aggregate functions
- NULLs are grouped together

Syntax:

sql

```sql
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1
ORDER BY column1;
```

Practical Examples:

sql

```sql
-- Example 1: Basic GROUP BY
SELECT
    dept_id,
    COUNT(*) AS employee_count
FROM employees
GROUP BY dept_id
```

```sql
ORDER BY dept_id;

-- Example 2: GROUP BY with multiple columns
SELECT
    dept_id,
    job_title,
    COUNT(*) AS count,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id, job_title
ORDER BY dept_id, job_title;

-- Example 3: GROUP BY with JOIN
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS employee_count,
    AVG(e.salary) AS avg_salary,
    MAX(e.salary) AS max_salary
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY employee_count DESC;

-- Example 4: GROUP BY with WHERE (filter before grouping)
SELECT
    dept_id,
    AVG(salary) AS avg_salary
FROM employees
WHERE hire_date >= '2020-01-01'  -- Only recent hires
GROUP BY dept_id;

-- Example 5: GROUP BY with expressions
SELECT
    YEAR(hire_date) AS hire_year,
    COUNT(*) AS hires,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY YEAR(hire_date)
ORDER BY hire_year;

-- Example 6: GROUP BY with CASE
SELECT
```

```sql
    CASE
        WHEN salary < 60000 THEN 'Low'
        WHEN salary BETWEEN 60000 AND 80000 THEN 'Medium'
        ELSE 'High'
    END AS salary_category,
    COUNT(*) AS employee_count,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY salary_category
ORDER BY salary_category;

-- Example 7: GROUP BY with ROLLUP (MySQL, PostgreSQL)
SELECT
    dept_id,
    job_title,
    COUNT(*) AS count,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id, job_title WITH ROLLUP;
-- Adds subtotal and total rows

-- Example 8: GROUP BY with CUBE (more comprehensive)
-- SELECT
--     dept_id,
--     job_title,
--     COUNT(*) AS count
-- FROM employees
-- GROUP BY CUBE(dept_id, job_title);

-- Example 9: GROUP BY with multiple aggregates
SELECT
    dept_id,
    COUNT(*) AS emp_count,
    SUM(salary) AS total_salary,
    AVG(salary) AS avg_salary,
    MIN(salary) AS min_salary,
    MAX(salary) AS max_salary,
    MAX(salary) - MIN(salary) AS salary_range
FROM employees
GROUP BY dept_id;

-- Example 10: GROUP BY with string aggregation
```

```sql
-- MySQL: GROUP_CONCAT
SELECT
    dept_id,
    COUNT(*) AS emp_count,
    GROUP_CONCAT(first_name) AS employee_names
FROM employees
GROUP BY dept_id;

-- PostgreSQL: STRING_AGG
-- SELECT
--     dept_id,
--     COUNT(*),
--     STRING_AGG(first_name, ', ') AS employee_names
-- FROM employees
-- GROUP BY dept_id;

-- Example 11: GROUP BY with filtered aggregates
SELECT
    dept_id,
    COUNT(*) AS total,
    SUM(CASE WHEN salary > 70000 THEN 1 ELSE 0 END) AS high_earners,
    SUM(CASE WHEN salary <= 70000 THEN 1 ELSE 0 END) AS low_earners
FROM employees
GROUP BY dept_id;

-- Example 12: GROUP BY with percentage calculations
SELECT
    dept_id,
    COUNT(*) AS count,
    COUNT(*) * 100.0 / (SELECT COUNT(*) FROM employees) AS percentage
FROM employees
GROUP BY dept_id;

-- Example 13: GROUP BY with date truncation
SELECT
    DATE_FORMAT(hire_date, '%Y-%m') AS hire_month,  -- MySQL
    COUNT(*) AS hires
FROM employees
GROUP BY DATE_FORMAT(hire_date, '%Y-%m')
ORDER BY hire_month;

-- PostgreSQL:
```

```sql
-- SELECT
--     DATE_TRUNC('month', hire_date) AS hire_month,
--     COUNT(*) AS hires
-- FROM employees
-- GROUP BY DATE_TRUNC('month', hire_date)
-- ORDER BY hire_month;

-- Example 14: GROUP BY with multiple levels
SELECT
    d.location,
    d.dept_name,
    COUNT(e.emp_id) AS emp_count,
    AVG(e.salary) AS avg_salary
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.location, d.dept_name
ORDER BY d.location, d.dept_name;

-- Example 15: GROUP BY with HAVING (see next section)
SELECT
    dept_id,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 70000;

-- Example 16: GROUP BY with DISTINCT vs GROUP BY
-- These are different:
SELECT DISTINCT dept_id FROM employees;  -- Just unique values
SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id;  -- With aggregation

-- Example 17: GROUP BY with NULL handling
INSERT INTO employees (emp_id, first_name, dept_id) VALUES (113, 'Test', NULL);
SELECT
    dept_id,
    COUNT(*) AS count
FROM employees
GROUP BY dept_id;  -- NULL forms its own group

-- Example 18: GROUP BY with ORDER BY aggregate
SELECT
    dept_id,
```

```sql
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
ORDER BY avg_salary DESC;

-- Example 19: GROUP BY with LIMIT
SELECT
    dept_id,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
ORDER BY avg_salary DESC
LIMIT 3;  -- Top 3 departments by average salary

-- Example 20: Complex GROUP BY with multiple tables
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS employee_count,
    COALESCE(SUM(e.salary), 0) AS total_salary,
    COALESCE(AVG(e.salary), 0) AS avg_salary,
    COUNT(DISTINCT e.job_title) AS unique_jobs
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(e.emp_id) > 0 OR total_salary > 0;
```

---

7. HAVING Clause

Filters groups after GROUP BY (similar to WHERE for rows).

Theory:

- Used with GROUP BY
- Filters groups based on aggregate results
- WHERE filters rows before grouping
- HAVING filters groups after grouping

- Can use aggregate functions in condition

Syntax:

sql

```sql
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1
HAVING aggregate_condition
ORDER BY column1;
```

Practical Examples:

sql

```sql
-- Example 1: Basic HAVING
-- Find departments with more than 2 employees
SELECT
    dept_id,
    COUNT(*) AS employee_count
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 2;

-- Example 2: HAVING with average condition
-- Find departments with average salary > 70000
SELECT
    dept_id,
    AVG(salary) AS avg_salary,
    COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 70000;

-- Example 3: HAVING with multiple conditions
SELECT
    dept_id,
    COUNT(*) AS emp_count,
    AVG(salary) AS avg_salary,
    SUM(salary) AS total_salary
```

```sql
FROM employees
GROUP BY dept_id
HAVING COUNT(*) >= 2 AND AVG(salary) > 65000;

-- Example 4: HAVING vs WHERE comparison
-- WHERE filters rows before grouping
-- HAVING filters groups after grouping

-- Find departments with average salary > 70000,
-- but only consider employees hired after 2020
SELECT
    dept_id,
    AVG(salary) AS avg_salary,
    COUNT(*) AS emp_count
FROM employees
WHERE hire_date >= '2020-01-01'  -- Filter rows first
GROUP BY dept_id
HAVING AVG(salary) > 70000;      -- Then filter groups

-- Example 5: HAVING with MIN/MAX
-- Find departments where max salary is more than double min salary
SELECT
    dept_id,
    MIN(salary) AS min_salary,
    MAX(salary) AS max_salary,
    MAX(salary) / MIN(salary) AS ratio
FROM employees
GROUP BY dept_id
HAVING MAX(salary) > 2 * MIN(salary);

-- Example 6: HAVING with SUM
-- Find departments with total salary > 150000
SELECT
    dept_id,
    SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id
HAVING SUM(salary) > 150000;

-- Example 7: HAVING with COUNT and JOIN
-- Find departments with at least 2 employees and show their average salary
SELECT
```

```sql
    d.dept_name,
    COUNT(e.emp_id) AS emp_count,
    AVG(e.salary) AS avg_salary
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(e.emp_id) >= 2;

-- Example 8: HAVING with complex condition
-- Find job titles that appear in multiple departments
SELECT
    job_title,
    COUNT(DISTINCT dept_id) AS dept_count,
    COUNT(*) AS total_count,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY job_title
HAVING COUNT(DISTINCT dept_id) > 1;

-- Example 9: HAVING with date condition on aggregate
-- Find years where we hired more than 2 people
SELECT
    YEAR(hire_date) AS hire_year,
    COUNT(*) AS hire_count,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY YEAR(hire_date)
HAVING COUNT(*) > 2;

-- Example 10: HAVING with percentage of total
-- Find departments that have more than 20% of employees
SELECT
    dept_id,
    COUNT(*) AS count,
    COUNT(*) * 100.0 / (SELECT COUNT(*) FROM employees) AS percentage
FROM employees
GROUP BY dept_id
HAVING COUNT(*) * 100.0 / (SELECT COUNT(*) FROM employees) > 20;

-- Example 11: HAVING with NULL check
-- Find departments where everyone has commission (no NULLs)
SELECT
```

```sql
    dept_id,
    COUNT(*) AS total,
    COUNT(commission) AS with_commission
FROM employees
GROUP BY dept_id
HAVING COUNT(*) = COUNT(commission);

-- Example 12: HAVING with BETWEEN
-- Find departments with average salary between 60000 and 80000
SELECT
    dept_id,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) BETWEEN 60000 AND 80000;

-- Example 13: HAVING with IN
-- Find departments with specific average salary ranges
SELECT
    dept_id,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) IN (65000, 70000, 75000);

-- Example 14: HAVING with string aggregate
-- Find job titles where all employees have email
SELECT
    job_title,
    COUNT(*) AS total,
    COUNT(email) AS with_email
FROM employees
GROUP BY job_title
HAVING COUNT(*) = COUNT(email);

-- Example 15: HAVING with multiple aggregate comparisons
-- Find departments where:
-- - More than 1 employee
-- - Average salary > 60000
-- - Total salary < 200000
SELECT
    dept_id,
```

```sql
    COUNT(*) AS emp_count,
    AVG(salary) AS avg_salary,
    SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 1
   AND AVG(salary) > 60000
   AND SUM(salary) < 200000;

-- Example 16: HAVING with subquery
-- Find departments with average salary > overall average
SELECT
    dept_id,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);

-- Example 17: HAVING with EXISTS
-- Find departments that have at least one employee with salary > 80000
SELECT
    d.dept_id,
    d.dept_name,
    COUNT(e.emp_id) AS total_employees
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING EXISTS (
    SELECT 1 FROM employees e2
    WHERE e2.dept_id = d.dept_id AND e2.salary > 80000
);

-- Example 18: HAVING with NOT
-- Find departments with no high earners (salary > 90000)
SELECT
    dept_id,
    MAX(salary) AS max_salary
FROM employees
GROUP BY dept_id
HAVING MAX(salary) <= 90000;

-- Example 19: HAVING with ROLLUP (finding groups with totals)
```

```sql
SELECT
    COALESCE(dept_id, -1) AS dept_id,
    COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id WITH ROLLUP
HAVING COUNT(*) > 2;  -- Includes the total row if >2

-- Example 20: Complex reporting query with HAVING
SELECT
    d.dept_name,
    YEAR(e.hire_date) AS hire_year,
    COUNT(*) AS hires,
    AVG(e.salary) AS avg_salary,
    SUM(CASE WHEN e.salary > 70000 THEN 1 ELSE 0 END) AS high_earners
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
WHERE e.hire_date >= '2019-01-01'
GROUP BY d.dept_name, YEAR(e.hire_date)
HAVING COUNT(*) >= 2
   AND AVG(e.salary) > 60000
ORDER BY d.dept_name, hire_year;
```

Part 2: SQL Joins (VERY IMPORTANT)

Joins combine rows from two or more tables based on related columns.

Introduction to Joins

Why Joins?

- Relational databases store data in multiple tables
- Joins reconstruct the complete picture
- Avoid data redundancy

- Maintain data integrity

Types of Joins:

Join Type	Description
INNER JOIN	Returns only matching rows from both tables
LEFT JOIN	All rows from left table, matching from right
RIGHT JOIN	All rows from right table, matching from left
FULL JOIN	All rows from both tables
CROSS JOIN	Cartesian product (every row with every row)
SELF JOIN	Table joined with itself

Venn Diagram Representation:

text

```
INNER JOIN:        LEFT JOIN:          RIGHT JOIN:         FULL JOIN:
┌─────┐            ┌─────┐             ┌─────┐             ┌─────┐
│  A  │            │  A  │             │  A  │             │  A  │
│  ∩  │            │  ∪  │             │     │             │  ∪  │
│  B  │            │  B  │             │  B  │             │  B  │
└─────┘            └─────┘             └─────┘             └─────┘
Only matching   All A + matching B   All B + matching A   All rows
```

Sample Data for Join Examples

sql

```sql
-- Create customers table
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    city VARCHAR(50)
);

-- Create orders table
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    amount DECIMAL(10,2),
    status VARCHAR(20),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- Create products table
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2),
    category VARCHAR(50)
);

-- Create order_items table
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT,
    price DECIMAL(10,2),
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- Insert sample data
INSERT INTO customers VALUES
(1, 'John Doe', 'john@email.com', 'New York'),
(2, 'Jane Smith', 'jane@email.com', 'Los Angeles'),
(3, 'Bob Johnson', 'bob@email.com', 'Chicago'),
```

```sql
(4, 'Alice Brown', 'alice@email.com', 'Houston'),
(5, 'Charlie Wilson', 'charlie@email.com', 'Phoenix');

INSERT INTO orders VALUES
(101, 1, '2024-01-15', 250.00, 'completed'),
(102, 2, '2024-01-20', 450.00, 'completed'),
(103, 1, '2024-02-01', 120.00, 'pending'),
(104, 3, '2024-02-05', 890.00, 'completed'),
(105, 5, '2024-02-10', 350.00, 'shipped'),
(106, 8, '2024-02-15', 175.00, 'pending');  -- Customer 8 doesn't exist

INSERT INTO products VALUES
(201, 'Laptop', 999.99, 'Electronics'),
(202, 'Mouse', 29.99, 'Electronics'),
(203, 'Desk', 349.99, 'Furniture'),
(204, 'Chair', 199.99, 'Furniture'),
(205, 'Monitor', 299.99, 'Electronics'),
(206, 'Headphones', 89.99, 'Electronics');

INSERT INTO order_items VALUES
(101, 201, 1, 999.99),
(101, 202, 2, 29.99),
(102, 203, 1, 349.99),
(102, 204, 2, 199.99),
(103, 205, 1, 299.99),
(104, 201, 2, 999.99),
(104, 206, 1, 89.99),
(105, 202, 3, 29.99);
```

---

1. INNER JOIN

Returns only rows that have matching values in both tables.

Theory:

- Most common join type
- Returns only matched records

- If no match, row is excluded
- Equivalent to JOIN (INNER is default)

Syntax:

sql

```sql
SELECT columns
FROM table1
INNER JOIN table2 ON table1.column = table2.column;

-- Or using WHERE (old syntax, not recommended)
SELECT columns
FROM table1, table2
WHERE table1.column = table2.column;
```

Practical Examples:

sql

```sql
-- Example 1: Basic INNER JOIN
-- Get all orders with customer information
SELECT
    o.order_id,
    o.order_date,
    o.amount,
    c.name AS customer_name,
    c.email
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;

-- Result: Only orders with valid customer_id (customer 8 excluded)

-- Example 2: INNER JOIN with specific columns
SELECT
    c.customer_id,
    c.name,
    o.order_id,
    o.amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

```sql
-- Example 3: INNER JOIN with WHERE clause
-- Get completed orders with customer info
SELECT
    c.name,
    o.order_id,
    o.order_date,
    o.amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'completed';

-- Example 4: Multiple INNER JOINs
-- Get order details with customer and product info
SELECT
    c.name AS customer,
    o.order_id,
    o.order_date,
    p.product_name,
    oi.quantity,
    oi.price
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN order_items oi ON o.order_id = oi.order_id
INNER JOIN products p ON oi.product_id = p.product_id
ORDER BY o.order_id, p.product_name;

-- Example 5: INNER JOIN with aggregation
-- Get total spent per customer
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) AS order_count,
    SUM(o.amount) AS total_spent
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC;

-- Example 6: INNER JOIN with table aliases
SELECT
    c.name,
    o.order_id,
```

```sql
    o.amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;  -- INNER is default

-- Example 7: INNER JOIN with USING (if column names are same)
-- MySQL/PostgreSQL support USING
SELECT
    c.name,
    o.order_id,
    o.amount
FROM customers c
INNER JOIN orders o USING (customer_id);  -- Assumes both have customer_id

-- Example 8: INNER JOIN with compound conditions
SELECT
    c.name,
    o.order_id,
    o.amount
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
    AND o.amount > 300
    AND o.status = 'completed';

-- Example 9: INNER JOIN with date range
SELECT
    c.name,
    o.order_id,
    o.order_date,
    o.amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date BETWEEN '2024-01-01' AND '2024-01-31';

-- Example 10: Self-contained subquery vs JOIN
-- Find customers who have placed orders (using IN)
SELECT * FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);

-- Same with JOIN (usually more efficient)
SELECT DISTINCT c.*
FROM customers c
```

```sql
INNER JOIN orders o ON c.customer_id = o.customer_id;

-- Example 11: INNER JOIN with calculated fields
SELECT
    c.name,
    o.order_id,
    o.amount,
    o.amount * 0.1 AS tax_amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;

-- Example 12: INNER JOIN with conditional logic
SELECT
    c.name,
    o.order_id,
    o.amount,
    CASE
        WHEN o.amount > 500 THEN 'Large'
        WHEN o.amount > 200 THEN 'Medium'
        ELSE 'Small'
    END AS order_size
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;

-- Example 13: INNER JOIN with multiple conditions in ON
SELECT
    c.name,
    o.order_id,
    o.amount,
    p.product_name
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id AND o.status = 'completed'
INNER JOIN order_items oi ON o.order_id = oi.order_id
INNER JOIN products p ON oi.product_id = p.product_id AND p.category = 'Electronic
s';

-- Example 14: INNER JOIN with COUNT and GROUP BY
SELECT
    p.category,
    COUNT(DISTINCT o.order_id) AS order_count,
    SUM(oi.quantity) AS items_sold,
    SUM(oi.quantity * oi.price) AS revenue
```

```sql
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
INNER JOIN orders o ON oi.order_id = o.order_id
GROUP BY p.category;

-- Example 15: Find customers and their most recent order
SELECT
    c.name,
    MAX(o.order_date) AS last_order_date,
    COUNT(o.order_id) AS total_orders
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

---

2. LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table, and matching rows from the right table.

Theory:

- All records from left table are included
- If no match in right table, NULLs are shown
- Essential for finding missing relationships
- OUTER keyword is optional

Syntax:

sql

```sql
SELECT columns
FROM table1
LEFT JOIN table2 ON table1.column = table2.column;
```

Practical Examples:

sql

```sql
-- Example 1: Basic LEFT JOIN
-- Get all customers and their orders (even customers with no orders)
SELECT
    c.customer_id,
    c.name,
    o.order_id,
    o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
ORDER BY c.customer_id;

-- Customer 4 (Alice) will show with NULL order_id, NULL amount

-- Example 2: Find customers with no orders
SELECT
    c.customer_id,
    c.name,
    c.email
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- Example 3: LEFT JOIN with COUNT
-- Count orders per customer (including zero)
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) AS order_count  -- COUNT(column) ignores NULLs
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY order_count DESC;

-- Example 4: LEFT JOIN with COALESCE for NULL handling
SELECT
    c.name,
    COALESCE(COUNT(o.order_id), 0) AS order_count,
    COALESCE(SUM(o.amount), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name;
```

```sql
-- Example 5: LEFT JOIN with multiple tables
-- Get all customers, their orders, and order items
SELECT
    c.name,
    o.order_id,
    o.amount,
    p.product_name,
    oi.quantity
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON o.order_id = oi.order_id
LEFT JOIN products p ON oi.product_id = p.product_id
ORDER BY c.name, o.order_id;

-- Example 6: LEFT JOIN with WHERE on right table (careful!)
-- This becomes INNER JOIN because WHERE filters out NULLs
SELECT
    c.name,
    o.order_id,
    o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.amount > 300;  -- Removes customers with NULL orders

-- Correct way: put condition in JOIN
SELECT
    c.name,
    o.order_id,
    o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id AND o.amount > 300;

-- Example 7: LEFT JOIN with date filter
-- All customers and their orders from January 2024
SELECT
    c.name,
    o.order_id,
    o.order_date,
    o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
    AND o.order_date BETWEEN '2024-01-01' AND '2024-01-31';
```

```sql
-- Example 8: LEFT JOIN with aggregation and condition
-- Customers with total spending, including those with none
SELECT
    c.name,
    COALESCE(SUM(o.amount), 0) AS total_spent,
    COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name
HAVING total_spent < 500 OR order_count = 0;

-- Example 9: LEFT JOIN to find unmatched records
-- Find products that have never been ordered
SELECT
    p.product_id,
    p.product_name,
    p.category
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.order_id IS NULL;

-- Example 10: LEFT JOIN with multiple conditions
SELECT
    c.name,
    o.order_id,
    o.amount,
    o.status
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
    AND o.status IN ('completed', 'shipped')
WHERE c.city = 'New York';

-- Example 11: LEFT JOIN with calculated fields
SELECT
    c.name,
    COUNT(o.order_id) AS order_count,
    AVG(o.amount) AS avg_order_amount,
    CASE
        WHEN COUNT(o.order_id) = 0 THEN 'No orders'
        WHEN AVG(o.amount) > 500 THEN 'High value'
        ELSE 'Regular'
```

```sql
    END AS customer_category
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name;

-- Example 12: LEFT JOIN with date difference
-- Customers and days since last order (or NULL if never ordered)
SELECT
    c.name,
    MAX(o.order_date) AS last_order,
    DATEDIFF(CURDATE(), MAX(o.order_date)) AS days_since_last_order
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name;

-- Example 13: LEFT JOIN with UNION for completeness
-- Show all customers and orders, with indicator
SELECT
    c.name,
    o.order_id,
    'Has orders' AS status
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
UNION ALL
SELECT
    c.name,
    NULL,
    'No orders'
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- Example 14: LEFT JOIN with ranking
-- Customers and their top order (if any)
SELECT
    c.name,
    o.order_id,
    o.amount AS highest_order
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
LEFT JOIN orders o2 ON c.customer_id = o2.customer_id AND o.amount < o2.amount
WHERE o2.order_id IS NULL OR o.order_id IS NULL;
```

```sql
-- Example 15: LEFT JOIN with percentage calculation
-- Percentage of customers who have ordered
SELECT
    (SELECT COUNT(DISTINCT customer_id) FROM orders) * 100.0 / COUNT(*) AS percent
_with_orders,
    100 - (SELECT COUNT(DISTINCT customer_id) FROM orders) * 100.0 / COUNT(*) AS p
ercent_without_orders
FROM customers;
```

---

3. RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table, and matching rows from the left table.

Theory:

- Mirror of LEFT JOIN
- All records from right table are included
- If no match in left table, NULLs are shown
- Less common than LEFT JOIN (can always use LEFT JOIN by swapping tables)

Syntax:

sql

```sql
SELECT columns
FROM table1
RIGHT JOIN table2 ON table1.column = table2.column;
```

Practical Examples:

sql

```sql
-- Example 1: Basic RIGHT JOIN
-- Get all orders and their customer information (even orders with invalid custome
r_id)
SELECT
```

```sql
    o.order_id,
    o.amount,
    c.name AS customer_name,
    c.email
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;

-- This shows order 106 with NULL customer (since customer 8 doesn't exist)

-- Example 2: Same as LEFT JOIN with tables swapped
-- These are equivalent:
SELECT * FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;
SELECT * FROM orders o RIGHT JOIN customers c ON o.customer_id = c.customer_id;

-- Example 3: Find orders with invalid customers
SELECT
    o.order_id,
    o.customer_id,
    o.amount,
    c.name
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;

-- Example 4: RIGHT JOIN with aggregation
-- Get order statistics, including orders without valid customers
SELECT
    COALESCE(c.name, 'Deleted Customer') AS customer,
    COUNT(o.order_id) AS order_count,
    SUM(o.amount) AS total_amount
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name;

-- Example 5: RIGHT JOIN with multiple tables
SELECT
    o.order_id,
    o.amount,
    c.name AS customer,
    p.product_name
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
```

```sql
LEFT JOIN order_items oi ON o.order_id = oi.order_id
LEFT JOIN products p ON oi.product_id = p.product_id;

-- Example 6: RIGHT JOIN with WHERE
-- All orders, only show customer info if they're from New York
SELECT
    o.order_id,
    o.amount,
    c.name,
    c.city
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id AND c.city = 'New York';

-- Example 7: RIGHT JOIN with COUNT
-- Count orders per customer, including orders with missing customers
SELECT
    COALESCE(c.name, 'Unknown') AS customer_name,
    COUNT(*) AS order_count
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY customer_name;

-- Example 8: RIGHT JOIN with conditional logic
SELECT
    o.order_id,
    o.amount,
    CASE
        WHEN c.name IS NULL THEN 'Customer not found'
        ELSE c.name
    END AS customer
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;

-- Example 9: RIGHT JOIN with date filter
-- All orders in 2024, with customer info if available
SELECT
    o.order_id,
    o.order_date,
    o.amount,
    c.name
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
```

```sql
WHERE YEAR(o.order_date) = 2024;

-- Example 10: RIGHT JOIN with subquery
SELECT
    o.order_id,
    o.amount,
    (SELECT COUNT(*) FROM order_items oi WHERE oi.order_id = o.order_id) AS item_c
ount,
    c.name
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;

-- Example 11: RIGHT JOIN with UNION (same as FULL JOIN simulation)
-- All customers and all orders
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
UNION
SELECT c.name, o.order_id
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;

-- Example 12: RIGHT JOIN with aggregation and HAVING
-- Orders with total amount > 500, including customer info
SELECT
    o.order_id,
    SUM(oi.quantity * oi.price) AS total,
    c.name
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, c.name
HAVING total > 500;

-- Example 13: RIGHT JOIN with COALESCE for display
SELECT
    o.order_id,
    o.order_date,
    COALESCE(c.name, 'MISSING CUSTOMER') AS customer,
    COALESCE(c.email, 'no-email@unknown.com') AS email
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;
```

```sql
-- Example 14: RIGHT JOIN with ranking
-- Rank orders by amount, include customer if exists
SELECT
    o.order_id,
    o.amount,
    RANK() OVER (ORDER BY o.amount DESC) AS amount_rank,
    c.name
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;

-- Example 15: RIGHT JOIN vs LEFT JOIN preference
-- Most developers prefer LEFT JOIN for readability
-- Instead of:
SELECT * FROM table1 RIGHT JOIN table2 ON table1.id = table2.id;
-- Use:
SELECT * FROM table2 LEFT JOIN table1 ON table2.id = table1.id;
```

---

4. FULL JOIN (FULL OUTER JOIN)

Returns all rows when there's a match in either left or right table.

Theory:

- Combines LEFT JOIN and RIGHT JOIN
- All rows from both tables are included
- NULLs where no match exists
- Not supported directly in MySQL (need UNION)
- Supported in PostgreSQL, SQL Server, Oracle

Syntax:

sql

```sql
SELECT columns
FROM table1
```

```sql
FULL OUTER JOIN table2 ON table1.column = table2.column;
```

Practical Examples:

sql

```sql
-- Example 1: Basic FULL JOIN (PostgreSQL/SQL Server)
-- Get all customers and all orders
-- SELECT
--     c.customer_id,
--     c.name,
--     o.order_id,
--     o.amount
-- FROM customers c
-- FULL OUTER JOIN orders o ON c.customer_id = o.customer_id
-- ORDER BY c.customer_id, o.order_id;

-- Example 2: FULL JOIN simulation in MySQL using UNION
-- All customers and all orders
SELECT
    c.customer_id,
    c.name,
    o.order_id,
    o.amount,
    'Matched' AS join_type
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
UNION ALL
SELECT
    c.customer_id,
    c.name,
    o.order_id,
    o.amount,
    'Unmatched Right' AS join_type
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;

-- Example 3: Find orphaned records on both sides
-- Customers with no orders AND orders with no customers
-- SELECT
--     COALESCE(c.customer_id, o.customer_id) AS id,
```

```sql
--     c.name,
--     o.order_id
-- FROM customers c
-- FULL OUTER JOIN orders o ON c.customer_id = o.customer_id
-- WHERE c.customer_id IS NULL OR o.customer_id IS NULL;

-- MySQL simulation:
SELECT
    c.customer_id,
    c.name,
    NULL AS order_id,
    NULL AS amount,
    'Customer only' AS type
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
UNION ALL
SELECT
    NULL,
    NULL,
    o.order_id,
    o.amount,
    'Order only' AS type
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;

-- Example 4: FULL JOIN with aggregation
-- Complete customer-order summary
-- SELECT
--     COALESCE(c.customer_id, o.customer_id) AS entity_id,
--     c.name,
--     COUNT(o.order_id) AS order_count,
--     SUM(o.amount) AS total_spent
-- FROM customers c
-- FULL OUTER JOIN orders o ON c.customer_id = o.customer_id
-- GROUP BY COALESCE(c.customer_id, o.customer_id), c.name;

-- Example 5: FULL JOIN with COALESCE for display
-- SELECT
--     COALESCE(c.customer_id, o.customer_id) AS customer_id,
--     COALESCE(c.name, 'DELETED CUSTOMER') AS customer_name,
```

```
--     o.order_id,
--     o.amount
-- FROM customers c
-- FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;

-- Example 6: Compare data between tables
-- Find mismatches between customers and orders
-- SELECT
--     CASE
--         WHEN c.customer_id IS NULL THEN 'In orders only'
--         WHEN o.customer_id IS NULL THEN 'In customers only'
--         ELSE 'In both'
--     END AS status,
--     COALESCE(c.customer_id, o.customer_id) AS id,
--     c.name,
--     o.order_id
-- FROM customers c
-- FULL OUTER JOIN orders o ON c.customer_id = o.customer_id
-- WHERE c.customer_id IS NULL OR o.customer_id IS NULL;

-- Example 7: FULL JOIN with date filter
-- SELECT
--     c.name,
--     o.order_id,
--     o.order_date
-- FROM customers c
-- FULL OUTER JOIN orders o ON c.customer_id = o.customer_id
-- WHERE o.order_date >= '2024-01-01' OR o.order_date IS NULL;

-- Example 8: FULL JOIN with multiple conditions
-- SELECT
--     c.*,
--     o.*
-- FROM customers c
-- FULL OUTER JOIN orders o ON c.customer_id = o.customer_id
--     AND o.amount > 300;

-- Example 9: FULL JOIN for data reconciliation
-- Check which customers and orders are in both systems
-- SELECT
--     COALESCE(c.customer_id, o.customer_id) AS id,
--     CASE
```

```
--         WHEN c.customer_id IS NOT NULL AND o.customer_id IS NOT NULL THEN 'Comp
lete'
--         WHEN c.customer_id IS NOT NULL THEN 'Missing orders'
--         ELSE 'Missing customer'
--     END AS status
-- FROM customers c
-- FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;

-- Example 10: FULL JOIN with ranking
-- Rank all customers and orders together
-- SELECT
--     COALESCE(c.name, 'NO CUSTOMER') AS name,
--     o.order_id,
--     o.amount,
--     RANK() OVER (ORDER BY o.amount DESC NULLS LAST) AS rank
-- FROM customers c
-- FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;
```

---

5. CROSS JOIN

Returns the Cartesian product of two tables (every row from first table combined with every row from second table).

Theory:

- No join condition needed
- Result rows = rows_in_table1 × rows_in_table2
- Can be very large (use with caution)
- Useful for generating combinations

Syntax:

sql

```sql
SELECT columns
FROM table1
```

```sql
CROSS JOIN table2;

-- Or (old syntax)
SELECT columns
FROM table1, table2;
```

Practical Examples:

sql

```sql
-- Example 1: Basic CROSS JOIN
-- Combine every customer with every product
SELECT
    c.name AS customer,
    p.product_name
FROM customers c
CROSS JOIN products p
ORDER BY c.name, p.product_name;

-- Result: 5 customers × 6 products = 30 rows

-- Example 2: CROSS JOIN with WHERE (effectively becomes INNER JOIN)
-- Not recommended - use INNER JOIN instead
SELECT
    c.name,
    p.product_name,
    p.price
FROM customers c
CROSS JOIN products p
WHERE c.city = 'New York';  -- Still Cartesian product, then filtered

-- Better:
SELECT c.name, p.product_name
FROM customers c, products p
WHERE c.city = 'New York';  -- Old syntax, still Cartesian

-- Example 3: Generate all possible combinations
-- All possible department and job title combinations
SELECT
    d.dept_name,
    e.job_title
FROM departments d
```

```sql
CROSS JOIN (SELECT DISTINCT job_title FROM employees) e
ORDER BY d.dept_name, e.job_title;

-- Example 4: Create a calendar grid
-- Generate all dates in January 2024
SELECT
    dates.date,
    p.product_name
FROM (
    SELECT '2024-01-01' + INTERVAL (n) DAY AS date
    FROM (
        SELECT 0 AS n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3
        UNION SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7
        UNION SELECT 8 UNION SELECT 9 UNION SELECT 10
    ) numbers
    WHERE '2024-01-01' + INTERVAL n DAY <= '2024-01-31'
) dates
CROSS JOIN products p
ORDER BY dates.date, p.product_name;

-- Example 5: Generate test data
-- Create sample order combinations
SELECT
    c.customer_id,
    p.product_id,
    ROUND(RAND() * 10, 0) AS quantity
FROM customers c
CROSS JOIN products p
LIMIT 20;

-- Example 6: CROSS JOIN for matrix calculations
-- Multiply two sets of numbers
WITH numbers1 AS (SELECT 1 AS num UNION SELECT 2 UNION SELECT 3),
     numbers2 AS (SELECT 1 AS num UNION SELECT 2 UNION SELECT 3)
SELECT
    n1.num AS a,
    n2.num AS b,
    n1.num * n2.num AS product
FROM numbers1 n1
CROSS JOIN numbers2 n2
ORDER BY a, b;
```

```sql
-- Example 7: CROSS JOIN with aggregation
-- Average price per category for each customer
SELECT
    c.name,
    p.category,
    AVG(p.price) AS avg_category_price
FROM customers c
CROSS JOIN products p
GROUP BY c.name, p.category
ORDER BY c.name, p.category;

-- Example 8: Generate all time periods
-- Create all month-product combinations
SELECT
    months.month,
    p.product_name
FROM (
    SELECT 1 AS month UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
    UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8
    UNION SELECT 9 UNION SELECT 10 UNION SELECT 11 UNION SELECT 12
) months
CROSS JOIN products p;

-- Example 9: CROSS JOIN with LIMIT
-- Get random combinations
SELECT
    c.name,
    p.product_name
FROM customers c
CROSS JOIN products p
ORDER BY RAND()
LIMIT 10;

-- Example 10: Performance warning
-- Be careful with large tables!
-- If table1 has 10,000 rows and table2 has 10,000 rows
-- CROSS JOIN produces 100,000,000 rows!
-- Always test with LIMIT first

-- Example 11: CROSS JOIN for permissions matrix
-- Generate user-permission combinations
SELECT
```

```sql
    u.username,
    p.permission_name
FROM users u
CROSS JOIN permissions p
WHERE u.is_active = 1;

-- Example 12: CROSS JOIN with CASE for conditional combinations
SELECT
    c.name,
    p.product_name,
    CASE
        WHEN p.category = 'Electronics' AND c.city = 'New York' THEN 'Featured'
        ELSE 'Regular'
    END AS promotion
FROM customers c
CROSS JOIN products p;

-- Example 13: Generate all possible pairs for matching
-- Find all possible employee-manager combinations
SELECT
    e1.first_name AS employee,
    e2.first_name AS potential_manager
FROM employees e1
CROSS JOIN employees e2
WHERE e1.emp_id != e2.emp_id
LIMIT 20;

-- Example 14: CROSS JOIN with date ranges
-- Create daily inventory records
SELECT
    dates.date,
    p.product_id,
    p.stock AS initial_stock
FROM (
    SELECT DISTINCT order_date AS date FROM orders
    UNION
    SELECT CURDATE()
) dates
CROSS JOIN products p;

-- Example 15: CROSS JOIN for survey combinations
-- Generate all possible answer combinations
```

```sql
SELECT
    q.question_text,
    a.answer_text
FROM questions q
CROSS JOIN answers a
WHERE q.question_id = a.question_id;  -- Actually becomes INNER JOIN
```

---

6. Self Join

Joining a table with itself.

Theory:

- Table appears twice in query
- Must use table aliases
- Useful for hierarchical data (employees-managers)
- Can find pairs, duplicates, comparisons

Syntax:

sql

```sql
SELECT columns
FROM table t1
JOIN table t2 ON t1.column = t2.column;
```

Practical Examples:

sql

```sql
-- First, add manager_id to employees if not exists
-- Already have manager_id in our employees table
-- Update some employees to have managers
UPDATE employees SET manager_id = 105 WHERE emp_id IN (101, 103, 112);  -- Charlie
manages
UPDATE employees SET manager_id = 102 WHERE emp_id = 106;  -- Jane manages
UPDATE employees SET manager_id = 104 WHERE emp_id = 107;  -- Alice manages
```

```sql
UPDATE employees SET manager_id = 109 WHERE emp_id = 110;  -- Grace manages
UPDATE employees SET manager_id = 108 WHERE emp_id = 111;  -- Frank manages

-- Example 1: Basic Self Join - Find employees and their managers
SELECT
    e.first_name AS employee,
    e.last_name AS emp_last,
    m.first_name AS manager,
    m.last_name AS manager_last
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id
ORDER BY manager, employee;

-- Example 2: Find employees who earn more than their managers
SELECT
    e.first_name AS employee,
    e.salary AS emp_salary,
    m.first_name AS manager,
    m.salary AS manager_salary
FROM employees e
INNER JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;

-- Example 3: Find employees with same manager
SELECT
    m.first_name AS manager,
    GROUP_CONCAT(e.first_name) AS reports
FROM employees e
INNER JOIN employees m ON e.manager_id = m.emp_id
GROUP BY m.first_name, m.emp_id;

-- Example 4: Find employees hired in same year
SELECT
    e1.first_name AS emp1,
    e2.first_name AS emp2,
    YEAR(e1.hire_date) AS hire_year
FROM employees e1
INNER JOIN employees e2 ON YEAR(e1.hire_date) = YEAR(e2.hire_date)
    AND e1.emp_id < e2.emp_id  -- Avoid duplicates and self matches
ORDER BY hire_year, emp1, emp2;

-- Example 5: Find employees with same salary
```

```sql
SELECT
    e1.first_name AS emp1,
    e1.salary,
    e2.first_name AS emp2
FROM employees e1
INNER JOIN employees e2 ON e1.salary = e2.salary
    AND e1.emp_id < e2.emp_id
ORDER BY e1.salary;

-- Example 6: Hierarchical query (employees, managers, grand-managers)
SELECT
    e.first_name AS employee,
    m.first_name AS manager,
    gm.first_name AS grand_manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id
LEFT JOIN employees gm ON m.manager_id = gm.emp_id
ORDER BY e.emp_id;

-- Example 7: Find employees who are also managers
SELECT DISTINCT
    m.first_name AS manager_name,
    COUNT(e.emp_id) AS report_count
FROM employees e
RIGHT JOIN employees m ON e.manager_id = m.emp_id
GROUP BY m.emp_id, m.first_name
HAVING COUNT(e.emp_id) > 0;

-- Example 8: Find pairs of employees in same department
SELECT
    d.dept_name,
    e1.first_name AS emp1,
    e2.first_name AS emp2
FROM employees e1
INNER JOIN employees e2 ON e1.dept_id = e2.dept_id AND e1.emp_id < e2.emp_id
INNER JOIN departments d ON e1.dept_id = d.dept_id
ORDER BY d.dept_name, emp1, emp2;

-- Example 9: Find employees with overlapping responsibilities
-- If we had projects table, but using department as proxy
SELECT
    e1.first_name AS emp1,
```

```sql
    e2.first_name AS emp2,
    e1.dept_id
FROM employees e1
INNER JOIN employees e2 ON e1.dept_id = e2.dept_id
    AND e1.emp_id < e2.emp_id
    AND e1.job_title != e2.job_title;

-- Example 10: Recursive self join for org chart (CTE)
-- Find all employees under a specific manager (Charlie - ID 105)
WITH RECURSIVE org_chart AS (
    -- Anchor: the manager
    SELECT emp_id, first_name, last_name, manager_id, 1 AS level
    FROM employees
    WHERE emp_id = 105

    UNION ALL

    -- Recursive: employees who report to someone in the org_chart
    SELECT e.emp_id, e.first_name, e.last_name, e.manager_id, oc.level + 1
    FROM employees e
    INNER JOIN org_chart oc ON e.manager_id = oc.emp_id
)
SELECT
    CONCAT(REPEAT('  ', level - 1), first_name, ' ', last_name) AS employee,
    level
FROM org_chart
ORDER BY level, emp_id;

-- Example 11: Find employees without managers (top level)
SELECT first_name, last_name
FROM employees
WHERE manager_id IS NULL;

-- Example 12: Find employees with no direct reports
SELECT
    e.first_name,
    e.last_name
FROM employees e
LEFT JOIN employees r ON e.emp_id = r.manager_id
WHERE r.emp_id IS NULL
ORDER BY e.first_name;
```

```sql
-- Example 13: Compare employees with colleagues
-- Who has higher salary in same department
SELECT
    e1.first_name AS emp1,
    e2.first_name AS emp2,
    e1.dept_id,
    e1.salary AS salary1,
    e2.salary AS salary2,
    e1.salary - e2.salary AS salary_diff
FROM employees e1
INNER JOIN employees e2 ON e1.dept_id = e2.dept_id AND e1.emp_id < e2.emp_id
ORDER BY e1.dept_id, salary_diff DESC;

-- Example 14: Find potential mentor-mentee pairs
-- Senior (hired earlier) and junior (hired later) in same department
SELECT
    senior.first_name AS senior,
    senior.hire_date AS senior_hired,
    junior.first_name AS junior,
    junior.hire_date AS junior_hired,
    senior.dept_id
FROM employees senior
INNER JOIN employees junior ON senior.dept_id = junior.dept_id
    AND senior.hire_date < junior.hire_date
    AND senior.emp_id != junior.emp_id
ORDER BY senior.dept_id, senior.hire_date, junior.hire_date;

-- Example 15: Find duplicate emails (if any)
SELECT
    e1.email,
    e1.first_name AS emp1,
    e2.first_name AS emp2
FROM employees e1
INNER JOIN employees e2 ON e1.email = e2.email AND e1.emp_id < e2.emp_id
WHERE e1.email IS NOT NULL;

-- Example 16: Organizational depth
SELECT
    e.first_name,
    e.last_name,
    COUNT(m.emp_id) AS management_level
FROM employees e
```

```sql
LEFT JOIN employees m ON e.manager_id = m.emp_id
LEFT JOIN employees m2 ON m.manager_id = m2.emp_id
LEFT JOIN employees m3 ON m2.manager_id = m3.emp_id
-- Continue as needed, but better with recursive CTE
GROUP BY e.emp_id, e.first_name, e.last_name;

-- Example 17: Find salary ranking within department
SELECT
    e1.first_name,
    e1.dept_id,
    e1.salary,
    COUNT(e2.salary) + 1 AS rank_in_dept
FROM employees e1
LEFT JOIN employees e2 ON e1.dept_id = e2.dept_id AND e1.salary < e2.salary
GROUP BY e1.emp_id, e1.first_name, e1.dept_id, e1.salary
ORDER BY e1.dept_id, rank_in_dept;

-- Example 18: Find employees who joined after their manager
SELECT
    e.first_name AS employee,
    e.hire_date AS emp_hired,
    m.first_name AS manager,
    m.hire_date AS manager_hired
FROM employees e
INNER JOIN employees m ON e.manager_id = m.emp_id
WHERE e.hire_date < m.hire_date;

-- Example 19: Find all possible pairs for team building
-- All combinations of employees (not including self)
SELECT
    CONCAT(e1.first_name, ' ', e1.last_name) AS person1,
    CONCAT(e2.first_name, ' ', e2.last_name) AS person2
FROM employees e1
CROSS JOIN employees e2
WHERE e1.emp_id < e2.emp_id
ORDER BY person1, person2
LIMIT 20;

-- Example 20: Find indirect reports (two levels down)
SELECT
    m.first_name AS manager,
    e.first_name AS direct_report,
```

```sql
    i.first_name AS indirect_report
FROM employees m
INNER JOIN employees e ON m.emp_id = e.manager_id
INNER JOIN employees i ON e.emp_id = i.manager_id
ORDER BY m.first_name, e.first_name, i.first_name;
```

Summary: Join Types Comparison

Join Type	Result	Use Case
INNER JOIN	Only matching rows	When you need only related data
LEFT JOIN	All left + matching right	When you need all from main table
RIGHT JOIN	All right + matching left	Same as LEFT with tables swapped
FULL JOIN	All from both	When you need complete picture
CROSS JOIN	Cartesian product	Generating combinations
SELF JOIN	Table with itself	Hierarchical data, comparisons

Performance Tips:

1. **Always use proper indexes** on join columns
2. **Join order matters** - put smaller result sets first
3. **Use EXPLAIN** to analyze query execution
4. **Avoid joining too many tables** in one query
5. **Consider denormalization** for frequently joined data
6. **Use appropriate join types** - don't use LEFT JOIN when INNER JOIN suffices
7. **Be careful with NULLs** in join conditions

Subqueries and Set Operations: Complete Theory Notes with Practical Examples

Part 1: Subqueries

A subquery is a query nested inside another query. It's used to return data that will be used in the main query as a condition or as a data source.

Introduction to Subqueries

What is a Subquery?

- A query within another query
- Also called inner query or nested query
- Enclosed in parentheses
- Can return single value, single row, single column, or table

Types of Subqueries by Return Value:

Type	Returns	Used In
Scalar	Single value (one row, one column)	SELECT, WHERE, HAVING
Row	Single row (multiple columns)	WHERE with row constructors
Column	Single column (multiple rows)	WHERE with IN, ANY, ALL
Table	Multiple rows and columns	FROM clause (derived table)

Types of Subqueries by Correlation:

Type	Description
Non-correlated	Independent of outer query, runs once
Correlated	Depends on outer query, runs for each outer row

Sample Data for Subquery Examples

sql

```sql
-- Using the same company_db database from previous examples
-- Add more data for richer examples

-- Add products table if not exists
CREATE TABLE IF NOT EXISTS products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    category VARCHAR(50),
    unit_price DECIMAL(10,2),
    stock_quantity INT
);

-- Add sales table
CREATE TABLE IF NOT EXISTS sales (
    sale_id INT PRIMARY KEY,
    product_id INT,
    sale_date DATE,
    quantity INT,
    discount DECIMAL(5,2),
    customer_id INT,
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- Insert products
INSERT INTO products VALUES
```

```sql
(1, 'Laptop Pro', 'Electronics', 1299.99, 50),
(2, 'Wireless Mouse', 'Electronics', 29.99, 200),
(3, 'Mechanical Keyboard', 'Electronics', 89.99, 75),
(4, 'Office Desk', 'Furniture', 349.99, 25),
(5, 'Ergonomic Chair', 'Furniture', 299.99, 30),
(6, 'Monitor 27"', 'Electronics', 329.99, 40),
(7, 'USB-C Hub', 'Accessories', 49.99, 150),
(8, 'Laptop Stand', 'Accessories', 39.99, 100);

-- Insert sales
INSERT INTO sales VALUES
(1001, 1, '2024-01-15', 2, 0, 101),
(1002, 2, '2024-01-15', 5, 10, 102),
(1003, 3, '2024-01-16', 1, 0, 103),
(1004, 1, '2024-01-17', 1, 5, 104),
(1005, 4, '2024-01-18', 1, 0, 105),
(1006, 5, '2024-01-18', 2, 15, 101),
(1007, 2, '2024-01-19', 3, 0, 102),
(1008, 6, '2024-01-20', 1, 0, 103),
(1009, 7, '2024-01-21', 4, 20, 106),
(1010, 8, '2024-01-22', 2, 0, 107),
(1011, 1, '2024-01-23', 1, 0, 108),
(1012, 3, '2024-01-24', 2, 10, 109),
(1013, 5, '2024-01-25', 1, 0, 110),
(1014, 2, '2024-01-26', 10, 5, 111),
(1015, 6, '2024-01-27', 2, 0, 112);
```

---

1. Scalar Subquery

A scalar subquery returns exactly one value (one row, one column).

Theory:

- Must return exactly one value
- If returns no rows, result is NULL
- If returns multiple rows, error occurs

- Can be used anywhere a single value is expected

Practical Examples:

```sql
-- Example 1: Scalar subquery in SELECT
-- Get employee details with company average salary
SELECT
    first_name,
    last_name,
    salary,
    (SELECT AVG(salary) FROM employees) AS company_avg_salary,
    salary - (SELECT AVG(salary) FROM employees) AS diff_from_avg
FROM employees;

-- Example 2: Scalar subquery in WHERE
-- Find employees who earn more than average
SELECT
    first_name,
    last_name,
    salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Example 3: Scalar subquery in HAVING
-- Find departments with above-average salary
SELECT
    dept_id,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);

-- Example 4: Scalar subquery in ORDER BY
-- Order employees by how far they are from average
SELECT
    first_name,
    last_name,
    salary
FROM employees
ORDER BY ABS(salary - (SELECT AVG(salary) FROM employees));
```

```sql
-- Example 5: Multiple scalar subqueries
-- Get product sales with comparisons
SELECT
    p.product_name,
    p.unit_price,
    (SELECT AVG(unit_price) FROM products) AS avg_price,
    (SELECT MAX(unit_price) FROM products) AS max_price,
    (SELECT MIN(unit_price) FROM products) AS min_price
FROM products p;

-- Example 6: Scalar subquery with aggregation
-- Get each product's sales compared to average
SELECT
    p.product_name,
    SUM(s.quantity * p.unit_price) AS total_revenue,
    (SELECT AVG(total)
     FROM (SELECT SUM(quantity * unit_price) AS total
           FROM sales
           JOIN products USING (product_id)
           GROUP BY product_id) AS product_totals
    ) AS avg_product_revenue
FROM products p
LEFT JOIN sales s ON p.product_id = s.product_id
GROUP BY p.product_id, p.product_name;

-- Example 7: Scalar subquery with CASE
-- Categorize products based on price
SELECT
    product_name,
    unit_price,
    CASE
        WHEN unit_price > (SELECT AVG(unit_price) FROM products) THEN 'Above Avera
ge'
        WHEN unit_price < (SELECT AVG(unit_price) FROM products) THEN 'Below Avera
ge'
        ELSE 'Exactly Average'
    END AS price_category
FROM products;

-- Example 8: Scalar subquery in UPDATE
-- Give bonus to employees below average salary
```

```sql
UPDATE employees
SET salary = salary * 1.05
WHERE salary < (SELECT AVG(salary) FROM employees);

-- Example 9: Scalar subquery in INSERT
-- Insert a summary record
INSERT INTO sales_summary (report_date, total_sales, avg_sale, product_count)
VALUES (
    CURDATE(),
    (SELECT SUM(quantity * unit_price) FROM sales JOIN products USING (product_id)
),
    (SELECT AVG(quantity * unit_price) FROM sales JOIN products USING (product_id)
),
    (SELECT COUNT(DISTINCT product_id) FROM sales)
);

-- Example 10: Scalar subquery for validation
-- Check if any product has zero stock
SELECT
    'Inventory Status' AS report,
    CASE
        WHEN EXISTS (SELECT 1 FROM products WHERE stock_quantity = 0)
        THEN 'Some products out of stock'
        ELSE 'All products in stock'
    END AS status;

-- Example 11: Scalar subquery with date functions
-- Get sales compared to same day last month
SELECT
    s.sale_date,
    SUM(s.quantity * p.unit_price) AS daily_sales,
    (SELECT SUM(s2.quantity * p2.unit_price)
     FROM sales s2
     JOIN products p2 ON s2.product_id = p2.product_id
     WHERE s2.sale_date = DATE_SUB(s.sale_date, INTERVAL 1 MONTH)
    ) AS same_day_last_month
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY s.sale_date;

-- Example 12: Scalar subquery with LIMIT
-- Get each product's price compared to second most expensive
```

```sql
SELECT
    product_name,
    unit_price,
    (SELECT unit_price
     FROM products
     ORDER BY unit_price DESC
     LIMIT 1 OFFSET 1) AS second_highest_price
FROM products;

-- Example 13: Scalar subquery with NULL handling
-- Handle case when subquery returns NULL
SELECT
    first_name,
    salary,
    COALESCE(
        (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id),
        (SELECT AVG(salary) FROM employees)
    ) AS dept_or_company_avg
FROM employees e;

-- Example 14: Scalar subquery for running calculations
-- Running total using scalar subquery (inefficient but educational)
SELECT
    s1.sale_date,
    s1.product_id,
    s1.quantity,
    (SELECT SUM(s2.quantity * p2.unit_price)
     FROM sales s2
     JOIN products p2 ON s2.product_id = p2.product_id
     WHERE s2.sale_date <= s1.sale_date) AS running_total
FROM sales s1
JOIN products p1 ON s1.product_id = p1.product_id
ORDER BY s1.sale_date;

-- Example 15: Multiple scalar subqueries in one query
-- Comprehensive product report
SELECT
    p.product_name,
    p.unit_price,
    p.stock_quantity,
    (SELECT COUNT(*) FROM sales WHERE product_id = p.product_id) AS times_sold,
```

```
    (SELECT SUM(quantity) FROM sales WHERE product_id = p.product_id) AS total_uni
ts_sold,
    (SELECT SUM(quantity * p.unit_price) FROM sales WHERE product_id = p.product_i
d) AS revenue,
    (SELECT AVG(quantity) FROM sales WHERE product_id = p.product_id) AS avg_quant
ity_per_sale,
    CASE
        WHEN (SELECT SUM(quantity) FROM sales WHERE product_id = p.product_id) > p
.stock_quantity
        THEN 'High Demand'
        ELSE 'Normal'
    END AS demand_status
FROM products p;
```

---

2. Correlated Subquery

A correlated subquery references columns from the outer query and executes once for each row processed by the outer query.

Theory:

- Depends on outer query for values
- Executes repeatedly (once per outer row)
- Can be less efficient than joins
- Useful for row-by-row comparisons
- Can be used in SELECT, WHERE, EXISTS

Practical Examples:

sql

```sql
-- Example 1: Basic correlated subquery in WHERE
-- Find employees who earn more than average in their department
SELECT
    e1.first_name,
    e1.last_name,
```

```sql
    e1.department,
    e1.salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id  -- Correlation
)
ORDER BY e1.dept_id, e1.salary;

-- Example 2: Correlated subquery in SELECT
-- Get each employee with their department average
SELECT
    e1.first_name,
    e1.last_name,
    e1.dept_id,
    e1.salary,
    (SELECT AVG(salary)
     FROM employees e2
     WHERE e2.dept_id = e1.dept_id) AS dept_avg
FROM employees e1;

-- Example 3: Find products with above-average sales in their category
SELECT
    p1.product_name,
    p1.category,
    p1.unit_price,
    (SELECT SUM(quantity)
     FROM sales s
     WHERE s.product_id = p1.product_id) AS total_sold
FROM products p1
WHERE (SELECT SUM(quantity)
       FROM sales s
       WHERE s.product_id = p1.product_id) > (
    SELECT AVG(product_sales)
    FROM (
        SELECT SUM(s2.quantity) AS product_sales
        FROM products p2
        LEFT JOIN sales s2 ON p2.product_id = s2.product_id
        WHERE p2.category = p1.category
        GROUP BY p2.product_id
    ) AS cat_sales
```

```sql
);

-- Example 4: EXISTS with correlated subquery
-- Find customers who have placed at least one order
SELECT
    c.customer_id,
    c.name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id  -- Correlation
);

-- Example 5: NOT EXISTS with correlated subquery
-- Find customers who have never placed an order
SELECT
    c.customer_id,
    c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);

-- Example 6: Correlated subquery with UPDATE
-- Update product stock based on sales
UPDATE products p
SET stock_quantity = stock_quantity - (
    SELECT COALESCE(SUM(quantity), 0)
    FROM sales s
    WHERE s.product_id = p.product_id
    AND s.sale_date >= '2024-01-01'
);

-- Example 7: Correlated subquery with DELETE
-- Delete products that have never been sold
DELETE FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM sales s
```

```sql
    WHERE s.product_id = p.product_id
);

-- Example 8: Find employees with highest salary in their department
SELECT
    e1.first_name,
    e1.last_name,
    e1.dept_id,
    e1.salary
FROM employees e1
WHERE e1.salary = (
    SELECT MAX(salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);

-- Example 9: Running total with correlated subquery
SELECT
    s1.sale_date,
    s1.product_id,
    s1.quantity,
    (
        SELECT SUM(s2.quantity)
        FROM sales s2
        WHERE s2.sale_date <= s1.sale_date
        AND s2.product_id = s1.product_id
    ) AS running_total
FROM sales s1
ORDER BY s1.product_id, s1.sale_date;

-- Example 10: Find products that outsell all products in their category
SELECT
    p1.product_name,
    p1.category,
    (
        SELECT SUM(s.quantity)
        FROM sales s
        WHERE s.product_id = p1.product_id
    ) AS total_sold
FROM products p1
WHERE (
    SELECT SUM(s.quantity)
```

```sql
    FROM sales s
    WHERE s.product_id = p1.product_id
) > ALL (
    SELECT COALESCE(SUM(s2.quantity), 0)
    FROM products p2
    LEFT JOIN sales s2 ON p2.product_id = s2.product_id
    WHERE p2.category = p1.category
    AND p2.product_id != p1.product_id
    GROUP BY p2.product_id
);

-- Example 11: Correlated subquery with percentage calculation
-- What percentage of company sales does each product represent?
SELECT
    p.product_name,
    (
        SELECT SUM(s.quantity * p.unit_price)
        FROM sales s
        WHERE s.product_id = p.product_id
    ) AS product_revenue,
    (
        SELECT SUM(s.quantity * p.unit_price) * 100.0 /
               (SELECT SUM(s2.quantity * p2.unit_price)
                FROM sales s2
                JOIN products p2 ON s2.product_id = p2.product_id)
        FROM sales s
        WHERE s.product_id = p.product_id
    ) AS percentage_of_total
FROM products p
WHERE product_revenue > 0;

-- Example 12: Find employees who joined after their manager
SELECT
    e1.first_name AS employee,
    e1.hire_date AS emp_hire,
    e2.first_name AS manager,
    e2.hire_date AS mgr_hire
FROM employees e1
INNER JOIN employees e2 ON e1.manager_id = e2.emp_id
WHERE e1.hire_date < e2.hire_date;

-- Alternative with correlated subquery
```

```sql
SELECT
    e.first_name,
    e.last_name,
    e.hire_date
FROM employees e
WHERE e.hire_date < (
    SELECT hire_date
    FROM employees m
    WHERE m.emp_id = e.manager_id
);

-- Example 13: Find days with above-average sales for that month
SELECT
    s1.sale_date,
    SUM(s1.quantity * p1.unit_price) AS daily_total
FROM sales s1
JOIN products p1 ON s1.product_id = p1.product_id
GROUP BY s1.sale_date
HAVING SUM(s1.quantity * p1.unit_price) > (
    SELECT AVG(daily_total)
    FROM (
        SELECT
            s2.sale_date,
            SUM(s2.quantity * p2.unit_price) AS daily_total
        FROM sales s2
        JOIN products p2 ON s2.product_id = p2.product_id
        WHERE MONTH(s2.sale_date) = MONTH(s1.sale_date)
        AND YEAR(s2.sale_date) = YEAR(s1.sale_date)
        GROUP BY s2.sale_date
    ) AS month_days
);

-- Example 14: Correlated subquery with ROW_NUMBER simulation
-- Simulate ROW_NUMBER() without window functions
SELECT
    e1.first_name,
    e1.last_name,
    e1.dept_id,
    e1.salary,
    (
        SELECT COUNT(*)
        FROM employees e2
```

```sql
        WHERE e2.dept_id = e1.dept_id
        AND e2.salary >= e1.salary
    ) AS salary_rank_in_dept
FROM employees e1
ORDER BY e1.dept_id, salary_rank_in_dept;

-- Example 15: Complex correlated subquery for trend analysis
-- Compare each product's sales to its previous month's sales
SELECT
    p.product_name,
    YEAR(s.sale_date) AS sale_year,
    MONTH(s.sale_date) AS sale_month,
    SUM(s.quantity) AS monthly_sales,
    (
        SELECT SUM(s2.quantity)
        FROM sales s2
        WHERE s2.product_id = p.product_id
        AND YEAR(s2.sale_date) = YEAR(s.sale_date)
        AND MONTH(s2.sale_date) = MONTH(s.sale_date) - 1
    ) AS prev_month_sales,
    CASE
        WHEN (
            SELECT SUM(s2.quantity)
            FROM sales s2
            WHERE s2.product_id = p.product_id
            AND YEAR(s2.sale_date) = YEAR(s.sale_date)
            AND MONTH(s2.sale_date) = MONTH(s.sale_date) - 1
        ) IS NOT NULL
        THEN SUM(s.quantity) * 100.0 / (
            SELECT SUM(s2.quantity)
            FROM sales s2
            WHERE s2.product_id = p.product_id
            AND YEAR(s2.sale_date) = YEAR(s.sale_date)
            AND MONTH(s2.sale_date) = MONTH(s.sale_date) - 1
        ) - 100
        ELSE NULL
    END AS growth_percentage
FROM products p
JOIN sales s ON p.product_id = s.product_id
GROUP BY p.product_id, p.product_name, YEAR(s.sale_date), MONTH(s.sale_date)
ORDER BY p.product_name, sale_year, sale_month;
```

---

3. Nested Subquery

A nested subquery is a subquery inside another subquery (multiple levels deep).

Theory:

- Subqueries can be nested multiple levels
- Innermost subquery executes first
- Results passed to outer subquery
- Can be difficult to read - consider CTEs
- Performance can suffer with deep nesting

Practical Examples:

sql

```sql
-- Example 1: Three-level nested subquery
-- Find products that have sold more than the average of top-selling products
SELECT
    product_name,
    category,
    unit_price
FROM products
WHERE product_id IN (
    SELECT product_id
    FROM sales
    GROUP BY product_id
    HAVING SUM(quantity) > (
        SELECT AVG(total_quantity)
        FROM (
            SELECT
                product_id,
                SUM(quantity) AS total_quantity
            FROM sales
            GROUP BY product_id
        ) AS product_totals
    )
```

```sql
);

-- Example 2: Find employees who earn more than the average of managers' salaries
SELECT
    first_name,
    last_name,
    salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE emp_id IN (
        SELECT DISTINCT manager_id
        FROM employees
        WHERE manager_id IS NOT NULL
    )
);

-- Example 3: Find the most popular product category by revenue
SELECT
    category,
    total_revenue
FROM (
    SELECT
        category,
        SUM(quantity * unit_price) AS total_revenue
    FROM products p
    JOIN sales s ON p.product_id = s.product_id
    GROUP BY category
) AS category_revenue
WHERE total_revenue = (
    SELECT MAX(total_revenue)
    FROM (
        SELECT
            category,
            SUM(quantity * unit_price) AS total_revenue
        FROM products p
        JOIN sales s ON p.product_id = s.product_id
        GROUP BY category
    ) AS cat_rev
);
```

```sql
-- Example 4: Nested subquery with EXISTS and NOT EXISTS
-- Find customers who bought all products in the Electronics category
SELECT
    c.name,
    c.email
FROM customers c
WHERE NOT EXISTS (
    SELECT p.product_id
    FROM products p
    WHERE p.category = 'Electronics'
    AND NOT EXISTS (
        SELECT 1
        FROM orders o
        JOIN order_items oi ON o.order_id = oi.order_id
        WHERE o.customer_id = c.customer_id
        AND oi.product_id = p.product_id
    )
);

-- Example 5: Three-level subquery with aggregation
-- Find days when sales were above the average of top 3 days
SELECT
    sale_date,
    daily_total
FROM (
    SELECT
        sale_date,
        SUM(quantity * unit_price) AS daily_total
    FROM sales s
    JOIN products p ON s.product_id = p.product_id
    GROUP BY sale_date
) AS daily_sales
WHERE daily_total > (
    SELECT AVG(top_daily_total)
    FROM (
        SELECT
            SUM(quantity * unit_price) AS top_daily_total
        FROM sales s
        JOIN products p ON s.product_id = p.product_id
        GROUP BY sale_date
        ORDER BY top_daily_total DESC
        LIMIT 3
```

```sql
    ) AS top_days
);

-- Example 6: Find products with price higher than any product in a subcategory
SELECT
    product_name,
    unit_price
FROM products
WHERE unit_price > ALL (
    SELECT unit_price
    FROM products
    WHERE category IN (
        SELECT category
        FROM products
        GROUP BY category
        HAVING AVG(unit_price) < 100
    )
);

-- Example 7: Nested subquery in HAVING
-- Find categories where average price is above the overall average
-- of categories that have at least 2 products
SELECT
    category,
    AVG(unit_price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > (
    SELECT AVG(category_avg)
    FROM (
        SELECT
            category,
            AVG(unit_price) AS category_avg
        FROM products
        GROUP BY category
        HAVING COUNT(*) >= 2
    ) AS qualifying_categories
);

-- Example 8: Find employees who have the same salary as someone
-- who earns more than the average of their department
SELECT
```

```sql
    first_name,
    last_name,
    salary
FROM employees
WHERE salary IN (
    SELECT salary
    FROM employees e1
    WHERE salary > (
        SELECT AVG(salary)
        FROM employees e2
        WHERE e2.dept_id = e1.dept_id
    )
);

-- Example 9: Complex nested subquery with multiple levels
-- Find products that have sold more than the average of products
-- in categories that have above-average sales
SELECT
    p.product_name,
    p.category,
    SUM(s.quantity) AS total_sold
FROM products p
JOIN sales s ON p.product_id = s.product_id
GROUP BY p.product_id, p.product_name, p.category
HAVING SUM(s.quantity) > (
    SELECT AVG(category_avg)
    FROM (
        SELECT
            p2.category,
            AVG(product_sales) AS category_avg
        FROM (
            SELECT
                p3.product_id,
                p3.category,
                COALESCE(SUM(s3.quantity), 0) AS product_sales
            FROM products p3
            LEFT JOIN sales s3 ON p3.product_id = s3.product_id
            GROUP BY p3.product_id, p3.category
        ) AS product_sales_by_cat
        GROUP BY p2.category
        HAVING AVG(product_sales) > (
            SELECT AVG(product_sales)
```

```sql
            FROM (
                SELECT COALESCE(SUM(s4.quantity), 0) AS product_sales
                FROM products p4
                LEFT JOIN sales s4 ON p4.product_id = s4.product_id
                GROUP BY p4.product_id
            ) AS all_product_sales
        )
    ) AS top_categories
);

-- Example 10: Nested subquery for percentile calculation
-- Find products in the top 25% by price
SELECT
    product_name,
    unit_price
FROM products
WHERE unit_price >= (
    SELECT unit_price
    FROM products
    ORDER BY unit_price
    LIMIT 1
    OFFSET (
        SELECT ROUND(COUNT(*) * 0.75) - 1
        FROM products
    )
);

-- Example 11: Nested subquery with window function simulation
-- Find employees with salary in top 3 of their department
SELECT
    e1.first_name,
    e1.last_name,
    e1.dept_id,
    e1.salary
FROM employees e1
WHERE (
    SELECT COUNT(*)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
    AND e2.salary > e1.salary
) < 3
ORDER BY e1.dept_id, e1.salary DESC;
```

```sql
-- Example 12: Find customers who spent more than average
-- of customers who bought from the same category
SELECT
    c.name,
    c.customer_id,
    (
        SELECT SUM(oi.quantity * oi.price)
        FROM orders o
        JOIN order_items oi ON o.order_id = oi.order_id
        WHERE o.customer_id = c.customer_id
    ) AS total_spent
FROM customers c
WHERE (
    SELECT SUM(oi.quantity * oi.price)
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.customer_id = c.customer_id
) > (
    SELECT AVG(customer_spent)
    FROM (
        SELECT
            c2.customer_id,
            SUM(oi2.quantity * oi2.price) AS customer_spent
        FROM customers c2
        JOIN orders o2 ON c2.customer_id = o2.customer_id
        JOIN order_items oi2 ON o2.order_id = oi2.order_id
        JOIN products p2 ON oi2.product_id = p2.product_id
        WHERE p2.category IN (
            SELECT DISTINCT p3.category
            FROM orders o3
            JOIN order_items oi3 ON o3.order_id = oi3.order_id
            JOIN products p3 ON oi3.product_id = p3.product_id
            WHERE o3.customer_id = c.customer_id
        )
        GROUP BY c2.customer_id
    ) AS category_customers
);

-- Example 13: Nested subquery for data validation
-- Find potential data inconsistencies
SELECT
```

```sql
    'Sales with no matching product' AS issue,
    COUNT(*) AS count
FROM sales
WHERE product_id NOT IN (
    SELECT product_id
    FROM products
)
UNION ALL
SELECT
    'Orders with no matching customer',
    COUNT(*)
FROM orders
WHERE customer_id NOT IN (
    SELECT customer_id
    FROM customers
    WHERE customer_id IS NOT NULL
)
UNION ALL
SELECT
    'Order items with no matching order',
    COUNT(*)
FROM order_items
WHERE order_id NOT IN (
    SELECT order_id
    FROM orders
);

-- Example 14: Hierarchical query with nested subqueries
-- Find all employees who report to managers who report to executives
SELECT
    first_name,
    last_name
FROM employees
WHERE manager_id IN (
    SELECT emp_id
    FROM employees
    WHERE manager_id IN (
        SELECT emp_id
        FROM employees
        WHERE manager_id IS NULL
    )
);
```

```sql
-- Example 15: Complex nested subquery for market basket analysis
-- Find products often bought together with Laptops
SELECT
    p2.product_name,
    COUNT(*) AS times_bought_together
FROM order_items oi1
JOIN order_items oi2 ON oi1.order_id = oi2.order_id
JOIN products p1 ON oi1.product_id = p1.product_id
JOIN products p2 ON oi2.product_id = p2.product_id
WHERE p1.product_name = 'Laptop Pro'
AND p2.product_name != 'Laptop Pro'
AND oi1.order_id IN (
    SELECT DISTINCT order_id
    FROM order_items
    WHERE product_id IN (
        SELECT product_id
        FROM products
        WHERE category = 'Electronics'
    )
)
GROUP BY p2.product_name
ORDER BY times_bought_together DESC;
```

---

4. Subquery in SELECT

Subqueries in the SELECT clause must return a single value (scalar subquery).

Theory:

- Also called scalar subqueries
- Must return exactly one value
- Can be correlated or non-correlated
- Often used for calculations and comparisons
- Can be used with aggregate functions

Practical Examples:

sql

```sql
-- Example 1: Basic subquery in SELECT
-- Get employee details with department name
SELECT
    e.first_name,
    e.last_name,
    e.salary,
    (SELECT dept_name
     FROM departments d
     WHERE d.dept_id = e.dept_id) AS department_name
FROM employees e;

-- Example 2: Multiple subqueries in SELECT
-- Comprehensive employee report
SELECT
    e.first_name,
    e.last_name,
    e.salary,
    (SELECT dept_name FROM departments d WHERE d.dept_id = e.dept_id) AS departmen
t,
    (SELECT AVG(salary) FROM employees) AS company_avg,
    (SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e.dept_id) AS dept_av
g,
    (SELECT COUNT(*) FROM employees) AS total_employees,
    (SELECT COUNT(*) FROM employees e2 WHERE e2.dept_id = e.dept_id) AS dept_count
FROM employees e;

-- Example 3: Subquery with arithmetic in SELECT
-- Calculate variance from averages
SELECT
    e.first_name,
    e.last_name,
    e.salary,
    (SELECT AVG(salary) FROM employees) AS company_avg,
    e.salary - (SELECT AVG(salary) FROM employees) AS diff_from_company_avg,
    (SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e.dept_id) AS dept_av
g,
    e.salary - (SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e.dept_id)
AS diff_from_dept_avg
FROM employees e;
```

```sql
-- Example 4: Subquery with conditional logic in SELECT
-- Categorize employees
SELECT
    e.first_name,
    e.last_name,
    e.salary,
    CASE
        WHEN e.salary > (SELECT AVG(salary) FROM employees) THEN 'Above Average'
        WHEN e.salary = (SELECT AVG(salary) FROM employees) THEN 'Exactly Average'
        ELSE 'Below Average'
    END AS salary_category
FROM employees e;

-- Example 5: Subquery with aggregate in SELECT
-- Product sales report
SELECT
    p.product_name,
    p.unit_price,
    (SELECT COALESCE(SUM(s.quantity), 0)
     FROM sales s
     WHERE s.product_id = p.product_id) AS total_sold,
    (SELECT COALESCE(SUM(s.quantity * p.unit_price), 0)
     FROM sales s
     WHERE s.product_id = p.product_id) AS total_revenue,
    (SELECT COALESCE(AVG(s.quantity), 0)
     FROM sales s
     WHERE s.product_id = p.product_id) AS avg_quantity_per_sale,
    (SELECT COUNT(*)
     FROM sales s
     WHERE s.product_id = p.product_id) AS number_of_sales
FROM products p;

-- Example 6: Subquery with date functions in SELECT
-- Sales analysis with comparisons
SELECT
    s.sale_date,
    s.product_id,
    s.quantity,
    (SELECT SUM(s2.quantity)
     FROM sales s2
     WHERE s2.sale_date = s.sale_date) AS total_quantity_that_day,
```

```sql
    (SELECT AVG(s3.quantity)
     FROM sales s3
     WHERE s3.product_id = s.product_id) AS avg_quantity_for_product
FROM sales s;

-- Example 7: Subquery with ranking in SELECT
-- Simulate ranking without window functions
SELECT
    e.first_name,
    e.last_name,
    e.salary,
    (SELECT COUNT(*) + 1
     FROM employees e2
     WHERE e2.salary > e.salary) AS salary_rank
FROM employees e
ORDER BY salary_rank;

-- Example 8: Subquery with percentage in SELECT
-- Market share calculation
SELECT
    p.product_name,
    (SELECT COALESCE(SUM(s.quantity * p.unit_price), 0)
     FROM sales s
     WHERE s.product_id = p.product_id) AS revenue,
    (SELECT COALESCE(SUM(s.quantity * p.unit_price), 0) * 100.0 /
            (SELECT SUM(s2.quantity * p2.unit_price)
             FROM sales s2
             JOIN products p2 ON s2.product_id = p2.product_id)
     FROM sales s
     WHERE s.product_id = p.product_id) AS percentage_of_total,
    (SELECT COALESCE(SUM(s.quantity * p.unit_price), 0) * 100.0 /
            (SELECT SUM(s2.quantity * p2.unit_price)
             FROM sales s2
             JOIN products p2 ON s2.product_id = p2.product_id
             WHERE p2.category = p.category)
     FROM sales s
     WHERE s.product_id = p.product_id) AS percentage_of_category
FROM products p;

-- Example 9: Subquery with string aggregation in SELECT
-- Customer summary with order details
SELECT
```

```sql
    c.customer_id,
    c.name,
    (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.customer_id) AS order_c
ount,
    (SELECT COALESCE(SUM(o.amount), 0)
     FROM orders o
     WHERE o.customer_id = c.customer_id) AS total_spent,
    (SELECT GROUP_CONCAT(o.order_id ORDER BY o.order_date DESC SEPARATOR ', ')
     FROM orders o
     WHERE o.customer_id = c.customer_id) AS order_ids,
    (SELECT MAX(o.order_date)
     FROM orders o
     WHERE o.customer_id = c.customer_id) AS last_order_date
FROM customers c;

-- Example 10: Subquery with JSON in SELECT (MySQL 5.7+)
SELECT
    p.product_name,
    (SELECT JSON_ARRAYAGG(
        JSON_OBJECT('sale_date', s.sale_date,
                    'quantity', s.quantity,
                    'customer_id', s.customer_id)
     )
     FROM sales s
     WHERE s.product_id = p.product_id) AS sales_history
FROM products p;

-- Example 11: Subquery for trend analysis in SELECT
-- Month-over-month growth
SELECT
    sale_month,
    monthly_sales,
    previous_month_sales,
    CASE
        WHEN previous_month_sales IS NULL THEN NULL
        ELSE (monthly_sales - previous_month_sales) * 100.0 / previous_month_sales
    END AS growth_percentage
FROM (
    SELECT
        DATE_FORMAT(s.sale_date, '%Y-%m') AS sale_month,
        SUM(s.quantity * p.unit_price) AS monthly_sales,
        (SELECT SUM(s2.quantity * p2.unit_price)
```

```sql
        FROM sales s2
        JOIN products p2 ON s2.product_id = p2.product_id
        WHERE DATE_FORMAT(s2.sale_date, '%Y-%m') =
              DATE_FORMAT(DATE_SUB(s.sale_date, INTERVAL 1 MONTH), '%Y-%m')
       ) AS previous_month_sales
    FROM sales s
    JOIN products p ON s.product_id = p.product_id
    GROUP BY DATE_FORMAT(s.sale_date, '%Y-%m')
) AS monthly_data;

-- Example 12: Subquery for inventory valuation in SELECT
-- FIFO inventory valuation simulation
SELECT
    p.product_name,
    p.stock_quantity,
    p.unit_price AS current_price,
    (SELECT SUM(s.quantity * s.price)
     FROM order_items s
     WHERE s.product_id = p.product_id
     AND s.order_id IN (
         SELECT o.order_id
         FROM orders o
         WHERE o.order_date <= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
     )) AS sales_last_30_days_value,
    (SELECT AVG(s.price)
     FROM order_items s
     WHERE s.product_id = p.product_id) AS avg_selling_price
FROM products p;

-- Example 13: Subquery with window function alternative in SELECT
-- Without window functions
SELECT
    e1.first_name,
    e1.last_name,
    e1.dept_id,
    e1.salary,
    (SELECT COUNT(*)
     FROM employees e2
     WHERE e2.dept_id = e1.dept_id
     AND e2.salary >= e1.salary) AS rank_in_dept,
    (SELECT COUNT(*)
     FROM employees e2
```

```sql
     WHERE e2.salary >= e1.salary) AS overall_rank
FROM employees e1
ORDER BY e1.dept_id, rank_in_dept;

-- Example 14: Subquery for data completeness check in SELECT
-- Check for missing information
SELECT
    e.emp_id,
    e.first_name,
    e.last_name,
    CASE WHEN e.email IS NULL THEN 'Missing' ELSE 'OK' END AS email_status,
    CASE WHEN e.manager_id IS NULL THEN 'Missing' ELSE 'OK' END AS manager_status,
    CASE WHEN EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = e.emp_id)
         THEN 'Has Orders'
         ELSE 'No Orders'
    END AS order_status
FROM employees e;

-- Example 15: Complex subquery in SELECT with multiple levels
-- Comprehensive performance metrics
SELECT
    p.product_id,
    p.product_name,
    p.category,
    p.unit_price,
    (SELECT COUNT(*) FROM sales s WHERE s.product_id = p.product_id) AS sale_count
,
    (SELECT COALESCE(SUM(s.quantity), 0)
     FROM sales s
     WHERE s.product_id = p.product_id) AS units_sold,
    (SELECT COALESCE(SUM(s.quantity * p.unit_price), 0)
     FROM sales s
     WHERE s.product_id = p.product_id) AS revenue,
    (SELECT COALESCE(SUM(s.quantity * p.unit_price), 0) * 100.0 /
            (SELECT SUM(s2.quantity * p2.unit_price)
             FROM sales s2
             JOIN products p2 ON s2.product_id = p2.product_id)
     FROM sales s
     WHERE s.product_id = p.product_id) AS pct_of_total_revenue,
    (SELECT AVG(s.quantity)
     FROM sales s
     WHERE s.product_id = p.product_id) AS avg_quantity_per_sale,
```

```sql
    (SELECT MAX(s.sale_date)
     FROM sales s
     WHERE s.product_id = p.product_id) AS last_sale_date,
    CASE
        WHEN (SELECT COALESCE(SUM(s.quantity), 0)
              FROM sales s
              WHERE s.product_id = p.product_id) > p.stock_quantity * 2
        THEN 'High Demand'
        WHEN (SELECT COALESCE(SUM(s.quantity), 0)
              FROM sales s
              WHERE s.product_id = p.product_id) > p.stock_quantity
        THEN 'Good Demand'
        ELSE 'Low Demand'
    END AS demand_category
FROM products p;
```

---

5. Subquery in WHERE

Subqueries in the WHERE clause filter rows based on results from another query.

Theory:

- Most common location for subqueries
- Can use operators: =, >, <, >=, <=, <>, IN, NOT IN, EXISTS, NOT EXISTS, ANY, ALL
- Can be correlated or non-correlated
- IN and EXISTS are common for membership tests

Practical Examples:

sql

```sql
-- Example 1: Subquery with = (equality)
-- Find employees in the same department as 'John Doe'
SELECT
    first_name,
    last_name,
    dept_id
```

```sql
FROM employees
WHERE dept_id = (
    SELECT dept_id
    FROM employees
    WHERE first_name = 'John' AND last_name = 'Doe'
);

-- Example 2: Subquery with > (greater than)
-- Find products with price above average
SELECT
    product_name,
    unit_price
FROM products
WHERE unit_price > (
    SELECT AVG(unit_price)
    FROM products
);

-- Example 3: Subquery with IN
-- Find employees in departments located in New York
SELECT
    first_name,
    last_name,
    dept_id
FROM employees
WHERE dept_id IN (
    SELECT dept_id
    FROM departments
    WHERE location = 'New York'
);

-- Example 4: Subquery with NOT IN
-- Find employees NOT in IT or Finance departments
SELECT
    first_name,
    last_name,
    dept_id
FROM employees
WHERE dept_id NOT IN (
    SELECT dept_id
    FROM departments
    WHERE dept_name IN ('IT', 'Finance')
```

```sql
);

-- Example 5: Subquery with EXISTS
-- Find customers who have placed at least one order
SELECT
    c.customer_id,
    c.name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);

-- Example 6: Subquery with NOT EXISTS
-- Find customers who have never placed an order
SELECT
    c.customer_id,
    c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);

-- Example 7: Subquery with ANY
-- Find employees who earn more than any employee in department 2
SELECT
    first_name,
    last_name,
    salary
FROM employees
WHERE salary > ANY (
    SELECT salary
    FROM employees
    WHERE dept_id = 2
);
-- Returns true if salary > any salary from dept 2 (i.e., > min of dept 2)

-- Example 8: Subquery with ALL
-- Find employees who earn more than all employees in department 2
```

```sql
SELECT
    first_name,
    last_name,
    salary
FROM employees
WHERE salary > ALL (
    SELECT salary
    FROM employees
    WHERE dept_id = 2
);
-- Returns true if salary > max of dept 2

-- Example 9: Subquery with comparisons and aggregates
-- Find products that have sold more than the average quantity
SELECT
    p.product_name,
    p.unit_price
FROM products p
WHERE (
    SELECT COALESCE(SUM(s.quantity), 0)
    FROM sales s
    WHERE s.product_id = p.product_id
) > (
    SELECT AVG(product_quantity)
    FROM (
        SELECT COALESCE(SUM(s.quantity), 0) AS product_quantity
        FROM products p2
        LEFT JOIN sales s ON p2.product_id = s.product_id
        GROUP BY p2.product_id
    ) AS product_sales
);

-- Example 10: Subquery with BETWEEN
-- Find employees hired between the first and last hire of department 1
SELECT
    first_name,
    last_name,
    hire_date
FROM employees
WHERE hire_date BETWEEN (
    SELECT MIN(hire_date)
    FROM employees
```

```sql
    WHERE dept_id = 1
) AND (
    SELECT MAX(hire_date)
    FROM employees
    WHERE dept_id = 1
);

-- Example 11: Subquery with multiple columns (row subquery)
-- Find employees with same (dept_id, salary) as a specific employee
SELECT
    first_name,
    last_name,
    dept_id,
    salary
FROM employees
WHERE (dept_id, salary) = (
    SELECT dept_id, salary
    FROM employees
    WHERE first_name = 'John' AND last_name = 'Doe'
);

-- Example 12: Subquery with correlated comparison
-- Find employees who earn more than average in their department
SELECT
    e1.first_name,
    e1.last_name,
    e1.salary,
    e1.dept_id
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);

-- Example 13: Subquery with NOT IN and NULL handling
-- Careful with NULLs in NOT IN
SELECT
    first_name,
    last_name
FROM employees
WHERE emp_id NOT IN (
```

```sql
    SELECT manager_id
    FROM employees
    WHERE manager_id IS NOT NULL  -- Important to exclude NULLs
);

-- Example 14: Subquery with IN and multiple conditions
-- Find products that have been sold in high quantity
SELECT
    product_name,
    unit_price
FROM products
WHERE product_id IN (
    SELECT product_id
    FROM sales
    GROUP BY product_id
    HAVING SUM(quantity) > 10
    AND AVG(quantity) > 2
);

-- Example 15: Subquery with EXISTS and correlated conditions
-- Find employees who have at least one subordinate
SELECT
    e1.first_name,
    e1.last_name
FROM employees e1
WHERE EXISTS (
    SELECT 1
    FROM employees e2
    WHERE e2.manager_id = e1.emp_id
);

-- Example 16: Subquery with NOT EXISTS and correlated conditions
-- Find employees who are not managers
SELECT
    e1.first_name,
    e1.last_name
FROM employees e1
WHERE NOT EXISTS (
    SELECT 1
    FROM employees e2
    WHERE e2.manager_id = e1.emp_id
);
```

```sql
-- Example 17: Subquery with ANY and correlated conditions
-- Find employees who have at least one colleague with higher salary
SELECT
    e1.first_name,
    e1.last_name,
    e1.salary
FROM employees e1
WHERE e1.salary < ANY (
    SELECT e2.salary
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
    AND e2.emp_id != e1.emp_id
);

-- Example 18: Subquery with ALL and correlated conditions
-- Find employees who have the highest salary in their department
SELECT
    e1.first_name,
    e1.last_name,
    e1.salary
FROM employees e1
WHERE e1.salary >= ALL (
    SELECT e2.salary
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);

-- Example 19: Subquery with multiple levels
-- Find products that have been sold to customers who also bought Laptops
SELECT DISTINCT
    p.product_name
FROM products p
WHERE p.product_id IN (
    SELECT oi.product_id
    FROM order_items oi
    WHERE oi.order_id IN (
        SELECT o.order_id
        FROM orders o
        WHERE o.customer_id IN (
            SELECT o2.customer_id
            FROM orders o2
```

```
            JOIN order_items oi2 ON o2.order_id = oi2.order_id
            JOIN products p2 ON oi2.product_id = p2.product_id
            WHERE p2.product_name = 'Laptop Pro'
        )
    )
);

-- Example 20: Subquery with date range
-- Find sales that happened after the last sale of product 1
SELECT
    s.sale_id,
    s.product_id,
    s.sale_date,
    s.quantity
FROM sales s
WHERE s.sale_date > (
    SELECT MAX(s2.sale_date)
    FROM sales s2
    WHERE s2.product_id = 1
);
```

Part 2: Set Operations

Set operations combine results from multiple queries into a single result set.

Introduction to Set Operations

What are Set Operations?

- Combine results from two or more SELECT statements
- Each SELECT must have same number of columns
- Corresponding columns must have compatible data types
- Column names come from first SELECT

Types of Set Operations:

Operation	Description
UNION	Combines results, removes duplicates
UNION ALL	Combines results, keeps duplicates
INTERSECT	Returns common rows from both queries
EXCEPT (or MINUS)	Returns rows in first query not in second

Rules:

- Same number of columns in all queries
- Compatible data types
- ORDER BY only at the end
- Column names from first query

---

1. UNION

Combines results from multiple queries and removes duplicates.

Theory:

- Returns distinct rows from combined queries
- Slower than UNION ALL due to duplicate removal
- Useful for combining similar data from different tables
- Sorts results to remove duplicates

Syntax:

sql

```sql
SELECT column1, column2 FROM table1
UNION
SELECT column1, column2 FROM table2
ORDER BY column1;
```

Practical Examples:

sql

```sql
-- Example 1: Basic UNION
-- Get all unique department names from both employees and departments?
-- Actually, let's get all unique job titles from employees and all department nam
es
SELECT job_title AS name, 'Job Title' AS type
FROM employees
WHERE job_title IS NOT NULL
UNION
SELECT dept_name, 'Department'
FROM departments
ORDER BY name;

-- Example 2: UNION with different tables
-- Get all customer and employee names
SELECT
    name AS person_name,
    'Customer' AS person_type
FROM customers
UNION
SELECT
    CONCAT(first_name, ' ', last_name) AS person_name,
    'Employee' AS person_type
FROM employees
ORDER BY person_name;

-- Example 3: UNION for complete list of IDs
-- Get all IDs from multiple tables
SELECT customer_id AS id FROM customers
UNION
SELECT emp_id FROM employees
```

```sql
UNION
SELECT product_id FROM products
UNION
SELECT order_id FROM orders
ORDER BY id;

-- Example 4: UNION with calculated columns
-- Get sales summary with indicators
SELECT
    'High Value' AS category,
    COUNT(*) AS count,
    SUM(amount) AS total
FROM orders
WHERE amount > 500
UNION
SELECT
    'Medium Value' AS category,
    COUNT(*) AS count,
    SUM(amount) AS total
FROM orders
WHERE amount BETWEEN 200 AND 500
UNION
SELECT
    'Low Value' AS category,
    COUNT(*) AS count,
    SUM(amount) AS total
FROM orders
WHERE amount < 200;

-- Example 5: UNION with different filters
-- Get active and inactive counts
SELECT
    'Active Customers' AS status,
    COUNT(*) AS count
FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders)
UNION
SELECT
    'Inactive Customers' AS status,
    COUNT(*) AS count
FROM customers
```

```sql
WHERE customer_id NOT IN (SELECT DISTINCT customer_id FROM orders WHERE customer_id IS NOT NULL);

-- Example 6: UNION for pivot-like operation
-- Sales by product category summary
SELECT
    'Electronics' AS category,
    COUNT(*) AS sales_count,
    SUM(quantity) AS total_units
FROM sales s
JOIN products p ON s.product_id = p.product_id
WHERE p.category = 'Electronics'
UNION
SELECT
    'Furniture' AS category,
    COUNT(*) AS sales_count,
    SUM(quantity) AS total_units
FROM sales s
JOIN products p ON s.product_id = p.product_id
WHERE p.category = 'Furniture'
UNION
SELECT
    'Accessories' AS category,
    COUNT(*) AS sales_count,
    SUM(quantity) AS total_units
FROM sales s
JOIN products p ON s.product_id = p.product_id
WHERE p.category = 'Accessories'
ORDER BY sales_count DESC;

-- Example 7: UNION with date aggregation
-- Monthly sales report with totals
SELECT
    DATE_FORMAT(sale_date, '%Y-%m') AS period,
    'Monthly' AS period_type,
    SUM(quantity * unit_price) AS total
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY DATE_FORMAT(sale_date, '%Y-%m')
UNION
SELECT
    'TOTAL' AS period,
```

```sql
    'Overall' AS period_type,
    SUM(quantity * unit_price) AS total
FROM sales s
JOIN products p ON s.product_id = p.product_id
ORDER BY period;

-- Example 8: UNION with constant values
-- Get enumeration of possible statuses
SELECT 'pending' AS order_status, 1 AS sort_order
UNION
SELECT 'processing', 2
UNION
SELECT 'shipped', 3
UNION
SELECT 'delivered', 4
UNION
SELECT 'cancelled', 5
ORDER BY sort_order;

-- Example 9: UNION for completeness check
-- Find all referenced IDs that don't exist in parent tables
SELECT
    'Orders with invalid customer' AS issue,
    order_id AS id
FROM orders
WHERE customer_id NOT IN (SELECT customer_id FROM customers WHERE customer_id IS N
OT NULL)
UNION
SELECT
    'Order items with invalid product',
    order_id
FROM order_items
WHERE product_id NOT IN (SELECT product_id FROM products)
UNION
SELECT
    'Order items with invalid order',
    order_id
FROM order_items
WHERE order_id NOT IN (SELECT order_id FROM orders);

-- Example 10: UNION with TOP/LIMIT
-- Get top 3 products by revenue and top 3 by quantity
```

```sql
(SELECT
    'Revenue' AS metric,
    p.product_name,
    SUM(s.quantity * p.unit_price) AS value
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.product_name
ORDER BY value DESC
LIMIT 3)
UNION
(SELECT
    'Quantity' AS metric,
    p.product_name,
    SUM(s.quantity) AS value
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.product_name
ORDER BY value DESC
LIMIT 3);

-- Example 11: UNION with COUNT and GROUP BY
-- Get counts by different dimensions
SELECT
    'By Department' AS dimension,
    d.dept_name AS value,
    COUNT(e.emp_id) AS count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
UNION
SELECT
    'By Location' AS dimension,
    d.location AS value,
    COUNT(e.emp_id) AS count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.location
UNION
SELECT
    'By Job Title' AS dimension,
    e.job_title AS value,
    COUNT(*) AS count
```

```sql
FROM employees e
WHERE e.job_title IS NOT NULL
GROUP BY e.job_title
ORDER BY dimension, count DESC;

-- Example 12: UNION with CASE
-- Categorize products by price range
SELECT
    'Budget' AS price_range,
    COUNT(*) AS product_count,
    AVG(unit_price) AS avg_price
FROM products
WHERE unit_price < 50
UNION
SELECT
    'Mid-range',
    COUNT(*),
    AVG(unit_price)
FROM products
WHERE unit_price BETWEEN 50 AND 200
UNION
SELECT
    'Premium',
    COUNT(*),
    AVG(unit_price)
FROM products
WHERE unit_price > 200;

-- Example 13: UNION for hierarchical data
-- Get organizational structure
SELECT
    emp_id AS id,
    CONCAT(first_name, ' ', last_name) AS name,
    'Employee' AS level
FROM employees
WHERE manager_id IS NOT NULL
UNION
SELECT
    emp_id,
    CONCAT(first_name, ' ', last_name),
    'Manager'
FROM employees
```

```sql
WHERE emp_id IN (SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT
NULL)
UNION
SELECT
    emp_id,
    CONCAT(first_name, ' ', last_name),
    'Top Level'
FROM employees
WHERE manager_id IS NULL
ORDER BY level, name;

-- Example 14: UNION with string functions
-- Get all email contacts
SELECT
    'Customer' AS source,
    name AS contact_name,
    email
FROM customers
WHERE email IS NOT NULL
UNION
SELECT
    'Employee',
    CONCAT(first_name, ' ', last_name),
    email
FROM employees
WHERE email IS NOT NULL
ORDER BY contact_name;

-- Example 15: UNION with multiple aggregations
-- Sales dashboard metrics
SELECT
    'Total Revenue' AS metric,
    SUM(quantity * unit_price) AS value
FROM sales s
JOIN products p ON s.product_id = p.product_id
UNION
SELECT
    'Total Units Sold',
    SUM(quantity)
FROM sales
UNION
SELECT
```

```sql
    'Average Order Value',
    AVG(order_total)
FROM (
    SELECT order_id, SUM(quantity * unit_price) AS order_total
    FROM sales s
    JOIN products p ON s.product_id = p.product_id
    GROUP BY order_id
) AS orders
UNION
SELECT
    'Unique Customers',
    COUNT(DISTINCT customer_id)
FROM sales
UNION
SELECT
    'Unique Products Sold',
    COUNT(DISTINCT product_id)
FROM sales;
```

---

2. UNION ALL

Combines results from multiple queries and keeps all duplicates.

Theory:

- Returns all rows from combined queries
- Faster than UNION (no duplicate removal)
- Use when duplicates are acceptable or desired
- No sorting performed

Syntax:

sql

```sql
SELECT column1, column2 FROM table1
UNION ALL
SELECT column1, column2 FROM table2
```

```sql
ORDER BY column1;
```

Practical Examples:

sql

```sql
-- Example 1: Basic UNION ALL
-- Get all names from multiple tables (including duplicates)
SELECT name AS person_name FROM customers
UNION ALL
SELECT CONCAT(first_name, ' ', last_name) FROM employees
ORDER BY person_name;

-- Example 2: UNION ALL for audit log
-- Combine multiple audit sources
SELECT
    'Order' AS source,
    order_id AS record_id,
    'Created' AS action,
    created_at AS action_date
FROM orders
UNION ALL
SELECT
    'Order',
    order_id,
    'Status Changed',
    updated_at
FROM order_status_history
UNION ALL
SELECT
    'Customer',
    customer_id,
    'Updated',
    updated_at
FROM customer_audit
ORDER BY action_date DESC
LIMIT 100;

-- Example 3: UNION ALL for pagination across tables
-- Get all records with pagination
(SELECT
    'Customer' AS type,
```

```sql
    customer_id AS id,
    name AS name,
    created_at
FROM customers
ORDER BY created_at DESC
LIMIT 10)
UNION ALL
(SELECT
    'Employee',
    emp_id,
    CONCAT(first_name, ' ', last_name),
    hire_date
FROM employees
ORDER BY hire_date DESC
LIMIT 10)
ORDER BY created_at DESC
LIMIT 15;

-- Example 4: UNION ALL with running totals
-- Combine sales data with returns (if we had returns table)
SELECT
    sale_date,
    'Sale' AS transaction_type,
    quantity * unit_price AS amount
FROM sales s
JOIN products p ON s.product_id = p.product_id
UNION ALL
SELECT
    return_date,
    'Return',
    -quantity * unit_price  -- Negative amount for returns
FROM returns r
JOIN products p ON r.product_id = p.product_id
ORDER BY sale_date;

-- Example 5: UNION ALL for data distribution analysis
-- Get all numeric values for histogram
SELECT salary AS value, 'Employee Salary' AS source FROM employees
UNION ALL
SELECT unit_price, 'Product Price' FROM products
UNION ALL
SELECT amount, 'Order Amount' FROM orders
```

```sql
WHERE amount IS NOT NULL;

-- Example 6: UNION ALL with multiple conditions
-- Get all orders and order items in one result set
SELECT
    order_id,
    'Order Header' AS level,
    amount AS value,
    order_date AS date
FROM orders
UNION ALL
SELECT
    oi.order_id,
    'Order Item',
    oi.quantity * oi.price,
    o.order_date
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
ORDER BY order_id, level DESC;

-- Example 7: UNION ALL for batch processing
-- Get records to process in batches
SELECT 'A' AS batch, customer_id, name FROM customers WHERE customer_id % 4 = 0
UNION ALL
SELECT 'B', customer_id, name FROM customers WHERE customer_id % 4 = 1
UNION ALL
SELECT 'C', customer_id, name FROM customers WHERE customer_id % 4 = 2
UNION ALL
SELECT 'D', customer_id, name FROM customers WHERE customer_id % 4 = 3
ORDER BY batch, customer_id;

-- Example 8: UNION ALL with CTE
-- Combine hierarchical data
WITH RECURSIVE org_chart AS (
    SELECT emp_id, first_name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.first_name, e.manager_id, oc.level + 1
    FROM employees e
    JOIN org_chart oc ON e.manager_id = oc.emp_id
)
```

```sql
SELECT
    CONCAT(REPEAT('  ', level - 1), first_name) AS employee,
    level
FROM org_chart
UNION ALL
SELECT 'Total Employees:', COUNT(*) FROM employees
ORDER BY
    CASE WHEN employee LIKE 'Total%' THEN 999 ELSE level END,
    employee;

-- Example 9: UNION ALL for data quality checks
-- List all potential data issues
SELECT 'Missing email' AS issue, COUNT(*) AS count FROM customers WHERE email IS N
ULL
UNION ALL
SELECT 'Missing phone', COUNT(*) FROM customers WHERE phone IS NULL
UNION ALL
SELECT 'Orders with no items', COUNT(*) FROM orders o
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.order_id = o.order_id)
UNION ALL
SELECT 'Products never sold', COUNT(*) FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id)
UNION ALL
SELECT 'Employees with no department', COUNT(*) FROM employees WHERE dept_id IS NU
LL;

-- Example 10: UNION ALL with window functions
-- Get rankings from multiple categories
SELECT
    'Sales' AS category,
    p.product_name,
    ROW_NUMBER() OVER (ORDER BY SUM(s.quantity) DESC) AS rank
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.product_name
UNION ALL
SELECT
    'Revenue',
    p.product_name,
    ROW_NUMBER() OVER (ORDER BY SUM(s.quantity * p.unit_price) DESC)
FROM sales s
JOIN products p ON s.product_id = p.product_id
```

```sql
GROUP BY p.product_name
UNION ALL
SELECT
    'Price',
    product_name,
    ROW_NUMBER() OVER (ORDER BY unit_price DESC)
FROM products
ORDER BY category, rank;

-- Example 11: UNION ALL for time series with missing data
-- Fill in missing dates with zeros
WITH RECURSIVE dates AS (
    SELECT MIN(sale_date) AS date FROM sales
    UNION ALL
    SELECT DATE_ADD(date, INTERVAL 1 DAY)
    FROM dates
    WHERE date < (SELECT MAX(sale_date) FROM sales)
)
SELECT
    d.date,
    COALESCE(SUM(s.quantity * p.unit_price), 0) AS daily_sales
FROM dates d
LEFT JOIN sales s ON d.date = s.sale_date
LEFT JOIN products p ON s.product_id = p.product_id
GROUP BY d.date
UNION ALL
SELECT
    'TOTAL',
    SUM(s.quantity * p.unit_price)
FROM sales s
JOIN products p ON s.product_id = p.product_id
ORDER BY date;

-- Example 12: UNION ALL for pivot tables
-- Create a pivot-like report
SELECT
    'Total Sales' AS metric,
    SUM(CASE WHEN MONTH(sale_date) = 1 THEN quantity * unit_price END) AS Jan,
    SUM(CASE WHEN MONTH(sale_date) = 2 THEN quantity * unit_price END) AS Feb,
    SUM(CASE WHEN MONTH(sale_date) = 3 THEN quantity * unit_price END) AS Mar
FROM sales s
JOIN products p ON s.product_id = p.product_id
```

```sql
WHERE YEAR(sale_date) = 2024
UNION ALL
SELECT
    'Units Sold',
    SUM(CASE WHEN MONTH(sale_date) = 1 THEN quantity END),
    SUM(CASE WHEN MONTH(sale_date) = 2 THEN quantity END),
    SUM(CASE WHEN MONTH(sale_date) = 3 THEN quantity END)
FROM sales s
WHERE YEAR(sale_date) = 2024;

-- Example 13: UNION ALL for error logging
-- Combine error logs from multiple sources
SELECT
    'Application' AS source,
    error_time,
    error_message
FROM app_errors
WHERE error_time >= CURDATE()
UNION ALL
SELECT
    'Database',
    log_time,
    message
FROM db_logs
WHERE log_level = 'ERROR'
UNION ALL
SELECT
    'User Feedback',
    feedback_date,
    CONCAT('User reported: ', issue_description)
FROM user_feedback
WHERE issue_type = 'bug'
ORDER BY error_time DESC;

-- Example 14: UNION ALL with JSON generation
-- Generate JSON structure from multiple tables
SELECT JSON_ARRAYAGG(
    JSON_OBJECT(
        'id', id,
        'name', name,
        'type', type
    )
```

```sql
) AS json_output
FROM (
    SELECT customer_id AS id, name, 'customer' AS type FROM customers
    UNION ALL
    SELECT product_id, product_name, 'product' FROM products
    UNION ALL
    SELECT emp_id, CONCAT(first_name, ' ', last_name), 'employee' FROM employees
) AS all_entities;

-- Example 15: UNION ALL for ETL change data capture
-- Track changes across time
SELECT
    'Current' AS snapshot,
    COUNT(*) AS customer_count,
    SUM(balance) AS total_balance
FROM customers
UNION ALL
SELECT
    'Yesterday',
    COUNT(*),
    SUM(balance)
FROM customers_history
WHERE snapshot_date = DATE_SUB(CURDATE(), INTERVAL 1 DAY)
UNION ALL
SELECT
    'Last Week',
    COUNT(*),
    SUM(balance)
FROM customers_history
WHERE snapshot_date = DATE_SUB(CURDATE(), INTERVAL 7 DAY)
UNION ALL
SELECT
    'Last Month',
    COUNT(*),
    SUM(balance)
FROM customers_history
WHERE snapshot_date = DATE_SUB(CURDATE(), INTERVAL 1 MONTH);
```

---

3. INTERSECT

Returns rows that appear in both result sets.

Theory:

- Returns only common rows from both queries
- Removes duplicates
- Symmetrical (order doesn't matter)
- Not supported in MySQL (use IN or EXISTS instead)

Syntax:

sql

```sql
SELECT column1, column2 FROM table1
INTERSECT
SELECT column1, column2 FROM table2
ORDER BY column1;
```

Practical Examples (with MySQL alternatives):

sql

```sql
-- Example 1: Basic INTERSECT (PostgreSQL/SQL Server)
-- Find customers who are also employees
-- PostgreSQL/SQL Server:
-- SELECT customer_id, name FROM customers
-- INTERSECT
-- SELECT emp_id, CONCAT(first_name, ' ', last_name) FROM employees;

-- MySQL alternative using IN
SELECT c.customer_id, c.name
FROM customers c
WHERE c.customer_id IN (
    SELECT e.emp_id FROM employees e
);

-- Example 2: INTERSECT with multiple columns
-- Find matching (department, location) combinations
```

```sql
-- PostgreSQL/SQL Server:
-- SELECT dept_id, location FROM departments
-- INTERSECT
-- SELECT dept_id, 'New York' FROM departments WHERE location = 'New York';

-- MySQL alternative
SELECT DISTINCT d1.dept_id, d1.location
FROM departments d1
WHERE EXISTS (
    SELECT 1
    FROM departments d2
    WHERE d2.dept_id = d1.dept_id
    AND d2.location = d1.location
    AND d2.location = 'New York'
);

-- Example 3: Find products that have been both sold and returned
-- If we had returns table
-- PostgreSQL/SQL Server:
-- SELECT product_id FROM sales
-- INTERSECT
-- SELECT product_id FROM returns;

-- MySQL alternative
SELECT DISTINCT s.product_id
FROM sales s
WHERE s.product_id IN (
    SELECT r.product_id FROM returns r
);

-- Example 4: Find employees who are also managers
-- PostgreSQL/SQL Server:
-- SELECT emp_id FROM employees
-- INTERSECT
-- SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT NULL;

-- MySQL alternative
SELECT e.emp_id
FROM employees e
WHERE e.emp_id IN (
    SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT NULL
);
```

```sql
-- Example 5: Find dates that have both sales and new customer signups
-- PostgreSQL/SQL Server:
-- SELECT DATE(sale_date) FROM sales
-- INTERSECT
-- SELECT DATE(created_at) FROM customers;

-- MySQL alternative
SELECT DISTINCT DATE(s.sale_date) AS common_date
FROM sales s
WHERE DATE(s.sale_date) IN (
    SELECT DATE(created_at) FROM customers WHERE created_at IS NOT NULL
);

-- Example 6: Find customers who ordered in both January and February
-- Using INTERSECT conceptually
SELECT customer_id
FROM orders
WHERE MONTH(order_date) = 1
INTERSECT  -- Conceptually
SELECT customer_id
FROM orders
WHERE MONTH(order_date) = 2;

-- MySQL alternative
SELECT DISTINCT o1.customer_id
FROM orders o1
WHERE MONTH(o1.order_date) = 1
AND o1.customer_id IN (
    SELECT o2.customer_id
    FROM orders o2
    WHERE MONTH(o2.order_date) = 2
);

-- Example 7: Find products with price in top 25% and bottom 25% of sales
-- This is impossible with INTERSECT (contradictory), but showing syntax
-- SELECT product_id FROM products WHERE price > (SELECT PERCENTILE_CONT(0.75) ...
)
-- INTERSECT
-- SELECT product_id FROM products WHERE price < (SELECT PERCENTILE_CONT(0.25) ...
)
-- Returns empty set (by definition)
```

```sql
-- Example 8: Find employees who have both sales and support roles
-- If we had roles table
-- SELECT emp_id FROM employee_roles WHERE role = 'Sales'
-- INTERSECT
-- SELECT emp_id FROM employee_roles WHERE role = 'Support';

-- Example 9: Find orders that contain both Electronics and Furniture
SELECT oi1.order_id
FROM order_items oi1
JOIN products p1 ON oi1.product_id = p1.product_id
WHERE p1.category = 'Electronics'
INTERSECT  -- Conceptually
SELECT oi2.order_id
FROM order_items oi2
JOIN products p2 ON oi2.product_id = p2.product_id
WHERE p2.category = 'Furniture';

-- MySQL alternative
SELECT DISTINCT oi1.order_id
FROM order_items oi1
JOIN products p1 ON oi1.product_id = p1.product_id
WHERE p1.category = 'Electronics'
AND oi1.order_id IN (
    SELECT oi2.order_id
    FROM order_items oi2
    JOIN products p2 ON oi2.product_id = p2.product_id
    WHERE p2.category = 'Furniture'
);

-- Example 10: Find locations that have both IT and HR departments
SELECT DISTINCT d1.location
FROM departments d1
WHERE d1.dept_name = 'IT'
AND d1.location IN (
    SELECT d2.location
    FROM departments d2
    WHERE d2.dept_name = 'HR'
);

-- Example 11: INTERSECT with multiple tables
-- PostgreSQL/SQL Server:
```

```sql
-- SELECT customer_id FROM customers WHERE city = 'New York'
-- INTERSECT
-- SELECT customer_id FROM orders WHERE amount > 500
-- INTERSECT
-- SELECT customer_id FROM loyalty_program;

-- MySQL alternative
SELECT DISTINCT c.customer_id
FROM customers c
WHERE c.city = 'New York'
AND c.customer_id IN (
    SELECT o.customer_id FROM orders o WHERE o.amount > 500
)
AND c.customer_id IN (
    SELECT l.customer_id FROM loyalty_program l
);

-- Example 12: Find months that have both high and low sales
SELECT
    MONTH(sale_date) AS sale_month
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY MONTH(sale_date)
HAVING MAX(s.quantity * p.unit_price) > 1000
INTERSECT  -- Conceptually
SELECT
    MONTH(sale_date)
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY MONTH(sale_date)
HAVING MIN(s.quantity * p.unit_price) < 100;

-- MySQL alternative
SELECT DISTINCT MONTH(s1.sale_date) AS sale_month
FROM sales s1
JOIN products p1 ON s1.product_id = p1.product_id
WHERE s1.quantity * p1.unit_price > 1000
AND MONTH(s1.sale_date) IN (
    SELECT MONTH(s2.sale_date)
    FROM sales s2
    JOIN products p2 ON s2.product_id = p2.product_id
    WHERE s2.quantity * p2.unit_price < 100
```

```sql
);

-- Example 13: Find customers who bought in all quarters
SELECT customer_id
FROM orders
WHERE QUARTER(order_date) = 1
INTERSECT
SELECT customer_id
FROM orders
WHERE QUARTER(order_date) = 2
INTERSECT
SELECT customer_id
FROM orders
WHERE QUARTER(order_date) = 3
INTERSECT
SELECT customer_id
FROM orders
WHERE QUARTER(order_date) = 4;

-- MySQL alternative
SELECT o1.customer_id
FROM orders o1
WHERE QUARTER(o1.order_date) = 1
AND o1.customer_id IN (SELECT o2.customer_id FROM orders o2 WHERE QUARTER(o2.order
_date) = 2)
AND o1.customer_id IN (SELECT o3.customer_id FROM orders o3 WHERE QUARTER(o3.order
_date) = 3)
AND o1.customer_id IN (SELECT o4.customer_id FROM orders o4 WHERE QUARTER(o4.order
_date) = 4)
GROUP BY o1.customer_id;

-- Example 14: Find employees who have the same job title in different departments
SELECT e1.job_title
FROM employees e1
WHERE e1.dept_id = 1
INTERSECT
SELECT e2.job_title
FROM employees e2
WHERE e2.dept_id = 2;

-- MySQL alternative
SELECT DISTINCT e1.job_title
```

```sql
FROM employees e1
WHERE e1.dept_id = 1
AND e1.job_title IN (
    SELECT e2.job_title
    FROM employees e2
    WHERE e2.dept_id = 2
);

-- Example 15: INTERSECT with GROUP BY
-- Find categories that have both budget and premium products
SELECT p1.category
FROM products p1
WHERE p1.unit_price < 50
INTERSECT
SELECT p2.category
FROM products p2
WHERE p2.unit_price > 200;

-- MySQL alternative
SELECT DISTINCT p1.category
FROM products p1
WHERE p1.unit_price < 50
AND p1.category IN (
    SELECT p2.category
    FROM products p2
    WHERE p2.unit_price > 200
);
```

---

4. EXCEPT (or MINUS)

Returns rows from the first query that are not in the second query.

Theory:

- Returns rows in first result set but not in second
- Removes duplicates
- Order matters (A EXCEPT B ≠ B EXCEPT A)

- Called MINUS in Oracle
- Not supported in MySQL (use NOT IN or NOT EXISTS)

Syntax:

sql

```sql
SELECT column1, column2 FROM table1
EXCEPT
SELECT column1, column2 FROM table2
ORDER BY column1;
```

Practical Examples (with MySQL alternatives):

sql

```sql
-- Example 1: Basic EXCEPT
-- Find customers who are not employees
-- PostgreSQL/SQL Server:
-- SELECT customer_id, name FROM customers
-- EXCEPT
-- SELECT emp_id, CONCAT(first_name, ' ', last_name) FROM employees;

-- MySQL alternative using NOT IN
SELECT c.customer_id, c.name
FROM customers c
WHERE c.customer_id NOT IN (
    SELECT e.emp_id FROM employees e WHERE e.emp_id IS NOT NULL
);

-- Example 2: Find products that have never been sold
-- PostgreSQL/SQL Server:
-- SELECT product_id, product_name FROM products
-- EXCEPT
-- SELECT DISTINCT product_id, p.product_name FROM order_items oi
-- JOIN products p ON oi.product_id = p.product_id;

-- MySQL alternative
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.order_id IS NULL;
```

```sql
-- Or using NOT EXISTS
SELECT p.product_id, p.product_name
FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id
);

-- Example 3: Find employees who are not managers
-- PostgreSQL/SQL Server:
-- SELECT emp_id, first_name, last_name FROM employees
-- EXCEPT
-- SELECT DISTINCT manager_id, first_name, last_name FROM employees WHERE manager_
id IS NOT NULL;

-- MySQL alternative
SELECT e.emp_id, e.first_name, e.last_name
FROM employees e
WHERE e.emp_id NOT IN (
    SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT NULL
);

-- Example 4: Find customers who have not placed any orders
-- PostgreSQL/SQL Server:
-- SELECT customer_id, name FROM customers
-- EXCEPT
-- SELECT DISTINCT customer_id, c.name FROM orders o
-- JOIN customers c ON o.customer_id = c.customer_id;

-- MySQL alternative
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- Example 5: Find departments with no employees
-- PostgreSQL/SQL Server:
-- SELECT dept_id, dept_name FROM departments
-- EXCEPT
-- SELECT DISTINCT dept_id, d.dept_name FROM employees e
-- JOIN departments d ON e.dept_id = d.dept_id;
```

```sql
-- MySQL alternative
SELECT d.dept_id, d.dept_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;

-- Example 6: Find products that have been sold but are now out of stock
-- PostgreSQL/SQL Server:
-- SELECT product_id, product_name FROM products WHERE stock_quantity = 0
-- INTERSECT
-- SELECT DISTINCT product_id, p.product_name FROM order_items oi
-- JOIN products p ON oi.product_id = p.product_id;

-- This is INTERSECT, but showing EXCEPT variant:
-- Find products in stock that have never been sold
SELECT p.product_id, p.product_name
FROM products p
WHERE p.stock_quantity > 0
AND NOT EXISTS (
    SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id
);

-- Example 7: Find months with no sales
WITH RECURSIVE months AS (
    SELECT 1 AS month_num
    UNION ALL
    SELECT month_num + 1
    FROM months
    WHERE month_num < 12
)
SELECT month_num
FROM months
WHERE month_num NOT IN (
    SELECT DISTINCT MONTH(sale_date)
    FROM sales
    WHERE YEAR(sale_date) = 2024
);

-- Example 8: Find customers who ordered in January but not in February
-- PostgreSQL/SQL Server:
-- SELECT customer_id FROM orders WHERE MONTH(order_date) = 1
-- EXCEPT
```

```sql
-- SELECT customer_id FROM orders WHERE MONTH(order_date) = 2;

-- MySQL alternative
SELECT DISTINCT o1.customer_id
FROM orders o1
WHERE MONTH(o1.order_date) = 1
AND o1.customer_id NOT IN (
    SELECT o2.customer_id
    FROM orders o2
    WHERE MONTH(o2.order_date) = 2
);

-- Example 9: Find employees with salary above average but not in top 10%
SELECT e.emp_id, e.first_name, e.last_name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees)
AND e.emp_id NOT IN (
    SELECT emp_id
    FROM employees
    ORDER BY salary DESC
    LIMIT (SELECT CEIL(COUNT(*) * 0.1) FROM employees)
);

-- Example 10: Find products that are more expensive than average but have below-a
verage sales
SELECT p.product_id, p.product_name, p.unit_price
FROM products p
WHERE p.unit_price > (SELECT AVG(unit_price) FROM products)
AND p.product_id NOT IN (
    SELECT oi.product_id
    FROM order_items oi
    GROUP BY oi.product_id
    HAVING SUM(oi.quantity) > (
        SELECT AVG(product_quantity)
        FROM (
            SELECT SUM(quantity) AS product_quantity
            FROM order_items
            GROUP BY product_id
        ) AS product_sales
    )
);
```

```sql
-- Example 11: Find locations that have departments but no employees
SELECT DISTINCT d.location
FROM departments d
WHERE d.location NOT IN (
    SELECT DISTINCT d2.location
    FROM departments d2
    JOIN employees e ON d2.dept_id = e.dept_id
);

-- Example 12: Find customers who have spent money but never had a large order (>$
1000)
SELECT c.customer_id, c.name
FROM customers c
WHERE c.customer_id IN (
    SELECT DISTINCT customer_id FROM orders
)
AND c.customer_id NOT IN (
    SELECT customer_id
    FROM orders
    WHERE amount > 1000
);

-- Example 13: Find employees who have the same job title as someone in IT
-- but don't work in IT themselves
SELECT DISTINCT e1.emp_id, e1.first_name, e1.last_name, e1.job_title
FROM employees e1
WHERE e1.dept_id != 1
AND e1.job_title IN (
    SELECT DISTINCT e2.job_title
    FROM employees e2
    WHERE e2.dept_id = 1
);

-- Example 14: Find products that have been ordered by everyone except a specific
customer
SELECT p.product_id, p.product_name
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM customers c
    WHERE c.customer_id = 101
    AND NOT EXISTS (
```

```sql
        SELECT 1
        FROM orders o
        JOIN order_items oi ON o.order_id = oi.order_id
        WHERE o.customer_id = c.customer_id
        AND oi.product_id = p.product_id
    )
);

-- Example 15: Complex EXCEPT with multiple conditions
-- Find customers who:
-- - Have placed orders
-- - Live in New York
-- - Have never ordered Electronics
-- - Have spent more than $500 total
SELECT c.customer_id, c.name
FROM customers c
WHERE c.city = 'New York'
AND c.customer_id IN (
    SELECT o.customer_id
    FROM orders o
    GROUP BY o.customer_id
    HAVING SUM(o.amount) > 500
)
AND c.customer_id NOT IN (
    SELECT DISTINCT o2.customer_id
    FROM orders o2
    JOIN order_items oi ON o2.order_id = oi.order_id
    JOIN products p ON oi.product_id = p.product_id
    WHERE p.category = 'Electronics'
);
```

Summary: Set Operations Comparison

Operation	Description	Duplicates	Performance	MySQL Support
UNION	Combines, removes duplicates	Removed	Slower	Yes
UNION ALL	Combines, keeps duplicates	Kept	Faster	Yes
INTERSECT	Common rows only	Removed	Medium	No (use IN/EXISTS)
EXCEPT	Rows in first not in second	Removed	Medium	No (use NOT IN/EXISTS)

Performance Tips:

1. Use UNION ALL when duplicates are acceptable
2. For INTERSECT/EXCEPT in MySQL, use EXISTS for better performance with large datasets
3. Always check for NULLs when using NOT IN
4. Consider query optimization and indexes on columns used in set operations
5. Set operations can be combined with other clauses (WHERE, GROUP BY, ORDER BY)

Indexing and Database Normalization: Complete Theory Notes (Continued)

---

2. Clustered Index (Continued)

Practical Example of Clustered Index Behavior

sql

```sql
-- Create table with clustered index on employee_id
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,  -- This creates a clustered index
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    dept_id INT,
    salary DECIMAL(10,2),
    hire_date DATE
);

-- Insert data in random order
INSERT INTO employees VALUES
(105, 'Charlie', 'Wilson', 1, 90000, '2018-09-05'),
(101, 'John', 'Doe', 1, 75000, '2020-01-15'),
(103, 'Bob', 'Johnson', 1, 82000, '2021-06-10'),
(102, 'Jane', 'Smith', 2, 65000, '2019-03-20'),
(104, 'Alice', 'Brown', 3, 70000, '2020-11-01');

-- Despite inserting in random order, data is physically stored sorted by emp_id
SELECT * FROM employees;
-- Returns:
-- 101 John Doe ...
-- 102 Jane Smith ...
-- 103 Bob Johnson ...
-- 104 Alice Brown ...
-- 105 Charlie Wilson ...
-- The data comes out sorted because that's how it's stored!
```

When to Use Clustered Index

Good For	Not Good For
Primary key lookups	Columns with frequent updates
Range queries (BETWEEN, >, <)	Wide keys (large columns)
Sorting by indexed column	Columns with many duplicates
Retrieving consecutive rows	Frequently updated columns

---

3. Non-clustered Index

Definition

A non-clustered index is a separate structure from the data table that contains index key values and pointers to the actual data rows. A table can have multiple non-clustered indexes (up to 999 in SQL Server).

Key Characteristics

Feature	Description
Physical Order	Separate structure, doesn't affect data storage
Count per Table	Multiple (typically up to 999)
Leaf Nodes	Contain index keys + pointers to data rows

Feature	Description
Storage	Requires additional disk space
Speed	Fast for lookups, but slower than clustered (requires pointer lookup)

Visual Representation

text

```
Non-clustered Index on last_name:

Index Structure:                    Actual Data Table:
┌──────────────────┐                ┌──────────────────────────┐
│ Root Page        │                │ emp_id  last_name  ...   │
│ Brown  → RID4    │                │ 101     Doe              │
│ Davis  → RID7    │                │ 102     Smith            │
│ Doe    → RID1    │                │ 103     Johnson          │
│ Ford   → RID10   │                │ 104     Brown  ←─────────┐
│ Johnson→ RID3    │                │ 105     Wilson           │
│ Lee    → RID9    │                │ 106     Miller           │
│ Miller → RID6    │                │ 107     Davis  ←──────┐  │
│ Smith  → RID2    │                │ 108     Thomas        │  │
│ Thomas → RID8    │                │ 109     Lee    ←───┐  │  │
│ Wilson → RID5    │                │ 110     Ford   ←───┐│  │  │
└──────────────────┘                └────────────────────┼┼┼──┘
                                                         │||
Process for finding "Davis":                            |||
1. Search index for "Davis" → found at RID7 ────────────┼┼┘
2. Use RID7 (Row Identifier) to locate actual row ──────┼┘
3. Retrieve the full row data ──────────────────────────┘
```

Example: Creating and Using Non-clustered Index

sql

```sql
-- Create table
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,  -- Clustered index
    first_name VARCHAR(50),
```

```sql
    last_name VARCHAR(50),
    email VARCHAR(100),
    dept_id INT,
    salary DECIMAL(10,2)
);

-- Create non-clustered indexes
CREATE INDEX idx_last_name ON employees(last_name);
CREATE INDEX idx_dept_salary ON employees(dept_id, salary);
CREATE UNIQUE INDEX idx_email ON employees(email);

-- Query that uses the index
SELECT emp_id, first_name, last_name
FROM employees
WHERE last_name = 'Smith';
-- Process:
-- 1. Database searches idx_last_name for 'Smith'
-- 2. Finds pointer to data row
-- 3. Retrieves the row using the pointer

-- Check if index is being used
EXPLAIN SELECT * FROM employees WHERE last_name = 'Smith';
-- Will show "index scan" or "index seek" instead of "table scan"
```

Clustered vs Non-clustered Comparison

Aspect	Clustered Index	Non-clustered Index
Storage	Data is the index	Separate structure
Count	One per table	Multiple per table
Speed	Fastest for retrieval	Fast, but extra step
Disk Space	No extra space	Requires additional space
Insert/Update	Can be slow (reordering)	Faster maintenance

Aspect	Clustered Index	Non-clustered Index
Leaf Nodes	Contain actual data	Contain pointers to data

---

4. Composite Index

Definition

A composite index (also called multi-column index) is an index created on multiple columns of a table. The column order matters significantly.

Key Concepts

- **Column Order**: The leftmost columns are most important
- **Prefix Matching**: Can use leftmost columns without using all
- **Covering Index**: If index contains all needed columns, no table access needed

Visual Representation

```text
Composite Index on (last_name, first_name):

Index Structure:
┌──────────────────────────────────┐
│ Adams, John    → Pointer to Row 5  │
│ Adams, Robert  → Pointer to Row 12 │
│ Brown, Alice   → Pointer to Row 3  │
│ Brown, Charles → Pointer to Row 8  │
│ Davis, Mary    → Pointer to Row 1  │
│ Davis, Michael → Pointer to Row 15 │
│ Smith, Jane    → Pointer to Row 2  │
│ Smith, John    → Pointer to Row 7  │
│ Smith, Robert  → Pointer to Row 10 │
```

```
└──────────────────────────────────────┘

Search patterns:
- WHERE last_name = 'Smith' → USES INDEX (full scan of Smith entries)
- WHERE last_name = 'Smith' AND first_name = 'John' → USES INDEX (direct hit)
- WHERE first_name = 'John' → CANNOT USE INDEX (need last_name first)
```

Examples of Composite Index Usage

sql

```sql
-- Create composite indexes
CREATE INDEX idx_name ON employees(last_name, first_name);
CREATE INDEX idx_dept_salary ON employees(dept_id, salary);
CREATE INDEX idx_location ON employees(city, state, zip);

-- Queries that can use these indexes
-- 1. Uses full index
SELECT * FROM employees
WHERE last_name = 'Smith' AND first_name = 'John';

-- 2. Uses leftmost column only
SELECT * FROM employees
WHERE last_name = 'Smith';  -- Still uses index efficiently

-- 3. Uses dept_id for filtering and salary for sorting
SELECT * FROM employees
WHERE dept_id = 1
ORDER BY salary;

-- 4. Uses dept_id and salary range
SELECT * FROM employees
WHERE dept_id = 1 AND salary > 70000;

-- Queries that CANNOT use this index effectively
-- 5. Missing leftmost column
SELECT * FROM employees
WHERE first_name = 'John';  -- Cannot use idx_name

-- 6. Skipped column in middle
SELECT * FROM employees
WHERE last_name = 'Smith' AND city = 'NYC';  -- Uses only last_name part
```

```sql
-- 7. Different order
SELECT * FROM employees
WHERE first_name = 'John' AND last_name = 'Smith';
-- Can still use index (optimizer reorders conditions)
```

Composite Index Design Guidelines

Index Columns	Good For	Example
Equality first, then range	Columns with =, then >, <	(dept_id, salary)
High cardinality first	Most unique values first	(email, last_name
Frequently used together	Common query patterns	(city, state)
Covering queries	All needed columns in index	(id, name, status)

---

5. When to Use Indexes

Golden Rules for Index Creation

DO Create Indexes When:

1. **Columns used in WHERE clause frequently**
   sql
   ```sql
   -- Good candidate for index
   SELECT * FROM orders WHERE customer_id = 123;
   -- Create: CREATE INDEX idx_customer ON orders(customer_id);
   ```

2. **Columns used in JOIN conditions**
   sql

```sql
-- Good candidate for index
SELECT * FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
-- Create: CREATE INDEX idx_dept_id ON employees(dept_id);
```

3. **Columns used in ORDER BY**

```sql
-- Good candidate for index
SELECT * FROM employees ORDER BY hire_date DESC;
-- Create: CREATE INDEX idx_hire_date ON employees(hire_date);
```

4. **Columns used in GROUP BY**

```sql
-- Good candidate for index
SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id;
-- Create: CREATE INDEX idx_dept_id ON employees(dept_id);
```

5. **Columns with high cardinality** (many unique values)

```sql
-- Good: email has many unique values
CREATE INDEX idx_email ON users(email);

-- Not as good: gender has only 2-3 unique values
-- Index on gender rarely helps
```

6. **Foreign key columns**

```sql
-- Always index foreign keys
CREATE INDEX idx_customer_id ON orders(customer_id);
```

Index Selection Guide

Query Pattern	Index Recommendation
`WHERE col = value`	Single-column index on col
`WHERE col1 = v1 AND col2 = v2`	Composite index on (col1, col2)

Query Pattern	Index Recommendation
`WHERE col1 = v1 OR col2 = v2`	Two separate indexes
`WHERE col1 = v1 ORDER BY col2`	Composite index on (col1, col2)
`WHERE col1 > v1`	Single-column index on col1
`WHERE col1 IN (v1, v2, v3)`	Single-column index on col1
`JOIN ON table1.col = table2.col`	Index on both columns

---

6. When Indexes Hurt Performance

Index Overhead and Negative Impacts

1. Slow INSERT Operations

sql

```sql
-- Table with 5 indexes
CREATE TABLE users (
    id INT PRIMARY KEY,           -- 1 index
    email VARCHAR(100) UNIQUE,     -- 2nd index
    username VARCHAR(50) UNIQUE,   -- 3rd index
    created_at TIMESTAMP,
    status VARCHAR(20),
    last_login TIMESTAMP
);

CREATE INDEX idx_created ON users(created_at);      -- 4th index
CREATE INDEX idx_status ON users(status);           -- 5th index
```

```sql
-- Every INSERT must now update:
-- 1. The table itself
-- 2. Primary key index
-- 3. Unique email index
-- 4. Unique username index
-- 5. Created_at index
-- 6. Status index
-- That's 6 writes instead of 1!
```

2. Slow UPDATE Operations

sql

```sql
-- Updating an indexed column
UPDATE users
SET email = 'newemail@example.com'
WHERE id = 100;

-- This must:
-- 1. Update the table data
-- 2. Update the email index (delete old, insert new)
-- 3. Primary key index unchanged (good)
-- 4. Other indexes unchanged

-- Updating multiple indexed columns is even worse
UPDATE users
SET email = 'new@example.com',
    username = 'newuser',
    status = 'active'
WHERE id = 100;
-- Updates 3 different indexes!
```

3. Slow DELETE Operations

sql

```sql
-- Deleting a row with many indexes
DELETE FROM users WHERE id = 100;
-- Must remove entry from ALL indexes
-- More indexes = slower delete
```

4. Storage Overhead

```text
Table Size: 100 MB
Indexes:
- Primary key: 20 MB
- Email unique: 15 MB
- Username unique: 15 MB
- Created_at: 12 MB
- Status: 8 MB
Total with indexes: 170 MB (70% overhead!)
```

5. Index Maintenance During Bulk Operations

```sql
-- Bulk insert of 10,000 rows
INSERT INTO logs (message, created_at)
SELECT 'Log message', NOW()
FROM source_table
LIMIT 10000;

-- If logs has 5 indexes, this becomes 50,000 index writes!
-- Better to drop indexes before bulk insert, then recreate
```

Signs of Index Problems

Symptom	Likely Cause
INSERTs getting slower over time	Too many indexes
UPDATEs on certain columns slow	Those columns are heavily indexed
Disk space growing fast	Unused or duplicate indexes
Query optimizer ignores your index	Outdated statistics or wrong index choice

Index Maintenance Best Practices

1. **Drop unused indexes**
   sql

```sql
-- Find unused indexes (MySQL)
SELECT * FROM sys.schema_unused_indexes;

-- Drop if not needed
DROP INDEX idx_unused ON table_name;
```

2. **Avoid over-indexing**
   sql

```sql
-- BAD: Too many single-column indexes
CREATE INDEX idx_dept ON employees(dept_id);
CREATE INDEX idx_salary ON employees(salary);
CREATE INDEX idx_status ON employees(status);
CREATE INDEX idx_hire ON employees(hire_date);

-- GOOD: Composite indexes for common queries
CREATE INDEX idx_dept_salary ON employees(dept_id, salary);
-- Single indexes only on frequently used columns
```

3. **Update statistics regularly**
   sql

```sql
-- MySQL
ANALYZE TABLE employees;

-- PostgreSQL
ANALYZE employees;

-- SQL Server
UPDATE STATISTICS employees;
```

4. **Rebuild fragmented indexes**
   sql

```sql
-- MySQL
ALTER TABLE employees ENGINE=InnoDB;

-- SQL Server
ALTER INDEX ALL ON employees REBUILD;
```

Index Decision Matrix

Table Size	Read/Write Ratio	Index Recommendation
Small (<1000 rows)	Any	Minimal indexes, full scans are fine
Medium (1K-100K)	Mostly reads	Index important WHERE columns
Medium (1K-100K)	Mostly writes	Index only essential columns
Large (>100K)	Mostly reads	Comprehensive indexing strategy
Large (>100K)	Mostly writes	Careful index selection, batch operations
Very Large (>1M)	Any	Partitioning + strategic indexes

Part 2: Database Normalization

Database normalization is the process of organizing data to reduce redundancy and improve integrity.

What is Normalization?

Definition

Normalization is a systematic approach of decomposing tables to eliminate data redundancy and avoid anomalies during data operations.

Goals of Normalization

1. **Eliminate redundant data** (save storage, ensure consistency)
2. **Ensure data dependencies make sense** (only store related data together)
3. **Avoid update anomalies** (update once, not in multiple places)
4. **Avoid insert anomalies** (can't insert without other data)
5. **Avoid delete anomalies** (don't lose unrelated data when deleting)

The Normal Forms

```text
Normal Form Progression:
    1NF
     ↓
    2NF (must be in 1NF)
     ↓
    3NF (must be in 2NF)
     ↓
    BCNF (must be in 3NF)
     ↓
    4NF, 5NF (rarely needed)
```

---

1. First Normal Form (1NF)

Definition

A table is in 1NF if:

1. **Atomic values**: Each column contains a single value (no arrays, lists, or sets)
2. **Unique column names**: No repeating groups of columns
3. **Unique rows**: Each row is unique (has a primary key)

Violation Example

sql

```sql
-- VIOLATES 1NF: Multiple values in one column
CREATE TABLE student_courses_bad (
    student_id INT,
    student_name VARCHAR(50),
    courses VARCHAR(200)  -- Stores "Math,Physics,Chemistry" - NOT ATOMIC!
);

-- VIOLATES 1NF: Repeating columns
CREATE TABLE student_courses_bad2 (
    student_id INT,
    student_name VARCHAR(50),
    course1 VARCHAR(50),
    course2 VARCHAR(50),
    course3 VARCHAR(50)  -- Repeating groups - BAD DESIGN!
);
```

1NF Compliant Design

sql

```sql
-- GOOD: 1NF Compliant
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50) NOT NULL
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL
);

CREATE TABLE student_enrollment (
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

Visualization of 1NF Transformation

```text
BEFORE (Violates 1NF):
┌──────────────┬──────────────────────────────┐
│ student_id   │ courses                      │
├──────────────┼──────────────────────────────┤
│ 1            │ Math, Physics, Chemistry     │ ← Multiple values
│ 2            │ Biology, Math                │ ← Multiple values
└──────────────┴──────────────────────────────┘

AFTER (1NF Compliant):
┌──────────────┬──────────────┐
│ student_id   │ course_name  │
├──────────────┼──────────────┤
│ 1            │ Math         │ ← Atomic values
│ 1            │ Physics      │ ← Atomic values
│ 1            │ Chemistry    │ ← Atomic values
│ 2            │ Biology      │ ← Atomic values
│ 2            │ Math         │ ← Atomic values
└──────────────┴──────────────┘
```

---

2. Second Normal Form (2NF)

Definition

A table is in 2NF if:

1. It is in 1NF
2. **No partial dependency**: All non-key columns must depend on the ENTIRE primary key (not just part of it)

Prerequisite

2NF only applies to tables with composite primary keys. Tables with single-column primary keys are automatically in 2NF if they're in 1NF.

Violation Example

sql

```sql
-- VIOLATES 2NF: Partial dependencies
CREATE TABLE order_details_bad (
    order_id INT,
    product_id INT,
    product_name VARCHAR(100),    -- Depends only on product_id, not on order_id
    quantity INT,                  -- Depends on both (order_id, product_id)
    unit_price DECIMAL(10,2),      -- Depends only on product_id, not on order_id
    order_date DATE,                -- Depends only on order_id, not on product_id
    customer_name VARCHAR(100),     -- Depends only on order_id, not on product_id
    PRIMARY KEY (order_id, product_id)
);

-- Problems:
-- product_name depends only on product_id (partial dependency)
-- unit_price depends only on product_id (partial dependency)
-- order_date depends only on order_id (partial dependency)
-- customer_name depends only on order_id (partial dependency)
-- This causes redundancy and update anomalies
```

Visualization of Partial Dependency

text

```text
Composite PK: (order_id, product_id)

┌──────────┬──────────┬──────────────┬──────────┬──────────┬──────────┐
│ order_id │ product_id│ product_name │ quantity │ order_date │ customer │
├──────────┼──────────┼──────────────┼──────────┼──────────┼──────────┤
│ 101      │ 1        │ Laptop       │ 2        │ 2024-01-15 │ John Doe │
│ 101      │ 2        │ Mouse        │ 5        │ 2024-01-15 │ John Doe │ ← Re
dundant
│ 102      │ 1        │ Laptop       │ 1        │ 2024-01-16 │ Jane Smith │
```

```
│ 102       │ 3          │ Keyboard       │ 1         │ 2024-01-16 │ Jane Smith │ ← Re
dundant
└──────────┴──────────┴────────────────┴──────────┴────────────┴────────────┘

Dependencies:
- product_name → product_id (partial)
- order_date, customer → order_id (partial)
- quantity → (order_id, product_id) (full)
```

2NF Compliant Design

sql

```sql
-- GOOD: 2NF Compliant - Split into multiple tables

-- Orders table (depends on order_id only)
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    order_date DATE NOT NULL,
    customer_name VARCHAR(100) NOT NULL
);

-- Products table (depends on product_id only)
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL
);

-- Order items table (depends on entire composite key)
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Visualization of 2NF Transformation

text

```
BEFORE (2NF Violation):
One big table with redundancy

AFTER (2NF Compliant):

orders table:
┌──────────┬────────────┬──────────┐
│ order_id │ order_date │ customer │
├──────────┼────────────┼──────────┤
│ 101      │ 2024-01-15 │ John Doe │
│ 102      │ 2024-01-16 │ Jane Smith │
└──────────┴────────────┴──────────┘

products table:
┌──────────┬──────────┬──────────┐
│ product_id│ product_name │ unit_price │
├──────────┼──────────┼──────────┤
│ 1        │ Laptop   │ 999.99   │
│ 2        │ Mouse    │ 29.99    │
│ 3        │ Keyboard │ 89.99    │
└──────────┴──────────┴──────────┘

order_items table:
┌──────────┬──────────┬──────────┐
│ order_id │ product_id│ quantity │
├──────────┼──────────┼──────────┤
│ 101      │ 1        │ 2        │
│ 101      │ 2        │ 5        │
│ 102      │ 1        │ 1        │
│ 102      │ 3        │ 1        │
└──────────┴──────────┴──────────┘

No more redundancy!
```

---

3. Third Normal Form (3NF)

Definition

A table is in 3NF if:

1. It is in 2NF
2. **No transitive dependency**: Non-key columns must not depend on other non-key columns

Transitive Dependency Explained

```text
Transitive dependency: A → B → C
If A determines B, and B determines C, then C transitively depends on A

Example:
employee_id → department_id → department_location
- employee_id determines department_id
- department_id determines department_location
- Therefore, department_location transitively depends on employee_id
```

Violation Example

```sql
-- VIOLATES 3NF: Transitive dependencies
CREATE TABLE employees_bad (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100),
    dept_id INT,
    dept_name VARCHAR(50),        -- Depends on dept_id, not directly on emp_id
    dept_location VARCHAR(50),     -- Depends on dept_id, not directly on emp_id
    manager_name VARCHAR(100)      -- Depends on dept_id, not directly on emp_id
);

-- Problems:
-- dept_name depends on dept_id (transitive)
-- dept_location depends on dept_id (transitive)
-- manager_name depends on dept_id (transitive)
```

```sql
-- If department changes location, must update all employees!
```

Visualization of Transitive Dependency

```text
Table with transitive dependencies:

┌─────────┬──────────┬─────────┬──────────┬───────────────┬──────────────┐
│ emp_id  │ emp_name │ dept_id │ dept_name │ dept_location │ manager_name │
├─────────┼──────────┼─────────┼──────────┼───────────────┼──────────────┤
│ 1       │ John     │ 10      │ IT       │ New York      │ Alice        │
│ 2       │ Jane     │ 10      │ IT       │ New York      │ Alice        │ ← Redun
dant
│ 3       │ Bob      │ 10      │ IT       │ New York      │ Alice        │ ← Redun
dant
│ 4       │ Alice    │ 20      │ HR       │ London        │ Bob          │
│ 5       │ Charlie  │ 20      │ HR       │ London        │ Bob          │ ← Redun
dant
└─────────┴──────────┴─────────┴──────────┴───────────────┴──────────────┘

Dependency chain:
emp_id → dept_id → (dept_name, dept_location, manager_name)
```

3NF Compliant Design

```sql
-- GOOD: 3NF Compliant - Remove transitive dependencies

-- Employees table (depends directly on emp_id)
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(100) NOT NULL,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

-- Departments table (department attributes depend on dept_id)
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL,
```

```sql
    dept_location VARCHAR(50),
    manager_id INT,
    FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
);

-- Now department information is stored once, not repeated for each employee
```

Visualization of 3NF Transformation

```text
BEFORE (3NF Violation):
One table with transitive dependencies

AFTER (3NF Compliant):

employees table:
┌─────────┬──────────┬─────────┐
│ emp_id │ emp_name │ dept_id │
├─────────┼──────────┼─────────┤
│ 1      │ John     │ 10      │
│ 2      │ Jane     │ 10      │
│ 3      │ Bob      │ 10      │
│ 4      │ Alice    │ 20      │
│ 5      │ Charlie  │ 20      │
└─────────┴──────────┴─────────┘

departments table:
┌─────────┬──────────┬─────────────┬──────────┐
│ dept_id │ dept_name │ dept_location │ manager_id │
├─────────┼──────────┼─────────────┼──────────┤
│ 10      │ IT       │ New York    │ 3        │
│ 20      │ HR       │ London      │ 4        │
└─────────┴──────────┴─────────────┴──────────┘

No more redundancy! Department info stored once.
```

---

4. Boyce-Codd Normal Form (BCNF)

Definition

BCNF is a stricter version of 3NF. A table is in BCNF if:

1. It is in 3NF
2. **Every determinant must be a candidate key** (for every functional dependency X → Y, X must be a superkey)

BCNF vs 3NF

- 3NF allows some dependencies where the left side is not a key, as long as the right side is a prime attribute
- BCNF eliminates all such exceptions

Violation Example

sql

```sql
-- VIOLATES BCNF (but might satisfy 3NF)
CREATE TABLE student_subject_teacher (
    student_id INT,
    subject_id INT,
    teacher_id INT,
    PRIMARY KEY (student_id, subject_id),
    UNIQUE (teacher_id, subject_id)  -- Each teacher teaches one subject
);

-- Functional dependencies:
-- 1. (student_id, subject_id) → teacher_id (from PK)
-- 2. teacher_id → subject_id (from unique constraint)
--    (This is the problem: teacher_id determines subject_id, but teacher_id is no
t a superkey)

-- Problems:
-- If a teacher changes subjects, we must update multiple rows
-- teacher_id is not a candidate key (can have multiple rows with same teacher_id)
```

Visualization of BCNF Violation

```text
Table: student_subject_teacher

┌──────────────┬────────────┬──────────────┐
│ student_id │ subject_id│ teacher_id │
├──────────────┼────────────┼──────────────┤
│ 1          │ Math       │ 101          │
│ 2          │ Math       │ 101          │ ← Teacher 101 teaches Math
│ 3          │ Physics    │ 102          │
│ 4          │ Physics    │ 102          │ ← Teacher 102 teaches Physics
│ 5          │ Math       │ 101          │
│ 6          │ Chemistry  │ 103          │
└──────────────┴────────────┴──────────────┘

Functional Dependency: teacher_id → subject_id
- teacher_id = 101 always means subject_id = Math
- teacher_id = 102 always means subject_id = Physics
- But teacher_id is not a candidate key (repeats)

Update anomaly: If teacher 101 switches to teaching Physics,
we must update ALL rows with teacher_id = 101!
```

BCNF Compliant Design

```sql
-- GOOD: BCNF Compliant - Split into multiple tables

-- Teachers and their subjects
CREATE TABLE teachers (
    teacher_id INT PRIMARY KEY,
    teacher_name VARCHAR(100),
    subject_id INT NOT NULL,
    FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
);

-- Subjects
CREATE TABLE subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(50)
```

```sql
);

-- Student subject enrollment
CREATE TABLE student_subject (
    student_id INT,
    subject_id INT,
    PRIMARY KEY (student_id, subject_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
);

-- Now teacher_id determines subject_id only in teachers table,
-- and the relationship is properly modeled
```

Visualization of BCNF Transformation

```text
BEFORE (BCNF Violation):

student_subject_teacher table:
┌──────────────┬────────────┬──────────────┐
│ student_id   │ subject_id│ teacher_id   │
├──────────────┼────────────┼──────────────┤
│ 1            │ Math       │ 101          │
│ 2            │ Math       │ 101          │
│ 3            │ Physics    │ 102          │
│ 4            │ Physics    │ 102          │
└──────────────┴────────────┴──────────────┘

AFTER (BCNF Compliant):

teachers table:
┌──────────────┬────────────────┬────────────┐
│ teacher_id   │ teacher_name   │ subject_id│
├──────────────┼────────────────┼────────────┤
│ 101          │ Mr. Smith      │ Math       │
│ 102          │ Ms. Jones      │ Physics    │
└──────────────┴────────────────┴────────────┘

student_subject table:
┌──────────────┬────────────┐
```

```
│ student_id │ subject_id│
├────────────┼──────────┤
│ 1          │ Math     │
│ 2          │ Math     │
│ 3          │ Physics  │
│ 4          │ Physics  │
└────────────┴──────────┘
```

```
Now teacher_id → subject_id exists in teachers table,
and teacher_id is a primary key there.
```

---

5. Denormalization

Definition

Denormalization is the process of intentionally introducing redundancy into a database by combining tables or adding redundant columns to improve read performance.

Why Denormalize?

- **Performance**: Reduce JOINs for frequently accessed queries
- **Simplicity**: Simplify complex queries for reporting
- **Aggregation**: Pre-calculate and store aggregate values

When to Denormalize

Scenario	Denormalization Strategy
Frequent reporting queries with many JOINs	Combine related tables
Calculated fields used often	Store pre-calculated values

Scenario	Denormalization Strategy
Read-heavy systems	Add redundancy for faster reads
Data warehouse/OLAP	Star/snowflake schemas

Denormalization Examples

Example 1: Adding Redundant Columns

sql

```sql
-- Normalized Design
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE
);

CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100),
    customer_email VARCHAR(100)
);

-- Query needs JOIN every time
SELECT o.order_id, o.order_date, c.customer_name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;

-- Denormalized Design (add redundant column)
CREATE TABLE orders_denormalized (
    order_id INT PRIMARY KEY,
    customer_id INT,
    customer_name VARCHAR(100),  -- Redundant, copied from customers
    order_date DATE
);

-- Now query is simpler and faster (no JOIN)
SELECT order_id, order_date, customer_name
```

```sql
FROM orders_denormalized;
```

Example 2: Pre-calculated Aggregates

sql

```sql
-- Normalized: Calculate on the fly
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.quantity * oi.price) AS total_revenue
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name;

-- Denormalized: Store pre-calculated values
ALTER TABLE products ADD COLUMN total_revenue DECIMAL(15,2) DEFAULT 0;

-- Update periodically (daily batch job)
UPDATE products p
SET total_revenue = (
    SELECT SUM(oi.quantity * oi.price)
    FROM order_items oi
    WHERE oi.product_id = p.product_id
);

-- Now queries are instant
SELECT product_id, product_name, total_revenue
FROM products;
```

Example 3: Combining Tables for Reporting

sql

```sql
-- Normalized Star Schema (Data Warehouse)
CREATE TABLE fact_sales (
    sale_id INT PRIMARY KEY,
    date_id INT,
    product_id INT,
    customer_id INT,
    quantity INT,
    price DECIMAL
);
```

```sql
CREATE TABLE dim_date (...);
CREATE TABLE dim_product (...);
CREATE TABLE dim_customer (...);

-- Reporting query needs 4 JOINs
SELECT ... FROM fact_sales f
JOIN dim_date d ON f.date_id = d.date_id
JOIN dim_product p ON f.product_id = p.product_id
JOIN dim_customer c ON f.customer_id = c.customer_id;

-- Denormalized: Single wide table for specific report
CREATE TABLE monthly_sales_report (
    report_date DATE,
    product_name VARCHAR(100),
    product_category VARCHAR(50),
    customer_segment VARCHAR(50),
    total_quantity INT,
    total_revenue DECIMAL,
    -- All needed fields in one table
    -- No JOINs needed for reporting
);
```

Denormalization Trade-offs

Advantage	Disadvantage
Faster reads (fewer JOINs)	Slower writes (more updates)
Simpler queries	Data redundancy
Pre-calculated values	Risk of inconsistency
Better for reporting	More storage space
Reduced query complexity	Update anomalies possible

Common Denormalization Patterns

1. **Pre-joined Tables**

```text
Normalized:        Denormalized:
customers ─┐       customer_orders
           └─┐     - customer_id
orders    ──┼─┐    - customer_name
            │ │    - order_id
order_items ─┘ │   - order_date
              │   - product_name
products ──────┘   - quantity
                   - price
All data in one table for faster reads
```

2. **Pre-calculated Summary Tables**

```sql
-- Daily summary table
CREATE TABLE daily_sales_summary (
    sale_date DATE PRIMARY KEY,
    total_orders INT,
    total_revenue DECIMAL,
    unique_customers INT,
    avg_order_value DECIMAL
);

-- Updated nightly from detailed tables
-- Reporting queries use this instead of scanning millions of rows
```

3. **Vertical Partitioning (Splitting wide tables)**

```sql
-- Original wide table
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100),
    -- Frequently accessed
    department VARCHAR(50),
    salary DECIMAL,
    -- Rarely accessed
    bio TEXT,
    profile_picture BLOB,
```

```sql
    tax_documents TEXT
);

-- Denormalized by splitting into hot and cold tables
CREATE TABLE employees_main (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50),
    salary DECIMAL
);

CREATE TABLE employees_details (
    emp_id INT PRIMARY KEY,
    bio TEXT,
    profile_picture BLOB,
    tax_documents TEXT,
    FOREIGN KEY (emp_id) REFERENCES employees_main(emp_id)
);
```

Denormalization Guidelines

Do Denormalize When	Don't Denormalize When
Read performance is critical	Write performance is critical
Data is mostly static	Data changes frequently
Queries are predictable	Data must be strictly consistent
Storage is cheap	Storage is limited
Reporting/analytics use	OLTP transactional systems
Can manage consistency via application	Can't ensure data integrity

---

Summary: Normalization Cheat Sheet

Normal Form	Rule	Example Violation
1NF	Atomic values, no repeating groups	Column with comma-separated va
2NF	No partial dependencies (with composite keys)	Product name in order details
3NF	No transitive dependencies	Department location in employee
BCNF	Every determinant must be a candidate key	Teacher determines subject in enr

Normalization Levels Decision Guide

text

```
For OLTP (Transaction Processing):
Aim for 3NF or BCNF
- Many small, fast transactions
- Data integrity critical
- Minimize redundancy

For OLAP (Reporting/Analytics):
Consider denormalization
- Complex read queries
- Periodic batch updates
- Performance over strict normalization

Typical Enterprise Database:
- OLTP systems: 3NF/BCNF
- Reporting views: Denormalized
- Data warehouse: Star schema (denormalized)
- Caches: Highly denormalized
```

Transactions and Views: Complete Theory Notes

Part 1: Transactions

A transaction is a unit of work that is performed against a database. Transactions are used to ensure data integrity and consistency.

---

1. ACID Properties

ACID is an acronym that describes the four essential properties of a database transaction.

Definition

ACID properties ensure that database transactions are processed reliably and maintain data integrity even in case of system failures, crashes, or errors.

The Four ACID Properties

Property	Full Name	Description
A	Atomicity	All or nothing - transaction completes fully or not at all
C	Consistency	Data follows all rules and constraints after transaction
I	Isolation	Concurrent transactions don't interfere with each other
D	Durability	Committed data persists even after system failure

---

Atomicity (All or Nothing)

Definition: A transaction is treated as a single, indivisible unit. Either all of its operations are successfully completed, or none of them are applied.

Real-World Analogy: ATM Transfer

text

```text
You want to transfer $100 from Savings to Checking:
1. Debit $100 from Savings
2. Credit $100 to Checking

Atomicity ensures:
- BOTH operations succeed → Transfer complete
- If credit fails → Debit is rolled back (money returns to Savings)
- NEVER a situation where money is deducted but not added
```

Visual Representation:

text

```text
BEGIN TRANSACTION
    ├── Step 1: UPDATE savings SET balance = balance - 100
    ├── Step 2: UPDATE checking SET balance = balance + 100
    └── If both succeed → COMMIT
        If any fails → ROLLBACK

Result: Either both happen, or neither happens!
```

Example:

sql

```sql
-- Atomic transaction example
BEGIN TRANSACTION;

-- Take money from savings
UPDATE accounts
SET balance = balance - 100
WHERE account_id = 'SAVINGS-123' AND balance >= 100;

-- Check if sufficient funds
IF ROW_COUNT() = 0 THEN
    ROLLBACK;  -- Not enough money, abort everything
```

```sql
    SELECT 'Insufficient funds';
ELSE
    -- Add money to checking
    UPDATE accounts
    SET balance = balance + 100
    WHERE account_id = 'CHECKING-123';

    COMMIT;  -- Both operations succeed together
    SELECT 'Transfer complete';
END IF;
```

Consistency (Data Integrity)

Definition: A transaction brings the database from one valid state to another, maintaining all predefined rules, constraints, and relationships.

What Consistency Ensures:

- All constraints are enforced (Primary Key, Foreign Key, Check, Unique)
- Data types are respected
- Triggers fire correctly
- No orphaned records

Visual Representation:

```text
Before Transaction:           After Transaction (Valid):
┌──────────────────┐          ┌──────────────────┐
│ Accounts         │          │ Accounts         │
│ ID: 1, Bal: 500  │          │ ID: 1, Bal: 400  │
│ ID: 2, Bal: 300  │          │ ID: 2, Bal: 400  │
└──────────────────┘          └──────────────────┘
Total: 800                   Total: 800 (preserved)

Rules maintained:
- No negative balances (CHECK constraint)
- Total money conserved
```

- Valid account IDs (PK constraint)

Example with Constraints:

sql

```sql
-- Create table with constraints
CREATE TABLE accounts (
    account_id INT PRIMARY KEY,
    account_holder VARCHAR(100) NOT NULL,
    balance DECIMAL(10,2) CHECK (balance >= 0),  -- No negative balances
    account_type VARCHAR(20) CHECK (account_type IN ('SAVINGS', 'CHECKING'))
);

-- Consistent transaction
BEGIN TRANSACTION;

-- This UPDATE will be checked against constraints
UPDATE accounts
SET balance = balance - 500
WHERE account_id = 1;

-- If this makes balance negative, constraint violation!
-- Transaction will be rolled back automatically

UPDATE accounts
SET balance = balance + 500
WHERE account_id = 2;

COMMIT;  -- Only commits if all constraints satisfied
```

Consistency Violation Examples:

sql

```sql
-- These would violate consistency and cause rollback:
UPDATE accounts SET balance = -100;  -- CHECK constraint violation
INSERT INTO orders VALUES (1, 999);  -- FOREIGN KEY violation (customer 999 doesn't exist)
INSERT INTO accounts VALUES (1, NULL, 500);  -- NOT NULL violation
```

Isolation (Concurrency Control)

Definition: Concurrent execution of transactions results in a state that is equivalent to some serial execution of the same transactions. Transactions don't interfere with each other.

The Isolation Problem:

```text
Transaction A:                    Transaction B:
BEGIN;                            BEGIN;
READ balance (1000)
                                  READ balance (1000)
                                  UPDATE balance = 800
                                  COMMIT;
UPDATE balance = 900
COMMIT;

What should final balance be? Without isolation, problems occur!
```

Isolation Problems:

Problem	Description	Example
Dirty Read	Reading uncommitted data	T2 reads data that T1 hasn't com yet
Non-repeatable Read	Same query returns different values in same transaction	T1 reads row, T2 updates it, T1 re again
Phantom Read	New rows appear/disappear	T1 runs query, T2 inserts row, T1 same query again

Visual Representation of Isolation Levels:

```text
Isolation Level Spectrum:
```

```
Lowest Isolation ──────────────────────────────────────→ Highest Isolation
     READ UNCOMMITTED    READ COMMITTED    REPEATABLE READ    SERIALIZABLE
     (Dirty reads)       (No dirty reads)  (Repeatable)       (Phantom-free)

Problems Allowed:
Dirty Read        ✓                  ✗                  ✗                  ✗
Non-repeatable    ✓                  ✓                  ✗                  ✗
Phantom Read      ✓                  ✓                  ✓                  ✗
```

Durability (Persistence)

Definition: Once a transaction is committed, its changes are permanent and survive any subsequent system failures.

What Durability Means:

- Data is written to non-volatile storage (disk)
- Survives power outages, crashes, restarts
- Can be recovered after system failure
- Transaction logs ensure no committed data is lost

How Durability Works:

```text
Transaction Flow:
1. BEGIN
2. Make changes in memory
3. Write to transaction log (WAL - Write Ahead Logging)
4. COMMIT → Log entry marked as committed
5. Later, data written to actual database files
6. If crash occurs, replay log to recover committed transactions

┌──────────┐     ┌──────────┐     ┌──────────┐
│ Memory   │────▶│ Log File │────▶│ Disk     │
│ Cache    │     │ (WAL)    │     │ Storage  │
└──────────┘     └──────────┘     └──────────┘
      ↑               ↑               ↑
```

```
    Changes         Permanent       Eventual
    made            on COMMIT       persistence
```

Example of Durability in Action:

sql

```sql
-- Even if power fails immediately after COMMIT
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
COMMIT;  -- ← Power fails right here!

-- After restart, database recovery will:
-- 1. Read transaction log
-- 2. Find committed transaction
-- 3. Re-apply changes if needed
-- 4. Balance transfer is preserved!
```

---

2. BEGIN, COMMIT, ROLLBACK

These are the basic transaction control commands.

BEGIN TRANSACTION

Definition: Marks the starting point of a transaction.

Syntax:

sql

```sql
-- Various syntaxes across databases
BEGIN TRANSACTION;        -- SQL Server, PostgreSQL
START TRANSACTION;        -- MySQL
BEGIN;                    -- PostgreSQL
BEGIN WORK;               -- Standard SQL
```

COMMIT

Definition: Makes all changes permanent and ends the transaction.

Syntax:

sql

```sql
COMMIT;
COMMIT WORK;              -- Alternative syntax
COMMIT TRANSACTION;       -- SQL Server
```

ROLLBACK

Definition: Undoes all changes made since the BEGIN and ends the transaction.

Syntax:

sql

```sql
ROLLBACK;
ROLLBACK WORK;            -- Alternative syntax
ROLLBACK TRANSACTION;     -- SQL Server
```

Practical Examples

Example 1: Basic Bank Transfer

sql

```sql
-- Start a transaction
BEGIN TRANSACTION;

-- Step 1: Withdraw from savings
UPDATE accounts
SET balance = balance - 500
WHERE account_id = 1 AND account_type = 'SAVINGS';

-- Check if withdrawal was successful
IF @@ROWCOUNT = 0  -- No rows updated (insufficient funds or wrong account)
BEGIN
    ROLLBACK;
    PRINT 'Withdrawal failed';
```

```sql
    RETURN;
END

-- Step 2: Deposit to checking
UPDATE accounts
SET balance = balance + 500
WHERE account_id = 2 AND account_type = 'CHECKING';

IF @@ROWCOUNT = 0  -- Deposit failed
BEGIN
    ROLLBACK;  -- This also undoes the withdrawal!
    PRINT 'Deposit failed';
    RETURN;
END

-- If we get here, both operations succeeded
COMMIT;
PRINT 'Transfer completed successfully';
```

Example 2: Order Processing

sql

```sql
BEGIN TRANSACTION;

-- 1. Create order
INSERT INTO orders (order_id, customer_id, order_date, status)
VALUES (1001, 123, GETDATE(), 'PENDING');

-- 2. Add order items
INSERT INTO order_items (order_id, product_id, quantity, price)
VALUES
    (1001, 101, 2, 29.99),
    (1001, 102, 1, 499.99);

-- 3. Update inventory
UPDATE products
SET stock_quantity = stock_quantity - 2
WHERE product_id = 101;

UPDATE products
SET stock_quantity = stock_quantity - 1
WHERE product_id = 102;
```

```sql
-- 4. Check if any product is out of stock
IF EXISTS (SELECT 1 FROM products WHERE product_id IN (101, 102) AND stock_quantit
y < 0)
BEGIN
    ROLLBACK;  -- Cancel entire order
    PRINT 'Order failed: insufficient stock';
END
ELSE
BEGIN
    COMMIT;  -- Order successful
    PRINT 'Order placed successfully';
END
```

Example 3: Multi-step Business Process

sql

```sql
-- Enroll student in course with payment
BEGIN TRANSACTION;

-- 1. Check prerequisites
DECLARE @prereq_met BIT;
SELECT @prereq_met = dbo.CheckPrerequisites(101, 'MATH201');

IF @prereq_met = 0
BEGIN
    ROLLBACK;
    PRINT 'Prerequisites not met';
    RETURN;
END

-- 2. Check available seats
DECLARE @seats_available INT;
SELECT @seats_available = available_seats FROM courses WHERE course_id = 'MATH201'
;

IF @seats_available <= 0
BEGIN
    ROLLBACK;
    PRINT 'No seats available';
    RETURN;
END
```

```sql
-- 3. Process payment
UPDATE student_balance
SET balance = balance - 500
WHERE student_id = 101;

IF @@ROWCOUNT = 0 OR (SELECT balance FROM student_balance WHERE student_id = 101)
< 0
BEGIN
    ROLLBACK;
    PRINT 'Payment failed';
    RETURN;
END

-- 4. Enroll student
INSERT INTO enrollments (student_id, course_id, enrollment_date, status)
VALUES (101, 'MATH201', GETDATE(), 'ENROLLED');

-- 5. Update available seats
UPDATE courses
SET available_seats = available_seats - 1
WHERE course_id = 'MATH201';

-- All steps successful
COMMIT;
PRINT 'Enrollment completed';
```

---

3. Savepoints

Definition: Savepoints allow you to roll back part of a transaction without aborting the entire transaction.

Why Use Savepoints?

- Roll back to a specific point within a transaction
- Handle errors gracefully without aborting everything

- Implement complex business logic with partial rollbacks

Syntax

sql

```sql
SAVEPOINT savepoint_name;        -- Create a savepoint
ROLLBACK TO SAVEPOINT savepoint_name;  -- Roll back to savepoint
RELEASE SAVEPOINT savepoint_name;      -- Remove savepoint
```

Visual Representation

text

```text
Transaction Timeline:
BEGIN
    ├── Operation 1 (success)
    ├── SAVEPOINT sp1
    ├── Operation 2 (success)
    ├── SAVEPOINT sp2
    ├── Operation 3 (FAILS)
    ├── ROLLBACK TO sp2
    ├── Operation 3a (alternative, success)
    └── COMMIT

Result: Operations 1, 2, and 3a are committed.
        Operation 3 is rolled back.
```

Practical Examples

Example 1: Order with Multiple Items

sql

```sql
BEGIN TRANSACTION;

-- Insert main order
INSERT INTO orders (order_id, customer_id, order_date)
VALUES (1001, 123, GETDATE());

SAVEPOINT order_created;

-- Try to add first item
```

```sql
BEGIN TRY
    INSERT INTO order_items (order_id, product_id, quantity, price)
    VALUES (1001, 101, 2, 29.99);

    UPDATE products SET stock = stock - 2 WHERE product_id = 101;
END TRY
BEGIN CATCH
    -- If first item fails, roll back to savepoint
    ROLLBACK TO SAVEPOINT order_created;
    PRINT 'First item failed, skipping...';
END CATCH

SAVEPOINT first_item_added;

-- Try to add second item
BEGIN TRY
    INSERT INTO order_items (order_id, product_id, quantity, price)
    VALUES (1001, 102, 1, 499.99);

    UPDATE products SET stock = stock - 1 WHERE product_id = 102;
END TRY
BEGIN CATCH
    -- If second item fails, roll back to previous savepoint
    ROLLBACK TO SAVEPOINT first_item_added;
    PRINT 'Second item failed, skipping...';
END CATCH

-- Check if any items were added
IF EXISTS (SELECT 1 FROM order_items WHERE order_id = 1001)
    COMMIT;
ELSE
    ROLLBACK;  -- No items, cancel entire order
```

Example 2: Batch Processing with Savepoints

sql

```sql
BEGIN TRANSACTION;

DECLARE @counter INT = 1;
DECLARE @total INT = 10;

WHILE @counter <= @total
```

```sql
BEGIN
    -- Create savepoint for this iteration
    SAVEPOINT iteration_start;

    BEGIN TRY
        -- Process one record
        INSERT INTO processed_data (record_id, data, status)
        VALUES (@counter, 'Sample data ' + CAST(@counter AS VARCHAR), 'PROCESSED')
;

        -- Simulate occasional failure
        IF @counter = 5
            THROW 50000, 'Simulated error on record 5', 1;

        PRINT 'Record ' + CAST(@counter AS VARCHAR) + ' processed';
        SET @counter = @counter + 1;
    END TRY
    BEGIN CATCH
        -- Roll back only this iteration
        ROLLBACK TO SAVEPOINT iteration_start;
        PRINT 'Record ' + CAST(@counter AS VARCHAR) + ' failed, skipping...';
        SET @counter = @counter + 1;
        -- Continue with next record
    END CATCH
END

-- Commit all successful records
COMMIT;
```

Example 3: Nested Savepoints

sql

```sql
BEGIN TRANSACTION;

-- Level 1: Customer update
UPDATE customers SET status = 'GOLD' WHERE customer_id = 100;
SAVEPOINT customer_updated;

-- Level 2: Order creation
INSERT INTO orders (customer_id, order_date) VALUES (100, GETDATE());
SAVEPOINT order_created;
```

```sql
-- Level 3: Payment processing
BEGIN TRY
    UPDATE accounts SET balance = balance - 500 WHERE customer_id = 100;
    SAVEPOINT payment_processed;

    -- Level 4: Shipping
    INSERT INTO shipments (order_id, status) VALUES (SCOPE_IDENTITY(), 'PENDING');

    -- If shipping fails, roll back to payment_processed
    -- but keep order and customer changes
END TRY
BEGIN CATCH
    ROLLBACK TO SAVEPOINT payment_processed;
    -- Handle payment issue, maybe try different payment method
END CATCH

-- Decide whether to commit everything
IF (some_condition)
    COMMIT;
ELSE
    ROLLBACK TO SAVEPOINT customer_updated;  -- Keep customer change, rollback res
t
```

---

4. Transaction Isolation Levels

Definition: Isolation levels define how transaction integrity is visible to other concurrent transactions.

The Five Isolation Levels

Level	Dirty Read	Non-repeatable Read	Phantom Read	Performance
READ UNCOMMITTED	Possible	Possible	Possible	Best
READ COMMITTED	Prevented	Possible	Possible	Good
REPEATABLE READ	Prevented	Prevented	Possible	Medium
SERIALIZABLE	Prevented	Prevented	Prevented	Worst
SNAPSHOT	Prevented	Prevented	Prevented*	Good

1. READ UNCOMMITTED (Dirty Reads)

Definition: The lowest isolation level. Transactions can read data modified by other uncommitted transactions.

Characteristics:

- No locks are applied
- Can read "dirty" data (uncommitted)
- Highest performance, lowest consistency
- Rarely used in production

Example:

```sql
-- Transaction 1 (running)
```

```sql
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
-- Not committed yet!

-- Transaction 2 (concurrent)
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT balance FROM accounts WHERE account_id = 1;
-- Returns: 400 (the uncommitted value!)

-- If Transaction 1 rolls back:
ROLLBACK;
-- Transaction 2 read data that never existed (DIRTY READ!)
```

Visualization:

```text
Time ──────────────────────────────────────────────────►
T1: BEGIN ── UPDATE ── (still running) ── ROLLBACK
                │
T2:             │── SELECT (reads 400) ── (uses wrong data!)
                └── This read is DIRTY - data never committed!
```

2. READ COMMITTED (Default in many databases)

Definition: Guarantees that any data read was committed at the moment of reading.

Characteristics:

- Prevents dirty reads
- Non-repeatable reads possible
- Default in SQL Server, PostgreSQL, Oracle

Example:

```sql
-- Transaction 1
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
-- Not committed

-- Transaction 2
```

```sql
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT balance FROM accounts WHERE account_id = 1;
-- Waits (or sees old value depending on implementation)
-- Won't see uncommitted data!

-- Transaction 1 commits
COMMIT;

-- Transaction 2 runs again
SELECT balance FROM accounts WHERE account_id = 1;
-- Now sees 400 (committed value)
-- This is a NON-REPEATABLE READ (same query, different results)
```

Non-Repeatable Read Problem:

```sql
-- Transaction A
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRANSACTION;
SELECT balance FROM accounts WHERE account_id = 1;  -- Returns 500

-- Transaction B (concurrent)
UPDATE accounts SET balance = 400 WHERE account_id = 1;
COMMIT;

-- Transaction A continues
SELECT balance FROM accounts WHERE account_id = 1;  -- Now returns 400!
-- Same query, different result - NON-REPEATABLE READ
COMMIT;
```

3. REPEATABLE READ

Definition: Guarantees that if a transaction reads a row twice, it sees the same data both times.

Characteristics:

- Prevents dirty reads and non-repeatable reads
- Phantom reads possible
- Default in MySQL

Example:

sql

```sql
-- Transaction A
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION;
SELECT balance FROM accounts WHERE account_id = 1;  -- Returns 500

-- Transaction B (concurrent)
UPDATE accounts SET balance = 400 WHERE account_id = 1;
COMMIT;  -- This update will be blocked or delayed depending on implementation

-- Transaction A continues
SELECT balance FROM accounts WHERE account_id = 1;  -- Still returns 500!
-- Same query, same result - REPEATABLE READ guaranteed

COMMIT;

-- After Transaction A commits, Transaction B's update takes effect
```

Phantom Read Problem:

sql

```sql
-- Transaction A
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION;
SELECT COUNT(*) FROM accounts WHERE balance > 1000;  -- Returns 5

-- Transaction B (concurrent)
INSERT INTO accounts (account_id, balance) VALUES (999, 1500);
COMMIT;  -- New row inserted!

-- Transaction A continues
SELECT COUNT(*) FROM accounts WHERE balance > 1000;  -- Still returns 5
-- In REPEATABLE READ, this might still return 5 (no phantom)
-- But in some implementations, it might return 6 (PHANTOM READ)
COMMIT;
```

4. SERIALIZABLE

Definition: The highest isolation level. Transactions are completely isolated from each other.

Characteristics:

- Prevents dirty reads, non-repeatable reads, and phantom reads
- Transactions execute as if they were serial (one after another)
- Maximum consistency, minimum concurrency
- Can cause deadlocks and performance issues

Example:

```sql
-- Transaction A
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION;
SELECT * FROM accounts WHERE balance > 1000;
-- This places range locks on all rows with balance > 1000

-- Transaction B (concurrent)
INSERT INTO accounts (account_id, balance) VALUES (999, 1500);
-- This will be BLOCKED until Transaction A commits!
-- Because the new row would affect the range Transaction A read

-- Transaction A continues
-- Can perform additional operations with complete confidence
-- No other transactions can affect the data it has read
COMMIT;

-- After A commits, Transaction B can proceed
```

Visualization:

```text
SERIALIZABLE ensures transactions appear to run sequentially:

Actual concurrent execution:   Logical view (as if serial):
T1: ──READ──────┐               T1: ──────────READ─UPDATE─COMMIT
T2: ────INSERT──┘                T2:                     ──INSERT─COMMIT
```

```
T2's INSERT waits until T1 completes, avoiding any interference.
```

5. SNAPSHOT ISOLATION

Definition: Each transaction sees a snapshot of data as it existed at the start of the transaction.

Characteristics:

- Uses row versioning instead of locking
- No dirty, non-repeatable, or phantom reads
- Write conflicts detected at commit time
- Available in SQL Server, PostgreSQL, Oracle

Example:

sql

```sql
-- Enable snapshot isolation (SQL Server)
ALTER DATABASE MyDB SET ALLOW_SNAPSHOT_ISOLATION ON;

-- Transaction A
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
BEGIN TRANSACTION;
SELECT balance FROM accounts WHERE account_id = 1;  -- Sees snapshot (500)

-- Transaction B (concurrent)
UPDATE accounts SET balance = 400 WHERE account_id = 1;
COMMIT;  -- This succeeds

-- Transaction A continues
SELECT balance FROM accounts WHERE account_id = 1;  -- Still sees 500!
-- Using its original snapshot

UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
-- This will FAIL with an update conflict!
-- Because the data changed since the snapshot was taken
COMMIT;
```

Isolation Level Comparison Matrix

Level	Locks Read?	Locks Range?	Row Versioning?	Update Conflict?
READ UNCOMMITTED	No	No	No	N/A
READ COMMITTED	Yes (read only)	No	Optional	No
REPEATABLE READ	Yes (until end)	No	Optional	No
SERIALIZABLE	Yes	Yes	No	No
SNAPSHOT	No	No	Yes	Detected at commit

Setting Isolation Levels

```sql
-- MySQL
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
-- queries
COMMIT;

-- PostgreSQL
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- queries
COMMIT;

-- SQL Server
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
BEGIN TRANSACTION;
```

```sql
-- queries
COMMIT;

-- Oracle (only READ COMMITTED and SERIALIZABLE)
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- queries
COMMIT;
```

Choosing the Right Isolation Level

Use Case	Recommended Level	Reason
Reporting, Analytics	SNAPSHOT or READ UNCOMMITTED	Consistency not critical, performanc matters
Banking, Financial	SERIALIZABLE	Data integrity critical
E-commerce checkout	REPEATABLE READ	Prevent overselling
Social media posts	READ COMMITTED	Good balance
Inventory management	SERIALIZABLE	Prevent double-selling
Read-only reports	SNAPSHOT	Consistent view without blocking

Part 2: Views

A view is a virtual table based on the result set of a SELECT query. It contains no data itself but displays data from one or more tables.

---

1. Create View

Definition: A view is a saved SQL query that acts like a table. It doesn't store data physically but provides a way to look at data from one or more tables.

Why Use Views?

Benefit	Description
Simplicity	Hide complex queries behind simple view names
Security	Restrict access to specific columns or rows
Consistency	Provide consistent interface even if underlying tables change
Reusability	Write once, use everywhere

Syntax

sql

```sql
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Practical Examples

Example 1: Simple View

sql

```sql
-- Create a view that shows only active employees
CREATE VIEW active_employees AS
SELECT
    emp_id,
    first_name,
    last_name,
```

```sql
    email,
    dept_id,
    salary
FROM employees
WHERE status = 'ACTIVE';

-- Now query becomes simple
SELECT * FROM active_employees WHERE dept_id = 1;
-- Instead of: SELECT * FROM employees WHERE status = 'ACTIVE' AND dept_id = 1
```

Example 2: View with Calculated Columns

sql

```sql
-- Create view with computed columns
CREATE VIEW employee_summary AS
SELECT
    emp_id,
    CONCAT(first_name, ' ', last_name) AS full_name,
    email,
    salary,
    salary * 12 AS annual_salary,
    CASE
        WHEN salary < 50000 THEN 'Junior'
        WHEN salary < 80000 THEN 'Mid'
        ELSE 'Senior'
    END AS level,
    DATEDIFF(YEAR, hire_date, GETDATE()) AS years_employed
FROM employees;

-- Easy reporting
SELECT * FROM employee_summary WHERE level = 'Senior';
```

Example 3: View with JOINs

sql

```sql
-- Create view that joins multiple tables
CREATE VIEW order_details_view AS
SELECT
    o.order_id,
    o.order_date,
    c.customer_name,
    c.email AS customer_email,
```

```sql
    p.product_name,
    oi.quantity,
    oi.price,
    oi.quantity * oi.price AS line_total,
    e.first_name + ' ' + e.last_name AS sales_rep
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
LEFT JOIN employees e ON o.sales_rep_id = e.emp_id;

-- Now complex report becomes simple
SELECT * FROM order_details_view
WHERE order_date >= '2024-01-01'
ORDER BY order_date;
```

Example 4: View with Aggregation

sql

```sql
-- Create summary view
CREATE VIEW sales_summary_by_product AS
SELECT
    p.product_id,
    p.product_name,
    p.category,
    COUNT(DISTINCT o.order_id) AS number_of_orders,
    SUM(oi.quantity) AS total_units_sold,
    SUM(oi.quantity * oi.price) AS total_revenue,
    AVG(oi.price) AS avg_selling_price,
    MIN(oi.price) AS min_price,
    MAX(oi.price) AS max_price
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
LEFT JOIN orders o ON oi.order_id = o.order_id
GROUP BY p.product_id, p.product_name, p.category;

-- Easy sales analysis
SELECT * FROM sales_summary_by_product
WHERE total_revenue > 10000
ORDER BY total_revenue DESC;
```

Example 5: View with Filtered Data (Row-Level Security)

```sql
-- Create view that only shows data for specific department
CREATE VIEW dept_employees AS
SELECT
    emp_id,
    first_name,
    last_name,
    email,
    salary,
    hire_date
FROM employees
WHERE dept_id = DATABASE_PRINCIPAL_ID();  -- Only current user's department

-- Grant access to view instead of base table
GRANT SELECT ON dept_employees TO hr_users;
```

---

2. Update View

Definition: Some views can be updated, meaning you can perform INSERT, UPDATE, DELETE operations through the view.

Updateable View Requirements

- Must be based on a single table (no JOINs)
- Cannot use DISTINCT, GROUP BY, HAVING
- Cannot use aggregate functions
- Cannot use subqueries in SELECT list
- Must include all NOT NULL columns from base table

Practical Examples

Example 1: Updateable Simple View

```sql
-- Create a simple view
```

```sql
CREATE VIEW active_customers AS
SELECT
    customer_id,
    name,
    email,
    phone,
    status
FROM customers
WHERE status = 'ACTIVE';

-- Can UPDATE through view
UPDATE active_customers
SET phone = '555-0123'
WHERE customer_id = 101;

-- Can INSERT through view (if all required columns included)
INSERT INTO active_customers (customer_id, name, email, phone, status)
VALUES (201, 'New Customer', 'new@email.com', '555-9999', 'ACTIVE');

-- Can DELETE through view
DELETE FROM active_customers WHERE customer_id = 101;
```

Example 2: View with WITH CHECK OPTION

sql

```sql
-- Create view with CHECK OPTION
CREATE VIEW high_value_customers AS
SELECT
    customer_id,
    name,
    total_purchases
FROM customers
WHERE total_purchases > 10000
WITH CHECK OPTION;

-- This works
UPDATE high_value_customers
SET total_purchases = 15000
WHERE customer_id = 101;

-- This will FAIL with CHECK OPTION violation
UPDATE high_value_customers
```

```sql
SET total_purchases = 5000
WHERE customer_id = 101;
-- Error: The row would disappear from the view!

-- CHECK OPTION prevents updates that would make the row invisible through the vie
w
```

Example 3: Views with INSTEAD OF Triggers

sql

```sql
-- For complex views that can't be updated directly,
-- use INSTEAD OF triggers

CREATE VIEW customer_order_summary AS
SELECT
    c.customer_id,
    c.name AS customer_name,
    o.order_id,
    o.order_date,
    o.amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;

-- Create INSTEAD OF trigger for INSERT
CREATE TRIGGER tr_customer_order_summary_insert
ON customer_order_summary
INSTEAD OF INSERT
AS
BEGIN
    INSERT INTO customers (customer_id, name)
    SELECT DISTINCT customer_id, customer_name
    FROM inserted
    WHERE customer_id NOT IN (SELECT customer_id FROM customers);

    INSERT INTO orders (order_id, customer_id, order_date, amount)
    SELECT order_id, customer_id, order_date, amount
    FROM inserted
    WHERE order_id IS NOT NULL;
END;
```

---

3. Materialized Views

Definition: A materialized view is a view that physically stores the query results on disk. It's like a snapshot of data at a point in time.

Regular View vs Materialized View

Aspect	Regular View	Materialized View
Storage	No storage, just query	Stores data physically
Performance	Runs query each time	Fast retrieval, no query execution
Data Freshness	Always current	May be stale (needs refresh)
Indexes	Can't create indexes	Can create indexes like tables
Updates	Can update base tables	Must refresh periodically

Syntax (Database Specific)

Oracle/PostgreSQL:

sql

```sql
-- Create materialized view
CREATE MATERIALIZED VIEW mv_sales_summary
AS
SELECT
    product_id,
    SUM(quantity) AS total_quantity,
    SUM(quantity * price) AS total_revenue
FROM sales
GROUP BY product_id;

-- Refresh manually
REFRESH MATERIALIZED VIEW mv_sales_summary;
```

```sql
-- Create with automatic refresh
CREATE MATERIALIZED VIEW mv_sales_summary
REFRESH COMPLETE ON DEMAND
AS
SELECT ...;
```

SQL Server (uses Indexed Views):

```sql
-- Must have specific SET options
SET ANSI_NULLS ON;
SET ANSI_PADDING ON;
SET ANSI_WARNINGS ON;
SET CONCAT_NULL_YIELDS_NULL ON;
SET QUOTED_IDENTIFIER ON;
SET NUMERIC_ROUNDABORT OFF;

-- Create view with unique clustered index
CREATE VIEW vw_sales_summary
WITH SCHEMABINDING
AS
SELECT
    product_id,
    COUNT_BIG(*) AS sale_count,
    SUM(ISNULL(quantity, 0)) AS total_quantity,
    SUM(ISNULL(quantity * price, 0)) AS total_revenue
FROM dbo.sales
GROUP BY product_id;
GO

-- Create unique clustered index (materializes the view)
CREATE UNIQUE CLUSTERED INDEX IX_vw_sales_summary
ON vw_sales_summary (product_id);
```

Practical Examples

Example 1: Performance Improvement

```sql
-- Slow query (runs every time, aggregates millions of rows)
```

```sql
SELECT
    customer_id,
    COUNT(*) AS order_count,
    SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id;

-- Create materialized view (runs once, stores results)
CREATE MATERIALIZED VIEW mv_customer_summary AS
SELECT
    customer_id,
    COUNT(*) AS order_count,
    SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id;

-- Create indexes on materialized view for even faster access
CREATE INDEX idx_mv_spent ON mv_customer_summary(total_spent DESC);

-- Now queries are instant!
SELECT * FROM mv_customer_summary
WHERE total_spent > 10000;
```

Example 2: Data Warehouse Star Schema

sql

```sql
-- Materialize fact table aggregations
CREATE MATERIALIZED VIEW mv_monthly_sales AS
SELECT
    d.year,
    d.month,
    p.category,
    SUM(f.quantity) AS units_sold,
    SUM(f.revenue) AS total_revenue,
    COUNT(DISTINCT f.customer_id) AS unique_customers
FROM fact_sales f
JOIN dim_date d ON f.date_id = d.date_id
JOIN dim_product p ON f.product_id = p.product_id
GROUP BY d.year, d.month, p.category;

-- Refresh strategies:
-- 1. Complete refresh (rebuilds entire view)
```

```sql
REFRESH MATERIALIZED VIEW mv_monthly_sales;

-- 2. Incremental refresh (only new data)
-- Database specific implementation
```

Example 3: Refresh Strategies

```sql
-- Oracle: Automatic refresh
CREATE MATERIALIZED VIEW mv_product_stats
REFRESH COMPLETE START WITH SYSDATE NEXT SYSDATE + 1
AS
SELECT
    product_id,
    AVG(price) AS avg_price,
    COUNT(*) AS sale_count
FROM sales
GROUP BY product_id;

-- PostgreSQL: Concurrent refresh (allows reads during refresh)
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_product_stats;

-- SQL Server: Use indexed view with automatic updates
-- Updates happen automatically when base data changes
```

---

4. Security Benefits of Views

Views provide powerful security mechanisms to control data access.

Security Features

Feature	Description	Example
Column-Level Security	Hide sensitive columns	Show employees without salary

Feature	Description	Example
Row-Level Security	Show only relevant rows	Customers see only their orders
Data Masking	Transform sensitive data	Mask credit card numbers
Simplify Permissions	Grant on view, not tables	HR view for HR users only

Practical Security Examples

Example 1: Column-Level Security (Hide Sensitive Data)

sql

```sql
-- Base table contains sensitive information
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    ssn VARCHAR(11),        -- Sensitive
    salary DECIMAL(10,2),   -- Sensitive
    birth_date DATE,         -- Sensitive
    email VARCHAR(100),
    department VARCHAR(50)
);

-- Create view for general users - hides sensitive columns
CREATE VIEW public_employee_info AS
SELECT
    emp_id,
    first_name,
    last_name,
    email,
    department
FROM employees;

-- Create view for managers - shows more
CREATE VIEW manager_employee_info AS
SELECT
```

```sql
    emp_id,
    first_name,
    last_name,
    email,
    department,
    salary  -- Managers can see salary
FROM employees;

-- Create view for HR - shows everything except SSN
CREATE VIEW hr_employee_info AS
SELECT
    emp_id,
    first_name,
    last_name,
    email,
    department,
    salary,
    birth_date  -- HR can see birth date
FROM employees;

-- Grant appropriate permissions
GRANT SELECT ON public_employee_info TO public_users;
GRANT SELECT ON manager_employee_info TO managers;
GRANT SELECT ON hr_employee_info TO hr_staff;
```

Example 2: Row-Level Security

sql

```sql
-- Customers should only see their own orders
CREATE VIEW customer_orders_view AS
SELECT
    o.order_id,
    o.order_date,
    o.amount,
    o.status,
    p.product_name,
    oi.quantity
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.customer_id = CURRENT_USER_ID();  -- Database function returns current use
r
```

```sql
-- Grant access - users automatically see only their data
GRANT SELECT ON customer_orders_view TO authenticated_users;
```

Example 3: Data Masking Through Views

sql

```sql
-- Mask sensitive data
CREATE VIEW customer_service_view AS
SELECT
    customer_id,
    name,
    -- Mask email: show only first character
    CONCAT(LEFT(email, 1), '*****', RIGHT(email, CHARINDEX('@', REVERSE(email))))
AS masked_email,
    -- Mask phone: show only last 4 digits
    CONCAT('XXX-XXX-', RIGHT(phone, 4)) AS masked_phone,
    -- Show only year of birth
    YEAR(birth_date) AS birth_year,
    -- Mask credit card: show only last 4
    CONCAT('XXXX-XXXX-XXXX-', RIGHT(credit_card, 4)) AS masked_cc
FROM customers;

-- Support staff can help customers without seeing full sensitive data
SELECT * FROM customer_service_view WHERE customer_id = 101;
-- Returns: 101, John Doe, j*****@email.com, XXX-XXX-1234, 1985, XXXX-XXXX-XXXX-12
34
```

Example 4: Aggregated Access Without Details

sql

```sql
-- Give access to statistics without individual records
CREATE VIEW department_statistics AS
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS employee_count,
    AVG(e.salary) AS average_salary,
    MIN(e.salary) AS min_salary,
    MAX(e.salary) AS max_salary,
    SUM(e.salary) AS total_payroll
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
```

```sql
GROUP BY d.dept_id, d.dept_name;

-- Users can see department stats but not individual salaries
GRANT SELECT ON department_statistics TO all_employees;
```

Example 5: Multi-Tenant Data Isolation

sql

```sql
-- SaaS application with multiple clients (tenants)
CREATE TABLE tenant_data (
    id INT PRIMARY KEY,
    tenant_id INT,
    sensitive_data VARCHAR(500),
    created_at DATETIME
);

-- Create view that automatically filters by tenant
CREATE VIEW tenant_specific_data AS
SELECT
    id,
    sensitive_data,
    created_at
FROM tenant_data
WHERE tenant_id = CURRENT_TENANT_ID();  -- Function returns current tenant

-- Each tenant sees only their own data
-- Application code never needs to add WHERE tenant_id = ?
GRANT SELECT, INSERT, UPDATE ON tenant_specific_data TO app_users;

-- Insert through view (if updateable)
INSERT INTO tenant_specific_data (id, sensitive_data, created_at)
VALUES (1, 'Customer record', GETDATE());
-- The view's WHERE clause is automatically applied!
```

Security Best Practices with Views

Practice	Why	How
Grant on views, not tables	Control access without exposing structure	`GRANT SELECT ON view TO`
Use WITH CHECK OPTION	Prevent data leakage through updates	`CREATE VIEW ... WITH CHE` `OPTION`
Schema binding	Prevent accidental changes to underlying tables	`CREATE VIEW ... WITH` `SCHEMABINDING`
Encrypt view definition	Hide business logic (SQL Server)	`CREATE VIEW ... WITH ENC`
Regular review	Ensure views still meet security needs	Audit view definitions peri

Example: Complete Security Implementation

sql

```sql
-- 1. Create base tables (no direct access)
CREATE TABLE employees (...);
CREATE TABLE salaries (...);

-- 2. Create views with security
CREATE VIEW vw_employee_public
WITH SCHEMABINDING
AS
SELECT emp_id, name, department
FROM dbo.employees;

CREATE VIEW vw_employee_salaries
WITH SCHEMABINDING
AS
SELECT e.emp_id, e.name, s.salary, s.bonus
FROM dbo.employees e
JOIN dbo.salaries s ON e.emp_id = s.emp_id
```

```sql
WHERE e.manager_id = USER_ID();  -- Managers see direct reports only

-- 3. Grant permissions
REVOKE SELECT ON employees TO PUBLIC;
REVOKE SELECT ON salaries TO PUBLIC;

GRANT SELECT ON vw_employee_public TO all_users;
GRANT SELECT ON vw_employee_salaries TO managers;

-- 4. Create audit view
CREATE VIEW vw_access_log AS
SELECT * FROM security_log WHERE user_id = USER_ID();
```

Summary: Transactions and Views Quick Reference

Transactions

Command	Purpose
`BEGIN TRANSACTION`	Start a transaction
`COMMIT`	Save all changes permanently
`ROLLBACK`	Undo all changes since BEGIN
`SAVEPOINT name`	Create a point to roll back to
`ROLLBACK TO name`	Undo changes to a savepoint

Isolation Levels

Level	Dirty Read	Non-repeatable	Phantom
READ UNCOMMITTED	✓	✓	✓
READ COMMITTED	✗	✓	✓
REPEATABLE READ	✗	✗	✓
SERIALIZABLE	✗	✗	✗
SNAPSHOT	✗	✗	✗

Views

Type	Description	Use Case
Regular View	Virtual table, no data storage	Simplify complex queries
Updateable View	Can modify underlying data	Controlled data access
Materialized View	Physically stores data	Performance for aggregations
Indexed View	Materialized with indexes	Maximum query performance

Security with Views

- Hide sensitive columns
- Filter rows by user
- Mask sensitive data
- Provide aggregated access

- Simplify permission management

Stored Procedures and Triggers: Complete Theory Notes

Part 1: Stored Procedures

A stored procedure is a pre-compiled collection of SQL statements stored under a name and processed as a unit. They are stored in the database and can be executed repeatedly.

---

1. Creating Procedures

What is a Stored Procedure?

Definition: A stored procedure is a set of pre-compiled SQL statements that can be executed as a single unit. It's like a function or method in programming languages but runs inside the database.

Benefits of Stored Procedures

Benefit	Description
Performance	Pre-compiled and cached, faster execution
Reusability	Write once, use many times
Maintainability	Centralized business logic
Security	Grant execute without table access

Benefit	Description
Network Traffic	Send just procedure call, not multiple SQL statements
Consistency	Ensures same logic is applied everywhere

Syntax (Database Specific)

MySQL:

sql

```sql
DELIMITER //

CREATE PROCEDURE procedure_name ([parameters])
BEGIN
    -- SQL statements
END //

DELIMITER ;
```

SQL Server:

sql

```sql
CREATE PROCEDURE procedure_name
    [parameters]
AS
BEGIN
    -- SQL statements
END;
```

PostgreSQL:

sql

```sql
CREATE OR REPLACE PROCEDURE procedure_name ([parameters])
LANGUAGE plpgsql
AS $$
BEGIN
    -- SQL statements
END;
```

```
$$;
```

Practical Examples

Example 1: Simple Procedure - Get All Employees

sql

```sql
-- MySQL
DELIMITER //

CREATE PROCEDURE GetAllEmployees()
BEGIN
    SELECT * FROM employees ORDER BY last_name;
END //

DELIMITER ;

-- SQL Server
CREATE PROCEDURE GetAllEmployees
AS
BEGIN
    SELECT * FROM employees ORDER BY last_name;
END;
```

Example 2: Procedure with Business Logic

sql

```sql
-- MySQL: Calculate employee bonus
DELIMITER //

CREATE PROCEDURE CalculateAnnualBonus()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE emp_id INT;
    DECLARE emp_salary DECIMAL(10,2);
    DECLARE emp_performance VARCHAR(10);
    DECLARE bonus_amount DECIMAL(10,2);

    -- Cursor to loop through employees
    DECLARE emp_cursor CURSOR FOR
        SELECT employee_id, salary, performance_rating
```

```
        FROM employees WHERE status = 'ACTIVE';

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Create temp table for results
    CREATE TEMPORARY TABLE bonus_results (
        employee_id INT,
        bonus_amount DECIMAL(10,2),
        calculation_date DATETIME
    );

    OPEN emp_cursor;

    read_loop: LOOP
        FETCH emp_cursor INTO emp_id, emp_salary, emp_performance;

        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Calculate bonus based on performance
        CASE emp_performance
            WHEN 'EXCELLENT' THEN SET bonus_amount = emp_salary * 0.20;
            WHEN 'GOOD' THEN SET bonus_amount = emp_salary * 0.10;
            WHEN 'AVERAGE' THEN SET bonus_amount = emp_salary * 0.05;
            ELSE SET bonus_amount = 0;
        END CASE;

        -- Store result
        INSERT INTO bonus_results VALUES (emp_id, bonus_amount, NOW());
    END LOOP;

    CLOSE emp_cursor;

    -- Return results
    SELECT * FROM bonus_results;

    -- Clean up
    DROP TEMPORARY TABLE bonus_results;
END //

DELIMITER ;
```

Example 3: Procedure with Transaction

sql

```sql
-- SQL Server: Transfer money between accounts
CREATE PROCEDURE TransferMoney
    @FromAccount INT,
    @ToAccount INT,
    @Amount DECIMAL(10,2),
    @Result VARCHAR(100) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Check if source account has sufficient funds
        DECLARE @CurrentBalance DECIMAL(10,2);

        SELECT @CurrentBalance = balance
        FROM accounts
        WHERE account_id = @FromAccount;

        IF @CurrentBalance < @Amount
        BEGIN
            SET @Result = 'Insufficient funds';
            ROLLBACK;
            RETURN;
        END

        -- Deduct from source
        UPDATE accounts
        SET balance = balance - @Amount
        WHERE account_id = @FromAccount;

        -- Add to destination
        UPDATE accounts
        SET balance = balance + @Amount
        WHERE account_id = @ToAccount;

        -- Log transaction
        INSERT INTO transfer_log
```

```
            (from_account, to_account, amount, transfer_date, status)
        VALUES
            (@FromAccount, @ToAccount, @Amount, GETDATE(), 'SUCCESS');

        COMMIT;
        SET @Result = 'Transfer completed successfully';

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK;

        SET @Result = ERROR_MESSAGE();

        -- Log error
        INSERT INTO error_log
            (error_message, error_date, procedure_name)
        VALUES
            (ERROR_MESSAGE(), GETDATE(), 'TransferMoney');
    END CATCH
END;
```

Example 4: Complex Reporting Procedure

sql

```
-- PostgreSQL: Generate sales report
CREATE OR REPLACE PROCEDURE GenerateSalesReport(
    p_start_date DATE,
    p_end_date DATE,
    p_department_id INT DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_total_sales DECIMAL(15,2);
    v_avg_order_value DECIMAL(10,2);
    v_top_product VARCHAR(100);
BEGIN
    -- Create temporary report table
    CREATE TEMP TABLE sales_report AS
    SELECT
        DATE_TRUNC('day', o.order_date) AS sale_day,
```

```sql
    d.department_name,
    p.category,
    COUNT(DISTINCT o.order_id) AS order_count,
    SUM(oi.quantity) AS units_sold,
    SUM(oi.quantity * oi.price) AS revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
JOIN departments d ON o.department_id = d.department_id
WHERE o.order_date BETWEEN p_start_date AND p_end_date
    AND (p_department_id IS NULL OR o.department_id = p_department_id)
GROUP BY DATE_TRUNC('day', o.order_date), d.department_name, p.category;

-- Calculate totals
SELECT
    COALESCE(SUM(revenue), 0),
    COALESCE(AVG(revenue), 0)
INTO v_total_sales, v_avg_order_value
FROM sales_report;

-- Get top product
SELECT product_name INTO v_top_product
FROM (
    SELECT
        p.product_name,
        SUM(oi.quantity) as total_qty
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    JOIN products p ON oi.product_id = p.product_id
    WHERE o.order_date BETWEEN p_start_date AND p_end_date
        AND (p_department_id IS NULL OR o.department_id = p_department_id)
    GROUP BY p.product_name
    ORDER BY total_qty DESC
    LIMIT 1
) AS top;

-- Return detailed report
RETURN QUERY
SELECT * FROM sales_report;

-- Also return summary
RAISE NOTICE 'Total Sales: %, Avg Order Value: %, Top Product: %',
```

```
        v_total_sales, v_avg_order_value, v_top_product;

    -- Clean up
    DROP TABLE sales_report;
END;
$$;
```

---

2. Parameters

Stored procedures can accept parameters to make them flexible and reusable.

Types of Parameters

Parameter Type	Description	Direction
IN	Input parameter (default)	Pass values into procedure
OUT	Output parameter	Return values to caller
INOUT	Both input and output	Pass in and return value

Syntax Examples

MySQL:

sql

```sql
DELIMITER //

CREATE PROCEDURE GetEmployeesByDept(
    IN dept_id INT,                    -- Input parameter
    OUT emp_count INT,                  -- Output parameter
    INOUT min_salary DECIMAL(10,2)       -- INOUT parameter
)
BEGIN
```

```
    -- Get employees for department
    SELECT * FROM employees
    WHERE department_id = dept_id;

    -- Set output parameter
    SELECT COUNT(*) INTO emp_count
    FROM employees
    WHERE department_id = dept_id;

    -- Use and modify INOUT parameter
    IF min_salary IS NOT NULL THEN
        SELECT MIN(salary) INTO min_salary
        FROM employees
        WHERE department_id = dept_id
        AND salary > min_salary;
    END IF;
END //

DELIMITER ;
```

SQL Server:

sql

```
CREATE PROCEDURE GetEmployeeDetails
    @EmployeeID INT,
    @FirstName VARCHAR(50) OUTPUT,
    @LastName VARCHAR(50) OUTPUT,
    @Salary DECIMAL(10,2) OUTPUT,
    @Department VARCHAR(50) OUTPUT
AS
BEGIN
    SELECT
        @FirstName = first_name,
        @LastName = last_name,
        @Salary = salary,
        @Department = dept_name
    FROM employees e
    JOIN departments d ON e.dept_id = d.dept_id
    WHERE e.emp_id = @EmployeeID;

    IF @@ROWCOUNT = 0
    BEGIN
```

```
        RAISERROR('Employee not found', 16, 1);
        RETURN;
    END
END;
```

Practical Parameter Examples

Example 1: Search with Multiple Optional Parameters

sql

```sql
-- MySQL: Flexible search procedure
DELIMITER //

CREATE PROCEDURE SearchProducts(
    IN p_product_name VARCHAR(100),
    IN p_category VARCHAR(50),
    IN p_min_price DECIMAL(10,2),
    IN p_max_price DECIMAL(10,2),
    IN p_in_stock_only BOOLEAN,
    IN p_sort_by VARCHAR(20),
    IN p_page INT,
    IN p_page_size INT,
    OUT p_total_count INT
)
BEGIN
    -- Calculate offset
    DECLARE v_offset INT;
    SET v_offset = (p_page - 1) * p_page_size;

    -- Build dynamic WHERE clause
    SET @sql = 'SELECT * FROM products WHERE 1=1';

    IF p_product_name IS NOT NULL THEN
        SET @sql = CONCAT(@sql, ' AND product_name LIKE ''%', p_product_name, '%''
');
    END IF;

    IF p_category IS NOT NULL THEN
        SET @sql = CONCAT(@sql, ' AND category = ''', p_category, '''');
    END IF;
```

```sql
    IF p_min_price IS NOT NULL THEN
        SET @sql = CONCAT(@sql, ' AND price >= ', p_min_price);
    END IF;

    IF p_max_price IS NOT NULL THEN
        SET @sql = CONCAT(@sql, ' AND price <= ', p_max_price);
    END IF;

    IF p_in_stock_only THEN
        SET @sql = CONCAT(@sql, ' AND stock_quantity > 0');
    END IF;

    -- Get total count
    SET @count_sql = REPLACE(@sql, 'SELECT *', 'SELECT COUNT(*)');
    PREPARE stmt FROM @count_sql;
    EXECUTE stmt INTO p_total_count;
    DEALLOCATE PREPARE stmt;

    -- Add sorting
    CASE p_sort_by
        WHEN 'price_asc' THEN SET @sql = CONCAT(@sql, ' ORDER BY price ASC');
        WHEN 'price_desc' THEN SET @sql = CONCAT(@sql, ' ORDER BY price DESC');
        WHEN 'name' THEN SET @sql = CONCAT(@sql, ' ORDER BY product_name');
        ELSE SET @sql = CONCAT(@sql, ' ORDER BY product_id');
    END CASE;

    -- Add pagination
    SET @sql = CONCAT(@sql, ' LIMIT ', p_page_size, ' OFFSET ', v_offset);

    -- Execute search
    PREPARE stmt FROM @sql;
    EXECUTE stmt;
    DEALLOCATE PREPARE stmt;
END //

DELIMITER ;
```

Example 2: INOUT Parameter for Balance Update

sql

```sql
-- SQL Server: Update account with balance check
CREATE PROCEDURE UpdateAccountBalance
```

```sql
    @AccountID INT,
    @Amount DECIMAL(10,2),
    @NewBalance DECIMAL(10,2) OUTPUT,
    @Status VARCHAR(20) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Get current balance
        DECLARE @CurrentBalance DECIMAL(10,2);

        SELECT @CurrentBalance = balance
        FROM accounts
        WHERE account_id = @AccountID;

        IF @CurrentBalance IS NULL
        BEGIN
            SET @Status = 'Account not found';
            SET @NewBalance = NULL;
            ROLLBACK;
            RETURN;
        END

        -- Calculate new balance
        SET @NewBalance = @CurrentBalance + @Amount;

        -- Check if balance would become negative
        IF @NewBalance < 0
        BEGIN
            SET @Status = 'Insufficient funds';
            SET @NewBalance = @CurrentBalance;
            ROLLBACK;
            RETURN;
        END

        -- Update balance
        UPDATE accounts
        SET balance = @NewBalance
        WHERE account_id = @AccountID;
```

```sql
        -- Log transaction
        INSERT INTO account_transactions
            (account_id, amount, old_balance, new_balance, transaction_date)
        VALUES
            (@AccountID, @Amount, @CurrentBalance, @NewBalance, GETDATE());

        SET @Status = 'Success';

        COMMIT;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK;

        SET @Status = ERROR_MESSAGE();
        SET @NewBalance = NULL;
    END CATCH
END;
```

Example 3: Table-Valued Parameters (SQL Server)

sql

```sql
-- First, create a user-defined table type
CREATE TYPE OrderItemType AS TABLE (
    ProductID INT,
    Quantity INT,
    Price DECIMAL(10,2)
);

-- Procedure that accepts table-valued parameter
CREATE PROCEDURE PlaceBulkOrder
    @CustomerID INT,
    @OrderDate DATE,
    @Items OrderItemType READONLY,
    @OrderID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;
```

```sql
        -- Insert order header
        INSERT INTO orders (customer_id, order_date, status)
        VALUES (@CustomerID, @OrderDate, 'PENDING');

        SET @OrderID = SCOPE_IDENTITY();

        -- Insert order items
        INSERT INTO order_items (order_id, product_id, quantity, price)
        SELECT @OrderID, ProductID, Quantity, Price
        FROM @Items;

        -- Update inventory
        UPDATE p
        SET stock_quantity = p.stock_quantity - i.Quantity
        FROM products p
        INNER JOIN @Items i ON p.product_id = i.ProductID;

        -- Calculate total
        UPDATE orders
        SET total_amount = (
            SELECT SUM(quantity * price)
            FROM order_items
            WHERE order_id = @OrderID
        )
        WHERE order_id = @OrderID;

        COMMIT;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK;
        THROW;
    END CATCH
END;
```

---

3. Calling Procedures

Syntax for Executing/Calling Procedures

MySQL:

sql

```sql
-- Call without parameters
CALL GetAllEmployees();

-- Call with IN parameters
CALL GetEmployeesByDept(1);

-- Call with OUT parameters
CALL GetEmployeeCount(1, @count);
SELECT @count;

-- Call with INOUT parameter
SET @min_salary = 50000;
CALL UpdateMinSalary(1, @min_salary);
SELECT @min_salary;
```

SQL Server:

sql

```sql
-- Execute without parameters
EXEC GetAllEmployees;
-- or
EXECUTE GetAllEmployees;

-- Execute with parameters
EXEC GetEmployeesByDept @dept_id = 1;

-- Execute with output parameters
DECLARE @Count INT;
EXEC GetEmployeeCount @dept_id = 1, @emp_count = @Count OUTPUT;
SELECT @Count AS EmployeeCount;

-- Execute with named parameters
EXEC SearchProducts
    @p_category = 'Electronics',
```

```
    @p_min_price = 100,
    @p_max_price = 1000,
    @p_page = 1,
    @p_page_size = 20;
```

PostgreSQL:

sql

```sql
-- Call procedure
CALL TransferMoney(101, 102, 500);

-- Call with output parameters (PostgreSQL 14+)
CALL GetEmployeeStats(1, NULL, NULL);
```

Practical Calling Examples

Example 1: Application Layer Integration

python

```python
# Python example with MySQL
import mysql.connector

def search_products(category, min_price, max_price, page=1):
    conn = mysql.connector.connect(
        host="localhost",
        user="app_user",
        password="password",
        database="company_db"
    )

    cursor = conn.cursor()

    # Call stored procedure
    args = [
        None,  # product_name
        category,
        min_price,
        max_price,
        True,  # in_stock_only
        'price_asc',
        page,
```

```python
        20,     # page_size
        0       # output parameter
    ]

    result = cursor.callproc('SearchProducts', args)

    # Get output parameter (total count)
    total_count = result[8]

    # Fetch results
    results = []
    for row in cursor.stored_results():
        results = row.fetchall()

    cursor.close()
    conn.close()

    return {
        'products': results,
        'total_count': total_count,
        'page': page,
        'total_pages': (total_count + 19) // 20
    }

# Usage
products = search_products('Electronics', 100, 1000, 1)
print(f"Found {products['total_count']} products")
```

Example 2: Nested Procedure Calls

sql

```sql
-- Master procedure that calls multiple sub-procedures
CREATE PROCEDURE ProcessMonthlyPayroll
    @Month INT,
    @Year INT,
    @ProcessedCount INT OUTPUT,
    @TotalAmount DECIMAL(15,2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
```

```sql
        BEGIN TRANSACTION;

        -- Step 1: Calculate base salaries
        EXEC CalculateBaseSalaries @Month, @Year;

        -- Step 2: Calculate bonuses
        EXEC CalculateBonuses @Month, @Year;

        -- Step 3: Calculate deductions
        EXEC CalculateDeductions @Month, @Year;

        -- Step 4: Generate payroll
        EXEC GeneratePayrollRecords @Month, @Year,
             @ProcessedCount OUTPUT, @TotalAmount OUTPUT;

        -- Step 5: Send notifications
        EXEC SendPayrollNotifications @Month, @Year;

        COMMIT;

        -- Log success
        EXEC LogPayrollActivity @Month, @Year, 'SUCCESS',
             @ProcessedCount, @TotalAmount;

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK;

        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();

        -- Log error
        EXEC LogPayrollActivity @Month, @Year, 'FAILED',
             0, 0, @ErrorMessage;

        THROW;
    END CATCH
END;
```

Example 3: Dynamic Execution Based on Parameters

sql

```sql
-- Procedure that behaves differently based on input
CREATE PROCEDURE GetCustomerData
    @CustomerID INT = NULL,
    @Email VARCHAR(100) = NULL,
    @Phone VARCHAR(20) = NULL,
    @IncludeOrders BIT = 0
AS
BEGIN
    -- Return customer info
    IF @CustomerID IS NOT NULL
    BEGIN
        SELECT * FROM customers WHERE customer_id = @CustomerID;
    END
    ELSE IF @Email IS NOT NULL
    BEGIN
        SELECT * FROM customers WHERE email = @Email;
    END
    ELSE IF @Phone IS NOT NULL
    BEGIN
        SELECT * FROM customers WHERE phone = @Phone;
    END
    ELSE
    BEGIN
        -- Return all customers with pagination hint
        SELECT TOP 100 * FROM customers ORDER BY customer_id;
    END

    -- Optionally include orders
    IF @IncludeOrders = 1 AND @CustomerID IS NOT NULL
    BEGIN
        SELECT * FROM orders
        WHERE customer_id = @CustomerID
        ORDER BY order_date DESC;
    END
END;
```

---

4. Error Handling

Proper error handling in stored procedures ensures data integrity and provides meaningful feedback.

Error Handling Techniques

MySQL:

sql

```sql
DELIMITER //

CREATE PROCEDURE SafeTransfer(
    IN from_acc INT,
    IN to_acc INT,
    IN amount DECIMAL(10,2)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'An error occurred, transaction rolled back' AS message;
    END;

    DECLARE EXIT HANDLER FOR SQLWARNING
    BEGIN
        ROLLBACK;
        SELECT 'Warning occurred, transaction rolled back' AS message;
    END;

    DECLARE EXIT HANDLER FOR 1062  -- Duplicate key error
    BEGIN
        ROLLBACK;
        SELECT 'Duplicate key error' AS message;
    END;

    START TRANSACTION;

    -- Check balance
    IF (SELECT balance FROM accounts WHERE account_id = from_acc) < amount THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient funds';
    END IF;

    UPDATE accounts SET balance = balance - amount WHERE account_id = from_acc;
    UPDATE accounts SET balance = balance + amount WHERE account_id = to_acc;

    COMMIT;
    SELECT 'Transfer successful' AS message;
END //

DELIMITER ;
```

SQL Server Comprehensive Error Handling:

sql

```
CREATE PROCEDURE ProcessOrder
    @CustomerID INT,
    @ProductID INT,
    @Quantity INT,
    @OrderID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @ErrorSeverity INT;
    DECLARE @ErrorState INT;
    DECLARE @ErrorMessage NVARCHAR(4000);
    DECLARE @ErrorLine INT;

    BEGIN TRY
        -- Check for potential errors first
        IF NOT EXISTS (SELECT 1 FROM customers WHERE customer_id = @CustomerID)
        BEGIN
            RAISERROR('Customer %d does not exist', 16, 1, @CustomerID);
            RETURN;
        END

        IF NOT EXISTS (SELECT 1 FROM products WHERE product_id = @ProductID)
        BEGIN
            RAISERROR('Product %d does not exist', 16, 1, @ProductID);
            RETURN;
```

```sql
END

DECLARE @AvailableStock INT;
SELECT @AvailableStock = stock_quantity
FROM products
WHERE product_id = @ProductID;

IF @AvailableStock < @Quantity
BEGIN
    RAISERROR('Insufficient stock. Available: %d, Requested: %d',
             16, 1, @AvailableStock, @Quantity);
    RETURN;
END

BEGIN TRANSACTION;

-- Create order
INSERT INTO orders (customer_id, order_date, status)
VALUES (@CustomerID, GETDATE(), 'PENDING');

SET @OrderID = SCOPE_IDENTITY();

-- Add order item
INSERT INTO order_items (order_id, product_id, quantity, price)
SELECT @OrderID, @ProductID, @Quantity, unit_price
FROM products
WHERE product_id = @ProductID;

-- Update stock
UPDATE products
SET stock_quantity = stock_quantity - @Quantity
WHERE product_id = @ProductID;

-- Check for any issues
IF @@ROWCOUNT = 0
BEGIN
    RAISERROR('Stock update failed', 16, 1);
END

COMMIT TRANSACTION;

-- Return success message
```

```
        SELECT 'Order created successfully' AS Status, @OrderID AS OrderID;

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        -- Get error details
        SELECT
            @ErrorSeverity = ERROR_SEVERITY(),
            @ErrorState = ERROR_STATE(),
            @ErrorMessage = ERROR_MESSAGE(),
            @ErrorLine = ERROR_LINE();

        -- Log error
        INSERT INTO error_log (
            procedure_name,
            error_message,
            error_severity,
            error_state,
            error_line,
            error_time,
            parameters
        ) VALUES (
            'ProcessOrder',
            @ErrorMessage,
            @ErrorSeverity,
            @ErrorState,
            @ErrorLine,
            GETDATE(),
            CONCAT('Customer:', @CustomerID, ', Product:', @ProductID, ', Qty:', @
Quantity)
        );

        -- Re-raise with custom message
        DECLARE @UserMessage NVARCHAR(4000) =
            'Order processing failed: ' + @ErrorMessage;

        RAISERROR(@UserMessage, 16, 1);
    END CATCH
END;
```

PostgreSQL Error Handling:

sql

```sql
CREATE OR REPLACE PROCEDURE UpdateInventory(
    p_product_id INT,
    p_quantity_change INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_current_stock INT;
    v_product_name VARCHAR(100);
    v_error_context TEXT;
BEGIN
    -- Get current stock and product name
    SELECT stock_quantity, product_name
    INTO v_current_stock, v_product_name
    FROM products
    WHERE product_id = p_product_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Product % not found', p_product_id;
    END IF;

    -- Check if enough stock for negative changes
    IF v_current_stock + p_quantity_change < 0 THEN
        RAISE EXCEPTION 'Insufficient stock for %. Current: %, Change: %',
            v_product_name, v_current_stock, p_quantity_change;
    END IF;

    -- Perform update
    UPDATE products
    SET stock_quantity = stock_quantity + p_quantity_change,
        last_updated = CURRENT_TIMESTAMP
    WHERE product_id = p_product_id;

    -- Log the change
    INSERT INTO inventory_log (
        product_id,
        quantity_change,
        new_quantity,
        change_date,
```

```
        changed_by
    ) VALUES (
        p_product_id,
        p_quantity_change,
        v_current_stock + p_quantity_change,
        CURRENT_TIMESTAMP,
        CURRENT_USER
    );

EXCEPTION
    WHEN OTHERS THEN
        -- Get error context
        GET STACKED DIAGNOSTICS v_error_context = PG_EXCEPTION_CONTEXT;

        -- Log to error table
        INSERT INTO error_log (
            error_message,
            error_detail,
            error_hint,
            error_context,
            error_time,
            procedure_name,
            parameters
        ) VALUES (
            SQLERRM,
            SQLSTATE,
            NULL,
            v_error_context,
            CURRENT_TIMESTAMP,
            'UpdateInventory',
            format('product_id: %s, change: %s', p_product_id, p_quantity_change)
        );

        -- Re-raise
        RAISE;
END;
$$;
```

Error Handling Best Practices

Practice	Why	How
Validate inputs first	Fail fast, before transactions	Check parameters at start
Use transactions	Maintain data integrity	Wrap operations in BEGIN/COMMIT
Log errors	Debugging and auditing	Insert into error log table
Return meaningful messages	Help caller understand	Use custom error messages
Handle specific errors	Different responses for different errors	Multiple CATCH blocks
Clean up resources	Prevent leaks	Close cursors, release locks
Set appropriate isolation	Prevent concurrency issues	Use correct isolation level

---

Part 2: Triggers

A trigger is a special type of stored procedure that automatically executes when an event occurs in the database.

What are Triggers?

Definition: A trigger is a set of SQL statements that automatically "fires" or executes when certain events occur (INSERT, UPDATE, DELETE) on a table or view.

Trigger Types by Timing

Type	Description	Fires
BEFORE	Executes before the triggering event	Before INSERT/UPDATE/DELETE
AFTER	Executes after the triggering event	After INSERT/UPDATE/DELETE
INSTEAD OF	Replaces the triggering event	Instead of INSERT/UPDATE/DELET

Trigger Types by Event

Event	Description
INSERT	Fires when new rows are inserted
UPDATE	Fires when existing rows are updated
DELETE	Fires when rows are deleted

Special Tables in Triggers

Database	Inserted/New	Deleted/Old
MySQL	NEW	OLD

Database	Inserted/New	Deleted/Old
SQL Server	INSERTED	DELETED
PostgreSQL	NEW	OLD
Oracle	:NEW	:OLD

---

1. BEFORE Triggers

BEFORE triggers execute before the actual data modification occurs. They're useful for validation, default values, and preventing invalid operations.

MySQL BEFORE INSERT Trigger

sql

```sql
DELIMITER //

CREATE TRIGGER before_employee_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    -- Set default values
    IF NEW.hire_date IS NULL THEN
        SET NEW.hire_date = CURDATE();
    END IF;

    -- Auto-generate email if not provided
    IF NEW.email IS NULL THEN
        SET NEW.email = CONCAT(LOWER(NEW.first_name), '.',
                               LOWER(NEW.last_name), '@company.com');
    END IF;

    -- Validate age
```

```sql
    IF NEW.age < 18 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee must be at least 18 years old';
    END IF;

    -- Format name (capitalize first letter)
    SET NEW.first_name = CONCAT(UCASE(LEFT(NEW.first_name, 1)),
                                LCASE(SUBSTRING(NEW.first_name, 2)));
    SET NEW.last_name = CONCAT(UCASE(LEFT(NEW.last_name, 1)),
                               LCASE(SUBSTRING(NEW.last_name, 2)));
END //

DELIMITER ;
```

MySQL BEFORE UPDATE Trigger

sql

```sql
DELIMITER //

CREATE TRIGGER before_employee_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    -- Log old values before update
    INSERT INTO employee_audit (
        employee_id,
        action,
        old_salary,
        new_salary,
        changed_by,
        change_date
    ) VALUES (
        OLD.emp_id,
        'UPDATE',
        OLD.salary,
        NEW.salary,
        USER(),
        NOW()
    );

    -- Prevent salary decrease beyond certain threshold
```

```sql
    IF NEW.salary < OLD.salary * 0.9 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be decreased by more than 10%';
    END IF;

    -- Update timestamp
    SET NEW.last_updated = NOW();
END //

DELIMITER ;
```

SQL Server BEFORE Trigger (using INSTEAD OF)

sql

```sql
-- SQL Server doesn't have BEFORE triggers, use INSTEAD OF
CREATE TRIGGER tr_employees_insert
ON employees
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;

    -- Validate and modify data before insert
    INSERT INTO employees (
        emp_id, first_name, last_name, email,
        hire_date, salary, dept_id
    )
    SELECT
        i.emp_id,
        -- Capitalize names
        UPPER(LEFT(i.first_name, 1)) + LOWER(SUBSTRING(i.first_name, 2)),
        UPPER(LEFT(i.last_name, 1)) + LOWER(SUBSTRING(i.last_name, 2)),
        -- Generate email if NULL
        ISNULL(i.email,
               LOWER(i.first_name) + '.' + LOWER(i.last_name) + '@company.com'),
        -- Default hire date to today if NULL
        ISNULL(i.hire_date, GETDATE()),
        i.salary,
        i.dept_id
    FROM inserted i
    -- Validate age
```

```sql
    WHERE DATEDIFF(YEAR, i.birth_date, GETDATE()) >= 18;

    -- Handle invalid records
    IF @@ROWCOUNT < (SELECT COUNT(*) FROM inserted)
    BEGIN
        RAISERROR('Some records failed age validation', 16, 1);
    END
END;
```

PostgreSQL BEFORE Trigger

sql

```sql
-- First create a function for the trigger
CREATE OR REPLACE FUNCTION validate_employee()
RETURNS TRIGGER AS $$
BEGIN
    -- Auto-generate email if missing
    IF NEW.email IS NULL THEN
        NEW.email := LOWER(NEW.first_name) || '.' ||
                     LOWER(NEW.last_name) || '@company.com';
    END IF;

    -- Set hire date to now if missing
    IF NEW.hire_date IS NULL THEN
        NEW.hire_date := CURRENT_DATE;
    END IF;

    -- Validate salary
    IF NEW.salary < 0 THEN
        RAISE EXCEPTION 'Salary cannot be negative';
    END IF;

    -- Capitalize names
    NEW.first_name := INITCAP(NEW.first_name);
    NEW.last_name := INITCAP(NEW.last_name);

    -- Set audit fields
    NEW.created_at := CURRENT_TIMESTAMP;
    NEW.created_by := CURRENT_USER;

    RETURN NEW;
```

```sql
END;
$$ LANGUAGE plpgsql;

-- Create trigger
CREATE TRIGGER before_employee_insert
    BEFORE INSERT ON employees
    FOR EACH ROW
    EXECUTE FUNCTION validate_employee();
```

---

2. AFTER Triggers

AFTER triggers execute after the data modification has occurred. They're useful for auditing, cascading operations, and maintaining derived data.

MySQL AFTER INSERT Trigger

sql

```sql
DELIMITER //

CREATE TRIGGER after_order_insert
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
    -- Update customer's last order date
    UPDATE customers
    SET last_order_date = NEW.order_date,
        total_orders = total_orders + 1
    WHERE customer_id = NEW.customer_id;

    -- Create notification
    INSERT INTO notifications (
        user_id,
        notification_type,
        message,
        created_date
    ) VALUES (
        NEW.customer_id,
```

```sql
        'ORDER_CONFIRMATION',
        CONCAT('Your order #', NEW.order_id, ' has been placed'),
        NOW()
    );

    -- Update department sales summary
    UPDATE department_summary ds
    JOIN products p ON ds.dept_id = p.dept_id
    JOIN order_items oi ON p.product_id = oi.product_id
    SET ds.daily_sales = ds.daily_sales + (oi.quantity * oi.price)
    WHERE oi.order_id = NEW.order_id
    AND ds.summary_date = CURDATE();
END //

DELIMITER ;
```

MySQL AFTER UPDATE Trigger

sql

```sql
DELIMITER //

CREATE TRIGGER after_order_update
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    -- If status changed, log it
    IF OLD.status != NEW.status THEN
        INSERT INTO order_status_history (
            order_id,
            old_status,
            new_status,
            changed_by,
            changed_date
        ) VALUES (
            NEW.order_id,
            OLD.status,
            NEW.status,
            USER(),
            NOW()
        );
```

```sql
        -- If order is shipped, update inventory
        IF NEW.status = 'SHIPPED' THEN
            UPDATE products p
            JOIN order_items oi ON p.product_id = oi.product_id
            SET p.shipped_quantity = p.shipped_quantity + oi.quantity
            WHERE oi.order_id = NEW.order_id;
        END IF;

        -- If order is cancelled, restore inventory
        IF NEW.status = 'CANCELLED' AND OLD.status != 'CANCELLED' THEN
            UPDATE products p
            JOIN order_items oi ON p.product_id = oi.product_id
            SET p.stock_quantity = p.stock_quantity + oi.quantity
            WHERE oi.order_id = NEW.order_id;
        END IF;
    END IF;
END //

DELIMITER ;
```

SQL Server AFTER DELETE Trigger

sql

```sql
CREATE TRIGGER tr_employee_delete
ON employees
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- Archive deleted employees
    INSERT INTO employee_archive (
        emp_id,
        first_name,
        last_name,
        email,
        salary,
        dept_id,
        deleted_date,
        deleted_by
    )
```

```sql
    SELECT
        d.emp_id,
        d.first_name,
        d.last_name,
        d.email,
        d.salary,
        d.dept_id,
        GETDATE(),
        USER_NAME()
    FROM deleted d;

    -- Update department counts
    UPDATE departments
    SET employee_count = employee_count - 1
    WHERE dept_id IN (SELECT DISTINCT dept_id FROM deleted);

    -- Reassign any managed employees
    UPDATE employees
    SET manager_id = NULL
    WHERE manager_id IN (SELECT emp_id FROM deleted);
END;
```

PostgreSQL AFTER Trigger with Complex Logic

sql

```sql
-- Create function for after insert on sales
CREATE OR REPLACE FUNCTION process_new_sale()
RETURNS TRIGGER AS $$
DECLARE
    v_product RECORD;
    v_customer RECORD;
BEGIN
    -- Get product details
    SELECT * INTO v_product
    FROM products
    WHERE product_id = NEW.product_id;

    -- Update product statistics
    UPDATE product_stats
    SET
        total_sold = total_sold + NEW.quantity,
```

```sql
    last_sale_date = NEW.sale_date,
    revenue = revenue + (NEW.quantity * v_product.price)
WHERE product_id = NEW.product_id;

-- If this doesn't exist, insert it
IF NOT FOUND THEN
    INSERT INTO product_stats (
        product_id,
        total_sold,
        last_sale_date,
        revenue
    ) VALUES (
        NEW.product_id,
        NEW.quantity,
        NEW.sale_date,
        NEW.quantity * v_product.price
    );
END IF;

-- Update customer purchase summary
INSERT INTO customer_purchases (
    customer_id,
    total_spent,
    last_purchase_date,
    purchase_count
) VALUES (
    NEW.customer_id,
    NEW.quantity * v_product.price,
    NEW.sale_date,
    1
)
ON CONFLICT (customer_id) DO UPDATE
SET
    total_spent = customer_purchases.total_spent + EXCLUDED.total_spent,
    last_purchase_date = EXCLUDED.last_purchase_date,
    purchase_count = customer_purchases.purchase_count + 1;

-- Check for loyalty program eligibility
SELECT * INTO v_customer
FROM customers
WHERE customer_id = NEW.customer_id;
```

```
    IF v_customer.total_purchases > 10000 AND v_customer.loyalty_level = 'BRONZE'
THEN
        UPDATE customers
        SET loyalty_level = 'SILVER',
            loyalty_updated = CURRENT_TIMESTAMP
        WHERE customer_id = NEW.customer_id;

        -- Send notification
        INSERT INTO notifications (
            customer_id,
            type,
            message
        ) VALUES (
            NEW.customer_id,
            'LOYALTY_UPGRADE',
            'Congratulations! You''ve been upgraded to SILVER status!'
        );
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Create trigger
CREATE TRIGGER after_sale_insert
    AFTER INSERT ON sales
    FOR EACH ROW
    EXECUTE FUNCTION process_new_sale();
```

---

3. Audit Triggers

Audit triggers automatically track changes to data for compliance, security, and debugging purposes.

Comprehensive Audit Trail System

MySQL Audit Triggers:

sql

```sql
-- Create audit table
CREATE TABLE audit_log (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    table_name VARCHAR(100),
    record_id INT,
    action VARCHAR(10),
    old_data JSON,
    new_data JSON,
    changed_by VARCHAR(100),
    changed_at DATETIME
);

-- Audit trigger for employees table
DELIMITER //

CREATE TRIGGER audit_employees_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (
        table_name,
        record_id,
        action,
        new_data,
        changed_by,
        changed_at
    ) VALUES (
        'employees',
        NEW.emp_id,
        'INSERT',
        JSON_OBJECT(
            'emp_id', NEW.emp_id,
            'first_name', NEW.first_name,
            'last_name', NEW.last_name,
            'email', NEW.email,
            'salary', NEW.salary,
            'dept_id', NEW.dept_id
        ),
        USER(),
        NOW()
    );
```

```sql
END //

CREATE TRIGGER audit_employees_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (
        table_name,
        record_id,
        action,
        old_data,
        new_data,
        changed_by,
        changed_at
    ) VALUES (
        'employees',
        NEW.emp_id,
        'UPDATE',
        JSON_OBJECT(
            'emp_id', OLD.emp_id,
            'first_name', OLD.first_name,
            'last_name', OLD.last_name,
            'email', OLD.email,
            'salary', OLD.salary,
            'dept_id', OLD.dept_id
        ),
        JSON_OBJECT(
            'emp_id', NEW.emp_id,
            'first_name', NEW.first_name,
            'last_name', NEW.last_name,
            'email', NEW.email,
            'salary', NEW.salary,
            'dept_id', NEW.dept_id
        ),
        USER(),
        NOW()
    );
END //

CREATE TRIGGER audit_employees_delete
AFTER DELETE ON employees
FOR EACH ROW
```

```sql
BEGIN
    INSERT INTO audit_log (
        table_name,
        record_id,
        action,
        old_data,
        changed_by,
        changed_at
    ) VALUES (
        'employees',
        OLD.emp_id,
        'DELETE',
        JSON_OBJECT(
            'emp_id', OLD.emp_id,
            'first_name', OLD.first_name,
            'last_name', OLD.last_name,
            'email', OLD.email,
            'salary', OLD.salary,
            'dept_id', OLD.dept_id
        ),
        USER(),
        NOW()
    );
END //

DELIMITER ;
```

SQL Server Comprehensive Audit:

sql

```sql
-- Create audit tables
CREATE TABLE data_audit (
    audit_id BIGINT IDENTITY(1,1) PRIMARY KEY,
    audit_date DATETIME NOT NULL DEFAULT GETDATE(),
    audit_user VARCHAR(128) NOT NULL DEFAULT SUSER_NAME(),
    audit_app VARCHAR(128) NOT NULL DEFAULT APP_NAME(),
    audit_host VARCHAR(128) NOT NULL DEFAULT HOST_NAME(),
    table_name VARCHAR(128) NOT NULL,
    record_id INT NOT NULL,
    action_type CHAR(1) NOT NULL CHECK (action_type IN ('I','U','D')),
    column_name VARCHAR(128),
    old_value SQL_VARIANT,
```

```sql
    new_value SQL_VARIANT
);

-- Generic audit trigger for any table
CREATE TRIGGER tr_employees_audit
ON employees
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- Handle INSERTs
    INSERT INTO data_audit (
        table_name, record_id, action_type,
        column_name, new_value
    )
    SELECT
        'employees',
        i.emp_id,
        'I',
        c.name,
        CASE c.name
            WHEN 'emp_id' THEN i.emp_id
            WHEN 'first_name' THEN i.first_name
            WHEN 'last_name' THEN i.last_name
            WHEN 'email' THEN i.email
            WHEN 'salary' THEN i.salary
            WHEN 'dept_id' THEN i.dept_id
        END
    FROM inserted i
    CROSS JOIN (VALUES
        ('emp_id'), ('first_name'), ('last_name'),
        ('email'), ('salary'), ('dept_id')
    ) AS c(name)
    WHERE NOT EXISTS (SELECT 1 FROM deleted);

    -- Handle DELETEs
    INSERT INTO data_audit (
        table_name, record_id, action_type,
        column_name, old_value
    )
    SELECT
```

```sql
    'employees',
    d.emp_id,
    'D',
    c.name,
    CASE c.name
        WHEN 'emp_id' THEN d.emp_id
        WHEN 'first_name' THEN d.first_name
        WHEN 'last_name' THEN d.last_name
        WHEN 'email' THEN d.email
        WHEN 'salary' THEN d.salary
        WHEN 'dept_id' THEN d.dept_id
    END
FROM deleted d
CROSS JOIN (VALUES
    ('emp_id'), ('first_name'), ('last_name'),
    ('email'), ('salary'), ('dept_id')
) AS c(name)
WHERE NOT EXISTS (SELECT 1 FROM inserted);

-- Handle UPDATEs
INSERT INTO data_audit (
    table_name, record_id, action_type,
    column_name, old_value, new_value
)
SELECT
    'employees',
    i.emp_id,
    'U',
    c.name,
    CASE c.name
        WHEN 'emp_id' THEN d.emp_id
        WHEN 'first_name' THEN d.first_name
        WHEN 'last_name' THEN d.last_name
        WHEN 'email' THEN d.email
        WHEN 'salary' THEN d.salary
        WHEN 'dept_id' THEN d.dept_id
    END,
    CASE c.name
        WHEN 'emp_id' THEN i.emp_id
        WHEN 'first_name' THEN i.first_name
        WHEN 'last_name' THEN i.last_name
        WHEN 'email' THEN i.email
```

```sql
            WHEN 'salary' THEN i.salary
            WHEN 'dept_id' THEN i.dept_id
        END
    FROM inserted i
    JOIN deleted d ON i.emp_id = d.emp_id
    CROSS JOIN (VALUES
        ('emp_id'), ('first_name'), ('last_name'),
        ('email'), ('salary'), ('dept_id')
    ) AS c(name)
    WHERE
        (c.name = 'emp_id' AND i.emp_id != d.emp_id) OR
        (c.name = 'first_name' AND i.first_name != d.first_name) OR
        (c.name = 'last_name' AND i.last_name != d.last_name) OR
        (c.name = 'email' AND i.email != d.email) OR
        (c.name = 'salary' AND i.salary != d.salary) OR
        (c.name = 'dept_id' AND i.dept_id != d.dept_id);
END;
```

---

4. Event Triggers (DDL Triggers)

Event triggers fire in response to DDL (Data Definition Language) events like CREATE, ALTER, DROP.

SQL Server DDL Triggers

sql

```sql
-- Server-level trigger for database creation
CREATE TRIGGER tr_prevent_database_creation
ON ALL SERVER
FOR CREATE_DATABASE
AS
BEGIN
    PRINT 'Database creation attempted. Checking permissions...';

    IF IS_SRVROLEMEMBER('sysadmin') = 0
    BEGIN
        RAISERROR('Only sysadmins can create databases', 16, 1);
```

```
        ROLLBACK;
    END
END;

-- Database-level trigger for table changes
CREATE TRIGGER tr_audit_table_changes
ON DATABASE
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @EventData XML = EVENTDATA();

    INSERT INTO ddl_audit_log (
        event_type,
        object_name,
        object_type,
        login_name,
        command_text,
        event_time
    )
    VALUES (
        @EventData.value('(/EVENT_INSTANCE/EventType)[1]', 'VARCHAR(100)'),
        @EventData.value('(/EVENT_INSTANCE/ObjectName)[1]', 'VARCHAR(255)'),
        @EventData.value('(/EVENT_INSTANCE/ObjectType)[1]', 'VARCHAR(100)'),
        SUSER_NAME(),
        @EventData.value('(/EVENT_INSTANCE/TSQLCommand/CommandText)[1]', 'NVARCHAR
(MAX)'),
        GETDATE()
    );
END;

-- Prevent dropping important tables
CREATE TRIGGER tr_prevent_table_drop
ON DATABASE
FOR DROP_TABLE
AS
BEGIN
    DECLARE @EventData XML = EVENTDATA();
    DECLARE @TableName VARCHAR(255);
```

```
    SET @TableName = @EventData.value('(/EVENT_INSTANCE/ObjectName)[1]', 'VARCHAR(
255)');

    IF @TableName IN ('users', 'orders', 'products')
    BEGIN
        RAISERROR('Table %s is protected and cannot be dropped', 16, 1, @TableName
);
        ROLLBACK;
    END
END;
```

PostgreSQL Event Triggers

sql

```sql
-- Create event trigger function
CREATE OR REPLACE FUNCTION log_ddl_changes()
RETURNS event_trigger
LANGUAGE plpgsql
AS $$
DECLARE
    r RECORD;
BEGIN
    FOR r IN SELECT * FROM pg_event_trigger_ddl_commands()
    LOOP
        INSERT INTO ddl_audit_log (
            event_type,
            object_type,
            object_identity,
            command_tag,
            login_name,
            event_time
        ) VALUES (
            tg_event,
            r.object_type,
            r.object_identity,
            tg_tag,
            CURRENT_USER,
            CURRENT_TIMESTAMP
        );
    END LOOP;
END;
```

```sql
$$;

-- Create event trigger
CREATE EVENT TRIGGER log_ddl
    ON ddl_command_end
    EXECUTE FUNCTION log_ddl_changes();

-- Prevent certain DDL operations
CREATE OR REPLACE FUNCTION prevent_sensitive_ddl()
RETURNS event_trigger
LANGUAGE plpgsql
AS $$
DECLARE
    r RECORD;
BEGIN
    FOR r IN SELECT * FROM pg_event_trigger_ddl_commands()
    LOOP
        -- Prevent altering certain tables
        IF r.object_identity LIKE '%.users' OR
           r.object_identity LIKE '%.passwords' THEN
            RAISE EXCEPTION 'Modifying % is not allowed', r.object_identity;
        END IF;

        -- Prevent dropping tables during business hours
        IF tg_tag = 'DROP TABLE' AND
           EXTRACT(HOUR FROM CURRENT_TIME) BETWEEN 9 AND 17 THEN
            RAISE EXCEPTION 'Tables cannot be dropped during business hours';
        END IF;
    END LOOP;
END;
$$;

CREATE EVENT TRIGGER prevent_sensitive_changes
    ON ddl_command_start
    EXECUTE FUNCTION prevent_sensitive_ddl();
```

MySQL Event Triggers (Limited)

```sql
-- MySQL doesn't have full DDL triggers, but you can use
-- the audit plugin or simulate with events
```

```sql
-- Create a monitoring event
CREATE EVENT monitor_schema_changes
ON SCHEDULE EVERY 1 HOUR
DO
BEGIN
    -- Check for new tables
    INSERT INTO schema_audit (
        table_schema,
        table_name,
        create_time,
        detected_time
    )
    SELECT
        table_schema,
        table_name,
        create_time,
        NOW()
    FROM information_schema.tables
    WHERE table_schema = 'company_db'
    AND table_name NOT IN (
        SELECT table_name FROM schema_audit
        WHERE table_schema = 'company_db'
    );

    -- Check for dropped tables
    UPDATE schema_audit
    SET drop_detected_time = NOW(),
        is_active = FALSE
    WHERE is_active = TRUE
    AND table_schema = 'company_db'
    AND table_name NOT IN (
        SELECT table_name FROM information_schema.tables
        WHERE table_schema = 'company_db'
    );
END;
```

Trigger Best Practices

Do's and Don'ts

Do	Don't
Keep triggers simple and fast	Perform complex business logic
Use triggers for auditing	Create recursive triggers
Document trigger purpose	Assume execution order
Test thoroughly	Access remote data
Handle errors gracefully	Use cursors in triggers
Consider performance impact	Create too many triggers

Performance Considerations

sql

```sql
-- BAD: Slow trigger with cursor
CREATE TRIGGER slow_trigger
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE product_id INT;
    DECLARE cur CURSOR FOR SELECT product_id FROM old_products;
    -- This will be very slow!
END;

-- GOOD: Set-based operation
CREATE TRIGGER fast_trigger
AFTER INSERT ON orders
```

```sql
FOR EACH ROW
BEGIN
    UPDATE products
    SET stock = stock - (
        SELECT quantity
        FROM order_items
        WHERE order_id = NEW.order_id
        AND product_id = products.product_id
    )
    WHERE product_id IN (
        SELECT product_id
        FROM order_items
        WHERE order_id = NEW.order_id
    );
END;
```

Recursive Trigger Prevention

sql

```sql
-- MySQL: Prevent recursion
SET @@SESSION.max_sp_recursion_depth = 0;

-- SQL Server: Check for recursion
CREATE TRIGGER tr_prevent_recursion
ON employees
AFTER UPDATE
AS
BEGIN
    IF TRIGGER_NESTLEVEL() > 1
    BEGIN
        PRINT 'Trigger recursion detected';
        ROLLBACK;
        RETURN;
    END

    -- Actual trigger logic here
END;
```

Disabling/Enabling Triggers

sql

```sql
-- MySQL
DISABLE TRIGGER trigger_name;
ENABLE TRIGGER trigger_name;

-- SQL Server
DISABLE TRIGGER tr_employee_audit ON employees;
ENABLE TRIGGER tr_employee_audit ON employees;

-- Disable all triggers on a table
DISABLE TRIGGER ALL ON employees;

-- PostgreSQL
ALTER TABLE employees DISABLE TRIGGER trigger_name;
ALTER TABLE employees ENABLE TRIGGER trigger_name;
ALTER TABLE employees DISABLE TRIGGER ALL;
ALTER TABLE employees ENABLE TRIGGER ALL;
```

Summary: Stored Procedures vs Triggers

Aspect	Stored Procedures	Triggers
Execution	Explicitly called by user/application	Automatically fired by events
Parameters	Can accept parameters	No parameters
Return Values	Can return values	No return values
Transaction	Can manage transactions	Part of triggering transaction
Use Cases	Business logic, reports	Auditing, validation, cascading
Performance	Pre-compiled, efficient	Can impact DML performance

Aspect	Stored Procedures	Triggers
Maintenance	Easier to debug	Harder to debug
Flexibility	Highly flexible	Limited by event context

When to Use What

Scenario	Use
Complex business logic	Stored Procedure
Data validation	BEFORE Trigger
Auditing changes	AFTER Trigger
Reports and analytics	Stored Procedure
Maintain derived data	AFTER Trigger
Batch operations	Stored Procedure
Enforce referential integrity	Trigger
Application API	Stored Procedure

Window Functions and Advanced Query Techniques: Complete Theory Notes

Part 1: Window Functions (Modern SQL)

Window functions perform calculations across a set of rows related to the current row, without collapsing them into a single output row like GROUP BY.

Introduction to Window Functions

What are Window Functions?

Window functions perform calculations across a set of rows that are related to the current row. Unlike aggregate functions with GROUP BY, window functions do not collapse rows - each row retains its identity.

Key Concepts

Concept	Description
Window	The set of rows the function operates on
PARTITION BY	Divides rows into partitions (like GROUP BY)
ORDER BY	Defines the order within each partition
Frame	Subset of the partition (rows between boundaries)

Basic Syntax

sql

```sql
SELECT
    column1,
    column2,
    window_function() OVER (
        [PARTITION BY partition_columns]
        [ORDER BY order_columns]
        [frame_clause]
    ) AS alias
FROM table_name;
```

Sample Data for Examples

sql

```sql
-- Create sales table
CREATE TABLE sales (
    sale_id INT PRIMARY KEY,
    sale_date DATE,
    product_category VARCHAR(50),
    product_name VARCHAR(100),
    salesperson VARCHAR(50),
    region VARCHAR(50),
    quantity INT,
    unit_price DECIMAL(10,2),
    discount DECIMAL(5,2)
);

-- Insert sample data
INSERT INTO sales VALUES
(1, '2024-01-05', 'Electronics', 'Laptop', 'Alice', 'North', 2, 1200.00, 0),
(2, '2024-01-05', 'Electronics', 'Mouse', 'Bob', 'North', 10, 25.00, 5),
(3, '2024-01-06', 'Furniture', 'Desk', 'Alice', 'North', 1, 450.00, 10),
(4, '2024-01-06', 'Electronics', 'Laptop', 'Charlie', 'South', 1, 1200.00, 0),
(5, '2024-01-07', 'Furniture', 'Chair', 'Diana', 'South', 4, 200.00, 15),
(6, '2024-01-07', 'Electronics', 'Monitor', 'Alice', 'North', 2, 350.00, 0),
(7, '2024-01-08', 'Electronics', 'Mouse', 'Bob', 'North', 5, 25.00, 0),
(8, '2024-01-08', 'Furniture', 'Desk', 'Charlie', 'South', 2, 450.00, 10),
(9, '2024-01-09', 'Electronics', 'Laptop', 'Diana', 'South', 1, 1200.00, 0),
(10, '2024-01-09', 'Furniture', 'Chair', 'Alice', 'North', 3, 200.00, 10),
```

```
(11, '2024-01-10', 'Electronics', 'Monitor', 'Bob', 'North', 1, 350.00, 0),
(12, '2024-01-10', 'Electronics', 'Keyboard', 'Charlie', 'South', 3, 80.00, 5);
```

---

1. ROW_NUMBER()

Definition: Assigns a unique sequential number to each row within a partition, starting at 1.

Characteristics

- Always unique within partition
- No ties - even if values are equal, different numbers assigned
- Order determines which row gets which number

Basic Examples

Example 1: Simple Row Numbering

sql

```sql
SELECT
    sale_id,
    sale_date,
    product_name,
    salesperson,
    quantity,
    ROW_NUMBER() OVER (ORDER BY sale_date) AS row_num
FROM sales;

-- Result:
-- sale_id | sale_date  | product_name | salesperson | quantity | row_num
-- 1       | 2024-01-05 | Laptop       | Alice       | 2        | 1
-- 2       | 2024-01-05 | Mouse        | Bob         | 10       | 2
-- 3       | 2024-01-06 | Desk         | Alice       | 1        | 3
-- 4       | 2024-01-06 | Laptop       | Charlie     | 1        | 4
-- 5       | 2024-01-07 | Chair        | Diana       | 4        | 5
-- ...
```

Example 2: Row Numbering by Partition

sql

```sql
-- Number sales within each salesperson
SELECT
    salesperson,
    sale_date,
    product_name,
    quantity,
    ROW_NUMBER() OVER (PARTITION BY salesperson ORDER BY sale_date) AS sale_number
FROM sales
ORDER BY salesperson, sale_date;

-- Result:
-- salesperson | sale_date  | product_name | quantity | sale_number
-- Alice       | 2024-01-05 | Laptop       | 2        | 1
-- Alice       | 2024-01-06 | Desk         | 1        | 2
-- Alice       | 2024-01-07 | Monitor      | 2        | 3
-- Alice       | 2024-01-09 | Chair        | 3        | 4
-- Bob         | 2024-01-05 | Mouse        | 10       | 1
-- Bob         | 2024-01-08 | Mouse        | 5        | 2
-- Bob         | 2024-01-10 | Monitor      | 1        | 3
```

Example 3: Find Top N per Group

sql

```sql
-- Get the top 2 sales by quantity for each salesperson
WITH ranked_sales AS (
    SELECT
        salesperson,
        sale_date,
        product_name,
        quantity,
        ROW_NUMBER() OVER (PARTITION BY salesperson ORDER BY quantity DESC) AS ran
k
    FROM sales
)
SELECT *
FROM ranked_sales
WHERE rank <= 2
ORDER BY salesperson, rank;
```

```
-- Result:
-- salesperson | product_name | quantity | rank
-- Alice       | Laptop       | 2        | 1
-- Alice       | Chair        | 3        | 2? Wait, 3 > 2, so order matters!
-- Bob         | Mouse        | 10       | 1
-- Bob         | Mouse        | 5        | 2
```

Example 4: Pagination with ROW_NUMBER

sql

```sql
-- Get page 3 of sales (10 items per page)
WITH paginated AS (
    SELECT
        sale_id,
        sale_date,
        product_name,
        quantity,
        unit_price,
        ROW_NUMBER() OVER (ORDER BY sale_date DESC, sale_id) AS row_num
    FROM sales
)
SELECT *
FROM paginated
WHERE row_num BETWEEN 21 AND 30;  -- Page 3: rows 21-30
```

Example 5: Remove Duplicates

sql

```sql
-- Find and keep only the first occurrence of duplicate product names
WITH duplicates AS (
    SELECT
        product_name,
        sale_id,
        sale_date,
        ROW_NUMBER() OVER (PARTITION BY product_name ORDER BY sale_date) AS rn
    FROM sales
)
DELETE FROM sales
WHERE sale_id IN (
    SELECT sale_id
    FROM duplicates
    WHERE rn > 1
```

```
);
```

---

2. RANK()

Definition: Assigns a rank to each row within a partition, with gaps in ranking for ties.

Characteristics

- Same rank for tied values
- Next rank skips numbers (e.g., 1,1,3,4)
- Useful for competitions and leaderboards

Examples

Example 1: Basic Ranking

sql

```sql
SELECT
    product_name,
    quantity,
    RANK() OVER (ORDER BY quantity DESC) AS rank
FROM sales;

-- Result:
-- product_name | quantity | rank
-- Mouse        | 10       | 1
-- Mouse        | 5        | 2
-- Chair        | 4        | 3
-- Chair        | 3        | 4
-- Laptop       | 2        | 5
-- Laptop       | 2        | 5  (tied, same rank, next is 7)
-- Laptop       | 1        | 7
-- Monitor      | 2        | 5  (also tied for rank 5)
```

Example 2: Ranking with Partitions

sql

```sql
-- Rank salespeople by quantity within each region
SELECT
    region,
    salesperson,
    SUM(quantity) AS total_quantity,
    RANK() OVER (PARTITION BY region ORDER BY SUM(quantity) DESC) AS rank_in_regio
n
FROM sales
GROUP BY region, salesperson
ORDER BY region, rank_in_region;

-- Result:
-- region | salesperson | total_quantity | rank_in_region
-- North  | Alice       | 8              | 1
-- North  | Bob         | 16             | 2  (Alice actually has 2+1+2+3=8, Bob 1
0+5+1=16)
-- South  | Charlie     | 4              | 1
-- South  | Diana       | 5              | 2
```

Example 3: RANK vs ROW_NUMBER Comparison

sql

```sql
SELECT
    product_category,
    quantity,
    ROW_NUMBER() OVER (PARTITION BY product_category ORDER BY quantity DESC) AS ro
w_num,
    RANK() OVER (PARTITION BY product_category ORDER BY quantity DESC) AS rank_num
FROM sales
WHERE product_category = 'Electronics';

-- Result:
-- category    | quantity | row_num | rank_num
-- Electronics | 10       | 1       | 1
-- Electronics | 5        | 2       | 2
-- Electronics | 3        | 3       | 3
-- Electronics | 2        | 4       | 4
-- Electronics | 2        | 5       | 4  (tied, same rank, row_num still increment
s)
-- Electronics | 2        | 6       | 4
-- Electronics | 1        | 7       | 7
```

Example 4: Product Ranking by Revenue

sql

```sql
SELECT
    product_name,
    SUM(quantity * unit_price * (1 - discount/100)) AS revenue,
    RANK() OVER (ORDER BY SUM(quantity * unit_price * (1 - discount/100)) DESC) AS
revenue_rank
FROM sales
GROUP BY product_name
ORDER BY revenue_rank;

-- Result:
-- product_name | revenue     | revenue_rank
-- Laptop       | 3600        | 1
-- Desk         | 1215        | 2
-- Mouse        | 475         | 3
-- Chair        | 960         | 4  (Actually 4 chairs at $200 each)
-- Monitor      | 700         | 5
-- Keyboard     | 240         | 6
```

---

3. DENSE_RANK()

Definition: Assigns a rank to each row within a partition, with no gaps in ranking.

Characteristics

- Same rank for tied values
- Next rank is consecutive (e.g., 1,1,2,3)
- Useful when you want consecutive ranks

Examples

Example 1: DENSE_RANK vs RANK Comparison

sql

```sql
SELECT
    quantity,
    RANK() OVER (ORDER BY quantity DESC) AS rank,
    DENSE_RANK() OVER (ORDER BY quantity DESC) AS dense_rank
FROM sales;

-- Result:
-- quantity | rank | dense_rank
-- 10       | 1    | 1
-- 5        | 2    | 2
-- 4        | 3    | 3
-- 3        | 4    | 4
-- 2        | 5    | 5
-- 2        | 5    | 5  (same dense_rank as above)
-- 2        | 5    | 5
-- 1        | 8    | 6  (RANK skips to 8, DENSE_RANK goes to 6)
-- 1        | 8    | 6
-- 1        | 8    | 6
```

Example 2: Top Performance Tiers

sql

```sql
-- Create performance tiers based on quantity sold
SELECT
    salesperson,
    SUM(quantity) AS total_sold,
    DENSE_RANK() OVER (ORDER BY SUM(quantity) DESC) AS performance_tier
FROM sales
GROUP BY salesperson
ORDER BY performance_tier;

-- Result:
-- salesperson | total_sold | performance_tier
-- Bob         | 16         | 1
-- Alice       | 8          | 2
-- Diana       | 5          | 3
-- Charlie     | 4          | 4

-- Note: No gaps, even if there are ties
```

Example 3: Medal Distribution

sql

```sql
-- Assign medals based on scores
WITH athlete_scores AS (
    SELECT 'Athlete A' AS athlete, 95 AS score
    UNION SELECT 'Athlete B', 95
    UNION SELECT 'Athlete C', 90
    UNION SELECT 'Athlete D', 85
    UNION SELECT 'Athlete E', 85
    UNION SELECT 'Athlete F', 80
)
SELECT
    athlete,
    score,
    RANK() OVER (ORDER BY score DESC) AS rank_medal,
    DENSE_RANK() OVER (ORDER BY score DESC) AS dense_medal,
    CASE DENSE_RANK() OVER (ORDER BY score DESC)
        WHEN 1 THEN 'Gold'
        WHEN 2 THEN 'Silver'
        WHEN 3 THEN 'Bronze'
        ELSE 'No Medal'
    END AS medal
FROM athlete_scores
ORDER BY score DESC;

-- Result:
-- athlete   | score | rank_medal | dense_medal | medal
-- Athlete A | 95    | 1          | 1           | Gold
-- Athlete B | 95    | 1          | 1           | Gold
-- Athlete C | 90    | 3          | 2           | Silver
-- Athlete D | 85    | 4          | 3           | Bronze
-- Athlete E | 85    | 4          | 3           | Bronze
-- Athlete F | 80    | 6          | 4           | No Medal
```

---

4. LEAD() and LAG()

Definition: Access data from subsequent (LEAD) or preceding (LAG) rows without self-join.

Syntax

sql

```sql
LAG(column, offset, default) OVER (PARTITION BY ... ORDER BY ...)
LEAD(column, offset, default) OVER (PARTITION BY ... ORDER BY ...)
```

LAG Examples (Previous Row)

Example 1: Compare with Previous Sale

sql

```sql
SELECT
    sale_date,
    product_name,
    quantity,
    LAG(quantity) OVER (ORDER BY sale_date) AS prev_quantity,
    quantity - LAG(quantity) OVER (ORDER BY sale_date) AS quantity_change
FROM sales
ORDER BY sale_date;

-- Result:
-- sale_date  | product_name | quantity | prev_quantity | quantity_change
-- 2024-01-05 | Laptop       | 2        | NULL          | NULL
-- 2024-01-05 | Mouse        | 10       | 2             | 8
-- 2024-01-06 | Desk         | 1        | 10            | -9
-- 2024-01-06 | Laptop       | 1        | 1             | 0
-- 2024-01-07 | Chair        | 4        | 1             | 3
```

Example 2: Previous Sale by Same Salesperson

sql

```sql
SELECT
    salesperson,
    sale_date,
    product_name,
    quantity,
    LAG(product_name, 1, 'First Sale') OVER (
        PARTITION BY salesperson
        ORDER BY sale_date
    ) AS previous_product
FROM sales
```

```sql
ORDER BY salesperson, sale_date;

-- Result:
-- salesperson | sale_date  | product_name | quantity | previous_product
-- Alice       | 2024-01-05 | Laptop       | 2        | First Sale
-- Alice       | 2024-01-06 | Desk         | 1        | Laptop
-- Alice       | 2024-01-07 | Monitor      | 2        | Desk
-- Alice       | 2024-01-09 | Chair        | 3        | Monitor
```

Example 3: Year-over-Year Comparison

sql

```sql
WITH monthly_sales AS (
    SELECT
        DATE_TRUNC('month', sale_date) AS month,
        SUM(quantity * unit_price) AS revenue
    FROM sales
    GROUP BY DATE_TRUNC('month', sale_date)
)
SELECT
    month,
    revenue,
    LAG(revenue, 12) OVER (ORDER BY month) AS revenue_same_month_last_year,
    revenue - LAG(revenue, 12) OVER (ORDER BY month) AS yoy_change,
    (revenue - LAG(revenue, 12) OVER (ORDER BY month)) * 100.0 /
        LAG(revenue, 12) OVER (ORDER BY month) AS yoy_percentage
FROM monthly_sales;
```

LEAD Examples (Next Row)

Example 4: Compare with Next Sale

sql

```sql
SELECT
    sale_date,
    product_name,
    quantity,
    LEAD(quantity) OVER (ORDER BY sale_date) AS next_quantity,
    LEAD(quantity) OVER (ORDER BY sale_date) - quantity AS increase_to_next
FROM sales
ORDER BY sale_date;
```

```
-- Result:
-- sale_date   | product_name | quantity | next_quantity | increase_to_next
-- 2024-01-05 | Laptop       | 2        | 10            | 8
-- 2024-01-05 | Mouse        | 10       | 1             | -9
-- 2024-01-06 | Desk         | 1        | 1             | 0
-- 2024-01-06 | Laptop       | 1        | 4             | 3
```

Example 5: Next Sale Date for Customer

sql

```sql
SELECT
    salesperson,
    sale_date,
    product_name,
    LEAD(sale_date) OVER (PARTITION BY salesperson ORDER BY sale_date) AS next_sal
e_date,
    DATEDIFF(
        day,
        sale_date,
        LEAD(sale_date) OVER (PARTITION BY salesperson ORDER BY sale_date)
    ) AS days_until_next_sale
FROM sales
ORDER BY salesperson, sale_date;

-- Result:
-- salesperson | sale_date   | product_name | next_sale_date | days_until_next_sale
-- Alice       | 2024-01-05 | Laptop       | 2024-01-06     | 1
-- Alice       | 2024-01-06 | Desk         | 2024-01-07     | 1
-- Alice       | 2024-01-07 | Monitor      | 2024-01-09     | 2
-- Alice       | 2024-01-09 | Chair        | NULL           | NULL
```

Example 6: Find Gaps in Sales

sql

```sql
WITH daily_sales AS (
    SELECT DISTINCT sale_date
    FROM sales
    WHERE sale_date BETWEEN '2024-01-01' AND '2024-01-31'
)
SELECT
    sale_date,
```

```
    LEAD(sale_date) OVER (ORDER BY sale_date) AS next_sale_date,
    DATEDIFF(day, sale_date, LEAD(sale_date) OVER (ORDER BY sale_date)) AS gap_day
s
FROM daily_sales
WHERE DATEDIFF(day, sale_date, LEAD(sale_date) OVER (ORDER BY sale_date)) > 1;
```

---

5. OVER() with PARTITION BY

Definition: The OVER clause defines the window of rows the function operates on. PARTITION BY divides the result set into partitions.

Syntax Variations

sql

```sql
-- No PARTITION BY (entire result set)
ROW_NUMBER() OVER (ORDER BY column)

-- With PARTITION BY
ROW_NUMBER() OVER (PARTITION BY category ORDER BY column)

-- With multiple PARTITION BY columns
ROW_NUMBER() OVER (PARTITION BY category, region ORDER BY column)

-- With ORDER BY only (cumulative)
SUM(quantity) OVER (ORDER BY date)  -- Running total
```

Comprehensive Examples

Example 1: Multiple Window Functions Together

sql

```sql
SELECT
    salesperson,
    sale_date,
    product_name,
    quantity,
    ROW_NUMBER() OVER (PARTITION BY salesperson ORDER BY sale_date) AS sale_num,
```

```sql
    RANK() OVER (PARTITION BY salesperson ORDER BY quantity DESC) AS quantity_rank
,
    SUM(quantity) OVER (PARTITION BY salesperson) AS person_total,
    AVG(quantity) OVER (PARTITION BY salesperson) AS person_avg,
    quantity * 100.0 / SUM(quantity) OVER (PARTITION BY salesperson) AS pct_of_per
son_total
FROM sales
ORDER BY salesperson, sale_date;
```

Example 2: Partition by Multiple Columns

sql

```sql
-- Analyze sales by region and category
SELECT
    region,
    product_category,
    sale_date,
    quantity,
    SUM(quantity) OVER (PARTITION BY region, product_category) AS category_region_
total,
    RANK() OVER (PARTITION BY region, product_category ORDER BY quantity DESC) AS
rank_in_category
FROM sales
ORDER BY region, product_category, rank_in_category;
```

Example 3: Moving Window Frames

sql

```sql
-- Moving average of last 3 sales
SELECT
    sale_date,
    product_name,
    quantity,
    AVG(quantity) OVER (
        ORDER BY sale_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg_3
FROM sales;

-- Result:
-- sale_date  | product_name | quantity | moving_avg_3
-- 2024-01-05 | Laptop       | 2        | 2.0
```

```
-- 2024-01-05 | Mouse        | 10       | 6.0
-- 2024-01-06 | Desk         | 1        | 4.33
-- 2024-01-06 | Laptop       | 1        | 4.0
-- 2024-01-07 | Chair        | 4        | 2.0
```

Example 4: Different Frame Types

sql

```sql
SELECT
    sale_date,
    quantity,
    -- Default frame (RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
    SUM(quantity) OVER (ORDER BY sale_date) AS running_total_default,

    -- ROWS frame (physical rows)
    SUM(quantity) OVER (ORDER BY sale_date ROWS UNBOUNDED PRECEDING) AS running_to
tal_rows,

    -- RANGE frame (logical, based on ORDER BY values)
    SUM(quantity) OVER (ORDER BY sale_date RANGE UNBOUNDED PRECEDING) AS running_t
otal_range,

    -- Moving average of 3
    AVG(quantity) OVER (ORDER BY sale_date ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWIN
G) AS centered_avg_3
FROM sales;
```

---

6. Running Totals

Definition: Cumulative sums or aggregates that increase as you move through rows.

Examples

Example 1: Simple Running Total

sql

```sql
SELECT
    sale_date,
```

```sql
    product_name,
    quantity,
    SUM(quantity) OVER (ORDER BY sale_date) AS running_total_quantity,
    SUM(quantity * unit_price) OVER (ORDER BY sale_date) AS running_total_revenue
FROM sales
ORDER BY sale_date;

-- Result:
-- sale_date  | product_name | quantity | running_total_quantity | running_total_r
evenue
-- 2024-01-05 | Laptop       | 2        | 2                      | 2400.00
-- 2024-01-05 | Mouse        | 10       | 12                     | 2650.00
-- 2024-01-06 | Desk         | 1        | 13                     | 3100.00
-- 2024-01-06 | Laptop       | 1        | 14                     | 4300.00
-- 2024-01-07 | Chair        | 4        | 18                     | 5100.00
```

Example 2: Running Total by Category

sql

```sql
SELECT
    product_category,
    sale_date,
    product_name,
    quantity,
    SUM(quantity) OVER (
        PARTITION BY product_category
        ORDER BY sale_date
    ) AS category_running_total
FROM sales
ORDER BY product_category, sale_date;

-- Result:
-- category    | sale_date  | product_name | quantity | category_running_total
-- Electronics | 2024-01-05 | Laptop       | 2        | 2
-- Electronics | 2024-01-05 | Mouse        | 10       | 12
-- Electronics | 2024-01-06 | Laptop       | 1        | 13
-- Electronics | 2024-01-07 | Monitor      | 2        | 15
-- Electronics | 2024-01-08 | Mouse        | 5        | 20
-- Furniture   | 2024-01-06 | Desk         | 1        | 1
-- Furniture   | 2024-01-07 | Chair        | 4        | 5
-- Furniture   | 2024-01-08 | Desk         | 2        | 7
```

Example 3: Running Percentage

sql

```sql
WITH daily_totals AS (
    SELECT
        sale_date,
        SUM(quantity * unit_price) AS daily_revenue
    FROM sales
    GROUP BY sale_date
)
SELECT
    sale_date,
    daily_revenue,
    SUM(daily_revenue) OVER (ORDER BY sale_date) AS running_total,
    SUM(daily_revenue) OVER (ORDER BY sale_date) * 100.0 /
        SUM(daily_revenue) OVER () AS running_percentage
FROM daily_totals
ORDER BY sale_date;
```

Example 4: Running Total with Reset

sql

```sql
-- Running total that resets each month
SELECT
    sale_date,
    product_name,
    quantity,
    SUM(quantity) OVER (
        PARTITION BY YEAR(sale_date), MONTH(sale_date)
        ORDER BY sale_date
    ) AS monthly_running_total
FROM sales
ORDER BY sale_date;
```

Example 5: Complex Running Calculations

sql

```sql
SELECT
    salesperson,
    sale_date,
    product_name,
    quantity,
```

```sql
    unit_price,
    quantity * unit_price AS sale_amount,
    SUM(quantity * unit_price) OVER (
        PARTITION BY salesperson
        ORDER BY sale_date
    ) AS salesperson_running_total,
    AVG(quantity * unit_price) OVER (
        PARTITION BY salesperson
        ORDER BY sale_date
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg_sale,
    MAX(quantity * unit_price) OVER (
        PARTITION BY salesperson
        ORDER BY sale_date
        ROWS UNBOUNDED PRECEDING
    ) AS highest_so_far
FROM sales
ORDER BY salesperson, sale_date;
```

Part 2: Advanced Query Techniques

---

1. Common Table Expressions (CTE)

Definition: A CTE is a temporary result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement.

Benefits

- Improves readability
- Enables recursion
- Can be referenced multiple times
- Acts like a temporary view

Basic Syntax

sql

```sql
WITH cte_name AS (
    SELECT query
)
SELECT * FROM cte_name;
```

Multiple CTEs

sql

```sql
WITH
cte1 AS (
    SELECT ... FROM table1
),
cte2 AS (
    SELECT ... FROM table2
)
SELECT * FROM cte1 JOIN cte2 ON ...;
```

Practical Examples

Example 1: Simplify Complex Query

sql

```sql
-- Without CTE (hard to read)
SELECT
    d.dept_name,
    dept_stats.avg_salary,
    dept_stats.emp_count,
    dept_stats.avg_salary - company_stats.company_avg AS diff_from_company
FROM departments d
JOIN (
    SELECT
        dept_id,
        AVG(salary) AS avg_salary,
        COUNT(*) AS emp_count
    FROM employees
    GROUP BY dept_id
) dept_stats ON d.dept_id = dept_stats.dept_id
```

```sql
CROSS JOIN (
    SELECT AVG(salary) AS company_avg
    FROM employees
) company_stats;

-- With CTE (much cleaner)
WITH
dept_stats AS (
    SELECT
        dept_id,
        AVG(salary) AS avg_salary,
        COUNT(*) AS emp_count
    FROM employees
    GROUP BY dept_id
),
company_stats AS (
    SELECT AVG(salary) AS company_avg
    FROM employees
)
SELECT
    d.dept_name,
    dept_stats.avg_salary,
    dept_stats.emp_count,
    dept_stats.avg_salary - company_stats.company_avg AS diff_from_company
FROM departments d
JOIN dept_stats ON d.dept_id = dept_stats.dept_id
CROSS JOIN company_stats;
```

Example 2: Referencing Same CTE Multiple Times

sql

```sql
WITH high_earners AS (
    SELECT
        emp_id,
        first_name,
        last_name,
        salary,
        dept_id
    FROM employees
    WHERE salary > 70000
)
SELECT
```

```sql
    'High Earners' AS category,
    COUNT(*) AS count,
    AVG(salary) AS avg_salary
FROM high_earners
UNION ALL
SELECT
    'High Earners by Dept',
    COUNT(DISTINCT dept_id),
    NULL
FROM high_earners;
```

Example 3: CTE for Data Cleaning

sql

```sql
WITH cleaned_data AS (
    SELECT
        customer_id,
        UPPER(TRIM(name)) AS name,
        LOWER(TRIM(email)) AS email,
        COALESCE(phone, 'No Phone') AS phone,
        CASE
            WHEN age < 0 OR age > 120 THEN NULL
            ELSE age
        END AS valid_age
    FROM raw_customers
    WHERE customer_id IS NOT NULL
)
INSERT INTO customers (customer_id, name, email, phone, age)
SELECT * FROM cleaned_data
WHERE email LIKE '%@%.%';  -- Valid email format
```

Example 4: Hierarchical Data with CTE

sql

```sql
-- Find all products and their category paths
WITH category_path AS (
    SELECT
        category_id,
        category_name,
        parent_id,
        CAST(category_name AS VARCHAR(500)) AS path
    FROM categories
```

```sql
    WHERE parent_id IS NULL

    UNION ALL

    SELECT
        c.category_id,
        c.category_name,
        c.parent_id,
        CAST(cp.path || ' > ' || c.category_name AS VARCHAR(500))
    FROM categories c
    JOIN category_path cp ON c.parent_id = cp.category_id
)
SELECT * FROM category_path;
```

---

2. Recursive Queries

Definition: Recursive CTEs are CTEs that reference themselves, allowing you to traverse hierarchical or graph-structured data.

Syntax

sql

```sql
WITH RECURSIVE cte_name AS (
    -- Anchor member (initial query)
    SELECT ... FROM table WHERE base_condition

    UNION ALL

    -- Recursive member (references cte_name)
    SELECT ... FROM table
    JOIN cte_name ON ...
)
SELECT * FROM cte_name;
```

Examples

Example 1: Employee Hierarchy

sql

```sql
-- Find all employees under a manager (including indirect reports)
WITH RECURSIVE emp_hierarchy AS (
    -- Anchor: start with the top manager
    SELECT
        emp_id,
        first_name,
        last_name,
        manager_id,
        1 AS level,
        CAST(first_name AS VARCHAR(500)) AS path
    FROM employees
    WHERE emp_id = 105  -- Charlie's ID

    UNION ALL

    -- Recursive: find employees who report to someone in the hierarchy
    SELECT
        e.emp_id,
        e.first_name,
        e.last_name,
        e.manager_id,
        eh.level + 1,
        CAST(eh.path || ' > ' || e.first_name AS VARCHAR(500))
    FROM employees e
    JOIN emp_hierarchy eh ON e.manager_id = eh.emp_id
)
SELECT
    level,
    REPEAT('  ', level - 1) || first_name || ' ' || last_name AS indented_name,
    path
FROM emp_hierarchy
ORDER BY path;

-- Result:
-- level | indented_name          | path
-- 1     | Charlie Wilson         | Charlie
-- 2     |   John Doe            | Charlie > John
```

```
-- 2     |   Bob Johnson         | Charlie > Bob
-- 2     |   Jack Williams       | Charlie > Jack
-- 3     |     (reports to John) | Charlie > John > ...
```

Example 2: Organizational Chart with Levels

sql

```sql
WITH RECURSIVE org_chart AS (
    SELECT
        emp_id,
        first_name,
        last_name,
        manager_id,
        0 AS level,
        CAST(emp_id AS VARCHAR(1000)) AS path_ids
    FROM employees
    WHERE manager_id IS NULL  -- Top level

    UNION ALL

    SELECT
        e.emp_id,
        e.first_name,
        e.last_name,
        e.manager_id,
        oc.level + 1,
        CAST(oc.path_ids || ',' || e.emp_id AS VARCHAR(1000))
    FROM employees e
    JOIN org_chart oc ON e.manager_id = oc.emp_id
)
SELECT
    level,
    CONCAT(REPEAT('│   ', level), '├── ', first_name, ' ', last_name) AS org_tree,
    path_ids
FROM org_chart
ORDER BY path_ids;
```

Example 3: Bill of Materials (Product Structure)

sql

```sql
-- Create parts table
CREATE TABLE parts (
```

```sql
    part_id INT PRIMARY KEY,
    part_name VARCHAR(100),
    parent_part_id INT NULL,
    quantity_required INT
);

INSERT INTO parts VALUES
(1, 'Product A', NULL, 1),
(2, 'Subassembly B', 1, 2),
(3, 'Subassembly C', 1, 1),
(4, 'Part D', 2, 4),
(5, 'Part E', 2, 2),
(6, 'Part F', 3, 3),
(7, 'Raw Material G', 4, 2),
(8, 'Raw Material H', 5, 1);

-- Recursive BOM explosion
WITH RECURSIVE bom_explosion AS (
    SELECT
        part_id,
        part_name,
        parent_part_id,
        quantity_required,
        1 AS level,
        CAST(part_name AS VARCHAR(1000)) AS path,
        quantity_required AS total_quantity
    FROM parts
    WHERE parent_part_id IS NULL  -- Top level

    UNION ALL

    SELECT
        p.part_id,
        p.part_name,
        p.parent_part_id,
        p.quantity_required,
        be.level + 1,
        CAST(be.path || ' > ' || p.part_name AS VARCHAR(1000)),
        be.total_quantity * p.quantity_required AS total_quantity
    FROM parts p
    JOIN bom_explosion be ON p.parent_part_id = be.part_id
)
```

```sql
SELECT
    level,
    REPEAT('  ', level - 1) || part_name AS indented_name,
    quantity_required AS qty_per_parent,
    total_quantity AS total_qty_for_top_product,
    path
FROM bom_explosion
ORDER BY path;
```

Example 4: Generate Date Series

sql

```sql
WITH RECURSIVE dates AS (
    SELECT DATE('2024-01-01') AS date
    UNION ALL
    SELECT DATE_ADD(date, INTERVAL 1 DAY)
    FROM dates
    WHERE date < DATE('2024-12-31')
)
SELECT
    date,
    DAYOFWEEK(date) AS day_of_week,
    DAYOFYEAR(date) AS day_of_year,
    WEEK(date) AS week_number,
    QUARTER(date) AS quarter
FROM dates;
```

Example 5: Find Path Between Two Nodes

sql

```sql
-- Find connection between two employees
WITH RECURSIVE emp_connections AS (
    SELECT
        emp_id,
        manager_id,
        CAST(emp_id AS CHAR(500)) AS path,
        0 AS depth
    FROM employees
    WHERE emp_id = 101  -- Start employee

    UNION ALL
```

```sql
    SELECT
        e.emp_id,
        e.manager_id,
        CAST(CONCAT(ec.path, ',', e.emp_id) AS CHAR(500)),
        ec.depth + 1
    FROM employees e
    JOIN emp_connections ec ON e.manager_id = ec.emp_id
    WHERE ec.depth < 10  -- Limit depth
)
SELECT * FROM emp_connections
WHERE emp_id = 105;  -- Target manager
```

---

3. PIVOT / UNPIVOT

PIVOT rotates rows into columns, UNPIVOT rotates columns into rows. Different databases have different syntax.

SQL Server PIVOT

Example 1: Basic PIVOT - Sales by Quarter

sql

```sql
-- First, create sample data
CREATE TABLE sales_by_quarter (
    salesperson VARCHAR(50),
    quarter VARCHAR(10),
    sales_amount DECIMAL(10,2)
);

INSERT INTO sales_by_quarter VALUES
('Alice', 'Q1', 15000),
('Alice', 'Q2', 18000),
('Alice', 'Q3', 16500),
('Alice', 'Q4', 21000),
('Bob', 'Q1', 12000),
('Bob', 'Q2', 14000),
('Bob', 'Q3', 13500),
```

```sql
('Bob', 'Q4', 19000),
('Charlie', 'Q1', 9000),
('Charlie', 'Q2', 11000),
('Charlie', 'Q3', 12500),
('Charlie', 'Q4', 14500);

-- PIVOT: Convert quarters to columns
SELECT *
FROM (
    SELECT salesperson, quarter, sales_amount
    FROM sales_by_quarter
) AS source
PIVOT (
    SUM(sales_amount)
    FOR quarter IN ([Q1], [Q2], [Q3], [Q4])
) AS pivot_table;

-- Result:
-- salesperson | Q1     | Q2     | Q3     | Q4
-- Alice       | 15000  | 18000  | 16500  | 21000
-- Bob         | 12000  | 14000  | 13500  | 19000
-- Charlie     | 9000   | 11000  | 12500  | 14500
```

Example 2: PIVOT with Multiple Aggregates

sql

```sql
SELECT *
FROM (
    SELECT
        product_category,
        sale_date,
        quantity,
        unit_price
    FROM sales
) AS source
PIVOT (
    SUM(quantity) FOR product_category IN ([Electronics], [Furniture])
) AS qty_pivot
PIVOT (
    AVG(unit_price) FOR product_category IN ([Electronics], [Furniture])
) AS price_pivot;
```

MySQL PIVOT Simulation (using CASE)

sql

```sql
-- MySQL doesn't have PIVOT, use CASE with GROUP BY
SELECT
    salesperson,
    SUM(CASE WHEN quarter = 'Q1' THEN sales_amount ELSE 0 END) AS Q1,
    SUM(CASE WHEN quarter = 'Q2' THEN sales_amount ELSE 0 END) AS Q2,
    SUM(CASE WHEN quarter = 'Q3' THEN sales_amount ELSE 0 END) AS Q3,
    SUM(CASE WHEN quarter = 'Q4' THEN sales_amount ELSE 0 END) AS Q4,
    SUM(sales_amount) AS total
FROM sales_by_quarter
GROUP BY salesperson
ORDER BY salesperson;

-- Dynamic pivot (if quarters are unknown)
SET @sql = NULL;
SELECT
    GROUP_CONCAT(DISTINCT
        CONCAT(
            'SUM(CASE WHEN quarter = ''',
            quarter,
            ''' THEN sales_amount ELSE 0 END) AS `',
            quarter, '`'
        )
    ) INTO @sql
FROM sales_by_quarter;

SET @sql = CONCAT('SELECT salesperson, ', @sql, ', SUM(sales_amount) AS total
                  FROM sales_by_quarter GROUP BY salesperson');

PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;
```

UNPIVOT Examples

Example 3: SQL Server UNPIVOT

sql

```sql
-- First create pivoted table
```

```sql
CREATE TABLE monthly_sales (
    salesperson VARCHAR(50),
    Jan DECIMAL(10,2),
    Feb DECIMAL(10,2),
    Mar DECIMAL(10,2)
);

INSERT INTO monthly_sales VALUES
('Alice', 5000, 5500, 6000),
('Bob', 4500, 4700, 4900),
('Charlie', 3000, 3200, 3500);

-- UNPIVOT to get rows
SELECT salesperson, month, sales_amount
FROM monthly_sales
UNPIVOT (
    sales_amount FOR month IN (Jan, Feb, Mar)
) AS unpvt;

-- Result:
-- salesperson | month | sales_amount
-- Alice       | Jan   | 5000
-- Alice       | Feb   | 5500
-- Alice       | Mar   | 6000
-- Bob         | Jan   | 4500
-- Bob         | Feb   | 4700
-- Bob         | Mar   | 4900
```

Example 4: MySQL UNPIVOT Simulation

sql

```sql
SELECT salesperson, 'Jan' AS month, Jan AS sales_amount FROM monthly_sales
UNION ALL
SELECT salesperson, 'Feb', Feb FROM monthly_sales
UNION ALL
SELECT salesperson, 'Mar', Mar FROM monthly_sales
ORDER BY salesperson, month;
```

---

4. JSON Queries

Modern databases support JSON data types and querying.

MySQL JSON Functions

Example 1: Creating and Querying JSON

sql

```sql
-- Create table with JSON column
CREATE TABLE products_json (
    product_id INT PRIMARY KEY,
    product_data JSON
);

-- Insert JSON data
INSERT INTO products_json VALUES
(1, '{
    "name": "Laptop Pro",
    "category": "Electronics",
    "price": 1299.99,
    "specs": {
        "ram": "16GB",
        "storage": "512GB SSD",
        "processor": "Intel i7"
    },
    "tags": ["premium", "laptop", "workstation"],
    "reviews": [
        {"user": "John", "rating": 5, "comment": "Excellent"},
        {"user": "Jane", "rating": 4, "comment": "Good value"}
    ]
}');

-- Extract JSON values
SELECT
    product_id,
    JSON_EXTRACT(product_data, '$.name') AS name,
    JSON_EXTRACT(product_data, '$.price') AS price,
    JSON_EXTRACT(product_data, '$.specs.ram') AS ram
FROM products_json;
```

```sql
-- Extract with path operators (MySQL 8.0)
SELECT
    product_id,
    product_data->>'$.name' AS name,
    product_data->>'$.specs.processor' AS processor
FROM products_json
WHERE product_data->>'$.category' = 'Electronics';

-- Search in JSON arrays
SELECT * FROM products_json
WHERE JSON_CONTAINS(product_data->'$.tags', '"premium"');

-- Aggregate JSON
SELECT
    product_data->>'$.category' AS category,
    JSON_ARRAYAGG(product_data->>'$.name') AS products
FROM products_json
GROUP BY product_data->>'$.category';
```

Example 2: JSON Table Functions

sql

```sql
-- Convert JSON array to rows
SELECT
    p.product_id,
    p.product_data->>'$.name' AS product_name,
    r.*
FROM products_json p,
JSON_TABLE(
    p.product_data,
    '$.reviews[*]'
    COLUMNS(
        review_id FOR ORDINALITY,
        reviewer VARCHAR(50) PATH '$.user',
        rating INT PATH '$.rating',
        comment TEXT PATH '$.comment'
    )
) AS r;

-- Result:
-- product_id | product_name | review_id | reviewer | rating | comment
-- 1          | Laptop Pro   | 1         | John     | 5      | Excellent
```

```
-- 1          | Laptop Pro    | 2          | Jane      | 4       | Good value
```

PostgreSQL JSON Functions

sql

```sql
-- PostgreSQL JSONB examples
CREATE TABLE products_jsonb (
    product_id INT PRIMARY KEY,
    product_data JSONB
);

-- Query JSONB
SELECT
    product_id,
    product_data->>'name' AS name,
    product_data->'specs'->>'ram' AS ram
FROM products_jsonb
WHERE product_data @> '{"category": "Electronics"}';

-- Index JSONB for performance
CREATE INDEX idx_products_category ON products_jsonb
    USING GIN ((product_data->'category'));

-- Update JSON field
UPDATE products_jsonb
SET product_data = jsonb_set(
    product_data,
    '{price}',
    '1399.99'
)
WHERE product_id = 1;
```

SQL Server JSON

sql

```sql
-- SQL Server JSON support
SELECT
    product_id,
    JSON_VALUE(product_data, '$.name') AS name,
    JSON_VALUE(product_data, '$.specs.ram') AS ram,
```

```sql
    JSON_QUERY(product_data, '$.tags') AS tags
FROM products_json
WHERE JSON_VALUE(product_data, '$.category') = 'Electronics';

-- Modify JSON
UPDATE products_json
SET product_data = JSON_MODIFY(
    product_data,
    '$.price',
    1399.99
)
WHERE product_id = 1;

-- Convert JSON to table
SELECT *
FROM OPENJSON((SELECT product_data FROM products_json WHERE product_id = 1))
WITH (
    name VARCHAR(100) '$.name',
    price DECIMAL '$.price',
    ram VARCHAR(20) '$.specs.ram'
);
```

---

5. Dynamic SQL

Definition: Dynamic SQL is SQL code that is generated and executed at runtime.

Benefits and Risks

Benefits	Risks
Flexibility	SQL Injection
Handle dynamic structures	Harder to debug

Benefits	Risks
Build complex queries	Performance overhead
Runtime decision making	No compile-time checking

MySQL Dynamic SQL

sql

```sql
-- Basic dynamic SQL
SET @table_name = 'employees';
SET @sql = CONCAT('SELECT * FROM ', @table_name, ' LIMIT 10');

PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;

-- Dynamic WHERE clause
SET @column = 'salary';
SET @value = 70000;
SET @operator = '>';

SET @sql = CONCAT('SELECT * FROM employees WHERE ', @column, ' ', @operator, ' ?')
;

PREPARE stmt FROM @sql;
EXECUTE stmt USING @value;
DEALLOCATE PREPARE stmt;

-- Dynamic pivot generation
SET @sql = NULL;
SELECT
    GROUP_CONCAT(DISTINCT
        CONCAT(
            'MAX(CASE WHEN category = ''',
            category,
            ''' THEN amount END) AS `',
            category, '`'
        )
```

```sql
    ) INTO @sql
FROM expenses;

SET @sql = CONCAT('SELECT department, ', @sql, '
                   FROM expenses
                   GROUP BY department');

PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;
```

SQL Server Dynamic SQL

sql

```sql
-- SQL Server dynamic SQL with sp_executesql
DECLARE @sql NVARCHAR(MAX);
DECLARE @table_name NVARCHAR(128) = 'employees';
DECLARE @column_name NVARCHAR(128) = 'salary';
DECLARE @min_salary INT = 50000;

SET @sql = N'SELECT * FROM ' + QUOTENAME(@table_name) +
           N' WHERE ' + QUOTENAME(@column_name) + N' > @min';

EXEC sp_executesql @sql, N'@min INT', @min = @min_salary;

-- Dynamic search with multiple parameters
CREATE PROCEDURE SearchEmployees
    @FirstName NVARCHAR(50) = NULL,
    @LastName NVARCHAR(50) = NULL,
    @DeptID INT = NULL,
    @MinSalary DECIMAL = NULL,
    @MaxSalary DECIMAL = NULL
AS
BEGIN
    DECLARE @sql NVARCHAR(MAX);
    DECLARE @params NVARCHAR(MAX);
    DECLARE @where_clause NVARCHAR(MAX) = ' WHERE 1=1';

    SET @sql = 'SELECT * FROM employees';

    IF @FirstName IS NOT NULL
```

```
        SET @where_clause = @where_clause + ' AND first_name LIKE ''%' + @FirstNam
e + '%''';

    IF @LastName IS NOT NULL
        SET @where_clause = @where_clause + ' AND last_name LIKE ''%' + @LastName
+ '%''';

    IF @DeptID IS NOT NULL
        SET @where_clause = @where_clause + ' AND dept_id = @dept';

    IF @MinSalary IS NOT NULL
        SET @where_clause = @where_clause + ' AND salary >= @min';

    IF @MaxSalary IS NOT NULL
        SET @where_clause = @where_clause + ' AND salary <= @max';

    SET @sql = @sql + @where_clause + ' ORDER BY last_name, first_name';

    SET @params = N'@dept INT, @min DECIMAL, @max DECIMAL';

    EXEC sp_executesql @sql, @params,
        @dept = @DeptID,
        @min = @MinSalary,
        @max = @MaxSalary;
END;
```

PostgreSQL Dynamic SQL

sql

```sql
-- PostgreSQL dynamic SQL with EXECUTE
CREATE OR REPLACE FUNCTION search_employees(
    p_column TEXT,
    p_value TEXT
) RETURNS SETOF employees AS $$
DECLARE
    query TEXT;
BEGIN
    query := format('SELECT * FROM employees WHERE %I = %L', p_column, p_value);
    RETURN QUERY EXECUTE query;
END;
$$ LANGUAGE plpgsql;
```

```sql
-- Dynamic table creation
CREATE OR REPLACE FUNCTION create_monthly_table(
    p_year INT,
    p_month INT
) RETURNS VOID AS $$
DECLARE
    table_name TEXT;
    query TEXT;
BEGIN
    table_name := format('sales_%s_%s', p_year, LPAD(p_month::TEXT, 2, '0'));

    query := format('
        CREATE TABLE %I (
            sale_id SERIAL PRIMARY KEY,
            sale_date DATE NOT NULL,
            amount DECIMAL(10,2) NOT NULL,
            customer_id INT
        )', table_name);

    EXECUTE query;

    -- Add partition constraint
    query := format('
        ALTER TABLE %I ADD CONSTRAINT %I
        CHECK (EXTRACT(YEAR FROM sale_date) = %s
               AND EXTRACT(MONTH FROM sale_date) = %s)',
        table_name,
        format('check_%s', table_name),
        p_year, p_month);

    EXECUTE query;
END;
$$ LANGUAGE plpgsql;
```

SQL Injection Prevention

sql

```sql
-- BAD: Vulnerable to SQL injection
SET @user_input = '1; DROP TABLE users; --';
SET @sql = CONCAT('SELECT * FROM users WHERE user_id = ', @user_input);
```

```sql
PREPARE stmt FROM @sql;  -- DANGEROUS!

-- GOOD: Use parameterized queries
SET @sql = 'SELECT * FROM users WHERE user_id = ?';
PREPARE stmt FROM @sql;
EXECUTE stmt USING @user_input;  -- Safe, treated as value only

-- GOOD: Use QUOTENAME (SQL Server)
SET @sql = 'SELECT * FROM ' + QUOTENAME(@table_name) +
           ' WHERE ' + QUOTENAME(@column_name) + ' = @value';

-- GOOD: Use quote_ident (PostgreSQL)
EXECUTE format('SELECT * FROM %I WHERE %I = %L',
    @table_name, @column_name, @value);
```

Summary: Window Functions Quick Reference

Function	Description	Use Case
ROW_NUMBER()	Unique sequential number	Pagination, deduplication
RANK()	Rank with gaps	Leaderboards
DENSE_RANK()	Rank without gaps	Tiers, medals
LAG()	Previous row value	Compare with past
LEAD()	Next row value	Future comparison
SUM() OVER	Running total	Cumulative calculations
AVG() OVER	Moving average	Trend analysis

Window Function Frame Specifications

Frame	Description
`ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING`	Current row plus neighbors
`ROWS UNBOUNDED PRECEDING`	All rows from start to curren
`ROWS BETWEEN 3 PRECEDING AND CURRENT ROW`	Last 3 rows including curren
`RANGE BETWEEN INTERVAL 7 DAY PRECEDING AND CURRENT ROW`	Last 7 days

Summary: Advanced Techniques Quick Reference

Technique	Purpose	Example
CTE	Simplify complex queries	`WITH cte AS (SELECT ...)`
Recursive CTE	Hierarchical data	`WITH RECURSIVE org AS (...)`
PIVOT	Rows to columns	`SELECT ... PIVOT (...)`
UNPIVOT	Columns to rows	`SELECT ... UNPIVOT (...)`
JSON	Semi-structured data	`JSON_EXTRACT(data, '$.key')`
Dynamic SQL	Runtime query building	`PREPARE stmt FROM @sql`

Performance Optimization and Security in SQL: Complete Theory Notes

Part 1: Performance Optimization

Performance optimization is the process of improving query execution speed and resource utilization. It's crucial for maintaining responsive applications as data grows.

---

1. Query Execution Plan

What is an Execution Plan?

An execution plan is a roadmap that the database optimizer creates to execute a query. It shows the steps, operations, and order of operations the database will use.

Why Execution Plans Matter

Reason	Description
Understand Performance	See why queries are slow
Identify Bottlenecks	Find expensive operations
Verify Index Usage	Check if indexes are used
Compare Alternatives	Test different query approaches
Optimize Effectively	Make data-driven decisions

How to Get Execution Plans

MySQL:

sql

```sql
-- Explain a query
EXPLAIN SELECT * FROM employees WHERE salary > 70000;

-- More detailed (MySQL 8.0)
EXPLAIN ANALYZE SELECT * FROM employees WHERE salary > 70000;

-- Format as JSON
EXPLAIN FORMAT=JSON
SELECT * FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
WHERE e.salary > 70000;
```

SQL Server:

sql

```sql
-- Include actual execution plan
SET STATISTICS PROFILE ON;
SELECT * FROM employees WHERE salary > 70000;
SET STATISTICS PROFILE OFF;

-- Estimated plan
SET SHOWPLAN_XML ON;
SELECT * FROM employees WHERE salary > 70000;
SET SHOWPLAN_XML OFF;

-- Via query window (Ctrl+L)
```

PostgreSQL:

sql

```sql
-- Explain
EXPLAIN SELECT * FROM employees WHERE salary > 70000;

-- Explain with actual execution
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM employees WHERE salary > 70000;
```

```sql
-- Format JSON
EXPLAIN (FORMAT JSON)
SELECT * FROM employees WHERE salary > 70000;
```

Reading Execution Plans

Example Plan and Interpretation:

```sql
EXPLAIN SELECT e.first_name, e.last_name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
WHERE e.salary > 70000
ORDER BY e.last_name;

-- MySQL Output:
-- id | select_type | table | type   | possible_keys | key        | rows | Extra
-- 1  | SIMPLE      | e     | range  | idx_salary    | idx_salary| 500  | Using wh
ere; Using temporary; Using filesort
-- 1  | SIMPLE      | d     | eq_ref | PRIMARY       | PRIMARY    | 1    |

-- Interpretation:
-- 1. Uses index on salary to filter (good)
-- 2. Estimated 500 rows after filter
-- 3. Needs to create temporary table for sorting (bad)
-- 4. Department lookup uses primary key (excellent)
```

Key Execution Plan Terms

Term	Meaning	Good/Bad
Table Scan	Reading entire table	Bad for large tables
Index Seek	Looking up specific index entries	Excellent
Index Scan	Reading entire index	Good if covering

Term	Meaning	Good/Bad
Key Lookup	Fetching row after index find	Can be expensive
Nested Loops	Joining one row at a time	Good for small sets
Hash Join	Building hash table for join	Good for large sets
Merge Join	Using sorted inputs	Good for pre-sorted data
Sort	Explicit sorting operation	Expensive
Temporary Table	Creating temp storage	Often bad
Filesort	MySQL sorting without index	Bad for large sets

Using Execution Plans for Optimization

sql

```sql
-- Step 1: Run EXPLAIN on slow query
EXPLAIN
SELECT * FROM orders
WHERE customer_id = 123
ORDER BY order_date DESC
LIMIT 10;

-- If you see "Using filesort" or "Using temporary", optimize

-- Step 2: Check if proper indexes exist
SHOW INDEX FROM orders;

-- Step 3: Add appropriate index
CREATE INDEX idx_customer_date ON orders(customer_id, order_date DESC);

-- Step 4: Re-explain to verify improvement
EXPLAIN
```

```sql
SELECT * FROM orders
WHERE customer_id = 123
ORDER BY order_date DESC
LIMIT 10;
-- Should now show "Using index condition" or better
```

---

2. Query Tuning

Query tuning involves rewriting queries to execute more efficiently.

Golden Rules of Query Tuning

1. **Select Only Needed Columns**
   sql

```sql
-- BAD: Select all columns
SELECT * FROM orders WHERE customer_id = 123;

-- GOOD: Select only needed columns
SELECT order_id, order_date, total_amount
FROM orders
WHERE customer_id = 123;
```

2. **Use WHERE Clauses Early**
   sql

```sql
-- BAD: Filter after join
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '2024-01-01';

-- GOOD: Filter before join
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN (
    SELECT * FROM orders
    WHERE order_date >= '2024-01-01'
) o ON c.customer_id = o.customer_id;
```

3. **Avoid Functions on Indexed Columns**

sql

```sql
-- BAD: Function on column prevents index use
SELECT * FROM employees
WHERE YEAR(hire_date) = 2024;

-- GOOD: Rewrite to allow index use
SELECT * FROM employees
WHERE hire_date >= '2024-01-01'
  AND hire_date < '2025-01-01';
```

4. **Use EXISTS Instead of IN for Subqueries**

sql

```sql
-- SLOW with large lists
SELECT * FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM orders
    WHERE total_amount > 1000
);

-- FASTER (short-circuits when found)
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
    AND o.total_amount > 1000
);
```

5. **Avoid Wildcards at Start of LIKE**

sql

```sql
-- BAD: Cannot use index
SELECT * FROM products
WHERE product_name LIKE '%laptop%';

-- GOOD: Can use index (if exists)
SELECT * FROM products
WHERE product_name LIKE 'laptop%';
```

6. **Optimize OR Conditions**

sql

```sql
-- BAD: May not use indexes efficiently
```

```sql
SELECT * FROM orders
WHERE status = 'PENDING'
   OR customer_id = 123;

-- GOOD: Use UNION to leverage separate indexes
SELECT * FROM orders WHERE status = 'PENDING'
UNION ALL
SELECT * FROM orders WHERE customer_id = 123;
```

7. **Use Appropriate Data Types**

```sql
-- BAD: Implicit conversion
SELECT * FROM orders
WHERE order_id = '123';  -- String vs INT

-- GOOD: Match data types
SELECT * FROM orders
WHERE order_id = 123;    -- INT vs INT
```

8. **Limit Results Early**

```sql
-- BAD: Process all rows then limit
SELECT * FROM products
ORDER BY price DESC;  -- Then limit in app

-- GOOD: Limit in query
SELECT * FROM products
ORDER BY price DESC
LIMIT 10;
```

Query Tuning Examples

Example 1: Inefficient Query Transformation

```sql
-- Original slow query
SELECT
    c.customer_id,
    c.name,
    (SELECT COUNT(*) FROM orders o WHERE o.customer_id = c.customer_id) AS order_c
ount,
```

```sql
    (SELECT SUM(o.total_amount) FROM orders o WHERE o.customer_id = c.customer_id)
AS total_spent
FROM customers c
WHERE c.status = 'ACTIVE';

-- Optimized with JOIN and GROUP BY
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) AS order_count,
    SUM(o.total_amount) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.status = 'ACTIVE'
GROUP BY c.customer_id, c.name;
```

Example 2: Pagination Optimization

```sql
-- SLOW for large offsets (offset 100000, limit 20)
SELECT * FROM orders
ORDER BY order_date DESC
LIMIT 20 OFFSET 100000;

-- FAST using keyset pagination (seek method)
SELECT * FROM orders
WHERE order_date < (SELECT order_date FROM orders ORDER BY order_date DESC LIMIT 1
OFFSET 99999)
ORDER BY order_date DESC
LIMIT 20;

-- Even better with last seen value
SELECT * FROM orders
WHERE order_date < '2024-01-15'
   OR (order_date = '2024-01-15' AND order_id > 1234)
ORDER BY order_date DESC, order_id
LIMIT 20;
```

---

3. Index Tuning

Index tuning involves creating, modifying, or dropping indexes to improve query performance.

Index Selection Guidelines

When to Create Index:

- Columns in WHERE clause frequently
- Columns used in JOIN conditions
- Columns used in ORDER BY
- High cardinality columns (many unique values)
- Foreign key columns

When NOT to Create Index:

- Small tables (< 1000 rows)
- Columns rarely used in queries
- Low cardinality (e.g., gender with 2 values)
- Tables with high INSERT/UPDATE rates
- Columns with many NULL values

Analyzing Index Usage

MySQL:

sql

```sql
-- Show indexes on table
SHOW INDEX FROM employees;

-- Find unused indexes
SELECT * FROM sys.schema_unused_indexes;

-- Index statistics
SELECT * FROM information_schema.statistics
WHERE table_name = 'employees';
```

```sql
-- Index performance
SELECT * FROM performance_schema.table_io_waits_summary_by_index_usage
WHERE object_name = 'employees';
```

SQL Server:

```sql
-- Find missing indexes
SELECT * FROM sys.dm_db_missing_index_details;

-- Index usage statistics
SELECT
    OBJECT_NAME(s.object_id) AS table_name,
    i.name AS index_name,
    s.user_seeks,
    s.user_scans,
    s.user_lookups,
    s.user_updates
FROM sys.dm_db_index_usage_stats s
JOIN sys.indexes i ON s.object_id = i.object_id AND s.index_id = i.index_id
WHERE database_id = DB_ID()
ORDER BY s.user_seeks + s.user_scans + s.user_lookups DESC;

-- Fragmented indexes
SELECT
    OBJECT_NAME(ps.object_id) AS table_name,
    i.name AS index_name,
    ps.avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ps
JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id = i.index_id
WHERE ps.avg_fragmentation_in_percent > 5;
```

Index Maintenance

```sql
-- Reorganize index (less fragmentation)
ALTER INDEX idx_name ON employees REORGANIZE;

-- Rebuild index (more fragmentation)
ALTER INDEX idx_name ON employees REBUILD;
```

```sql
-- Rebuild all indexes on table
ALTER INDEX ALL ON employees REBUILD;

-- Update statistics
UPDATE STATISTICS employees;
UPDATE STATISTICS employees idx_name;

-- MySQL: Optimize table (rebuilds indexes)
OPTIMIZE TABLE employees;

-- PostgreSQL: Reindex
REINDEX INDEX idx_name;
REINDEX TABLE employees;
```

Composite Index Design Examples

Example 1: Index for Equality and Range

sql

```sql
-- Query: Find active customers in specific age range
SELECT * FROM customers
WHERE status = 'ACTIVE'
  AND age BETWEEN 25 AND 35;

-- Best index: (status, age)
-- Status first (equality), then age (range)
CREATE INDEX idx_status_age ON customers(status, age);
```

Example 2: Index for Sorting

sql

```sql
-- Query: Get recent orders for a customer
SELECT order_id, order_date, total_amount
FROM orders
WHERE customer_id = 123
ORDER BY order_date DESC;

-- Best index: (customer_id, order_date DESC)
CREATE INDEX idx_customer_date ON orders(customer_id, order_date DESC);
```

Example 3: Covering Index

sql

```sql
-- Query only needs specific columns
SELECT order_id, order_date, status
FROM orders
WHERE customer_id = 123
  AND order_date > '2024-01-01';

-- Covering index includes all needed columns
CREATE INDEX idx_covering ON orders(customer_id, order_date, status, order_id);
-- Query can be satisfied entirely from index (no table access)
```

---

4. Avoiding N+1 Queries

The N+1 query problem occurs when an application makes one query to get parent records, then N additional queries for each parent's children.

Understanding N+1 Problem

Example of N+1 Problem:

sql

```sql
-- Application code (Python/ORM style)
# ONE query to get all customers
customers = db.query("SELECT * FROM customers").fetchall()

# N queries for each customer's orders (N = number of customers)
for customer in customers:
    orders = db.query(f"SELECT * FROM orders WHERE customer_id = {customer['id']}"
)
    # Process orders...
```

Result: If you have 100 customers, you execute 1 + 100 = 101 queries!

Solutions to N+1

Solution 1: JOIN (Single Query)

sql

```sql
-- ONE query with JOIN
SELECT
    c.customer_id,
    c.name,
    o.order_id,
    o.order_date,
    o.total_amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
ORDER BY c.customer_id, o.order_date;

-- Application then processes in memory
```

Solution 2: Batch Loading (Two Queries)

sql

```sql
-- Query 1: Get all customers
SELECT * FROM customers WHERE status = 'ACTIVE';

-- Query 2: Get all orders for those customers in one query
SELECT * FROM orders
WHERE customer_id IN (1, 2, 3, 4, 5)  -- IDs from first query
ORDER BY customer_id, order_date;

-- Application matches orders to customers
```

Solution 3: Window Functions

sql

```sql
-- Get customers with their orders aggregated
SELECT
    c.customer_id,
    c.name,
    JSON_ARRAYAGG(
        JSON_OBJECT('order_id', o.order_id, 'date', o.order_date, 'amount', o.tota
l_amount)
    ) AS orders
```

```sql
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

ORM-Specific Solutions

SQLAlchemy (Python):

python

```python
# N+1 problem
customers = session.query(Customer).all()
for customer in customers:
    print(customer.orders)  # Triggers new query each time

# Solution: eager loading
customers = session.query(Customer).options(
    joinedload(Customer.orders)  # JOIN in one query
).all()

# Or select IN loading
customers = session.query(Customer).options(
    selectinload(Customer.orders)  # Two queries
).all()
```

Entity Framework (C#):

csharp

```csharp
// N+1 problem
var customers = context.Customers.ToList();
foreach (var customer in customers) {
    Console.WriteLine(customer.Orders.Count);  // Triggers new query
}

// Solution: Include
var customers = context.Customers
    .Include(c => c.Orders)
    .ToList();

// Solution: Projection
var customers = context.Customers
    .Select(c => new {
```

```
        Customer = c,
        OrderCount = c.Orders.Count
    })
    .ToList();
```

---

5. Partitioning

Partitioning divides large tables into smaller, more manageable pieces while still being queried as a single table.

Types of Partitioning

Type	Description	Example
Range	Partition by ranges of values	By date: 2024-Q1, 2024-Q2
List	Partition by discrete values	By region: North, South, East
Hash	Partition by hash function	Distribute data evenly
Composite	Combination of methods	Range then hash

MySQL Partitioning Examples

sql

```sql
-- Range Partitioning by Date
CREATE TABLE orders (
    order_id INT,
    order_date DATE,
    customer_id INT,
    total_amount DECIMAL(10,2)
)
PARTITION BY RANGE (YEAR(order_date)) (
```

```sql
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION p2023 VALUES LESS THAN (2024),
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p_future VALUES LESS THAN MAXVALUE
);

-- List Partitioning by Region
CREATE TABLE customers (
    customer_id INT,
    name VARCHAR(100),
    region VARCHAR(50)
)
PARTITION BY LIST COLUMNS(region) (
    PARTITION p_north VALUES IN ('North', 'Northeast', 'Northwest'),
    PARTITION p_south VALUES IN ('South', 'Southeast', 'Southwest'),
    PARTITION p_central VALUES IN ('Central', 'Midwest'),
    PARTITION p_other VALUES IN (DEFAULT)
);

-- Hash Partitioning for even distribution
CREATE TABLE logs (
    log_id INT,
    log_date DATETIME,
    message TEXT
)
PARTITION BY HASH(YEAR(log_date))
PARTITIONS 4;

-- Subpartitioning (composite)
CREATE TABLE sales (
    sale_id INT,
    sale_date DATE,
    region VARCHAR(20),
    amount DECIMAL(10,2)
)
PARTITION BY RANGE (YEAR(sale_date))
SUBPARTITION BY HASH (region)
SUBPARTITIONS 4 (
    PARTITION p2022 VALUES LESS THAN (2023),
    PARTITION p2023 VALUES LESS THAN (2024),
    PARTITION p2024 VALUES LESS THAN (2025)
);
```

PostgreSQL Partitioning

sql

```sql
-- Create parent table
CREATE TABLE orders (
    order_id INT,
    order_date DATE NOT NULL,
    customer_id INT,
    total_amount DECIMAL(10,2)
) PARTITION BY RANGE (order_date);

-- Create partitions
CREATE TABLE orders_2023_q1 PARTITION OF orders
FOR VALUES FROM ('2023-01-01') TO ('2023-04-01');

CREATE TABLE orders_2023_q2 PARTITION OF orders
FOR VALUES FROM ('2023-04-01') TO ('2023-07-01');

CREATE TABLE orders_2023_q3 PARTITION OF orders
FOR VALUES FROM ('2023-07-01') TO ('2023-10-01');

CREATE TABLE orders_2023_q4 PARTITION OF orders
FOR VALUES FROM ('2023-10-01') TO ('2024-01-01');

-- Automatic partition creation (PostgreSQL 12+)
CREATE TABLE orders_2024_q1 PARTITION OF orders
FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');
```

SQL Server Partitioning

sql

```sql
-- Create partition function
CREATE PARTITION FUNCTION pf_order_date (DATE)
AS RANGE RIGHT FOR VALUES (
    '2023-01-01', '2023-04-01', '2023-07-01', '2023-10-01', '2024-01-01'
);

-- Create partition scheme
CREATE PARTITION SCHEME ps_order_date
AS PARTITION pf_order_date
TO (fg1, fg2, fg3, fg4, fg5, fg6);  -- File groups
```

```sql
-- Create partitioned table
CREATE TABLE orders (
    order_id INT,
    order_date DATE,
    customer_id INT,
    total_amount DECIMAL(10,2)
) ON ps_order_date(order_date);
```

Benefits of Partitioning

Benefit	Description
Performance	Query only relevant partitions
Manageability	Archive old partitions easily
Availability	Maintain partitions independently
Parallelism	Scan partitions in parallel
Data Organization	Logical data separation

Partition Pruning

sql

```sql
-- Query that benefits from partition pruning
EXPLAIN SELECT * FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31';
-- Database will only scan partition(s) for Q1 2024

-- Query that cannot prune
SELECT * FROM orders
WHERE customer_id = 123;
-- Might scan all partitions (depends on partitioning key)
```

---

6. Caching Strategies

Caching stores frequently accessed data in fast-access memory to reduce database load.

Types of Caching

Cache Type	Location	Use Case
Query Cache	Database	Repeated identical queries
Application Cache	Application	Business objects, computed data
Distributed Cache	Separate service	Shared across app instances
CDN	Edge network	Static assets
Database Buffer Pool	Database memory	Frequently accessed data pages

Database-Level Caching

MySQL Query Cache (deprecated in 8.0):

sql

```sql
-- Enable query cache
SET GLOBAL query_cache_size = 1000000;
SET GLOBAL query_cache_type = 1;

-- Monitor query cache
SHOW STATUS LIKE 'Qcache%';
```

InnoDB Buffer Pool:

sql

```sql
-- Check buffer pool size
SHOW VARIABLES LIKE 'innodb_buffer_pool_size';

-- Buffer pool usage
SHOW ENGINE INNODB STATUS\G
```

PostgreSQL Shared Buffers:

```sql
-- Check settings
SHOW shared_buffers;
SHOW effective_cache_size;

-- Cache hit ratio
SELECT
    sum(heap_blks_read) as heap_read,
    sum(heap_blks_hit) as heap_hit,
    sum(heap_blks_hit) / (sum(heap_blks_hit) + sum(heap_blks_read))::float as hit_
ratio
FROM pg_statio_user_tables;
```

Application-Level Caching

Redis Example:

```python
import redis
import json

# Connect to Redis
cache = redis.Redis(host='localhost', port=6379, db=0)

def get_customer_orders(customer_id):
    # Try cache first
    cache_key = f"customer_orders:{customer_id}"
    cached = cache.get(cache_key)

    if cached:
        return json.loads(cached)

    # Cache miss - query database
```

```python
    orders = db.query("SELECT * FROM orders WHERE customer_id = %s", customer_id)

    # Store in cache for 5 minutes
    cache.setex(cache_key, 300, json.dumps(orders))

    return orders

def invalidate_customer_cache(customer_id):
    # Clear cache when data changes
    cache.delete(f"customer_orders:{customer_id}")
```

Memcached Example:

```python
import memcache

# Connect to Memcached
mc = memcache.Client(['127.0.0.1:11211'])

def get_product(product_id):
    # Try cache
    product = mc.get(f"product:{product_id}")

    if product:
        return product

    # Cache miss - query database
    product = db.query_one("SELECT * FROM products WHERE product_id = %s", product
_id)

    # Store in cache for 1 hour
    mc.set(f"product:{product_id}", product, time=3600)

    return product
```

Caching Strategies

Strategy 1: Cache-Aside (Lazy Loading)

```text
Application → Check Cache → If miss → Query DB → Store in Cache → Return
```

```
            ↑                                   ↓
            └──────────── If hit ─────────────────┘
```

Strategy 2: Write-Through

text

```
Application → Write to Cache → Cache writes to DB → Confirm
```

Strategy 3: Write-Behind

text

```
Application → Write to Cache → Acknowledge → Async write to DB
```

Strategy 4: Refresh-Ahead

text

```
Application → Check Cache → If near expiry → Async refresh from DB → Return cached
```

Cache Invalidation Strategies

Strategy	How It Works	When to Use
Time-based (TTL)	Expire after set time	Data that changes predictabl
Write-through	Update cache when DB updates	Critical consistency needed
Manual invalidation	Explicitly clear cache	When you know data change
Version-based	Use version keys	Distributed systems

python

```python
# Time-based TTL
cache.setex(key, 300, value)  # Expires in 5 minutes

# Version-based caching
def get_user_versioned(user_id, version=1):
    return cache.get(f"user:v{version}:{user_id}")

def update_user(user_id, data):
```

```python
    db.update("users", user_id, data)
    version = cache.incr("user_version")  # Increment global version
    cache.set(f"user:v{version}:{user_id}", data)
```

Part 2: Security in SQL

Security is critical to protect data from unauthorized access, modification, or theft.

---

1. SQL Injection Prevention

What is SQL Injection?

SQL injection is a code injection technique where attackers insert malicious SQL code into query parameters to manipulate the database.

SQL Injection Examples

Vulnerable Code:

python

```python
# DANGEROUS: Direct string concatenation
user_input = "1; DROP TABLE users; --"
query = f"SELECT * FROM users WHERE user_id = {user_input}"
# Executes: SELECT * FROM users WHERE user_id = 1; DROP TABLE users; --

user_input = "' OR '1'='1"
query = f"SELECT * FROM users WHERE username = '{user_input}' AND password = 'anything'"
# Executes: SELECT * FROM users WHERE username = '' OR '1'='1' AND password = 'anything'
# Returns all users (bypasses authentication)
```

Blind SQL Injection:

sql

```sql
-- Attacker asks true/false questions
' AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin')='a' --
-- If page loads normally, first character is 'a'
```

Prevention Techniques

Technique 1: Parameterized Queries (Prepared Statements)

python

```python
# Python with MySQL
cursor.execute(
    "SELECT * FROM users WHERE username = %s AND password = %s",
    (username, password)
)

# Python with SQLite
cursor.execute(
    "SELECT * FROM users WHERE username = ? AND password = ?",
    (username, password)
)

# Node.js with MySQL
connection.execute(
    "SELECT * FROM users WHERE username = ? AND password = ?",
    [username, password]
);

# Java with JDBC
PreparedStatement stmt = conn.prepareStatement(
    "SELECT * FROM users WHERE username = ? AND password = ?"
);
stmt.setString(1, username);
stmt.setString(2, password);
```

Technique 2: Stored Procedures

sql

```sql
-- Create stored procedure with parameters
CREATE PROCEDURE GetUser
    @Username NVARCHAR(50),
```

```
    @Password NVARCHAR(50)
AS
BEGIN
    SELECT * FROM users
    WHERE username = @Username AND password = @Password;
END;

-- Call safely
EXEC GetUser 'john', 'password123';
```

Technique 3: Input Validation and Whitelisting

python

```python
import re

def validate_user_input(input_string):
    # Allow only alphanumeric and underscore
    if re.match("^[a-zA-Z0-9_]+$", input_string):
        return input_string
    else:
        raise ValueError("Invalid input")

# For numeric IDs
def validate_numeric_id(id_string):
    if id_string.isdigit():
        return int(id_string)
    else:
        raise ValueError("ID must be numeric")

# For enumerated values
VALID_SORT_COLUMNS = ['name', 'date', 'price']
def validate_sort_column(column):
    if column in VALID_SORT_COLUMNS:
        return column
    else:
        return 'name'  # Default
```

Technique 4: Escaping (Last Resort)

python

```python
# MySQL
escaped_input = connection.escape_string(user_input)
```

```python
# PostgreSQL
escaped_input = connection.escape_string(user_input)

# Not as safe as parameterized queries!
```

ORM Protection

python

```python
# SQLAlchemy (safe by default)
user = session.query(User).filter(
    User.username == username,
    User.password == password
).first()

# Django ORM (safe)
user = User.objects.filter(username=username, password=password).first()

# Entity Framework (safe)
var user = context.Users
    .Where(u => u.Username == username && u.Password == password)
    .FirstOrDefault();
```

Defense in Depth

python

```python
def safe_login(username, password):
    # 1. Input validation
    if not username or not password:
        return None

    # 2. Limit length (prevent DoS)
    username = username[:50]
    password = password[:50]

    # 3. Parameterized query
    cursor.execute(
        "SELECT * FROM users WHERE username = %s",
        (username,)
    )
```

```python
    user = cursor.fetchone()

    # 4. Separate password verification
    if user and verify_password(password, user['password_hash']):
        return user

    return None
```

---

2. Prepared Statements

Prepared statements separate SQL logic from data, preventing injection and improving performance.

How Prepared Statements Work

```text
Step 1: Prepare
Client → "PREPARE stmt FROM 'SELECT * FROM users WHERE id = ?'"
Database → Parses, compiles, optimizes → Stores prepared statement

Step 2: Execute (multiple times)
Client → "EXECUTE stmt USING 123"
Database → Uses compiled plan, safely inserts 123 → Returns result

Client → "EXECUTE stmt USING 456"
Database → Reuses plan, safely inserts 456 → Returns result
```

Benefits of Prepared Statements

Benefit	Description
Security	Prevents SQL injection
Performance	Parse once, execute many

Benefit	Description
Separation	Logic vs data separation
Binary protocol	Efficient data transfer

Examples by Language

PHP (PDO):

```php
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = :email AND status = :stat
us");
$stmt->execute(['email' => $email, 'status' => 'active']);
$user = $stmt->fetch();

// Multiple executions
$stmt = $pdo->prepare("INSERT INTO logs (message, level) VALUES (?, ?)");
$stmt->execute(['User logged in', 'INFO']);
$stmt->execute(['Database error', 'ERROR']);
```

Python (MySQL Connector):

```python
# Named placeholders
cursor.execute(
    "SELECT * FROM products WHERE category = %(cat)s AND price > %(price)s",
    {'cat': 'Electronics', 'price': 100}
)

# Positional placeholders
cursor.execute(
    "INSERT INTO orders (customer_id, total) VALUES (%s, %s)",
    (123, 99.99)
)

# Batch insert
data = [(1, 'A'), (2, 'B'), (3, 'C')]
cursor.executemany(
```

```
    "INSERT INTO items (id, name) VALUES (%s, %s)",
    data
)
```

Java (JDBC):

```java
// Prepare once
PreparedStatement stmt = conn.prepareStatement(
    "UPDATE accounts SET balance = balance + ? WHERE account_id = ?"
);

// Execute multiple times
stmt.setBigDecimal(1, new BigDecimal("100.00"));
stmt.setInt(2, 123);
stmt.executeUpdate();

stmt.setBigDecimal(1, new BigDecimal("50.00"));
stmt.setInt(2, 456);
stmt.executeUpdate();

// With batch
stmt.addBatch();
```

---

3. Roles & Permissions

Database security uses roles and permissions to control access to objects and operations.

Principle of Least Privilege

Users and applications should have only the minimum permissions necessary to perform their functions.

Types of Permissions

Permission Level	What It Controls
Server Level	Create databases, manage logins
Database Level	Create tables, backup database
Schema Level	Create objects within schema
Object Level	SELECT, INSERT, UPDATE, DELETE on tables
Column Level	Access to specific columns
Row Level	Access to specific rows (via views)

Creating Users and Roles

MySQL:

```sql
-- Create users
CREATE USER 'app_user'@'localhost' IDENTIFIED BY 'secure_password';
CREATE USER 'report_user'@'%' IDENTIFIED BY 'report_password';

-- Create roles
CREATE ROLE 'app_role';
CREATE ROLE 'readonly_role';
CREATE ROLE 'admin_role';

-- Grant permissions to roles
GRANT SELECT, INSERT, UPDATE ON company_db.* TO 'app_role';
GRANT SELECT ON company_db.* TO 'readonly_role';
GRANT ALL PRIVILEGES ON company_db.* TO 'admin_role';

-- Assign roles to users
```

```sql
GRANT 'app_role' TO 'app_user'@'localhost';
GRANT 'readonly_role' TO 'report_user'@'%';

-- Set default roles
SET DEFAULT ROLE 'app_role' TO 'app_user'@'localhost';

-- Revoke permissions
REVOKE INSERT ON company_db.* FROM 'app_role';
```

PostgreSQL:

```sql
-- Create users
CREATE USER app_user WITH PASSWORD 'secure_password';
CREATE USER report_user WITH PASSWORD 'report_password';

-- Create groups (roles)
CREATE ROLE app_role;
CREATE ROLE readonly_role;

-- Grant permissions
GRANT CONNECT ON DATABASE company_db TO app_role;
GRANT USAGE ON SCHEMA public TO app_role;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO app_role;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_role;

-- Grant for future tables
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE ON TABLES TO app_role;

-- Assign users to roles
GRANT app_role TO app_user;
GRANT readonly_role TO report_user;
```

SQL Server:

```sql
-- Create login (server level)
CREATE LOGIN app_user WITH PASSWORD = 'secure_password';

-- Create user (database level)
USE company_db;
```

```sql
CREATE USER app_user FOR LOGIN app_user;

-- Create roles
CREATE ROLE app_role;
CREATE ROLE readonly_role;

-- Grant permissions
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo TO app_role;
GRANT SELECT ON SCHEMA::dbo TO readonly_role;

-- Add users to roles
ALTER ROLE app_role ADD MEMBER app_user;
ALTER ROLE readonly_role ADD MEMBER report_user;

-- Column-level security
GRANT SELECT ON employees(emp_id, first_name, last_name) TO readonly_role;
DENY SELECT ON employees(salary, ssn) TO readonly_role;
```

Row-Level Security

PostgreSQL Row-Level Security:

sql

```sql
-- Enable RLS on table
ALTER TABLE customers ENABLE ROW LEVEL SECURITY;

-- Create policy: users can only see customers in their region
CREATE POLICY customer_region_policy ON customers
    USING (region = current_setting('app.current_region'));

-- Create policy: managers can see all
CREATE POLICY manager_all_access ON customers
    FOR ALL
    TO manager_role
    USING (true);

-- Create policy: sales can only update their customers
CREATE POLICY sales_update_policy ON customers
    FOR UPDATE
    TO sales_role
    USING (sales_rep = current_user)
```

```sql
    WITH CHECK (sales_rep = current_user);
```

SQL Server Row-Level Security:

```sql
-- Create function to filter rows
CREATE FUNCTION dbo.fn_customer_security(@sales_rep AS sysname)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS result
WHERE @sales_rep = USER_NAME() OR USER_NAME() = 'manager';

-- Create security policy
CREATE SECURITY POLICY CustomerFilter
ADD FILTER PREDICATE dbo.fn_customer_security(sales_rep) ON dbo.customers,
ADD BLOCK PREDICATE dbo.fn_customer_security(sales_rep) ON dbo.customers;
```

Best Practices for Permissions

Practice	Why
Use roles, not individual users	Easier management
Least privilege	Minimize damage from compromise
Separate app and admin users	Different permissions for different needs
Regular audit	Review who has what access
Revoke unused permissions	Reduce attack surface
Use views for column/row security	Hide sensitive data
Password policies	Strong passwords, regular changes

Auditing Permissions

MySQL:

```sql
-- Show grants for user
SHOW GRANTS FOR 'app_user'@'localhost';

-- Show all users
SELECT user, host FROM mysql.user;

-- Show roles
SELECT * FROM mysql.roles_mapping;
```

PostgreSQL:

```sql
-- List roles
\du

-- Show grants
SELECT
    grantee,
    table_schema,
    table_name,
    privilege_type
FROM information_schema.role_table_grants
WHERE grantee = 'app_user';
```

---

4. Encryption

Encryption protects data at rest (stored) and in transit (network).

Types of Encryption

Type	Description	Use Case
Encryption in Transit	SSL/TLS	Network communication
Encryption at Rest	TDE, File-level	Data files on disk
Column-level Encryption	Application-level	Sensitive columns
Hashing	One-way	Passwords

Encryption in Transit (SSL/TLS)

MySQL SSL Setup:

```sql
-- Check SSL status
SHOW VARIABLES LIKE '%ssl%';

-- Require SSL for user
CREATE USER 'secure_user'@'%' IDENTIFIED BY 'password' REQUIRE SSL;

-- Or alter existing user
ALTER USER 'app_user'@'%' REQUIRE SSL;

-- Connection string with SSL
mysql -u app_user -p --ssl-ca=ca.pem --ssl-cert=client-cert.pem --ssl-key=client-key.pem
```

PostgreSQL SSL:

```sql
-- In postgresql.conf
ssl = on
ssl_cert_file = 'server.crt'
ssl_key_file = 'server.key'
```

```sql
-- Require SSL for user
ALTER USER app_user SET ssl = 'on';

-- Connection string
postgresql://app_user:password@host/db?sslmode=require
```

Encryption at Rest (TDE)

SQL Server TDE:

```sql
-- Create master key
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'StrongPassword123!';

-- Create certificate
CREATE CERTIFICATE TDECert WITH SUBJECT = 'TDE Certificate';

-- Create database encryption key
USE company_db;
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256
ENCRYPTION BY SERVER CERTIFICATE TDECert;

-- Enable encryption
ALTER DATABASE company_db SET ENCRYPTION ON;
```

MySQL Enterprise TDE:

```sql
-- Enable encryption for tablespace
CREATE TABLE sensitive_data (
    id INT,
    credit_card VARCHAR(256)
) ENCRYPTION='Y';

-- Set default encryption
SET GLOBAL default_table_encryption = ON;
```

Column-Level Encryption

MySQL:

sql

```sql
-- AES encryption functions
INSERT INTO users (username, credit_card)
VALUES (
    'john',
    AES_ENCRYPT('1234-5678-9012-3456', 'encryption_key')
);

SELECT
    username,
    AES_DECRYPT(credit_card, 'encryption_key') AS credit_card
FROM users;

-- With random initialization vector (more secure)
SET @iv = RANDOM_BYTES(16);
INSERT INTO secure_data (data, iv)
VALUES (AES_ENCRYPT('sensitive', 'key', @iv), @iv);
```

PostgreSQL:

sql

```sql
-- Enable pgcrypto extension
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Encrypt data
INSERT INTO users (username, ssn)
VALUES (
    'john',
    pgp_sym_encrypt('123-45-6789', 'encryption_key')
);

-- Decrypt data
SELECT
    username,
    pgp_sym_decrypt(ssn::bytea, 'encryption_key') AS ssn
FROM users;

-- Public/private key encryption
```

```sql
INSERT INTO documents (name, content)
VALUES (
    'secret.txt',
    pgp_pub_encrypt('Secret content', dearmor('public_key.asc'))
);
```

Password Hashing

NEVER store passwords in plain text!

Proper Password Hashing:

sql

```sql
-- MySQL
INSERT INTO users (username, password_hash)
VALUES (
    'john',
    SHA2('password123', 256)  -- Better: use application-level bcrypt
);

-- PostgreSQL with bcrypt
CREATE EXTENSION pgcrypto;

INSERT INTO users (username, password_hash)
VALUES (
    'john',
    crypt('password123', gen_salt('bf', 10))  -- bcrypt with cost 10
);

-- Verify
SELECT * FROM users
WHERE username = 'john'
  AND password_hash = crypt('password123', password_hash);
```

Application-Level Hashing (Recommended):

python

```python
import bcrypt

# Hash password
password = b"user_password"
```

```
salt = bcrypt.gensalt(rounds=12)  # Cost factor
hashed = bcrypt.hashpw(password, salt)

# Store hashed in database
cursor.execute(
    "INSERT INTO users (username, password_hash) VALUES (%s, %s)",
    (username, hashed)
)

# Verify password
stored_hash = cursor.execute(
    "SELECT password_hash FROM users WHERE username = %s",
    (username,)
).fetchone()[0]

if bcrypt.checkpw(password_input, stored_hash):
    print("Login successful")
```

java

```
import org.springframework.security.crypto.bcrypt.BCrypt;

// Hash password
String hashed = BCrypt.hashpw(password, BCrypt.gensalt(12));

// Verify password
if (BCrypt.checkpw(password, storedHash)) {
    // Authenticated
}
```

Data Masking

Dynamic Data Masking (SQL Server):

sql

```
-- Apply mask to column
ALTER TABLE customers
ALTER COLUMN credit_card ADD MASKED WITH (FUNCTION = 'partial(0, "XXXX-XXXX-XXXX-"
, 4)');

ALTER TABLE employees
ALTER COLUMN salary ADD MASKED WITH (FUNCTION = 'default()');
```

```sql
-- Unmasked users need permission
GRANT UNMASK TO manager_role;
```

View-Based Masking:

```sql
-- Create view with masked data
CREATE VIEW customer_public AS
SELECT
    customer_id,
    name,
    CONCAT(LEFT(email, 1), '*****', SUBSTRING(email, CHARINDEX('@', email))) AS ma
sked_email,
    CONCAT('XXX-XX-', RIGHT(ssn, 4)) AS masked_ssn
FROM customers;

-- Grant access to view, not base table
GRANT SELECT ON customer_public TO app_role;
```

Encryption Key Management

Best Practices:

1. **Separate key storage** - Keys not in database
2. **Key rotation** - Regular key changes
3. **Hardware Security Modules (HSM)** - Physical security
4. **Key hierarchy** - Master key encrypts data keys
5. **Backup keys securely** - Don't lose access

```python
# Key management service example (AWS KMS)
import boto3
from cryptography.fernet import Fernet

kms = boto3.client('kms')

# Generate data key
response = kms.generate_data_key(
    KeyId='alias/my-key',
    KeySpec='AES_256'
)
```

```python
plaintext_key = response['Plaintext']  # Use for encryption, discard after
encrypted_key = response['CiphertextBlob']  # Store in database

# Encrypt data
cipher = Fernet(plaintext_key)
encrypted = cipher.encrypt(b"sensitive data")

# Later, decrypt the key
response = kms.decrypt(CiphertextBlob=encrypted_key)
plaintext_key = response['Plaintext']
cipher = Fernet(plaintext_key)
decrypted = cipher.decrypt(encrypted)
```

Summary: Performance Optimization Quick Reference

Technique	When to Use	Impact
Execution Plans	Always when optimizing	Find bottlenecks
Index Tuning	Slow queries, large tables	High
Query Rewriting	Inefficient logic	Medium-High
Partitioning	Very large tables	High
Caching	Repeated queries	Very High
Denormalization	Read-heavy reporting	Medium

Summary: Security Quick Reference

Threat	Prevention
SQL Injection	Parameterized queries, input validation
Unauthorized Access	Strong authentication, least privilege
Data Theft	Encryption at rest and in transit
Privilege Escalation	Role separation, regular audits
Password Cracking	Strong hashing (bcrypt), MFA
Network Sniffing	SSL/TLS encryption

Security Checklist

- Use parameterized queries for all database access
- Encrypt sensitive data at rest
- Use SSL/TLS for all connections
- Implement least privilege access
- Regular security audits
- Strong password policies
- Secure key management
- Input validation on all user input
- Regular backups (encrypted)
- Monitor for suspicious activity

Database Design for Applications and Backend Integration: Complete Theory Notes

Part 1: Database Design for Applications

Database design is the process of structuring data to meet application requirements while ensuring data integrity, performance, and scalability.

---

1. ER Diagrams (Entity-Relationship Diagrams)

What is an ER Diagram?

An Entity-Relationship (ER) diagram is a visual representation of the database structure showing entities, attributes, and relationships.

Components of ER Diagrams

Component	Symbol	Description
Entity	Rectangle	Real-world object (table)
Attribute	Oval	Property of an entity (column)
Primary Key	Underlined attribute	Unique identifier
Relationship	Diamond	Association between entities
Cardinality	1, N, M	Number of occurrences

Notation Styles

Chen Notation:

text

```
    ┌──────────┐          1          ┌──────────┐
    │ CUSTOMER │──────────◊──────────│  ORDER   │
    └──────────┘        places        └──────────┘
       │                                   │
       │                                   │
       ○                                   ○
    name, id                            date, total
```

Crow's Foot Notation:

text

```
┌────────────┐                    ┌────────────┐
│ CUSTOMER   │──────────<┐        │   ORDER    │
├────────────┤          │         ├────────────┤
│ PK  id     │          │         │ PK  id     │
│ name       │          │         │ date       │
│ email      │          └─────────│ total      │
└────────────┘            placed  │ FK customer_id
                                  └────────────┘
```

Example: E-Commerce ER Diagram

text

```
┌──────────────┐                    ┌──────────────┐
│   CUSTOMER   │                    │    ORDER     │
├──────────────┤                    ├──────────────┤
│ PK id        │◄───────────────────│ PK id        │
│ name         │   1           *    │ date         │
│ email        │   places           │ total        │
│ phone        │                    │ status       │
│ address      │                    │ FK customer_id
└──────────────┘                    └──────────────┘
       │                                   │
       │                                   │
       │*                                  │*
       ▼                                   ▼
```

```
┌──────────────┐                  ┌──────────────┐
│   REVIEW     │                  │ ORDER_ITEM   │
├──────────────┤                  ├──────────────┤
│ PK id        │                  │ PK,FK order_id
│ rating       │                  │ PK,FK product_id
│ comment      │                  │ quantity     │
│ date         │                  │ price        │
│ FK customer │                  └──────────────┘
│ FK product   │                         │
└──────────────┘                         │*
       │*                                │
       └─────────────────────────────────┘
                         │
                         │
                   ┌─────▼──────┐
                   │ PRODUCT    │
                   ├────────────┤
                   │ PK id      │
                   │ name       │
                   │ price      │
                   │ stock      │
                   │ FK category
                   └────────────┘
                         │
                         │*
                   ┌─────▼──────┐
                   │ CATEGORY   │
                   ├────────────┤
                   │ PK id      │
                   │ name       │
                   └────────────┘
```

Creating ER Diagrams

Tools:

- **LucidChart** - Online diagramming
- Draw.io - Free, integrated with many platforms
- **MySQL Workbench** - Reverse engineering
- **pgAdmin** - PostgreSQL diagram tool
- dbdiagram.io - Code-based diagrams

Code-based ER Diagram (dbdiagram.io):

text

```
Table customers {
  id int [pk]
  name varchar
  email varchar [unique]
  phone varchar
  created_at timestamp
}

Table orders {
  id int [pk]
  customer_id int [ref: > customers.id]
  order_date timestamp
  total decimal
  status varchar
}

Table products {
  id int [pk]
  name varchar
  price decimal
  stock int
  category_id int [ref: > categories.id]
}

Table order_items {
  order_id int [ref: > orders.id]
  product_id int [ref: > products.id]
  quantity int
  price decimal

  indexes {
    (order_id, product_id) [pk]
  }
}

Table categories {
  id int [pk]
  name varchar
  parent_id int [ref: > categories.id]
```

```
}
```

---

2. One-to-One Relationship

Definition

A one-to-one (1:1) relationship means each record in Table A corresponds to exactly one record in Table B, and vice versa.

When to Use One-to-One

Scenario	Example
Extending tables	User + UserProfile
Splitting wide tables	Employee + EmployeeDetails
Sensitive data	User + UserCredentials
Optional data	Product + ProductWarranty

Implementation

Option 1: Same Primary Key

```sql
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE user_profiles (
```

```sql
    user_id INT PRIMARY KEY,
    full_name VARCHAR(100),
    avatar_url VARCHAR(255),
    bio TEXT,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

-- Each user has exactly one profile
-- Each profile belongs to exactly one user
```

Option 2: Unique Foreign Key

sql

```sql
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100)
);

CREATE TABLE employee_contracts (
    contract_id INT PRIMARY KEY,
    emp_id INT UNIQUE,  -- UNIQUE ensures one-to-one
    start_date DATE,
    salary DECIMAL(10,2),
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id)
);

-- Each employee has one contract
-- Each contract belongs to one employee
```

Visual Representation

text

```text
users                              user_profiles
┌──────────┐                       ┌──────────────┐
│ 1: John  │───────────────────────│ 1: John, bio │
├──────────┤       one-to-one      ├──────────────┤
│ 2: Jane  │───────────────────────│ 2: Jane, bio │
├──────────┤                       ├──────────────┤
│ 3: Bob   │───────────────────────│ 3: Bob, bio  │
└──────────┘                       └──────────────┘
```

Query Examples

```sql
-- Get user with profile
SELECT
    u.username,
    p.full_name,
    p.bio
FROM users u
JOIN user_profiles p ON u.user_id = p.user_id
WHERE u.user_id = 1;

-- Users without profiles
SELECT u.*
FROM users u
LEFT JOIN user_profiles p ON u.user_id = p.user_id
WHERE p.user_id IS NULL;
```

---

3. One-to-Many Relationship

Definition

A one-to-many (1:N) relationship means a record in Table A can have multiple related records in Table B, but each record in Table B belongs to only one record in Table A.

When to Use One-to-Many

Scenario	Example
Parent-child	Department → Employees
Master-detail	Order → Order Items

Scenario	Example
Ownership	User → Posts
Categorization	Category → Products

Implementation

sql

```sql
-- One side (parent)
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL,
    location VARCHAR(100)
);

-- Many side (child) with foreign key
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    salary DECIMAL(10,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

-- Index foreign key for performance
CREATE INDEX idx_emp_dept ON employees(dept_id);
```

Visual Representation

text

```text
departments (one)
┌──────────────────────┐
│ 1: IT                │────┐
├──────────────────────┤    │
│ 2: HR                │──┐ │
├──────────────────────┤  │ │
│ 3: Finance           │  │ │
```

```
└──────────────────────┘   │ │
                           │ │
employees (many)           │ │
┌──────────────────────┐   │ │
│ 101: Alice (dept=1)  │◄─┘ │
├──────────────────────┤    │
│ 102: Bob (dept=1)    │◄───┘
├──────────────────────┤
│ 103: Charlie (dept=2)│◄───┐
├──────────────────────┤    │
│ 104: Diana (dept=3)  │◄──┐│
└──────────────────────┘   ||
                           ||
dept 1 (IT) has 2 employees│
dept 2 (HR) has 1 employee │
dept 3 (Finance) has 1 employee
```

Query Examples

sql

```sql
-- Get department with employee count
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS employee_count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY employee_count DESC;

-- Get employees with their department names
SELECT
    e.name AS employee,
    d.dept_name AS department
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;

-- Find departments with no employees
SELECT d.*
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

---

4. Many-to-Many Relationship

Definition

A many-to-many (M:N) relationship means records in Table A can have multiple related records in Table B, and records in Table B can have multiple related records in Table A.

When to Use Many-to-Many

Scenario	Example
Students-Courses	Student enrolls in multiple courses, course has multiple students
Products-Orders	Order contains multiple products, product appears in multiple orders
Actors-Movies	Actor in multiple movies, movie has multiple actors
Users-Roles	User can have multiple roles, role can be assigned to multiple users

Implementation with Junction Table

sql

```sql
-- Table A
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);

-- Table B
CREATE TABLE courses (
```

```sql
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    credits INT
);

-- Junction table (associative entity)
CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    grade CHAR(2),
    PRIMARY KEY (student_id, course_id),  -- Composite primary key
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

-- Indexes for performance
CREATE INDEX idx_enroll_student ON enrollments(student_id);
CREATE INDEX idx_enroll_course ON enrollments(course_id);
```

Visual Representation

text

```text
students                                courses
┌──────────────┐                        ┌──────────────┐
│ 1: Alice     │                        │ 101: Math    │
├──────────────┤                        ├──────────────┤
│ 2: Bob       │                        │ 102: Physics│
├──────────────┤                        ├──────────────┤
│ 3: Charlie   │                        │ 103: Chem    │
├──────────────┤                        ├──────────────┤
│ 4: Diana     │                        │ 104: Bio     │
└──────────────┘                        └──────────────┘
       │          ▲                           ▲
       │          │                           │
       └──────────┼───────────────────────────┘
                  │
            enrollments (junction)
             ┌──────────────────────┐
             │ student_id │ course_id │
             ├────────────┼───────────┤
```

```
│ 1          │ 101        │  (Alice takes Math)
│ 1          │ 102        │  (Alice takes Physics)
│ 2          │ 101        │  (Bob takes Math)
│ 2          │ 103        │  (Bob takes Chem)
│ 3          │ 102        │  (Charlie takes Physics)
│ 3          │ 104        │  (Charlie takes Bio)
│ 4          │ 101        │  (Diana takes Math)
│ 4          │ 102        │  (Diana takes Physics)
│ 4          │ 103        │  (Diana takes Chem)
│ 4          │ 104        │  (Diana takes Bio)
└──────────────────────┘
```

Junction Table with Additional Data

The junction table can store additional attributes of the relationship:

sql

```sql
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    order_date DATE,
    customer_id INT
);

CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2)
);

-- Junction with additional columns
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,  -- Price at time of order
    discount DECIMAL(5,2) DEFAULT 0,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Query Examples

sql

```sql
-- Get all courses for a student
SELECT
    s.name AS student,
    c.course_name,
    e.enrollment_date,
    e.grade
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id
WHERE s.student_id = 1;

-- Get all students in a course
SELECT
    c.course_name,
    s.name AS student,
    e.grade
FROM courses c
JOIN enrollments e ON c.course_id = e.course_id
JOIN students s ON e.student_id = s.student_id
WHERE c.course_id = 101;

-- Find students taking both Math and Physics
SELECT s.name
FROM students s
WHERE s.student_id IN (
    SELECT e1.student_id
    FROM enrollments e1
    JOIN courses c1 ON e1.course_id = c1.course_id
    WHERE c1.course_name = 'Math'
) AND s.student_id IN (
    SELECT e2.student_id
    FROM enrollments e2
    JOIN courses c2 ON e2.course_id = c2.course_id
    WHERE c2.course_name = 'Physics'
);

-- Count enrollments per student
SELECT
    s.name,
```

```sql
    COUNT(e.course_id) AS course_count
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
ORDER BY course_count DESC;

-- Most popular courses
SELECT
    c.course_name,
    COUNT(e.student_id) AS student_count
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name
ORDER BY student_count DESC;
```

---

5. Junction Tables (Associative Entities)

Purpose of Junction Tables

Junction tables (also called associative tables, bridge tables, or mapping tables) are used to implement many-to-many relationships.

Design Considerations

Aspect	Best Practice
Primary Key	Composite of both foreign keys
Indexes	Index both foreign keys individually
Additional Attributes	Store relationship-specific data
Naming Convention	`table1_table2` or descriptive name

Aspect	Best Practice
Constraints	Ensure referential integrity

Types of Junction Tables

Simple Junction (no additional data):

sql

```sql
CREATE TABLE user_roles (
    user_id INT,
    role_id INT,
    PRIMARY KEY (user_id, role_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (role_id) REFERENCES roles(role_id)
);
```

Rich Junction (with relationship data):

sql

```sql
CREATE TABLE project_assignments (
    employee_id INT,
    project_id INT,
    start_date DATE NOT NULL,
    end_date DATE,
    role VARCHAR(50),
    hours_per_week DECIMAL(5,2),
    PRIMARY KEY (employee_id, project_id, start_date),
    FOREIGN KEY (employee_id) REFERENCES employees(emp_id),
    FOREIGN KEY (project_id) REFERENCES projects(project_id)
);
```

Self-Referential Junction:

sql

```sql
-- Friends relationship between users
CREATE TABLE friendships (
    user_id1 INT,
    user_id2 INT,
```

```sql
    friends_since DATE,
    status VARCHAR(20),
    PRIMARY KEY (user_id1, user_id2),
    FOREIGN KEY (user_id1) REFERENCES users(user_id),
    FOREIGN KEY (user_id2) REFERENCES users(user_id),
    CHECK (user_id1 < user_id2)  -- Prevent duplicates
);
```

Junction Table Patterns

Pattern 1: Simple Many-to-Many

text

```
[users]──<[user_roles]>──[roles]
```

Pattern 2: Many-to-Many with History

text

```
[employees]──<[assignments]>──[projects]
                    │
              [history table]
```

Pattern 3: Recursive Many-to-Many

text

```
[parts]──<[bill_of_materials]>──[parts]
           (parent_id, child_id)
```

Part 2: Backend Integration

---

1. Node.js + SQL

MySQL with Node.js

Installation:

bash

```bash
npm install mysql2  # MySQL client with promises
npm install mysql   # Traditional callback-based
```

Basic Connection:

javascript

```javascript
const mysql = require('mysql2/promise');

// Create connection pool
const pool = mysql.createPool({
    host: 'localhost',
    user: 'root',
    password: 'password',
    database: 'company_db',
    waitForConnections: true,
    connectionLimit: 10,
    queueLimit: 0
});

// Simple query
async function getUsers() {
    const [rows, fields] = await pool.query('SELECT * FROM users');
    return rows;
}

// Parameterized query (safe from SQL injection)
async function getUserById(id) {
    const [rows] = await pool.execute(
        'SELECT * FROM users WHERE id = ?',
        [id]
    );
    return rows[0];
}
```

```javascript
// Insert with prepared statement
async function createUser(name, email) {
    const [result] = await pool.execute(
        'INSERT INTO users (name, email) VALUES (?, ?)',
        [name, email]
    );
    return result.insertId;
}
```

PostgreSQL with Node.js

Installation:

```bash
npm install pg
```

Implementation:

```javascript
const { Pool } = require('pg');

const pool = new Pool({
    host: 'localhost',
    user: 'postgres',
    password: 'password',
    database: 'company_db',
    max: 20,
    idleTimeoutMillis: 30000,
    connectionTimeoutMillis: 2000,
});

async function getProducts() {
    const client = await pool.connect();
    try {
        const result = await client.query('SELECT * FROM products');
        return result.rows;
    } finally {
        client.release(); // Always release
    }
}
```

```
// Parameterized query (PostgreSQL uses $1, $2)
async function getProductsByCategory(category, minPrice) {
    const query = {
        text: 'SELECT * FROM products WHERE category = $1 AND price >= $2',
        values: [category, minPrice],
    };
    const result = await pool.query(query);
    return result.rows;
}

// Transaction
async function transferMoney(fromId, toId, amount) {
    const client = await pool.connect();
    try {
        await client.query('BEGIN');

        await client.query(
            'UPDATE accounts SET balance = balance - $1 WHERE id = $2',
            [amount, fromId]
        );

        await client.query(
            'UPDATE accounts SET balance = balance + $1 WHERE id = $2',
            [amount, toId]
        );

        await client.query('COMMIT');
        return { success: true };
    } catch (e) {
        await client.query('ROLLBACK');
        throw e;
    } finally {
        client.release();
    }
}
```

SQLite with Node.js

Installation:

bash

```
npm install sqlite3
```

Implementation:

```javascript
const sqlite3 = require('sqlite3').verbose();
const db = new sqlite3.Database('./database.sqlite');

// Wrapper for promises
const dbAsync = {
    run(sql, params = []) {
        return new Promise((resolve, reject) => {
            db.run(sql, params, function(err) {
                if (err) reject(err);
                else resolve({ lastID: this.lastID, changes: this.changes });
            });
        });
    },
    get(sql, params = []) {
        return new Promise((resolve, reject) => {
            db.get(sql, params, (err, result) => {
                if (err) reject(err);
                else resolve(result);
            });
        });
    },
    all(sql, params = []) {
        return new Promise((resolve, reject) => {
            db.all(sql, params, (err, rows) => {
                if (err) reject(err);
                else resolve(rows);
            });
        });
    }
};

// Usage
async function initializeDB() {
    await dbAsync.run(`
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
```

```
            email TEXT UNIQUE NOT NULL
        )
    `);
}

async function addUser(name, email) {
    return await dbAsync.run(
        'INSERT INTO users (name, email) VALUES (?, ?)',
        [name, email]
    );
}
```

---

2. ORM Concepts

What is ORM?

ORM (Object-Relational Mapping) is a technique that maps database tables to programming language objects.

Benefits of ORM

Benefit	Description
Productivity	Write code instead of SQL
Portability	Switch databases easily
Security	Built-in SQL injection protection
Relationships	Easy navigation between objects
Migrations	Version control for schema

Benefit	Description
Caching	Built-in caching mechanisms

Popular ORMs by Language

Language	ORM
JavaScript/TypeScript	Sequelize, TypeORM, Prisma, Knex
Python	SQLAlchemy, Django ORM, Peewee
Java	Hibernate, JPA
C#	Entity Framework, Dapper
PHP	Eloquent (Laravel), Doctrine
Ruby	ActiveRecord (Rails)

Sequelize (Node.js) Example

Installation:

bash

```bash
npm install sequelize mysql2
```

Model Definition:

javascript

```javascript
const { Sequelize, DataTypes } = require('sequelize');

// Connection
const sequelize = new Sequelize('database', 'username', 'password', {
```

```
    host: 'localhost',
    dialect: 'mysql'
});

// Define models
const User = sequelize.define('User', {
    id: {
        type: DataTypes.INTEGER,
        autoIncrement: true,
        primaryKey: true
    },
    name: {
        type: DataTypes.STRING,
        allowNull: false
    },
    email: {
        type: DataTypes.STRING,
        allowNull: false,
        unique: true,
        validate: {
            isEmail: true
        }
    },
    age: {
        type: DataTypes.INTEGER,
        validate: {
            min: 18,
            max: 100
        }
    }
});

const Post = sequelize.define('Post', {
    title: DataTypes.STRING,
    content: DataTypes.TEXT,
    published: {
        type: DataTypes.BOOLEAN,
        defaultValue: false
    }
});

// Define relationships
```

```
User.hasMany(Post, { foreignKey: 'userId' });
Post.belongsTo(User, { foreignKey: 'userId' });

// Sync with database
await sequelize.sync({ alter: true });
```

CRUD Operations:

```javascript
// Create
const user = await User.create({
    name: 'John Doe',
    email: 'john@example.com',
    age: 30
});

// Read
const users = await User.findAll({
    where: { age: { [Op.gt]: 25 } },
    order: [['name', 'ASC']],
    include: [Post]  // Eager loading
});

const user = await User.findByPk(1);

// Update
await User.update(
    { age: 31 },
    { where: { id: 1 } }
);

// Or
const user = await User.findByPk(1);
user.age = 31;
await user.save();

// Delete
await User.destroy({ where: { id: 1 } });
```

TypeORM (TypeScript) Example

```typescript
```

```
import { Entity, PrimaryGeneratedColumn, Column, OneToMany, createConnection } fro
m 'typeorm';

@Entity()
export class User {
    @PrimaryGeneratedColumn()
    id: number;

    @Column()
    name: string;

    @Column({ unique: true })
    email: string;

    @OneToMany(() => Post, post => post.user)
    posts: Post[];
}

@Entity()
export class Post {
    @PrimaryGeneratedColumn()
    id: number;

    @Column()
    title: string;

    @Column('text')
    content: string;

    @ManyToOne(() => User, user => user.posts)
    user: User;
}

// Connection
const connection = await createConnection({
    type: 'mysql',
    host: 'localhost',
    port: 3306,
    username: 'root',
    password: 'password',
    database: 'test',
    entities: [User, Post],
```

```
    synchronize: true,
});

// Repository pattern
const userRepository = connection.getRepository(User);
const users = await userRepository.find({
    where: { name: Like('%John%') },
    relations: ['posts']
});
```

Prisma (Modern TypeScript ORM)

Schema definition (`schema.prisma`):

```prisma
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  name      String?
  posts     Post[]
  profile   Profile?
}

model Post {
  id        Int      @id @default(autoincrement)
  title     String
  content   String?
  published Boolean  @default(false)
  author    User     @relation(fields: [authorId], references: [id])
  authorId  Int
}
```

```
model Profile {
  id     Int    @id @default(autoincrement())
  bio    String
  user   User   @relation(fields: [userId], references: [id])
  userId Int    @unique
}
```

Usage:

```typescript
import { PrismaClient } from '@prisma/client'

const prisma = new PrismaClient()

async function main() {
    // Create
    const user = await prisma.user.create({
        data: {
            email: 'alice@prisma.io',
            name: 'Alice',
            posts: {
                create: { title: 'Hello World' }
            }
        }
    })

    // Read with relations
    const users = await prisma.user.findMany({
        include: {
            posts: true,
            profile: true
        },
        where: {
            email: { contains: 'prisma' }
        }
    })

    // Update
    await prisma.post.update({
        where: { id: 1 },
        data: { published: true }
    })
```

```
}
```

---

3. Connection Pooling

What is Connection Pooling?

Connection pooling maintains a cache of database connections that can be reused across requests, avoiding the overhead of creating new connections for each request.

Why Connection Pooling?

Without Pooling	With Pooling
New connection per request	Reuse existing connections
High overhead (TCP handshake)	Minimal overhead
Database limit reached quickly	Controlled number of connections
Slow response time	Fast response time
Resource intensive	Efficient resource usage

Connection Pool Parameters

Parameter	Description	Example
max	Maximum connections in pool	20

Parameter	Description	Example
min	Minimum idle connections	5
idleTimeout	Close idle connections after	30000 ms
acquireTimeout	Wait for connection timeout	10000 ms
queueLimit	Max queue size	0 (unlimited)

Implementation Examples

Node.js MySQL2 Pool:

```javascript
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
    host: 'localhost',
    user: 'root',
    password: 'password',
    database: 'app_db',
    waitForConnections: true,
    connectionLimit: 10,      // Max connections
    queueLimit: 0,            // Unlimited queue
    enableKeepAlive: true,
    keepAliveInitialDelay: 0
});

// Use pool in routes
app.get('/users', async (req, res) => {
    try {
        // Get connection from pool
        const connection = await pool.getConnection();

        // Execute query
        const [rows] = await connection.query('SELECT * FROM users');
```

```javascript
        // Release back to pool
        connection.release();

        res.json(rows);
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: 'Database error' });
    }
});

// Or use pool directly (automatically releases)
app.get('/products', async (req, res) => {
    const [rows] = await pool.query('SELECT * FROM products');
    res.json(rows);
});

// Monitor pool
setInterval(() => {
    console.log({
        totalConnections: pool.pool._allConnections.length,
        idleConnections: pool.pool._freeConnections.length,
        queuedRequests: pool.pool._connectionQueue.length
    });
}, 5000);
```

Node.js PostgreSQL Pool:

```javascript
const { Pool } = require('pg');

const pool = new Pool({
    host: 'localhost',
    port: 5432,
    user: 'postgres',
    password: 'password',
    database: 'app_db',
    max: 20,                    // Max connections
    idleTimeoutMillis: 30000,  // Close idle after 30s
    connectionTimeoutMillis: 2000,  // Wait 2s for connection
    allowExitOnIdle: false
});
```

```javascript
// Event handlers
pool.on('connect', (client) => {
    console.log('New client connected');
});

pool.on('error', (err, client) => {
    console.error('Unexpected error on idle client', err);
});

// Query function with automatic release
async function query(text, params) {
    const start = Date.now();
    const res = await pool.query(text, params);
    const duration = Date.now() - start;
    console.log('executed query', { text, duration, rows: res.rowCount });
    return res;
}

// Use in routes
app.get('/users/:id', async (req, res) => {
    const { id } = req.params;
    const result = await query('SELECT * FROM users WHERE id = $1', [id]);
    res.json(result.rows[0]);
});
```

Pool Monitoring and Tuning

```javascript
// Metrics endpoint for monitoring
app.get('/metrics/pool', async (req, res) => {
    const poolStatus = {
        totalCount: pool.totalCount,
        idleCount: pool.idleCount,
        waitingCount: pool.waitingCount,
        // MySQL specific
        // ... more metrics
    };
    res.json(poolStatus);
});
```

```
// Connection pool best practices
class DatabaseService {
    constructor() {
        this.pool = mysql.createPool({
            // ... config
        });

        // Warm up pool
        this.warmUpPool();
    }

    async warmUpPool() {
        // Create minimum connections
        const connections = [];
        for (let i = 0; i < 5; i++) {
            connections.push(await this.pool.getConnection());
        }
        connections.forEach(conn => conn.release());
        console.log('Pool warmed up');
    }

    async withConnection(callback) {
        const connection = await this.pool.getConnection();
        try {
            return await callback(connection);
        } finally {
            connection.release(); // Always release
        }
    }

    async transaction(callback) {
        return this.withConnection(async (conn) => {
            await conn.beginTransaction();
            try {
                const result = await callback(conn);
                await conn.commit();
                return result;
            } catch (error) {
                await conn.rollback();
                throw error;
            }
        });
```

```
    }
}

// Usage
const db = new DatabaseService();
const result = await db.transaction(async (conn) => {
    await conn.query('UPDATE accounts SET balance = balance - 100 WHERE id = 1');
    await conn.query('UPDATE accounts SET balance = balance + 100 WHERE id = 2');
    return { success: true };
});
```

---

4. Migrations

What are Migrations?

Migrations are version-controlled scripts that define database schema changes over time. They allow teams to evolve the database schema consistently across environments.

Benefits of Migrations

Benefit	Description
Version Control	Track schema changes in Git
Reproducibility	Same schema everywhere
Rollback	Undo changes if needed
Collaboration	Team works on same schema
Deployment	Automated schema updates

Benefit	Description
Documentation	History of changes

Migration Tools

Language	Migration Tools
Node.js	Sequelize migrations, Knex.js, Umzug
Python	Alembic (SQLAlchemy), Django migrations
Ruby	ActiveRecord migrations
PHP	Laravel migrations, Doctrine migrations
Java	Flyway, Liquibase
Database	Custom scripts, Version control

Knex.js Migrations (Node.js)

Installation:

bash

```bash
npm install knex
npx knex init  # Creates knexfile.js
```

Migration File Structure:

javascript

```javascript
// migrations/20240317000100_create_users_table.js
exports.up = function(knex) {
    return knex.schema.createTable('users', function(table) {
```

```
        table.increments('id').primary();
        table.string('name').notNullable();
        table.string('email').unique().notNullable();
        table.string('password_hash').notNullable();
        table.boolean('is_active').defaultTo(true);
        table.timestamp('created_at').defaultTo(knex.fn.now());
        table.timestamp('updated_at').defaultTo(knex.fn.now());

        // Indexes
        table.index('email');
        table.index(['name', 'is_active']);
    });
};

exports.down = function(knex) {
    return knex.schema.dropTable('users');
};
```

Second Migration:

```javascript
// migrations/20240317000200_add_posts_table.js
exports.up = function(knex) {
    return knex.schema.createTable('posts', function(table) {
        table.increments('id');
        table.string('title').notNullable();
        table.text('content');
        table.integer('user_id').unsigned().notNullable();
        table.enu('status', ['draft', 'published', 'archived']).defaultTo('draft')
;
        table.timestamps(true, true);

        // Foreign key
        table.foreign('user_id').references('users.id').onDelete('CASCADE');

        // Indexes
        table.index('user_id');
        table.index('status');
    });
};

exports.down = function(knex) {
```

```
    return knex.schema.dropTable('posts');
};
```

Running Migrations:

```bash
# Run all pending migrations
npx knex migrate:latest

# Rollback last migration
npx knex migrate:rollback

# Rollback to specific version
npx knex migrate:down 20240317000100

# Check status
npx knex migrate:status

# Create new migration
npx knex migrate:make add_comments_table
```

Sequelize Migrations

Generate Migration:

```bash
npx sequelize-cli migration:generate --name create-users
```

Migration File:

```javascript
'use strict';

module.exports = {
    up: async (queryInterface, Sequelize) => {
        await queryInterface.createTable('users', {
            id: {
                type: Sequelize.INTEGER,
                autoIncrement: true,
                primaryKey: true
            },
```

```
            name: {
                type: Sequelize.STRING,
                allowNull: false
            },
            email: {
                type: Sequelize.STRING,
                allowNull: false,
                unique: true
            },
            createdAt: {
                type: Sequelize.DATE,
                allowNull: false,
                defaultValue: Sequelize.literal('CURRENT_TIMESTAMP')
            },
            updatedAt: {
                type: Sequelize.DATE,
                allowNull: false,
                defaultValue: Sequelize.literal('CURRENT_TIMESTAMP')
            }
        });

        await queryInterface.addIndex('users', ['email']);
    },

    down: async (queryInterface, Sequelize) => {
        await queryInterface.dropTable('users');
    }
};
```

Database Version Control Table

Migrations create a table to track applied migrations:

```sql
CREATE TABLE knex_migrations (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255),
    batch INT,
    migration_time TIMESTAMP
);
```

```sql
-- Or with Sequelize
CREATE TABLE SequelizeMeta (
    name VARCHAR(255) PRIMARY KEY
);
```

Advanced Migration Patterns

Data Migrations:

```javascript
exports.up = async function(knex) {
    // Schema change
    await knex.schema.table('users', table => {
        table.string('full_name');
    });

    // Data migration
    const users = await knex('users').select('id', 'first_name', 'last_name');
    for (const user of users) {
        await knex('users')
            .where('id', user.id)
            .update({
                full_name: `${user.first_name} ${user.last_name}`
            });
    }
};
```

Conditional Migrations:

```javascript
exports.up = async function(knex) {
    const hasTable = await knex.schema.hasTable('users');
    if (!hasTable) {
        await knex.schema.createTable('users', table => {
            // ...
        });
    }

    const hasColumn = await knex.schema.hasColumn('users', 'phone');
    if (!hasColumn) {
        await knex.schema.table('users', table => {
```

```
            table.string('phone');
        });
    }
};
```

Batch Processing for Large Tables:

```javascript
exports.up = async function(knex) {
    // Add column with default
    await knex.schema.table('large_table', table => {
        table.string('new_column').defaultTo('temp');
    });

    // Update in batches
    const batchSize = 1000;
    let offset = 0;
    let rowsUpdated;

    do {
        const result = await knex('large_table')
            .whereNull('new_column')
            .orWhere('new_column', 'temp')
            .limit(batchSize)
            .update({
                new_column: knex.raw('CONCAT(old_column, "_processed")'),
                updated_at: knex.fn.now()
            });

        rowsUpdated = result;
        offset += batchSize;
    } while (rowsUpdated === batchSize);
};
```

---

5. Transactions in Backend Code

Transaction Patterns

Pattern 1: Manual Transaction Control

javascript

```javascript
async function transferFunds(fromId, toId, amount) {
    const connection = await pool.getConnection();

    try {
        await connection.beginTransaction();

        // Check balance
        const [fromAccount] = await connection.query(
            'SELECT balance FROM accounts WHERE id = ? FOR UPDATE',
            [fromId]
        );

        if (fromAccount[0].balance < amount) {
            throw new Error('Insufficient funds');
        }

        // Deduct from source
        await connection.query(
            'UPDATE accounts SET balance = balance - ? WHERE id = ?',
            [amount, fromId]
        );

        // Add to destination
        await connection.query(
            'UPDATE accounts SET balance = balance + ? WHERE id = ?',
            [amount, toId]
        );

        // Log transaction
        await connection.query(
            'INSERT INTO transfers (from_id, to_id, amount, status) VALUES (?, ?,
?, ?)',
            [fromId, toId, amount, 'completed']
        );
```

```
        await connection.commit();
        return { success: true };

    } catch (error) {
        await connection.rollback();
        throw error;
    } finally {
        connection.release();
    }
}
```

Pattern 2: With Retry Logic

javascript

```
async function executeWithRetry(fn, maxRetries = 3) {
    let lastError;

    for (let i = 0; i < maxRetries; i++) {
        try {
            return await fn();
        } catch (error) {
            lastError = error;

            // Only retry on deadlock errors
            if (error.code === 'ER_LOCK_DEADLOCK' || error.code === '40001') {
                console.log(`Deadlock detected, retry ${i + 1}/${maxRetries}`);
                await new Promise(resolve => setTimeout(resolve, 100 * Math.pow(2,
i)));
                continue;
            }
            throw error;
        }
    }

    throw lastError;
}

// Usage
await executeWithRetry(() => transferFunds(1, 2, 100));
```

Pattern 3: Distributed Transactions (Saga Pattern)

```javascript
class OrderSaga {
    async createOrder(orderData) {
        const steps = [
            { name: 'reserveInventory', compensation: 'releaseInventory' },
            { name: 'processPayment', compensation: 'refundPayment' },
            { name: 'createOrder', compensation: 'cancelOrder' },
            { name: 'scheduleShipping', compensation: 'cancelShipping' }
        ];

        const completedSteps = [];

        try {
            for (const step of steps) {
                await this[step.name](orderData);
                completedSteps.push(step);
            }

            return { success: true };
        } catch (error) {
            // Compensation: rollback completed steps in reverse order
            for (const step of completedSteps.reverse()) {
                await this[step.compensation](orderData);
            }
            throw error;
        }
    }

    async reserveInventory(orderData) {
        // Reserve inventory
    }

    async releaseInventory(orderData) {
        // Release reserved inventory
    }
}
```

Transaction Isolation in Application

```javascript
async function updateInventory(productId, quantity) {
```

```
    const connection = await pool.getConnection();

    try {
        await connection.beginTransaction();

        // Set isolation level
        await connection.query('SET TRANSACTION ISOLATION LEVEL SERIALIZABLE');

        // Lock the row
        const [product] = await connection.query(
            'SELECT stock FROM products WHERE id = ? FOR UPDATE',
            [productId]
        );

        if (product[0].stock < quantity) {
            throw new Error('Insufficient stock');
        }

        await connection.query(
            'UPDATE products SET stock = stock - ? WHERE id = ?',
            [quantity, productId]
        );

        await connection.commit();

    } catch (error) {
        await connection.rollback();
        throw error;
    } finally {
        connection.release();
    }
}
```

ORM Transactions

Sequelize:

```javascript
const transaction = await sequelize.transaction();

try {
```

```
    const user = await User.create({
        name: 'John',
        email: 'john@example.com'
    }, { transaction });

    const profile = await Profile.create({
        userId: user.id,
        bio: 'Developer'
    }, { transaction });

    await transaction.commit();
} catch (error) {
    await transaction.rollback();
}

// Automatic transaction
await sequelize.transaction(async (t) => {
    const user = await User.create({ name: 'John' }, { transaction: t });
    await Profile.create({ userId: user.id, bio: 'Dev' }, { transaction: t });
});
```

TypeORM:

```typescript
await getConnection().transaction(async manager => {
    const user = manager.create(User, { name: 'John' });
    await manager.save(user);

    const profile = manager.create(Profile, { userId: user.id, bio: 'Dev' });
    await manager.save(profile);
});
```

---

6. Common SQL Databases in Industry

Comparison of Popular Databases

Feature	MySQL	PostgreSQL	SQL Server	SQLite
License	GPL/Commercial	PostgreSQL License	Commercial	Public Domain
ACID	Yes (with InnoDB)	Yes	Yes	Yes
Performance	Fast reads	Feature-rich	Enterprise	Lightweight
Scalability	Good	Excellent	Excellent	Limited
JSON Support	Good	Excellent	Good	Limited
Full-Text Search	Yes	Yes	Excellent	Yes
GIS/ Spatial	Yes	Excellent	Yes	No
Concurrency	MVCC	MVCC	Locking/MVCC	Locking
Replication	Master/Slave	Master/Slave	Multiple	None
Use Cases	Web apps	Complex queries	Enterprise	Mobile, embedded

MySQL

Best For: Web applications, read-heavy workloads, LAMP stack

Strengths:

- Fast read operations
- Easy to set up
- Large community
- Multiple storage engines
- Good replication

Weaknesses:

- ACID only with InnoDB
- Limited advanced features
- Less strict data validation

Configuration Example:

```sql
-- my.cnf
[mysqld]
innodb_buffer_pool_size = 2G
max_connections = 500
query_cache_type = 1
query_cache_size = 256M
innodb_log_file_size = 512M
```

PostgreSQL

Best For: Complex queries, data integrity, JSON data, GIS applications

Strengths:

- Full ACID compliance
- Advanced indexing (GIN, GiST, BRIN)
- Excellent JSON/JSONB support

- Extensible (custom functions)
- MVCC implemented well
- Strong community

Weaknesses:

- Slower for simple reads
- Higher memory usage
- Configuration complexity

Configuration Example:

```sql
-- postgresql.conf
shared_buffers = 2GB
work_mem = 32MB
maintenance_work_mem = 256MB
effective_cache_size = 6GB
wal_buffers = 16MB
max_connections = 200
```

Microsoft SQL Server

Best For: Enterprise applications, Windows ecosystem, BI integration

Strengths:

- Integration with Microsoft tools
- Excellent management studio
- Advanced analytics (SSAS, SSRS)
- In-memory OLTP
- Always On availability groups
- Strong security features

Weaknesses:

- Licensing cost
- Windows-only (except Linux version)

- Resource intensive

Configuration Example:

```sql
-- SQL Server
EXEC sp_configure 'max server memory', 4096;
EXEC sp_configure 'min server memory', 1024;
EXEC sp_configure 'max degree of parallelism', 4;
RECONFIGURE;
```

SQLite

Best For: Mobile apps, embedded systems, development, small applications

Strengths:

- Zero configuration
- Serverless
- Single file database
- Cross-platform
- ACID compliant
- Small footprint

Weaknesses:

- Limited concurrency (write lock)
- No user management
- Limited ALTER TABLE
- Not for high traffic

Configuration Example:

```sql
-- Enable WAL mode for better concurrency
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA cache_size = 10000;
PRAGMA foreign_keys = ON;
```

Choosing the Right Database

Application Type	Recommended Database
Traditional Web App	MySQL or PostgreSQL
Complex Data/Reporting	PostgreSQL
Enterprise/Windows	SQL Server
Mobile/Embedded	SQLite
High Concurrency Writes	PostgreSQL
Read-heavy	MySQL with replication
Geospatial	PostgreSQL with PostGIS
Full-Text Search	PostgreSQL or Elasticsearch
Microservices	PostgreSQL or MySQL
Development	SQLite (for local)

Summary: Database Design Best Practices

Relationship Summary

Relationship	Implementation	Example
One-to-One	Same PK or unique FK	User ↔ Profile

Relationship	Implementation	Example
One-to-Many	FK on many side	Department → Employees
Many-to-Many	Junction table	Students ↔ Courses

Backend Integration Summary

Concept	Purpose	Tools
ORM	Object-relational mapping	Sequelize, TypeORM, Prisma
Connection Pool	Reuse connections	Built-in drivers
Migrations	Schema version control	Knex, Alembic, Flyway
Transactions	Data integrity	BEGIN/COMMIT in code
Prepared Statements	Security & performance	Parameterized queries

Database Selection Guide

```text
Small App / MVP ──→ SQLite
Web App ──────────→ MySQL / PostgreSQL
Enterprise ────────→ SQL Server / Oracle
Geospatial ────────→ PostgreSQL + PostGIS
High Analytics ─────→ PostgreSQL / Column stores
Mobile ────────────→ SQLite / Realm
```

ORM Concepts and Practical Skills: Complete Theory Notes

Part 1: ORM Concepts (Important for Full-Stack)

Object-Relational Mapping (ORM) is a technique that bridges the gap between object-oriented programming languages and relational databases.

---

1. What is ORM?

Definition

ORM (Object-Relational Mapping) is a programming technique that converts data between incompatible type systems in object-oriented programming languages and relational databases. It creates a "virtual object database" that can be used from within the programming language.

The Impedance Mismatch

Object-Oriented	Relational	ORM Bridge
Classes	Tables	Maps class to table
Objects	Rows	Maps object to row
Attributes	Columns	Maps properties to columns
Inheritance	No direct equivalent	Various mapping strategies
Object references	Foreign keys	Maps references to relationships

How ORM Works

```text
Application Code (Objects)              Database (Tables)
┌──────────────────────┐                ┌──────────────────────┐
│ user = new User()    │                │ users table          │
│ user.name = "John"   │── ORM ────────►│ id | name | email    │
│ user.save()          │                │ 1  | John | ...      │
└──────────────────────┘                └──────────────────────┘
         │                                        ▲
         │                                        │
         └──────────── Object-Relational Mapping ─────┘
                    - Converts objects to SQL
                    - Manages relationships
                    - Handles transactions
                    - Provides query interface
```

Benefits of ORM

Benefit	Description	Example
Productivity	Write code instead of SQL	`User.find(1)` vs `SELECT * FROM users WHERE i`
Portability	Switch databases easily	Same code works with MySQL, PostgreSQL, S
Security	Built-in SQL injection protection	Parameterized queries automatically
Relationships	Easy navigation between objects	`user.posts` instead of JOINs
Migrations	Version control for schema	Programmatic schema changes
Caching	Built-in caching mechanisms	First-level, second-level cache
Lazy Loading	Load only when needed	`user.orders` loads on access

Drawbacks of ORM

Drawback	Description	Mitigation
Performance	Generated SQL may be suboptimal	Use raw queries for complex operati
Learning Curve	Must learn ORM-specific API	Start with simple operations
Complexity	Can hide what's happening	Monitor generated SQL
N+1 Problem	Can cause performance issues	Use eager loading
Less Control	Harder to optimize	Use query builder for complex queri

---

2. Entity Mapping

Entity mapping is the process of defining how classes and their properties map to database tables and columns.

Basic Mapping Concepts

```javascript
// Example: User entity mapping
@Entity({ tableName: 'users' })
class User {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({ name: 'first_name', length: 100 })
    firstName: string;

    @Column({ name: 'last_name', length: 100 })
    lastName: string;
```

```
    @Column({ unique: true })
    email: string;

    @Column({ type: 'date', nullable: true })
    birthDate: Date;

    @Column({ default: true })
    isActive: boolean;

    @CreateDateColumn()
    createdAt: Date;

    @UpdateDateColumn()
    updatedAt: Date;
}
```

Column Types and Options

Option	Description	Example
type	Database column type	`@Column({ type: 'text' })`
length	Maximum length	`@Column({ length: 255 })`
nullable	Can be NULL	`@Column({ nullable: true })`
unique	Unique constraint	`@Column({ unique: true })`
default	Default value	`@Column({ default: 0 })`
primary	Primary key	`@PrimaryColumn()`
autoincrement	Auto-increment	`@PrimaryGeneratedColumn()`

Option	Description	Example
enum	Enum values	`@Column({ enum: ['admin', 'user'] })`
precision/scale	Decimal precision	`@Column({ precision: 10, scale: 2 })`

Relationship Mapping

One-to-One:

```javascript
// User has one Profile
@Entity()
class User {
    @OneToOne(() => Profile)
    @JoinColumn() // Owning side
    profile: Profile;
}

@Entity()
class Profile {
    @OneToOne(() => User, user => user.profile)
    user: User; // Inverse side
}

// Usage
const user = await userRepository.findOne({
    where: { id: 1 },
    relations: ['profile']
});
```

One-to-Many / Many-to-One:

```javascript
// User has many Posts (One-to-Many)
@Entity()
class User {
    @OneToMany(() => Post, post => post.user)
    posts: Post[];
```

```
}

// Post belongs to one User (Many-to-One)
@Entity()
class Post {
    @ManyToOne(() => User, user => user.posts)
    @JoinColumn({ name: 'user_id' })
    user: User;
}

// Usage
const user = await userRepository.findOne({
    where: { id: 1 },
    relations: ['posts']
});
```

Many-to-Many:

javascript

```
// User has many Roles, Roles have many Users
@Entity()
class User {
    @ManyToMany(() => Role, role => role.users)
    @JoinTable({ // Owning side
        name: 'user_roles',
        joinColumn: { name: 'user_id', referencedColumnName: 'id' },
        inverseJoinColumn: { name: 'role_id', referencedColumnName: 'id' }
    })
    roles: Role[];
}

@Entity()
class Role {
    @ManyToMany(() => User, user => user.roles)
    users: User[]; // Inverse side
}

// Usage with junction table data
@Entity()
class UserRole {
    @ManyToOne(() => User)
    user: User;
```

```
    @ManyToOne(() => Role)
    role: Role;

    @Column()
    assignedAt: Date;

    @Column()
    assignedBy: string;
}
```

Inheritance Mapping Strategies

Single Table Inheritance:

```javascript
// All classes in one table
@Entity()
@Inheritance({ strategy: InheritanceType.SINGLE_TABLE })
@DiscriminatorColumn({ name: 'type', type: 'string' })
class Vehicle {
    @PrimaryGeneratedColumn()
    id: number;

    @Column()
    make: string;

    @Column()
    model: string;
}

@ChildEntity() // Discriminator value 'Car' by default
class Car extends Vehicle {
    @Column()
    doors: number;
}

@ChildEntity()
class Motorcycle extends Vehicle {
    @Column()
    hasSidecar: boolean;
```

```
}
```

Joined Table Inheritance:

```javascript
// Each class in its own table, joined via foreign key
@Entity()
@Inheritance({ strategy: InheritanceType.JOINED })
class Person {
    @PrimaryGeneratedColumn()
    id: number;

    @Column()
    name: string;
}

@ChildEntity()
class Employee extends Person {
    @Column()
    salary: number;

    @Column()
    hireDate: Date;
}
```

---

3. Lazy vs Eager Loading

Lazy Loading

Lazy loading defers loading of related entities until they are accessed.

```javascript
// Lazy loading (TypeORM)
@Entity()
class User {
    @OneToMany(() => Post, post => post.user, { lazy: true })
    posts: Promise<Post[]>; // Returns Promise, loads when accessed
}
```

```
// Usage
const user = await userRepository.findOne(1);
// posts not loaded yet

const posts = await user.posts; // Triggers database query
// Now posts are loaded
```

Lazy Loading Implementation:

```javascript
class LazyLoader {
    private _posts: Post[] | null = null;
    private _loaded = false;

    async getPosts(userId: number): Promise<Post[]> {
        if (!this._loaded) {
            this._posts = await database.query(
                'SELECT * FROM posts WHERE user_id = ?',
                [userId]
            );
            this._loaded = true;
        }
        return this._posts;
    }
}

// Usage
const user = await getUser(1);
const posts = await user.getPosts(); // Query executed here
```

Eager Loading

Eager loading loads related entities upfront in the same query.

```javascript
// Eager loading (TypeORM)
@Entity()
class User {
    @OneToMany(() => Post, post => post.user, { eager: true })
    posts: Post[]; // Loaded automatically with User
```

```
}

// Or specify at query time
const user = await userRepository.findOne({
    where: { id: 1 },
    relations: ['posts', 'profile', 'posts.comments']
});

// Equivalent SQL
SELECT
    u.*,
    p.*,
    pr.*,
    c.*
FROM users u
LEFT JOIN posts p ON u.id = p.user_id
LEFT JOIN profiles pr ON u.id = pr.user_id
LEFT JOIN comments c ON p.id = c.post_id
WHERE u.id = 1;
```

Comparison: Lazy vs Eager

Aspect	Lazy Loading	Eager Loading
When loaded	On access	Initially
Number of queries	1 + N (N+1 problem)	1 (with joins)
Memory usage	Lower initially	Higher initially
Flexibility	Load only what's needed	Predictable loading
Control	Application decides	Query decides
Best for	Optional relationships	Always-needed relationships

Solving N+1 Problem

Problem:

javascript

```javascript
// N+1 problem - BAD
const users = await userRepository.find(); // 1 query
for (const user of users) {
    const posts = await postRepository.find({ where: { userId: user.id } }); // N
queries
    // Process posts
}
// Total: 1 + N queries
```

Solution 1: Eager Loading:

javascript

```javascript
// Solution - Eager loading
const users = await userRepository.find({
    relations: ['posts'] // 1 query with JOIN
});
// Total: 1 query
```

Solution 2: Batch Loading:

javascript

```javascript
// Solution - Batch loading
const users = await userRepository.find(); // 1 query
const userIds = users.map(u => u.id);
const posts = await postRepository.find({
    where: { userId: In(userIds) } // 1 query with IN
});

// Group posts by user
const postsByUser = groupBy(posts, 'userId');
users.forEach(user => {
    user.posts = postsByUser[user.id] || [];
});
// Total: 2 queries
```

Solution 3: DataLoader Pattern:

javascript

```
// DataLoader for batching and caching
class DataLoader {
    constructor(batchFn) {
        this.batchFn = batchFn;
        this.cache = new Map();
        this.queue = [];
    }

    async load(key) {
        if (this.cache.has(key)) {
            return this.cache.get(key);
        }

        return new Promise((resolve) => {
            this.queue.push({ key, resolve });

            // Batch after event loop tick
            setTimeout(() => this.dispatch(), 0);
        });
    }

    async dispatch() {
        const queue = [...this.queue];
        this.queue = [];

        const keys = queue.map(item => item.key);
        const results = await this.batchFn(keys);

        queue.forEach((item, index) => {
            this.cache.set(item.key, results[index]);
            item.resolve(results[index]);
        });
    }
}

// Usage
const postLoader = new DataLoader(async (userIds) => {
    const posts = await postRepository.find({
        where: { userId: In(userIds) }
    });
    return userIds.map(id => posts.filter(p => p.userId === id));
});
```

```javascript
const users = await userRepository.find();
for (const user of users) {
    user.posts = await postLoader.load(user.id); // Batched
}
// Total: 2 queries (1 for users, 1 batched for posts)
```

---

4. Migration Tools

What are Migrations?

Migrations are version-controlled scripts that define and manage database schema changes over time.

Migration Workflow

```text
Local Development              Staging/Production
──────────────────────────────────────────────────

1. Create/change models
2. Generate migration
3. Apply migration locally
4. Test changes
5. Commit migration file
         │
         ▼
6. Pull changes
7. Run migrations
8. Database updated
```

Migration File Structure

Sequelize Example:

```javascript
// migrations/20240317000100-create-users.js
```

```
module.exports = {
    up: async (queryInterface, Sequelize) => {
        await queryInterface.createTable('users', {
            id: {
                type: Sequelize.INTEGER,
                autoIncrement: true,
                primaryKey: true
            },
            name: {
                type: Sequelize.STRING,
                allowNull: false
            },
            email: {
                type: Sequelize.STRING,
                allowNull: false,
                unique: true
            },
            createdAt: {
                type: Sequelize.DATE,
                allowNull: false
            },
            updatedAt: {
                type: Sequelize.DATE,
                allowNull: false
            }
        });

        await queryInterface.addIndex('users', ['email']);
    },

    down: async (queryInterface, Sequelize) => {
        await queryInterface.dropTable('users');
    }
};
```

Prisma Example:

```prisma
// schema.prisma - declarative schema
model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
```

```
  name      String?
  posts     Post[]
  profile   Profile?
}

model Post {
  id        Int      @id @default(autoincrement())
  title     String
  content   String?
  published Boolean  @default(false)
  author    User     @relation(fields: [authorId], references: [id])
  authorId  Int
}

// Generate migration:
// npx prisma migrate dev --name add_user_model
```

Migration Commands

Sequelize:

```bash
# Generate migration
npx sequelize-cli migration:generate --name create-users

# Run all pending migrations
npx sequelize-cli db:migrate

# Undo last migration
npx sequelize-cli db:migrate:undo

# Undo all migrations
npx sequelize-cli db:migrate:undo:all

# Check status
npx sequelize-cli db:migrate:status
```

TypeORM:

```bash
# Generate migration from entity changes
```

```bash
npx typeorm migration:generate -n CreateUsers

# Create empty migration
npx typeorm migration:create -n AddUserEmail

# Run migrations
npx typeorm migration:run

# Revert last migration
npx typeorm migration:revert

# Show migrations
npx typeorm migration:show
```

Prisma:

```bash
# Create migration from schema changes
npx prisma migrate dev --name init

# Apply to production
npx prisma migrate deploy

# Reset database
npx prisma migrate reset

# Check migration status
npx prisma migrate status
```

Advanced Migration Patterns

Data Migration:

```javascript
module.exports = {
    up: async (queryInterface, Sequelize) => {
        // Schema change
        await queryInterface.addColumn('users', 'full_name', {
            type: Sequelize.STRING
        });
```

```javascript
        // Data migration
        const users = await queryInterface.sequelize.query(
            'SELECT id, first_name, last_name FROM users',
            { type: Sequelize.QueryTypes.SELECT }
        );

        for (const user of users) {
            await queryInterface.sequelize.query(
                'UPDATE users SET full_name = :fullName WHERE id = :id',
                {
                    replacements: {
                        fullName: `${user.first_name} ${user.last_name}`,
                        id: user.id
                    }
                }
            );
        }
    },

    down: async (queryInterface, Sequelize) => {
        await queryInterface.removeColumn('users', 'full_name');
    }
};
```

Conditional Migrations:

```javascript
module.exports = {
    up: async (queryInterface, Sequelize) => {
        const tableExists = await queryInterface.tableExists('users');

        if (!tableExists) {
            await queryInterface.createTable('users', {
                // ... columns
            });
        }

        const columns = await queryInterface.describeTable('users');

        if (!columns.phone) {
            await queryInterface.addColumn('users', 'phone', {
                type: Sequelize.STRING
```

```
            });
        }
    }
};
```

Batch Migration for Large Tables:

```javascript
module.exports = {
    up: async (queryInterface, Sequelize) => {
        const batchSize = 1000;
        let offset = 0;
        let hasMore = true;

        while (hasMore) {
            const [results] = await queryInterface.sequelize.query(`
                UPDATE large_table
                SET processed = true
                WHERE id IN (
                    SELECT id FROM large_table
                    WHERE processed = false
                    LIMIT ${batchSize}
                )
                RETURNING id
            `);

            hasMore = results.length === batchSize;
            offset += batchSize;

            // Small delay to prevent lock contention
            await new Promise(resolve => setTimeout(resolve, 100));
        }
    }
};
```

---

5. ORM Examples

Sequelize (Node.js)

Installation and Setup:

bash

```bash
npm install sequelize mysql2
npx sequelize-cli init
```

Model Definition:

javascript

```javascript
// models/user.js
module.exports = (sequelize, DataTypes) => {
    const User = sequelize.define('User', {
        id: {
            type: DataTypes.INTEGER,
            autoIncrement: true,
            primaryKey: true
        },
        name: {
            type: DataTypes.STRING,
            allowNull: false,
            validate: {
                notEmpty: true,
                len: [2, 50]
            }
        },
        email: {
            type: DataTypes.STRING,
            allowNull: false,
            unique: true,
            validate: {
                isEmail: true
            }
        },
        age: {
            type: DataTypes.INTEGER,
            validate: {
                min: 18,
```

```
                max: 100
            }
        },
        status: {
            type: DataTypes.ENUM('active', 'inactive', 'pending'),
            defaultValue: 'pending'
        }
    }, {
        timestamps: true,
        paranoid: true, // soft delete
        underscored: true, // use snake_case in DB
        indexes: [
            {
                unique: true,
                fields: ['email']
            },
            {
                fields: ['status']
            }
        ]
    });

    User.associate = (models) => {
        User.hasMany(models.Post, {
            foreignKey: 'userId',
            as: 'posts'
        });

        User.belongsToMany(models.Role, {
            through: 'UserRoles',
            as: 'roles',
            foreignKey: 'userId'
        });
    };

    return User;
};
```

CRUD Operations:

```javascript
// Create
```

```javascript
const user = await User.create({
    name: 'John Doe',
    email: 'john@example.com',
    age: 30
});

// Read
const user = await User.findByPk(1);
const users = await User.findAll({
    where: {
        age: { [Op.gt]: 25 },
        status: 'active'
    },
    include: ['posts'],
    order: [['createdAt', 'DESC']],
    limit: 10
});

// Update
await User.update(
    { status: 'active' },
    { where: { id: 1 } }
);

// Or
const user = await User.findByPk(1);
user.age = 31;
await user.save();

// Delete
await User.destroy({ where: { id: 1 } }); // Soft delete if paranoid
await User.destroy({ where: { id: 1 }, force: true }); // Hard delete
```

Complex Queries:

```javascript
// Scopes
const User = sequelize.define('User', {
    // ...
}, {
    scopes: {
        active: {
```

```
            where: { status: 'active' }
        },
        withPosts: {
            include: ['posts']
        },
        olderThan(age) {
            return {
                where: {
                    age: { [Op.gt]: age }
                }
            };
        }
    }
});

// Usage
const activeUsers = await User.scope('active').findAll();
const activeWithPosts = await User.scope(['active', 'withPosts']).findAll();
const olderThan30 = await User.scope({ method: ['olderThan', 30] }).findAll();

// Raw queries
const [results, metadata] = await sequelize.query(
    'SELECT * FROM users WHERE age > :age',
    {
        replacements: { age: 25 },
        type: sequelize.QueryTypes.SELECT
    }
);
```

Prisma (Modern TypeScript ORM)

Schema Definition (`schema.prisma`):

```
prisma

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
```

```
}

model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  name      String?
  age       Int?
  status    Status   @default(PENDING)
  posts     Post[]
  profile   Profile?
  roles     Role[]   @relation("UserRoles")
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model Profile {
  id     Int    @id @default(autoincrement())
  bio    String
  avatar String?
  user   User   @relation(fields: [userId], references: [id])
  userId Int    @unique
}

model Post {
  id         Int        @id @default(autoincrement())
  title      String
  content    String?
  published  Boolean    @default(false)
  author     User       @relation(fields: [authorId], references: [id])
  authorId   Int
  categories Category[]
  createdAt  DateTime   @default(now())
  updatedAt  DateTime   @updatedAt
}

model Category {
  id    Int    @id @default(autoincrement())
  name  String @unique
  posts Post[]
}

model Role {
```

```
  id    Int    @id @default(autoincrement())
  name  String @unique
  users User[] @relation("UserRoles")
}

model UserRole {
  user      User     @relation(fields: [userId], references: [id])
  userId    Int
  role      Role     @relation(fields: [roleId], references: [id])
  roleId    Int
  assignedAt DateTime @default(now())
  assignedBy String

  @@id([userId, roleId])
}

enum Status {
  ACTIVE
  INACTIVE
  PENDING
}
```

Usage in Application:

```typescript
import { PrismaClient, Status } from '@prisma/client'

const prisma = new PrismaClient({
    log: ['query', 'info', 'warn', 'error']
})

// Create
const user = await prisma.user.create({
    data: {
        email: 'alice@prisma.io',
        name: 'Alice',
        profile: {
            create: { bio: 'I like turtles' }
        },
        posts: {
            create: [
                { title: 'Hello World' },
```

```
                { title: 'Second Post' }
            ]
        }
    },
    include: {
        profile: true,
        posts: true
    }
})

// Read with filters
const users = await prisma.user.findMany({
    where: {
        AND: [
            { age: { gt: 18 } },
            { status: 'ACTIVE' }
        ],
        OR: [
            { name: { contains: 'John' } },
            { email: { endsWith: '@example.com' } }
        ]
    },
    include: {
        posts: {
            where: { published: true },
            orderBy: { createdAt: 'desc' },
            take: 5
        }
    },
    orderBy: { name: 'asc' },
    skip: 10,
    take: 20
})

// Update
await prisma.user.update({
    where: { email: 'alice@prisma.io' },
    data: {
        age: 31,
        status: 'ACTIVE',
        posts: {
            updateMany: {
```

```
                where: { published: false },
                data: { published: true }
            }
        }
    }
})

// Delete
await prisma.user.delete({
    where: { email: 'alice@prisma.io' }
})

// Transaction
await prisma.$transaction([
    prisma.user.update({
        where: { id: 1 },
        data: { balance: { decrement: 100 } }
    }),
    prisma.user.update({
        where: { id: 2 },
        data: { balance: { increment: 100 } }
    })
])

// Raw query
const users = await prisma.$queryRaw`
    SELECT * FROM users
    WHERE age > ${minAge}
    ORDER BY created_at DESC
`

// Middleware
prisma.$use(async (params, next) => {
    const before = Date.now()
    const result = await next(params)
    const after = Date.now()
    console.log(`Query ${params.model}.${params.action} took ${after - before}ms`)
    return result
})
```

Hibernate (Java)

Entity Definition:

```java
import javax.persistence.*;
import java.util.*;

@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name", nullable = false, length = 100)
    private String name;

    @Column(unique = true, nullable = false)
    private String email;

    @Enumerated(EnumType.STRING)
    private Status status;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "created_at")
    private Date createdAt;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, fetch = FetchType.LAZ
Y)
    private List<Post> posts = new ArrayList<>();

    @ManyToMany
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles = new HashSet<>();

    // Getters and setters
}
```

```java
@Entity
@Table(name = "posts")
public class Post {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(length = 2000)
    private String content;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;

    // Getters and setters
}
```

Repository Layer:

```java
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    // Derived queries
    User findByEmail(String email);

    List<User> findByNameContainingAndAgeGreaterThan(String name, Integer age);

    @Query("SELECT u FROM User u WHERE u.status = :status ORDER BY u.createdAt DES
C")
    List<User> findByStatusWithCustomOrder(@Param("status") Status status);

    @Query(value = "SELECT * FROM users WHERE age > ?1", nativeQuery = true)
    List<User> findOlderThan(int age);

    @Modifying
    @Query("UPDATE User u SET u.status = :status WHERE u.id = :id")
    int updateUserStatus(@Param("id") Long id, @Param("status") Status status);
}
```

```java
@Service
@Transactional
public class UserService {
    @Autowired
    private UserRepository userRepository;

    public User createUser(UserDto dto) {
        User user = new User();
        user.setName(dto.getName());
        user.setEmail(dto.getEmail());
        return userRepository.save(user);
    }

    @Transactional(readOnly = true)
    public User getUserWithPosts(Long id) {
        User user = userRepository.findById(id).orElseThrow();
        Hibernate.initialize(user.getPosts()); // Force load
        return user;
    }
}
```

Part 2: Practical Skills Companies Expect

---

1. Design a Database Schema

Steps to Design a Schema

Step 1: Requirements Gathering

javascript

```javascript
// E-commerce example requirements
const requirements = {
    entities: ['User', 'Product', 'Order', 'Category', 'Review'],
```

```
    relationships: {
        'User has many Orders': 'one-to-many',
        'Order has many Products': 'many-to-many',
        'Product belongs to Category': 'many-to-one',
        'User writes many Reviews': 'one-to-many',
        'Product has many Reviews': 'one-to-many'
    },
    constraints: {
        'User email unique': true,
        'Order total > 0': true,
        'Review rating 1-5': true
    },
    queries: [
        'Find products by category',
        'Get user order history',
        'Calculate total sales per day',
        'Search products by name'
    ]
};
```

Step 2: Conceptual Design (ER Diagram)

text

```
┌──────────┐          ┌──────────┐
│  User    │          │  Order   │
├──────────┤          ├──────────┤
│PK id     │1       * │PK id     │
│ name     │◄─────────►│ date     │
│ email    │  places  │ total    │
│ address  │          │ status   │
└──────────┘          │FK user_id│
     │                └────┬─────┘
     │                     │
     │*                   *│
┌────▼─────┐          ┌────▼─────┐
│ Review   │          │OrderItem│
├──────────┤          ├──────────┤
│PK id     │          │PK,FK order_id
│ rating   │          │PK,FK product_id
│ comment  │          │ quantity
│ date     │          │ price
│FK user_id│          └──────┬────┘
```

```
│FK product_id│            │
└──────┬──────┘            │
       │                   │
       │*                  │*
┌──────▼──────┐           ┌──────▼──────┐
│ Product     │           │Category     │
├─────────────┤           ├─────────────┤
│PK id        │*        1│PK id        │
│ name        │◄──────────►│ name        │
│ price       │ belongs   │ description │
│ stock       │           └─────────────┘
│ description│
│FK category_id│
└─────────────┘
```

Step 3: Logical Design (Tables)

sql

```sql
-- Create schema
CREATE SCHEMA ecommerce;

-- Users table
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash CHAR(60) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    last_login TIMESTAMP NULL,
    is_active BOOLEAN DEFAULT TRUE,

    INDEX idx_email (email),
    INDEX idx_name (last_name, first_name)
) ENGINE=InnoDB;

-- Categories table
CREATE TABLE categories (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
```

```sql
    description TEXT,
    parent_id INT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (parent_id) REFERENCES categories(id),
    INDEX idx_parent (parent_id)
);

-- Products table
CREATE TABLE products (
    id INT PRIMARY KEY AUTO_INCREMENT,
    sku VARCHAR(50) NOT NULL UNIQUE,
    name VARCHAR(200) NOT NULL,
    slug VARCHAR(200) NOT NULL UNIQUE,
    description TEXT,
    price DECIMAL(10,2) NOT NULL CHECK (price > 0),
    cost DECIMAL(10,2) CHECK (cost > 0),
    stock_quantity INT NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
    category_id INT,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (category_id) REFERENCES categories(id),
    INDEX idx_category (category_id),
    INDEX idx_price (price),
    FULLTEXT INDEX idx_search (name, description)
);

-- Orders table
CREATE TABLE orders (
    id INT PRIMARY KEY AUTO_INCREMENT,
    order_number VARCHAR(50) NOT NULL UNIQUE,
    user_id INT NOT NULL,
    order_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status ENUM('pending', 'processing', 'shipped', 'delivered', 'cancelled')
           DEFAULT 'pending',
    subtotal DECIMAL(10,2) NOT NULL CHECK (subtotal >= 0),
    tax DECIMAL(10,2) NOT NULL DEFAULT 0,
    shipping_cost DECIMAL(10,2) NOT NULL DEFAULT 0,
    total DECIMAL(10,2) NOT NULL CHECK (total >= 0),
    shipping_address TEXT NOT NULL,
```

```sql
    payment_method VARCHAR(50),
    payment_status ENUM('pending', 'paid', 'failed') DEFAULT 'pending',
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id) REFERENCES users(id),
    INDEX idx_user (user_id),
    INDEX idx_status (status),
    INDEX idx_date (order_date)
);

-- Order items (junction table)
CREATE TABLE order_items (
    order_id INT,
    product_id INT,
    quantity INT NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(10,2) NOT NULL, -- Price at time of order
    discount DECIMAL(5,2) DEFAULT 0,
    total_price DECIMAL(10,2) GENERATED ALWAYS AS (quantity * unit_price * (1 - di
scount/100)) STORED,

    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(id),

    INDEX idx_product (product_id)
);

-- Reviews table
CREATE TABLE reviews (
    id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    product_id INT NOT NULL,
    rating TINYINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    title VARCHAR(200),
    comment TEXT,
    is_verified_purchase BOOLEAN DEFAULT FALSE,
    helpful_count INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (product_id) REFERENCES products(id),
```

```sql
    UNIQUE KEY unique_user_product (user_id, product_id), -- One review per user per product

    INDEX idx_product_rating (product_id, rating)
);
```

Step 4: Indexing Strategy

```sql
-- Performance indexes
CREATE INDEX idx_orders_user_status ON orders(user_id, status);
CREATE INDEX idx_orders_date_status ON orders(order_date, status);
CREATE INDEX idx_products_category_price ON products(category_id, price);
CREATE INDEX idx_reviews_product_helpful ON reviews(product_id, helpful_count DESC);

-- Full-text for search
CREATE FULLTEXT INDEX idx_product_search ON products(name, description);
```

---

2. Write Complex Joins

Multi-Table Joins

```sql
-- Get complete order details with user, items, and product info
SELECT
    o.order_number,
    o.order_date,
    CONCAT(u.first_name, ' ', u.last_name) AS customer_name,
    u.email,
    p.name AS product_name,
    oi.quantity,
    oi.unit_price,
    oi.discount,
    oi.total_price AS item_total,
    c.name AS category,
    o.total AS order_total,
    o.status
```

```sql
FROM orders o
JOIN users u ON o.user_id = u.id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
LEFT JOIN categories c ON p.category_id = c.id
WHERE o.order_date >= '2024-01-01'
    AND o.status IN ('shipped', 'delivered')
ORDER BY o.order_date DESC, o.order_number;
```

Aggregation with Joins

sql

```sql
-- Sales report by category with statistics
SELECT
    c.name AS category,
    COUNT(DISTINCT o.id) AS order_count,
    COUNT(DISTINCT oi.product_id) AS products_sold,
    SUM(oi.quantity) AS total_units,
    SUM(oi.total_price) AS total_revenue,
    AVG(oi.total_price) AS avg_order_value,
    MIN(oi.total_price) AS min_sale,
    MAX(oi.total_price) AS max_sale,
    SUM(oi.total_price) / SUM(oi.quantity) AS avg_unit_price
FROM categories c
LEFT JOIN products p ON c.id = p.category_id
LEFT JOIN order_items oi ON p.id = oi.product_id
LEFT JOIN orders o ON oi.order_id = o.id
WHERE o.status = 'delivered'
    AND o.order_date BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id, c.name
HAVING total_revenue > 1000
ORDER BY total_revenue DESC;
```

Self-Joins

sql

```sql
-- Category hierarchy
SELECT
    c1.name AS category,
    c2.name AS subcategory,
```

```sql
    c3.name AS sub_subcategory,
    COUNT(p.id) AS product_count
FROM categories c1
LEFT JOIN categories c2 ON c1.id = c2.parent_id
LEFT JOIN categories c3 ON c2.id = c3.parent_id
LEFT JOIN products p ON c3.id = p.category_id OR c2.id = p.category_id OR c1.id =
p.category_id
WHERE c1.parent_id IS NULL  -- Top-level categories
GROUP BY c1.name, c2.name, c3.name
ORDER BY c1.name, c2.name, c3.name;
```

Window Functions with Joins

sql

```sql
-- Customer purchase analysis
WITH customer_stats AS (
    SELECT
        u.id,
        u.first_name,
        u.last_name,
        COUNT(o.id) AS total_orders,
        SUM(o.total) AS lifetime_value,
        AVG(o.total) AS avg_order_value,
        MIN(o.order_date) AS first_order,
        MAX(o.order_date) AS last_order,
        RANK() OVER (ORDER BY SUM(o.total) DESC) AS value_rank
    FROM users u
    JOIN orders o ON u.id = o.user_id
    WHERE o.status = 'delivered'
    GROUP BY u.id, u.first_name, u.last_name
)
SELECT
    cs.*,
    DATEDIFF(cs.last_order, cs.first_order) AS customer_lifetime_days,
    CASE
        WHEN cs.last_order > DATE_SUB(NOW(), INTERVAL 30 DAY) THEN 'Active'
        WHEN cs.last_order > DATE_SUB(NOW(), INTERVAL 90 DAY) THEN 'At Risk'
        ELSE 'Churned'
    END AS customer_status,
    (SELECT COUNT(*) FROM customer_stats WHERE value_rank <= 10) AS top_10_count
FROM customer_stats cs
```

```sql
WHERE cs.lifetime_value > 0
ORDER BY cs.lifetime_value DESC;
```

---

3. Optimize Slow Queries

Query Analysis Process

sql

```sql
-- Step 1: Identify slow queries
-- MySQL
SELECT * FROM performance_schema.events_statements_summary_by_digest
ORDER BY SUM_TIMER_WAIT DESC
LIMIT 10;

-- Step 2: Explain the query
EXPLAIN ANALYZE
SELECT
    u.id,
    u.email,
    COUNT(o.id) AS order_count,
    SUM(o.total) AS total_spent
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE u.created_at > '2023-01-01'
GROUP BY u.id, u.email
HAVING total_spent > 1000;

-- Step 3: Analyze output
-- Look for:
-- - Full table scans
-- - Using temporary
-- - Using filesort
-- - Large row estimates
```

Optimization Techniques

Technique 1: Add Missing Indexes

sql

```sql
-- Before: Full table scan on orders
EXPLAIN SELECT * FROM orders WHERE user_id = 123 AND status = 'pending';
-- Type: ALL (full table scan)

-- After: Add composite index
CREATE INDEX idx_orders_user_status ON orders(user_id, status);

-- Now: Index lookup
EXPLAIN SELECT * FROM orders WHERE user_id = 123 AND status = 'pending';
-- Type: ref (index lookup)
```

Technique 2: Rewrite Subqueries as Joins

sql

```sql
-- Slow: Correlated subquery
SELECT
    p.*,
    (SELECT COUNT(*) FROM order_items oi WHERE oi.product_id = p.id) AS times_orde
red
FROM products p
WHERE p.category_id = 5;

-- Fast: Rewrite with LEFT JOIN and GROUP BY
SELECT
    p.*,
    COUNT(oi.id) AS times_ordered
FROM products p
LEFT JOIN order_items oi ON p.id = oi.product_id
WHERE p.category_id = 5
GROUP BY p.id;
```

Technique 3: Use EXISTS instead of IN for Large Datasets

sql

```sql
-- Slow for large datasets
SELECT * FROM users
WHERE id IN (SELECT user_id FROM orders WHERE total > 1000);

-- Fast with EXISTS (short-circuits)
SELECT * FROM users u
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id AND o.total > 1000);
```

Technique 4: Optimize Pagination

sql

```sql
-- Slow for large offsets
SELECT * FROM orders
ORDER BY id
LIMIT 100000, 20;

-- Fast with keyset pagination
SELECT * FROM orders
WHERE id > 100000
ORDER BY id
LIMIT 20;

-- Or with composite key
SELECT * FROM orders
WHERE (created_at, id) < ('2024-01-01', 100000)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Technique 5: Avoid Functions on Indexed Columns

sql

```sql
-- Bad: Function on indexed column
SELECT * FROM users
WHERE YEAR(created_at) = 2024;

-- Good: Range condition
SELECT * FROM users
WHERE created_at >= '2024-01-01'
  AND created_at < '2025-01-01';
```

Technique 6: Use Covering Indexes

sql

```sql
-- Query only needs specific columns
SELECT id, email, status FROM users WHERE status = 'active';

-- Create covering index
CREATE INDEX idx_users_status_covering ON users(status, id, email);
-- Now query can be satisfied entirely from index
```

Real-World Optimization Example

sql

```sql
-- Original slow query (takes 15 seconds)
SELECT
    u.email,
    u.first_name,
    u.last_name,
    COUNT(o.id) AS order_count,
    SUM(o.total) AS total_spent,
    AVG(r.rating) AS avg_rating
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
LEFT JOIN reviews r ON u.id = r.user_id
WHERE u.created_at > '2023-01-01'
  AND o.status = 'delivered'
GROUP BY u.id, u.email, u.first_name, u.last_name
HAVING total_spent > 500
ORDER BY total_spent DESC
LIMIT 100;

-- Optimized version (takes 0.5 seconds)
WITH user_orders AS (
    SELECT
        user_id,
        COUNT(*) AS order_count,
        SUM(total) AS total_spent
    FROM orders
    WHERE status = 'delivered'
      AND order_date > '2023-01-01'
    GROUP BY user_id
    HAVING SUM(total) > 500
),
user_reviews AS (
    SELECT
        user_id,
        AVG(rating) AS avg_rating
    FROM reviews
    GROUP BY user_id
)
SELECT
    u.email,
```

```sql
    u.first_name,
    u.last_name,
    COALESCE(uo.order_count, 0) AS order_count,
    uo.total_spent,
    ur.avg_rating
FROM users u
JOIN user_orders uo ON u.id = uo.user_id
LEFT JOIN user_reviews ur ON u.id = ur.user_id
WHERE u.created_at > '2023-01-01'
ORDER BY uo.total_spent DESC
LIMIT 100;

-- Indexes needed
CREATE INDEX idx_orders_status_date ON orders(status, order_date, user_id, total);
CREATE INDEX idx_reviews_user ON reviews(user_id, rating);
```

---

4. Write Transactions

Transaction Patterns

Pattern 1: Simple Transaction

javascript

```javascript
async function createOrder(userId, items) {
    const connection = await pool.getConnection();

    try {
        await connection.beginTransaction();

        // Calculate total
        let total = 0;
        for (const item of items) {
            const [product] = await connection.query(
                'SELECT price, stock FROM products WHERE id = ? FOR UPDATE',
                [item.productId]
            );
```

```
            if (product[0].stock < item.quantity) {
                throw new Error(`Insufficient stock for product ${item.productId}`
);
            }

            total += product[0].price * item.quantity;
        }

        // Create order
        const [orderResult] = await connection.query(
            'INSERT INTO orders (user_id, total, status) VALUES (?, ?, ?)',
            [userId, total, 'pending']
        );

        const orderId = orderResult.insertId;

        // Create order items and update stock
        for (const item of items) {
            await connection.query(
                'INSERT INTO order_items (order_id, product_id, quantity, price) V
ALUES (?, ?, ?, ?)',
                [orderId, item.productId, item.quantity, item.price]
            );

            await connection.query(
                'UPDATE products SET stock = stock - ? WHERE id = ?',
                [item.quantity, item.productId]
            );
        }

        await connection.commit();
        return orderId;

    } catch (error) {
        await connection.rollback();
        throw error;
    } finally {
        connection.release();
    }
}
```

Pattern 2: Transaction with Savepoints

```javascript
async function processBulkOrder(orders) {
    const connection = await pool.getConnection();

    try {
        await connection.beginTransaction();

        const results = [];

        for (let i = 0; i < orders.length; i++) {
            const order = orders[i];

            // Create savepoint for this order
            await connection.query(`SAVEPOINT order_${i}`);

            try {
                // Process individual order
                const orderId = await processSingleOrder(connection, order);
                results.push({ orderId, success: true });

            } catch (error) {
                // Rollback only this order
                await connection.query(`ROLLBACK TO SAVEPOINT order_${i}`);
                results.push({ order: order.id, success: false, error: error.messa
ge });

                // Continue with next order
            }
        }

        await connection.commit();
        return results;

    } catch (error) {
        await connection.rollback();
        throw error;
    } finally {
        connection.release();
    }
}
```

Pattern 3: Distributed Transaction (Saga Pattern)

```javascript
class OrderSaga {
    async execute(orderData) {
        const steps = [
            { name: 'reserveInventory', compensate: 'releaseInventory' },
            { name: 'processPayment', compensate: 'refundPayment' },
            { name: 'createOrder', compensate: 'cancelOrder' },
            { name: 'scheduleShipping', compensate: 'cancelShipping' }
        ];

        const executed = [];

        try {
            for (const step of steps) {
                await this[step.name](orderData);
                executed.push(step);
            }
            return { success: true };

        } catch (error) {
            // Compensation in reverse order
            for (const step of executed.reverse()) {
                await this[step.compensate](orderData);
            }
            throw error;
        }
    }

    async reserveInventory(data) {
        // Reserve inventory
    }

    async releaseInventory(data) {
        // Release reserved inventory
    }

    // Other steps...
}
```

Pattern 4: Optimistic Locking

```javascript
```

```
async function updateProductWithVersion(productId, updates) {
    const maxRetries = 3;

    for (let attempt = 0; attempt < maxRetries; attempt++) {
        const connection = await pool.getConnection();

        try {
            await connection.beginTransaction();

            // Get current version
            const [product] = await connection.query(
                'SELECT version FROM products WHERE id = ? FOR UPDATE',
                [productId]
            );

            if (!product.length) {
                throw new Error('Product not found');
            }

            const currentVersion = product[0].version;

            // Update with version check
            const [result] = await connection.query(
                'UPDATE products SET ?, version = version + 1 WHERE id = ? AND ver
sion = ?',
                [updates, productId, currentVersion]
            );

            if (result.affectedRows === 0) {
                // Version mismatch, retry
                await connection.rollback();
                continue;
            }

            await connection.commit();
            return { success: true };

        } catch (error) {
            await connection.rollback();
            throw error;
        } finally {
            connection.release();
```

```
        }
    }

    throw new Error('Max retries exceeded due to version conflicts');
}
```

---

5. Integrate SQL with Backend APIs

REST API Example (Node.js/Express)

javascript

```javascript
// server.js
const express = require('express');
const app = express();
app.use(express.json());

// Database pool
const pool = require('./db');

// GET /api/users - List users with pagination and filtering
app.get('/api/users', async (req, res) => {
    try {
        const { page = 1, limit = 20, search, status } = req.query;
        const offset = (page - 1) * limit;

        // Build query dynamically
        let query = 'SELECT * FROM users WHERE 1=1';
        const params = [];

        if (search) {
            query += ' AND (name LIKE ? OR email LIKE ?)';
            params.push(`%${search}%`, `%${search}%`);
        }

        if (status) {
            query += ' AND status = ?';
            params.push(status);
```

```
        }

        // Get total count
        const countQuery = query.replace('SELECT *', 'SELECT COUNT(*) as total');
        const [countResult] = await pool.query(countQuery, params);
        const total = countResult[0].total;

        // Add pagination
        query += ' ORDER BY created_at DESC LIMIT ? OFFSET ?';
        params.push(parseInt(limit), offset);

        const [rows] = await pool.query(query, params);

        res.json({
            data: rows,
            pagination: {
                page: parseInt(page),
                limit: parseInt(limit),
                total,
                pages: Math.ceil(total / limit)
            }
        });

    } catch (error) {
        console.error(error);
        res.status(500).json({ error: 'Internal server error' });
    }
});

// GET /api/users/:id - Get single user with relations
app.get('/api/users/:id', async (req, res) => {
    try {
        const { id } = req.params;

        // Get user
        const [users] = await pool.query(
            'SELECT * FROM users WHERE id = ?',
            [id]
        );

        if (!users.length) {
            return res.status(404).json({ error: 'User not found' });
```

```
        }

        const user = users[0];

        // Get user's orders
        const [orders] = await pool.query(
            'SELECT * FROM orders WHERE user_id = ? ORDER BY created_at DESC',
            [id]
        );

        // Get user's reviews
        const [reviews] = await pool.query(
            `SELECT r.*, p.name as product_name
             FROM reviews r
             JOIN products p ON r.product_id = p.id
             WHERE r.user_id = ?`,
            [id]
        );

        res.json({
            ...user,
            orders,
            reviews
        });

    } catch (error) {
        console.error(error);
        res.status(500).json({ error: 'Internal server error' });
    }
});

// POST /api/users - Create user
app.post('/api/users', async (req, res) => {
    try {
        const { email, firstName, lastName, password } = req.body;

        // Validate input
        if (!email || !firstName || !lastName || !password) {
            return res.status(400).json({ error: 'Missing required fields' });
        }

        // Check for existing email
```

```
        const [existing] = await pool.query(
            'SELECT id FROM users WHERE email = ?',
            [email]
        );

        if (existing.length) {
            return res.status(409).json({ error: 'Email already exists' });
        }

        // Hash password (using bcrypt)
        const hashedPassword = await bcrypt.hash(password, 10);

        // Insert user
        const [result] = await pool.query(
            `INSERT INTO users (email, first_name, last_name, password_hash, creat
ed_at)
             VALUES (?, ?, ?, ?, NOW())`,
            [email, firstName, lastName, hashedPassword]
        );

        // Get created user
        const [newUser] = await pool.query(
            'SELECT id, email, first_name, last_name, created_at FROM users WHERE
id = ?',
            [result.insertId]
        );

        res.status(201).json(newUser[0]);

    } catch (error) {
        console.error(error);
        res.status(500).json({ error: 'Internal server error' });
    }
});

// PUT /api/users/:id - Update user
app.put('/api/users/:id', async (req, res) => {
    const connection = await pool.getConnection();

    try {
        await connection.beginTransaction();
```

```
const { id } = req.params;
const updates = req.body;

// Get current user for version check
const [user] = await connection.query(
    'SELECT version FROM users WHERE id = ? FOR UPDATE',
    [id]
);

if (!user.length) {
    await connection.rollback();
    return res.status(404).json({ error: 'User not found' });
}

// Build dynamic update query
const fields = [];
const values = [];

Object.keys(updates).forEach(key => {
    if (['firstName', 'lastName', 'phone'].includes(key)) {
        fields.push(`${key} = ?`);
        values.push(updates[key]);
    }
});

if (fields.length === 0) {
    await connection.rollback();
    return res.status(400).json({ error: 'No valid fields to update' });
}

// Add version check for optimistic locking
fields.push('version = version + 1');
values.push(user[0].version, id);

const [result] = await connection.query(
    `UPDATE users SET ${fields.join(', ')} WHERE id = ? AND version = ?`,
    values
);

if (result.affectedRows === 0) {
    await connection.rollback();
```

```
            return res.status(409).json({ error: 'Version conflict, please retry'
});
        }

        await connection.commit();

        // Get updated user
        const [updated] = await pool.query(
            'SELECT id, email, first_name, last_name, phone, version FROM users WH
ERE id = ?',
            [id]
        );

        res.json(updated[0]);

    } catch (error) {
        await connection.rollback();
        console.error(error);
        res.status(500).json({ error: 'Internal server error' });
    } finally {
        connection.release();
    }
});

// DELETE /api/users/:id - Soft delete
app.delete('/api/users/:id', async (req, res) => {
    try {
        const { id } = req.params;

        // Soft delete
        const [result] = await pool.query(
            'UPDATE users SET is_active = FALSE, deleted_at = NOW() WHERE id = ? A
ND is_active = TRUE',
            [id]
        );

        if (result.affectedRows === 0) {
            return res.status(404).json({ error: 'User not found or already delete
d' });
        }

        res.status(204).send();
```

```
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: 'Internal server error' });
    }
});

// POST /api/orders - Create order with transaction
app.post('/api/orders', async (req, res) => {
    const connection = await pool.getConnection();

    try {
        await connection.beginTransaction();

        const { userId, items } = req.body;

        // Validate items and calculate total
        let total = 0;
        const orderItems = [];

        for (const item of items) {
            const [products] = await connection.query(
                'SELECT id, price, stock FROM products WHERE id = ? FOR UPDATE',
                [item.productId]
            );

            if (!products.length) {
                throw new Error(`Product ${item.productId} not found`);
            }

            const product = products[0];

            if (product.stock < item.quantity) {
                throw new Error(`Insufficient stock for product ${item.productId}`
);
            }

            const itemTotal = product.price * item.quantity;
            total += itemTotal;

            orderItems.push({
                ...item,
```

```
                price: product.price,
                total: itemTotal
            });
        }

        // Create order
        const [orderResult] = await connection.query(
            'INSERT INTO orders (user_id, total, status, created_at) VALUES (?, ?,
?, NOW())',
            [userId, total, 'pending']
        );

        const orderId = orderResult.insertId;

        // Create order items and update stock
        for (const item of orderItems) {
            await connection.query(
                'INSERT INTO order_items (order_id, product_id, quantity, price) V
ALUES (?, ?, ?, ?)',
                [orderId, item.productId, item.quantity, item.price]
            );

            await connection.query(
                'UPDATE products SET stock = stock - ? WHERE id = ?',
                [item.quantity, item.productId]
            );
        }

        await connection.commit();

        // Return created order
        const [order] = await pool.query(
            `SELECT o.*,
                    JSON_ARRAYAGG(
                        JSON_OBJECT('product_id', oi.product_id,
                                    'quantity', oi.quantity,
                                    'price', oi.price)
                    ) as items
             FROM orders o
             JOIN order_items oi ON o.id = oi.order_id
             WHERE o.id = ?
             GROUP BY o.id`,
```

```
            [orderId]
        );

        res.status(201).json(order[0]);

    } catch (error) {
        await connection.rollback();
        console.error(error);
        res.status(400).json({ error: error.message });
    } finally {
        connection.release();
    }
});

// GET /api/reports/sales - Complex reporting endpoint
app.get('/api/reports/sales', async (req, res) => {
    try {
        const { startDate, endDate, groupBy = 'day' } = req.query;

        let dateFormat;
        switch (groupBy) {
            case 'hour':
                dateFormat = '%Y-%m-%d %H:00:00';
                break;
            case 'day':
                dateFormat = '%Y-%m-%d';
                break;
            case 'month':
                dateFormat = '%Y-%m';
                break;
            case 'year':
                dateFormat = '%Y';
                break;
            default:
                dateFormat = '%Y-%m-%d';
        }

        const [rows] = await pool.query(
            `SELECT
                DATE_FORMAT(o.created_at, ?) as period,
                COUNT(DISTINCT o.id) as order_count,
                COUNT(DISTINCT o.user_id) as customer_count,
```

```
                SUM(oi.quantity) as units_sold,
                SUM(o.total) as revenue,
                AVG(o.total) as avg_order_value,
                SUM(o.total) / SUM(oi.quantity) as avg_unit_price
            FROM orders o
            JOIN order_items oi ON o.id = oi.order_id
            WHERE o.status = 'delivered'
              AND o.created_at BETWEEN ? AND ?
            GROUP BY DATE_FORMAT(o.created_at, ?)
            ORDER BY period DESC`,
            [dateFormat, startDate, endDate, dateFormat]
        );

        // Calculate totals
        const totals = rows.reduce((acc, row) => {
            acc.total_orders += row.order_count;
            acc.total_revenue += parseFloat(row.revenue);
            acc.total_units += row.units_sold;
            return acc;
        }, { total_orders: 0, total_revenue: 0, total_units: 0 });

        res.json({
            data: rows,
            summary: totals
        });

    } catch (error) {
        console.error(error);
        res.status(500).json({ error: 'Internal server error' });
    }
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

---

6. Debug Production Issues

Monitoring Queries

sql

```sql
-- MySQL: Find slow queries
-- Enable slow query log
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 2; -- Log queries taking > 2 seconds

-- View slow query log
SELECT * FROM mysql.slow_log ORDER BY start_time DESC LIMIT 10;

-- Current running queries
SHOW FULL PROCESSLIST;

-- Kill a stuck query
KILL QUERY 12345;

-- Query performance insights
SELECT * FROM performance_schema.events_statements_summary_by_digest
ORDER BY SUM_TIMER_WAIT DESC
LIMIT 20;
```

sql

```sql
-- PostgreSQL: Find slow queries
-- Enable logging
ALTER SYSTEM SET log_min_duration_statement = '2s';
SELECT pg_reload_conf();

-- View slow queries
SELECT
    query,
    calls,
    total_time / calls as avg_time,
    rows / calls as avg_rows,
    total_time
FROM pg_stat_statements
ORDER BY total_time DESC
LIMIT 20;
```

Connection Pool Issues

javascript

```javascript
// Monitor connection pool
class PoolMonitor {
    constructor(pool) {
        this.pool = pool;
        this.metrics = {
            totalConnections: 0,
            activeConnections: 0,
            idleConnections: 0,
            waitingClients: 0,
            maxConnections: 0
        };

        // Monitor every 10 seconds
        setInterval(() => this.checkPool(), 10000);
    }

    checkPool() {
        // MySQL2 pool
        if (this.pool.pool) {
            this.metrics = {
                totalConnections: this.pool.pool._allConnections.length,
                activeConnections: this.pool.pool._acquiringConnections.length,
                idleConnections: this.pool.pool._freeConnections.length,
                waitingClients: this.pool.pool._connectionQueue.length,
                maxConnections: this.pool.config.connectionLimit
            };
        }

        // Log if approaching limits
        if (this.metrics.activeConnections > this.metrics.maxConnections * 0.8) {
            console.warn('Connection pool near limit:', this.metrics);
        }

        // Log metrics to monitoring system
        this.sendMetrics();
    }

    async getPoolStatus() {
        return {
```

```
            ...this.metrics,
            utilization: (this.metrics.activeConnections / this.metrics.maxConnect
ions * 100).toFixed(2) + '%'
        };
    }
}

// Health check endpoint
app.get('/health/db', async (req, res) => {
    try {
        const start = Date.now();
        await pool.query('SELECT 1');
        const latency = Date.now() - start;

        const monitor = new PoolMonitor(pool);
        const poolStatus = await monitor.getPoolStatus();

        res.json({
            status: 'healthy',
            latency: `${latency}ms`,
            pool: poolStatus
        });
    } catch (error) {
        res.status(500).json({
            status: 'unhealthy',
            error: error.message
        });
    }
});
```

Deadlock Detection

javascript

```
// Deadlock detection and handling
async function executeWithDeadlockHandling(fn, maxRetries = 3) {
    for (let i = 0; i < maxRetries; i++) {
        try {
            return await fn();
        } catch (error) {
            // Check if it's a deadlock error
            const isDeadlock =
```

```
                error.code === 'ER_LOCK_DEADLOCK' || // MySQL
                error.code === '40001' || // PostgreSQL
                error.message.includes('deadlock'); // Generic

            if (isDeadlock && i < maxRetries - 1) {
                console.log(`Deadlock detected, retry ${i + 1}/${maxRetries}`);

                // Exponential backoff
                await new Promise(resolve =>
                    setTimeout(resolve, 100 * Math.pow(2, i))
                );
                continue;
            }

            throw error;
        }
    }
}

// Log deadlocks for analysis
process.on('deadlock', (error, query) => {
    console.error({
        event: 'deadlock',
        timestamp: new Date().toISOString(),
        error: error.message,
        query: query,
        stack: error.stack
    });

    // Send to monitoring service
    monitoring.captureException(error, {
        tags: { type: 'database_deadlock' },
        extra: { query }
    });
});
```

Query Performance Monitoring

```javascript
// Query performance middleware
app.use(async (req, res, next) => {
```

```
    const start = Date.now();

    // Store original json method
    const originalJson = res.json;

    // Override json to log after response
    res.json = function(data) {
        const duration = Date.now() - start;

        // Log slow queries (> 500ms)
        if (duration > 500) {
            console.warn({
                event: 'slow_request',
                method: req.method,
                path: req.path,
                duration: `${duration}ms`,
                query: req.query,
                user: req.user?.id
            });
        }

        // Store query stats for analysis
        if (req.dbQueries) {
            req.dbQueries.forEach(query => {
                if (query.duration > 200) {
                    console.warn({
                        event: 'slow_query',
                        sql: query.sql,
                        params: query.params,
                        duration: `${query.duration}ms`
                    });
                }
            });
        }

        return originalJson.call(this, data);
    };

    next();
});

// Query timing decorator
```

```
async function timedQuery(connection, sql, params) {
    const start = Date.now();
    try {
        const result = await connection.query(sql, params);
        return result;
    } finally {
        const duration = Date.now() - start;

        // Attach to request for logging
        if (connection.request) {
            if (!connection.request.dbQueries) {
                connection.request.dbQueries = [];
            }
            connection.request.dbQueries.push({
                sql,
                params,
                duration
            });
        }

        // Alert on very slow queries
        if (duration > 1000) {
            console.error({
                event: 'critical_slow_query',
                sql,
                params,
                duration: `${duration}ms`
            });

            // Send alert
            alerting.send({
                severity: 'warning',
                title: 'Critical slow query detected',
                message: `Query took ${duration}ms`,
                sql
            });
        }
    }
}
```

Debugging Checklist

```text
Production Debugging Checklist
═══════════════════════════════

☐ 1. Check database status
    ☐ Connection pool utilization
    ☐ Active connections
    ☐ Long-running queries
    ☐ Lock waits

☐ 2. Analyze slow queries
    ☐ Enable slow query log
    ☐ Get execution plans
    ☐ Check index usage
    ☐ Review table statistics

☐ 3. Check application logs
    ☐ Error logs
    ☐ Slow request logs
    ☐ Database error logs
    ☐ Transaction deadlocks

☐ 4. Monitor system resources
    ☐ CPU usage
    ☐ Memory usage
    ☐ Disk I/O
    ☐ Network latency

☐ 5. Verify configuration
    ☐ Connection pool settings
    ☐ Timeout values
    ☐ Max connections
    ☐ Buffer pool size

☐ 6. Test with production-like data
    ☐ Reproduce issue in staging
    ☐ Test fixes safely
    ✅ Validate before deploying
```

Common Production Issues and Solutions

Issue	Symptoms	Solution
Connection Exhaustion	"Too many connections" error	Increase pool size, fix connection leaks
Deadlocks	Transaction retry errors	Review transaction order, add retry log
Slow Queries	High response time	Add indexes, rewrite queries, denorma
Lock Contention	Queries waiting	Reduce transaction scope, use row-lev
Connection Leaks	Connections never released	Check try/finally blocks, add monitorin
Data Corruption	Inconsistent data	Add constraints, use transactions, audi
Memory Issues	Out of memory errors	Optimize queries, increase buffer pool
Replication Lag	Stale reads	Use read/write splitting, monitor lag

---

Summary: What Companies Expect

Technical Skills

Skill Level	What You Should Know
Beginner (0-1 year)	Basic CRUD, simple joins, indexes
Intermediate (1-3 years)	Complex joins, transactions, optimization, ORM basics

Skill Level	What You Should Know
Advanced (3-5 years)	Performance tuning, partitioning, replication, disaster recovery

Practical Abilities

1. **Design a database schema** for a new feature
2. **Write complex queries** involving multiple tables and aggregations
3. **Optimize slow queries** using EXPLAIN and indexes
4. **Implement transactions** with proper error handling
5. **Integrate SQL with backend APIs** efficiently
6. **Debug production issues** using logs and monitoring
7. **Use ORM effectively** while understanding generated SQL
8. **Write migrations** for schema changes
9. **Implement security** with prepared statements and encryption
10. **Monitor and tune** database performance

Interview Questions

```text
Expected to answer:
- "Design a database for an e-commerce site"
- "How would you optimize this slow query?"
- "Explain the N+1 problem and how to solve it"
- "How do transactions work? Explain ACID"
- "What's the difference between JOIN types?"
- "How do indexes work and when to use them?"
- "How would you handle a deadlock?"
- "Explain your experience with ORMs"
```